

O'REILLY®

2nd Edition

CockroachDB

The Definitive Guide

Distributed Data at Scale



Compliments of



CockroachDB

Guy Harrison, Jesse Seldess,
Ben Darnell & Rob Reid

SECOND EDITION

CockroachDB: The Definitive Guide

Distributed Data at Scale

Guy Harrison, Jesse Seldess, Ben Darnell & Rob Reid

O'REILLY®

Table of Contents

Preface.....	xvii
--------------	------

Part I. Introduction to CockroachDB

1. Introduction to CockroachDB.....	3
A Brief History of Databases	3
Pre-Relational Databases	4
The Relational Model	5
Implementing the Relational Model	7
Transactions	7
The SQL Language	8
The RDBMS Hegemony	8
Enter the Internet	9
The NoSQL Movement	11
The Emergence of Distributed SQL	11
The Advent of CockroachDB	13
CockroachDB Design Goals	14
CockroachDB Releases	16
CockroachDB in Action	17
CockroachDB at Netflix	17
CockroachDB at Devsisters	18
CockroachDB at DoorDash	18
CockroachDB at Bose	18
CockroachDB at Form3	19
CockroachDB at Hard Rock Digital	19
CockroachDB at Spreadly	19

CockroachDB at Route	20
Summary	20
2. CockroachDB Architecture.....	21
The CockroachDB Cluster Architecture	22
The CockroachDB Software Stack	24
The CockroachDB SQL Layer	25
From SQL to Key-Values	27
Tables as Represented in the KV Store	27
Column Families	28
Indexes in the KV Store	29
Inverted Indexes	30
The STORING Clause	30
Table Definitions and Schema Changes	31
The CockroachDB Transaction Layer	32
MVCC Principles	33
Transaction Workflow	34
Write Intents	35
Parallel Commits	36
Transaction Cleanup	37
Overview of Transaction Flow	37
Read/Write Conflicts	39
Clock Synchronization and Clock Skew	40
The CockroachDB Distribution Layer	41
Meta Ranges	41
Gossip	42
Leaseholders	42
Range Splits	43
Multiregion Distribution	44
The CockroachDB Replication Layer	45
Raft	45
Raft and Leaseholders	46
Closed Timestamps and Follower Reads	46
The CockroachDB Storage Layer	47
Log-Structured Merge Trees	47
SSTables and Bloom Filters	48
Deletes and Updates	49
Multiversion Concurrency Control	50
The Block Cache	50
Summary	50

3. Getting Started.....	53
Installation	53
Installing CockroachDB Software	53
Creating a CockroachDB Cloud Basic Instance	57
Starting a Local Single-Node Server	59
Starting Up CockroachDB in a Docker Container	60
Starting Up a Secure Server	62
Shutting Down the Server	63
Remote Connection	63
Creating a Kubernetes Cluster	64
Creating a Cluster with Terraform	66
Using a GUI Client	70
Exploring CockroachDB	71
Adding Some Data	71
Databases and Tables	73
Issuing SQL	74
The DB Console	75
Working with Programming Languages	75
Connecting to CockroachDB from Node.js	76
Connecting to CockroachDB from Java	77
Connecting to CockroachDB from Python	78
Connecting to CockroachDB from Go	79
Summary	81
 4. CockroachDB SQL.....	 83
SQL Language Compatibility	83
Querying Data with SELECT	84
The SELECT List	85
The FROM Clause	85
Joins	85
Anti-Joins	86
Cross Joins	87
Set Operations	87
Group Operations	88
Subqueries	88
Correlated Subquery	89
Lateral Subquery	89
The WHERE Clause	90
Common Table Expressions	90
ORDER BY	91
Window Functions	92
Other SELECT Clauses	93

CockroachDB Arrays	93
Working with JSON	95
Summary of SELECT	97
Creating Tables and Indexes	97
Column Definitions	99
Computed Columns	101
Data Types	101
Primary Keys	103
Constraints	103
Indexes	104
Vectors	107
CREATE TABLE AS SELECT	109
Altering Tables	109
Dropping Tables	110
Views	111
Functions	111
Procedures	114
Inserting Data	115
UPDATE	118
UPSERT	119
DELETE	120
TRUNCATE	122
IMPORT INTO	122
Transactional Statements	122
BEGIN Transaction	123
SAVEPOINT	124
COMMIT	124
ROLLBACK	124
SELECT FOR UPDATE	124
SELECT FOR SHARE	125
AS OF SYSTEM TIME	126
Other Data Definition Language Targets	127
Administrative Commands	128
The Information Schema	129
Summary	129

Part II. Developing Applications with CockroachDB

5. CockroachDB Schema Design.....	133
Logical Data Modeling	134
Normalization	135

Don't Go Too Far	136
Primary Key Choices	137
Special-Purpose Designs	138
Physical Design	138
Entities to Tables	139
Attributes to Columns	140
Primary Key Design	141
Foreign Key Constraints	146
Denormalization	147
Replicating Columns to Avoid Joins	147
Summary Tables	149
Vertical Partitioning	149
Horizontal Partitioning	150
Repeating Groups	151
JSON Document Models	151
JSON Document Antipatterns	153
Indexing JSON Attributes	153
Using JSON or Arrays to Avoid Joins	154
Indexes	156
Index Selectivity	157
Index Break-Even Point	157
Index Overhead	159
Composite Indexes	160
Covering Indexes	161
Composite and Covering Index Performance	162
Guidelines for Composite Indexes	163
Indexes and Null Values	163
Inverted Indexes	163
Partial Indexes	164
Sort-Optimizing Indexes	164
Expression Indexes	165
Full-Text Indexes	167
Spatial Indexes	169
Hash-Sharded Indexes	170
Measuring Index Effectiveness	170
Summary	172
6. Application Design and Implementation.....	175
CockroachDB Programming	175
Performing CRUD Operations	176
Connection Pools	179
Prepared and Parameterized Statements	183

Batch Inserts	185
Pagination of Results	188
Projections	191
Client-Side Caching	192
Managing Transactions	194
Transaction Retry Errors	195
Implementing Transaction Retries	197
Automatic Transaction Retries	198
Using FOR UPDATE to Avoid Transaction Retry Errors	199
Reducing Contention by Eliminating Hot Rows	201
Reducing Transaction Elapsed Time	202
Reordering Statements	203
Time Travel Queries	203
Ambiguous Transactions Errors	204
Deadlocks	205
Transaction Priorities	205
Working with ORM Frameworks	206
Row-Level TTL	210
Summary	212
7. Application Migration and Integration.....	213
Migration Objectives	214
Database Consolidation	214
Failover Regions	215
Fragile Data Integrations	216
Unnecessary Caching Tier	217
Unnecessary Data Warehouse Workloads	218
Application Silos	218
Loading Data	218
File Locations	219
Importing from userfile Storage	220
Importing from Cloud Storage	222
Import Performance	223
Migrating from Another Database	224
Extracting and Converting DDL	224
General Considerations When Converting DDL	230
Exporting Data	231
Migrating Schemas to CockroachDB	232
Loading Data into CockroachDB with MOLT Fetch	233
Loading Data into CockroachDB with IMPORT INTO	235
Directly Importing PostgreSQL or MySQL Dumps	236
Synchronizing and Switching Over	237

Updating Application Code	241
Mainframe Migrations	242
Exporting CockroachDB Data	243
Change Data Capture	245
Core Change Data Capture	245
Using the Changefeed Programmatically	248
Enterprise Change Data Capture	249
CDC Queries	250
Change Data Capture to Kafka	256
Change Data Capture to Pulsar	257
Change Data Capture to Snowflake	259
Summary	264
8. SQL Tuning.....	265
Finding Slow SQL	265
Explaining and Tracing SQL	267
EXPLAIN ANALYZE	272
EXPLAIN Options	273
EXPLAIN DEBUG	276
Changing SQL Execution	278
Optimizing Table Lookups	279
Optimizing Joins	289
Join Methods	290
Optimizing Sorting and Aggregation	300
Disk Sorts	303
Optimizing DML	305
Optimizing the Optimizer	307
Optimizer Statistics	307
Viewing Statistics	307
Automatic Statistics	308
Manually Collecting Statistics	309
Summary	310

Part III. Deploying and Administering CockroachDB

9. Planning a Deployment.....	313
Know Your Requirements	314
Comparison of Deployment Options	314
CockroachDB Cloud Basic Deployments	317
CockroachDB Cloud Standard Deployments	319
Single-Region Advanced Deployments	319

Common Planning Tasks—Advanced Deployments	319
Benchmarking and Capacity Planning	320
CockroachDB Cloud Deployments	322
Self-Hosted on a Cloud Platform	324
Self-Hosted “Bare-Metal” On-Premises	325
Other Self-Hosted Considerations	327
Self-Hosted Kubernetes	327
Configuring for Self-Hosted High Availability	329
Disk Failure	329
Node Failures	329
Network Failure	330
Zone and Region Topologies	331
Summary	332
10. Single-Region Deployment.....	335
Deploying On-Premises or On-Cloud	335
Firewall Configuration	336
Operating System Configuration	336
Clock Synchronization On-Premises	337
Clock Synchronization on Cloud Platforms	338
Creating Certificates	340
Configuring the Nodes	342
Creating a Ballast File	343
Initializing the Cluster	344
Creating the First User	345
Installing a Load Balancer (On-Premises)	347
Cloud Load Balancers	349
Configuring Regions and Zones	350
Deploying on Kubernetes	350
Initializing the Operator	351
Initializing the Cluster	352
Creating a Client Pod	354
Load Balancing	354
Other Kubernetes Tasks	356
Summary	356
11. Multiregion Deployment.....	359
Multiregion Concepts	359
Regions and Zones	359
Survival Goals	361
Locality Rules	363
Planning Your Multiregion Deployment	365

Deploying in Multiregion	365
Converting to a Multiregion Database	368
Configuring Regional by Row	369
Setting Regional Survival Goal	372
Super Regions	373
Summary	374
12. Backup and Disaster Recovery.....	375
Backups	376
The BACKUP Command	377
Backup Destinations	378
Full Backup	379
Table- and Database-Level Backups	379
Incremental Backups	380
AS OF SYSTEM TIME Backup	380
Encrypted Backups	381
WITH REVISION HISTORY	381
SHOW BACKUP	382
Managing Backup Jobs	383
Scheduling Backups	384
Locality-Aware Backups	387
Restoring Data	387
Backup Validation	390
Exporting Data	391
Disaster Recovery Best Practices	392
Backup Scheduling and Configuration	392
Recovering from Human Errors	393
Two-Data Center Replication	394
Physical Cluster Replication	394
Logical Data Replication	398
Summary	402
13. Security.....	403
Firewall Configuration	404
IP Allowlist with CockroachDB Cloud	404
Egress Perimeter Controls	405
Private Connectivity and VPC Peering	406
Native Linux Firewall	410
Configuring a Firewall in GCP	412
Configuring a Firewall in AWS	413
Configuring Ports for Microsoft Azure	414
Encryption and Server Certificates	415

Encryption at Rest	416
Customer-Managed Encryption Keys	418
Authentication Mechanisms	423
Standard Authentication	423
Advanced Authentication	424
Authorization	424
Managing Users	425
Managing Privileges	425
Fine-Grained Access Control with Views	427
Logging and Auditing	428
Security Best Practices	431
Summary	432

14. Administration and Troubleshooting. 433

Monitoring	433
CockroachDB Cloud Advanced Alerts	434
CockroachDB Cloud Standard Alerts	435
CockroachDB Cloud Basic Alerts	435
Availability Monitoring	436
The Cloud API	437
The Cluster API	441
Monitoring and Alerting with Prometheus	443
The Metrics Export Endpoint	444
Monitoring and Alerting with Datadog	445
Log Configuration	446
Log Channels	449
Log Format	450
Filter Levels	451
Log Destinations	452
Logging to Fluentd	452
Redaction	453
Logs in Cloud Deployments	453
Cluster Management	454
Upgrading the Cluster Version	454
Adding Nodes to a Cluster	456
Decommissioning Nodes	457
Troubleshooting	460
Clock Synchronization Errors	461
Node Liveness	461
Networking Issues	462
Loss of Client Connectivity	464
Running Out of Disk Space	464

Working with CockroachDB Support Resources	464
Summary	465
15. Cluster Optimization.....	467
Tuning Versus Firefighting	467
Workload Optimization	469
Detecting Problem Workloads	469
Review of Workload Optimization Strategies	472
Indexing	472
Ad Hoc or Analytic Queries	473
Cluster Balance	475
Causes of Imbalance	476
Admission Control	485
Network	486
Memory Optimization	488
Key-Value Cache	489
max-sql-memory	490
Host Memory	491
Disk I/O	491
Scaling Out	494
Summary	494
Index.....	497

Preface

Welcome to *CockroachDB: The Definitive Guide*, and thank you for being here! With this book, we want to help you learn to build and deploy applications with CockroachDB, the distributed SQL database that meets you where you're at on your journey—in the cloud, on-premises, or hybrid.

First, the question everyone asks: *Why the name CockroachDB?*

One immutable fact of engineering is that things break. At large enough scale, things are breaking all the time. The kinds of failures that might happen once a year on a single machine become daily occurrences when you're running thousands of them. A system that aspires to handle large scale must treat fault tolerance as a core responsibility. This was one of the key insights of Google's MapReduce framework. By requiring all computation to fit within a relatively restrictive framework, it became straightforward for the system to automatically rerun the necessary pieces of work after a failure.

We believe that this is how it should be: highly available replication should be the default state of a database from Day One, not the result of painstaking configuration work. And looking ahead to Day Two (or Day Two Hundred), the database must be able to grow along with the application so that runaway success is a cause for celebration, not panic.

When Cockroach Labs set out to build a relational database from scratch, we wanted to bring consistency, native resilience, data locality, and massive scale to modern applications. Our vision was of a system able to colonize any resource that you gave it and then relentlessly optimize itself. A database that would use available space and reach equilibrium across a coordinated set of distributed nodes so that it would not only incorporate new resources, but—if a machine or data center or even an entire region went down—the database would simply equalize the remaining available resources.

We aimed to build a database that would be impossible to eradicate...just like *periplaneta brunnea*, the pesky common cockroach. And so CockroachDB was born.

Building CockroachDB

CockroachDB's origin is a tale of necessity. Quite literally, Spencer Kimball, Peter Mattis, and I, Ben Darnell, set out to build the relational database that we ourselves needed.

After working together at Google, the three of us went in different directions for a while. Eventually we ended up back together (along with Spencer's brother Andy, a Microsoft SQL Server veteran) at a startup called Viewfinder, building an application for photo organization and social sharing. We, of course, hoped it would be successful, so we wanted to build for scale. After all, as Spencer says, "Sometimes shit happens. And, at scale, shit's always happening."

It turns out that, given the history of that project, we didn't do well enough to ever need massive scalability, but we wanted to plan for it from the beginning. We also didn't want to lock ourselves into a monolithic database, sharding MySQL or anything like that. We had been down that road before and knew it didn't lead where we wanted to go. This left us looking at NoSQL options as the best alternative and eventually we settled on DynamoDB.

DynamoDB ticked a lot of boxes: it's scalable, it's fast, it's predictable. It's very similar to Bigtable, which I had used as the backend for Google Reader while I was at Google. And so, on paper at least, we really liked that model. But as we got experience with it, we realized we had a couple of fundamental problems with NoSQL.

The biggest one was we found that we really needed secondary indexes. The Bigtable/DynamoDB model is really primary keys only; there's no concept of secondary indexes. To get them, we had to build our own partial implementation of transactions on top of DynamoDB, which was workable. Once we were using this system, however, we found a subsequent problem. DynamoDB on its own is very fast, predictable, and scalable—but once you start doing things that combine multiple records, as we were doing with our secondary indexes, then things got difficult.

Ultimately, we came up with a system that worked, but the layering of transactions and indexes on top of DynamoDB was inefficient, both in terms of performance and our engineering efforts.

During this time we did start talking about the idea of building a database ourselves, but Viewfinder was a startup with a small team. We decided, *You know, DynamoDB is not ideal, but it exists and we can use it today, so that's what we'll do.* But then a

year or two later Square bought Viewfinder, and we found out they were struggling with sharded MySQL. We realized then, *Okay, it's not just us, this is a real database need that's not getting met*. At Square we were now in a good position to try building a system that was both inherently scalable and supported secondary indexes. So that's where the effort really started in earnest.

Spencer started on it as a side project on nights and weekends at first. And then he got some more contributors, and eventually it even became an official project at Square. But we knew that the real destiny of this project was to serve the entire database market, so that's when we decided to leave Square and start a company to do just that: Cockroach Labs was founded in 2015.

Next Steps

Having set out to build a database that is as indestructible as an intrusion of cockroaches, the next question was, of course, what other capabilities it would need.

We believed that the right approach was to embrace distributed transactions from the start and make them an integral part of the architecture. And while Structured Query Language (SQL) is hardly anyone's idea of a perfect query language, it is the one that everyone knows, with a time-tested feature set. SQL's declarative schema management and statistics-based query optimization give operators powerful tools to manage their application's performance at runtime.

So the story of getting to CockroachDB version 1.0 was solving the new problem of scale plus transactions using SQL. The result is a globally distributed SQL database built on top of a transactional and consistent key-value (KV) store. The primary design goals are support for atomic, consistent, isolated, and durable (ACID) transactions; horizontal scalability; low latency/high availability; and survivability. CockroachDB implements a Raft consensus algorithm for consistency and aims to tolerate disk, machine, rack, and even data center failures with minimal disruption and no manual intervention.

Once we started building and deploying this, we found that the greatest interest in what we were doing came from people working with global deployments. It turns out this is something Cockroach is uniquely capable of handling, and so with version 2.0 we started building more and more features for global and geographic distribution.

It's been a long time since we launched CockroachDB 2.0 and multiregion. Since then, the biggest developments have been CockroachDB Cloud followed by CockroachDB Cloud Basic. These fulfilled our next mission: to make data easy.

Our Viewfinder startup days showed us the need for a database that can be used for projects from Day One and would also grow along with it, without breaking the bank. You can self-host CockroachDB locally on a laptop, on a corporate dev cluster or private cloud, as well as on any public cloud infrastructure, but the price of entry is still fairly substantial. The first step toward making CockroachDB accessible was to take away the operational burden from the user, and we achieved that with our cloud-hosted, fully managed service now called CockroachDB Cloud Advanced.

Now we have added CockroachDB Cloud Basic to give users the low-cost or even no-cost foot in the door that you can use from the beginning, and then it scales up affordably with your usage. It's now inexpensive enough for everyone to simply build multiregion by default.

What comes after that? Well, we are continuing to explore ways to use cloud native features to gain even greater efficiencies and drive down operating costs. Things like looking at the menu of various storage solutions that the clouds offer and choosing the appropriate ones for our different kinds of storage needs instead of using block storage devices for everything as we currently do.

Along the way, we've added user-defined functions and stored procedures. This is a lightning rod for discussion because people have strong opinions, but I think that when you combine user-defined functions and stored procedures with a scalable database, it gets really interesting. It can really transform the way you think about your application architecture because it lets you shift more of your logic into the database.

More speculatively? As CockroachDB's cofounder, I'd like to explore ways that you could host your entire application, just straight out of the database, using stored procedures. Even saying that out loud is going to be a bit controversial because that's the sort of thing that people tried to do back in, like, 1998 and, we admit, it didn't work well then.

However, I think things have evolved to the point that now may be the right time. That's still very speculative, just something we're thinking about and not something that is officially on the Cockroach Labs roadmap at this time. But we'd like to try to hack out some sort of prototype instead of just talking about it. Maybe now that this book is published, that will free up some time to experiment.

Why We Wrote This Book

This book aims to help you, the reader, understand the architecture and capabilities of CockroachDB as well as suitable use cases for CockroachDB. By the end of this book, you will be able to get started with CockroachDB, build effective applications on it, and, ultimately, run a cluster in production.

The chapters teach developers, architects, and DevOps teams how to build, optimize, and manage applications that run on CockroachDB. We wrote this book to give any developer, no matter how they want to work with CockroachDB, the knowledge and tools they need to do it effectively.

— Ben Darnell
March 2025
New York City

Who This Book Is For

Whether you're an architect designing systems, a developer crafting code, or a DevOps engineer streamlining deployments, this book will guide you through everything you need to know about CockroachDB—from fundamental concepts and basic operations to advanced features and production deployment strategies:

Innovative developers

CockroachDB: The Definitive Guide is for developers building modern applications, whether in a small startup or a large enterprise. This guide will teach developers how to build and ship apps with fewer obstacles using a powerful distributed SQL database that just works.

Architects

The Definitive Guide is for modern architects delivering scalable, resilient apps across their IT ecosystem. Architects will learn how to design distributed applications to provide low latency, high availability, and faster performance.

DevOps teams

The Definitive Guide is for strategic IT operators managing applications with data-intensive workloads. Operators will learn how to optimize CockroachDB's inherent abilities of scale and resilience, efficiently distributing data to meet any workload demand—wherever it's deployed.

Those already familiar with distributed systems will discover the benefits of strong data correctness and consistency guarantees as well as optimizations for delivering low-latency transactions to users anywhere in the world.

It includes specific guidance for anyone transitioning from a monolithic database (e.g., MySQL or PostgreSQL) to a distributed architecture, as well as practical examples for anyone more familiar with NoSQL systems.

How This Book Is Organized

This book contains 15 chapters divided into three sections:

- **Part I, “Introduction to CockroachDB”**, establishes the historical context of CockroachDB, covering the evolution of databases and the emergence of distributed cloud databases. Readers cover core concepts of distributed SQL, then dive into the architecture and capabilities of CockroachDB, followed by examinations of suitable use cases for CockroachDB. **Part I** ends with a hands-on guide to getting started, both with a local installation and Cockroach Labs’ free cloud service, and a thorough guide to CockroachDB’s SQL dialect.
- **Part II, “Developing Applications with CockroachDB”**, covers the fundamentals of CockroachDB schema design. Next, readers work through application design and implementation as well as integration with and/or migration from other databases. You will practice working with data in CockroachDB, including managing transactions, exploring change data capture, and building SQL tuning skills.
- **Part III, “Deploying and Administering CockroachDB”**, covers the planning and execution of single and multiregion deployments. Essential topics include backup and disaster recovery; security; monitoring, administration, and troubleshooting; and cluster optimization.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Accessing the Book's Images Online

Readers of the printed book can access full-color, large-format versions of the images online at <https://oreil.ly/cockroachDB-definitive-guide-figs>.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://oreil.ly/cockroachDB-definitive-guide-code>.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*CockroachDB: The Definitive Guide*, 2nd ed., by Guy Harrison, Jesse Seldess, Ben Darnell, and Rob Reid (O'Reilly). Copyright 2025 O'Reilly Media, Inc., 978-1-098-17984-7.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/cockroachDB-def-guide-2e>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

Writing this book was a truly collaborative venture, and the authors are thankful for the many people who helped along the way, especially Andy Pavlo, Sean Chittenden, and Matt Aslett, who provided their knowledgeable opinions around Distributed SQL and CockroachDB to be shared in (and on) this book.

The authors would like to thank the vital open source community that supports and contributes to CockroachDB, as well as the many Roachers who joined Cockroach Labs to help us continually extend and improve the database. Our special thanks go

to the reviewers who applied their subject matter expertise—gained from building CockroachDB—to careful technical review of each chapter. Andy Woods, Vy Ton, Piyush Singh, Liv Lobo, Abbey Russell, Keith McClellan, Chris Ireland, Greg Turnquist, Alicia Lu, Rohan Joshi, and David Bressler, all from Cockroach Labs, we are grateful for your thoughtful suggestions and keen-sighted corrections that helped improve every part of this book. We are equally grateful to external reviewers Alan Beaulieu, consultant, and software engineer and machine learning specialist Patrick Deziel from Rotational Labs, LLC. We would also like to thank Jessica Edwards for shepherding the process from Day One, and Michelle Gienow for then taking over and helping us cross the finish line.

We'd like to thank the team from O'Reilly whose consistent diligence and professionalism kept us on track and ensured we created a high-quality and useful book. Thanks especially to Angela Rufino, Andy Kwan, and Sharon Cordesse: it really was a pleasure to work with you. Additional thanks to the production team: Christopher Faucher, Gregory Hyman, Stephanie English, Dwight Ramsey, Kim Cofer, Johnna VanHoose Dinse, and Kate Dullea.

Guy Harrison

I'd like to thank all the folks at O'Reilly and CockroachDB who collaborated on the book, and particularly Ben and Jesse for giving me the opportunity to work with them to write this guide. As the “old man” of the team, I've been writing books on database technology for more than 20 years, and the opportunity to work at the cutting edge of modern database technology was really exciting. I'd, of course, most especially like to thank my wonderful wife, Jenny, who makes every day a good day, and who tolerated my distraction and absence during this project.

Jesse Seldess

I would like to thank everyone who has been part of the Cockroach Labs Documentation team over the years. Great docs make for self-sufficient and happy users, and I've been fortunate to work with dedicated, talented writers producing world-class docs that users can trust and rely on for their day-to-day work. Many, many thanks to Guy Harrison as well; your extensive database and development experience took this project to new levels, and your productive writing approach kept us in constructive discussion and on track. Finally, love and appreciation to my wife, Leonie, and kids, Selma and Paz, for helping me find space and clarity to do my part in this book, all my work at Cockroach Labs, and so much more.

Ben Darnell

I would like to thank Spencer Kimball and Peter Mattis for being colleagues, cofounders, and true collaborators in creating CockroachDB. It's been quite a journey

so far, and putting this guide together was a constant reminder of just how far we have come since we first came up with the crazy idea of building our own distributed database. Enormous thanks also to Guy Harrison, whose deep existing database knowledge paired with his diligence in learning Cockroach just as deeply contributed immeasurably to the scope and thoroughness of the book. I also wish to thank Brandi Evans for making it possible for me to find time to step away to work on this project. Most of all, I want to thank my wife, Juliet Moser, for her support and patience throughout the evenings and weekends it took to produce this book, as well as the many other reasons I am grateful for her.

Rob Reid

Whenever my Roacher colleagues head off for Thanksgiving, I always joke that I'll hold the fort (on account of my being British and not having anything to be thankful for). In reality of course, I'm bursting with appreciation. From the moment I discovered CockroachDB back in 2016, I knew it was special. Then, as a customer of Cockroach Labs in 2018, I realized that the company itself was also special. In early 2023, I joined Cockroach Labs as Technical Evangelist (it was only a matter of time) and this book's second edition is just one of the many ways I can now express my appreciation of this incredible technology. I'd like to thank all of my friends at Cockroach Labs (especially Greg Turnquist, Chris Ireland, Alicia Lu, David Bressler, Ray Austin, and Ryan Kelly for their reviews and support with this project) and the wonderful folks at O'Reilly (especially Angela Rufino for making this project so much fun). Last but certainly not least, I want to thank my wife, Emily, and my kids, Ruby and Rex, for putting up with my incessant enthusiasm for CockroachDB.

PART I

Introduction to CockroachDB

Introduction to CockroachDB

CockroachDB is a distributed, transactional, relational, cloud native SQL database system. That’s quite a mouthful! But in short, CockroachDB leverages both the strengths of the previous generation of relational database systems—strong consistency, the power of SQL, and the relational data model—and the strengths of modern distributed cloud principles. The result is a database system that is broadly compatible with other SQL-based transactional databases but delivers much greater scalability and availability.

In this chapter, we’ll review the history of database management systems (DBMSs) and discover how CockroachDB exploits technology advances of the last few decades to deliver on its ambitious goals.

A Brief History of Databases

Data storage and data processing are the “killer apps” of human civilization. Verbal language gave us an enormous advantage in cooperating as a community. Still, it was only when we developed data storage—e.g., written language—that each generation could build on the lessons of preceding generations.

The earliest written records—dating back almost 10,000 years—are agricultural accounting records. These cuneiform records, recorded on clay tablets ([Figure 1-1](#)), serve the same purpose as the databases that support modern accounting systems.

Information storage technologies over thousands of years progressed slowly. The use of cheap, portable, and reasonably durable paper media organized in libraries and cabinets represented best practices for almost a millennia.



Figure 1-1. Cuneiform tablet circa 3000 BC (Source: [Wikipedia](#))

The emergence of digital data processing has truly resulted in an information revolution. Within a single human life span, digital information systems have resulted in exponential growth in the volume and rate of information storage. Today, the vast bulk of human information is stored in digital formats, much of it within database systems.

Pre-Relational Databases

The first digital computers had negligible storage capacity and were used primarily for computation—for instance, generating ballistic tables, decrypting codes, and performing scientific calculations. However, as magnetic tape and disks became mainstream in the 1950s, it became increasingly possible to use computers to store and process volumes of information that would be unwieldy by other means.

Early applications used simple flat files for data storage. But it soon became obvious that the complexities of reliably and efficiently dealing with large amounts of data required specialized and dedicated software platforms—and these became the first data systems.

Early database systems ran within monolithic mainframe computers, which also were responsible for the application code. The applications were tightly coupled with the database systems and processed data directly using procedural language directives. By the 1970s, two models of database systems were vying for dominance: the *network* and *hierarchical* models. These models were represented by the major databases of

the day, *IMS* (Information Management System) and *IDMS* (Integrated Database Management System).

These systems were great advances on their predecessors but had significant drawbacks. Queries needed to be anticipated in advance of implementation, and only record-at-a-time processing was supported. Even the simplest report required programming resources to implement, and all IT departments suffered from a huge backlog of reporting requests.

The Relational Model

Probably no one has had more influence over database technology than Edgar Codd (whatever Larry Ellison might think). Codd was a “programming mathematician”—what we might today call a data scientist—who had worked at IBM on and off since 1949. In 1970, Codd wrote his seminal paper, “[A Relational Model of Data for Large Shared Data Banks](#)”. This paper outlined what Codd saw as fundamental issues in the design of existing database systems:

- Existing database systems merged physical and logical representations of data in a way that often complicated requests for data and created difficulties in satisfying requests that were not anticipated during database design.
- There was no formal standard for data representation—as a mathematician, Codd was familiar with theoretical models for representing data—and he believed these principles should be applied to database systems.
- Existing database systems were too hard to use. Only programmers were able to retrieve data from these systems, and the process of retrieving data was needlessly complex. Codd felt that there needed to be an easy-access method for data retrieval.

Codd’s relational model described a means of logically representing data that was independent of the underlying storage mechanism. It required a *query language* that could be used to answer any question that could be satisfied by the data.

The relational model defines the fundamental building blocks of a relational database:

- *Tuples* are a set of *attribute* values. Attributes are named scalar (single-dimensional) values. A tuple can be thought of as an individual “record” or “row.”
- A *relation* is a collection of distinct tuples of the same form. A relation represents a two-dimensional data set with a fixed number of attributes and an arbitrary number of tuples—a table in a database is an example of a relation.
- *Constraints* enforce consistency and define relationships between tuples.

- Various *operations* are defined, such as joins, projections, and unions. Operations on relations always return relations. For instance, when you join two relations, the result is itself a relation.
- A *key* consists of one or more attributes that can be used to identify a tuple. There can be more than one key, and a key can consist of multiple attributes.

The relational model furthermore defined a series of “normal forms” that represent reducing levels of redundancy in the model. A relation is in *third normal form* if all the data in each tuple is dependent on the entire primary key of that tuple and on no other attributes. We generally remember this by the adage, “The key, the whole key, and nothing but the key (so help me, Codd).”

Third normal form generally represents the starting point for the construction of an efficient and performant data model. We will come back to third normal form in [Chapter 5](#). [Figure 1-2](#) illustrates data in third normal form.

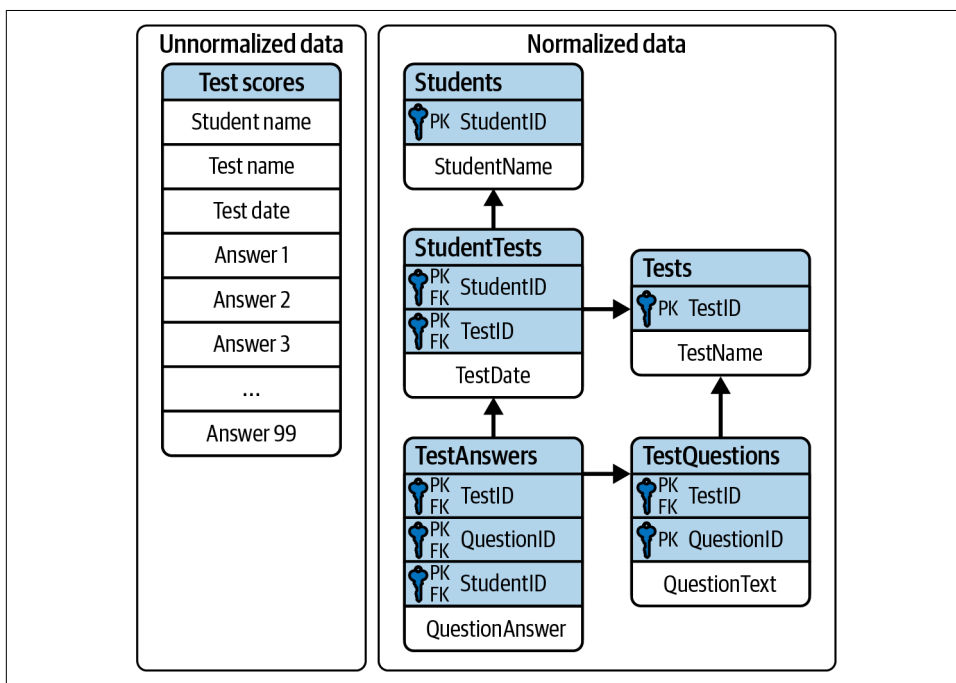


Figure 1-2. Data represented in a relational “third normal form” structure

Implementing the Relational Model

The relational model served as the foundation for the familiar structures present in all relational databases today. Tuples are represented as *rows* and relations as *tables*.

A table is a relation that has been given physical storage. The underlying storage may take different forms. In addition to the physical representation of the data, indexing and clustering schemes were introduced to facilitate efficient data processing and implement constraints.

Indexes and clustered storage were not a part of the relational model, but they were incorporated in relational databases to transparently enhance query performance without changing the types of queries that could be performed. Thus, the logical representation of the data as presented to the application was independent of the underlying physical model.

Indeed, in some relational implementations, a table might be implemented by multiple indexed structures, allowing different access paths to the data.

Transactions

A transaction is a logical unit of work that must succeed or fail as a unit. Transactions predated the relational model, but in pre-relational systems, transactions were often the responsibility of the application layer. In Codd's relational model, the database took formal responsibility for transactional processing.¹ In Codd's formulation, a relational system would provide explicit support for commencing a transaction and either committing or aborting that transaction.

The use of transactions to maintain consistency in application data was also used internally to maintain consistency between the various physical structures that represented tables. For instance, when a table is represented in multiple indexes, all of those indexes must be kept synchronized in a transactional manner.

Codd's relational model did not define all the aspects of transactional behavior that became common to most relational database systems. In 1981, Jim Gray articulated the core principles of transaction processing that we still use today. These principles later became known as *ACID*—atomic, consistent, isolated, and durable—transactions.

As Gray put it, “A transaction is a transformation of state which has the properties of *atomicity* (all or nothing), *durability* (effects survive failures) and *consistency* (a correct transformation).” The principle of *isolation*—added in a later revision—

¹ From “Rule 5” in Codd's 12 rules, which were published in the early 1980s.

required that one transaction should not be able to see the effects of other in-progress transactions.

Perfect isolation between transactions—*serializable* isolation—creates some restrictions on concurrent data processing. Many databases adopted lower levels of isolation or allowed applications to choose from various isolation levels. These implications will be discussed further in [Chapter 2](#).

The SQL Language

Codd specified that a relational system should support a “database sublanguage” to navigate and modify relational data. He proposed the Alpha language in 1971, which influenced the QUEL language designed by the creators of Ingres—an early relational database system developed at the University of California, which influenced the open source PostgreSQL database.

Meanwhile, researchers at IBM were developing System R, a prototype DBMS based on Codd’s relational model. They developed the SEQUEL language as the data sublanguage for the project. SEQUEL eventually was renamed SQL and was adopted in commercial IBM databases, including IBM DB2.

Oracle chose SQL as the query language for its pioneering Oracle relational database management system (RDBMS), and by the end of the 1970s, SQL had won out over QUEL as the relational query language and became an ANSI (American National Standards Institute) standard language in 1986.

SQL needs little introduction. Today, it’s one of the most widely used computer languages in the world. We will devote [Chapter 4](#) to the CockroachDB SQL implementation. However, it’s worth noting that the relative ease of use that SQL provided expanded the audience of database users dramatically. No longer did you need to be a highly experienced database programmer to retrieve data from a database—SQL could be taught to casual users of databases, such as analysts and statisticians. It’s fair to say that SQL brought databases within reach of business users.

The RDBMS Hegemony

The combination of the relational model, SQL language, and ACID transactions became the dominant model for new database systems from the early 1980s through the early 2000s. These systems became known generically as RDBMS.

The RDBMS came into prevalence around the same time as a seismic paradigm shift in application architectures. The world of mainframe applications was giving way to the *client/server* model. In the client/server model, application code ran on microcomputers (PCs) while the database ran on a minicomputer, increasingly running the Unix OS. During the migration to client/server, mainframe-based pre-relational databases were largely abandoned in favor of the new breed of RDBMSs.

By the end of the 20th century, the RDBMS reigned supreme. The leading commercial databases of the day—Oracle, Sybase, SQL Server, Informix, and DB2—competed on performance, functionality, or price, but all were virtually identical in their adoption of the relational model, SQL, and ACID transactions. As open source software grew in popularity, open source RDBMSs such as MySQL and PostgreSQL gained significant and growing traction.

Enter the Internet

Around the turn of the 21st century, an even more important shift in application architectures occurred. That shift was, of course, the internet. Initially, internet applications ran on a software stack not dissimilar to a client/server application. A single large server hosted the application's database, while application code ran on a “middle tier” server and end users interacted with the application through web browsers.

In the early days of the internet, this architecture sufficed—though often just barely. The monolithic database servers were often a performance bottleneck, and although standby databases were routinely deployed, a database failure was one of the most common causes of application failure.

As the web grew, the limitations of the centralized RDBMS became untenable. The emerging “web 2.0” social network and ecommerce sites had two characteristics that were increasingly difficult to support:

- These systems had a global or near-global scale. Users in multiple continents needed simultaneous access to the application.
- Any level of downtime was undesirable. The old model of “weekend upgrades” was no longer acceptable. There was no maintenance window that did not involve significant business disruption.

All parties agreed that the monolithic single database system would have to give way if the demands of the new breed of internet applications were to be met. It became recognized that a very significant and potentially immovable obstacle stood in the way: *CAP theorem*. CAP—or **Brewer's**—theorem states that you can have at most two of three desirable characteristics in a distributed system (illustrated in **Figure 1-3**):

Consistency

Every user sees the same view of the database state.

Availability

The database remains available unless all elements of the distributed system fail.

Partition tolerance

The system runs in an environment in which a network partition might divide the distributed system in two, or if two nodes in the network cannot

communicate, a partition-tolerant system will continue to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

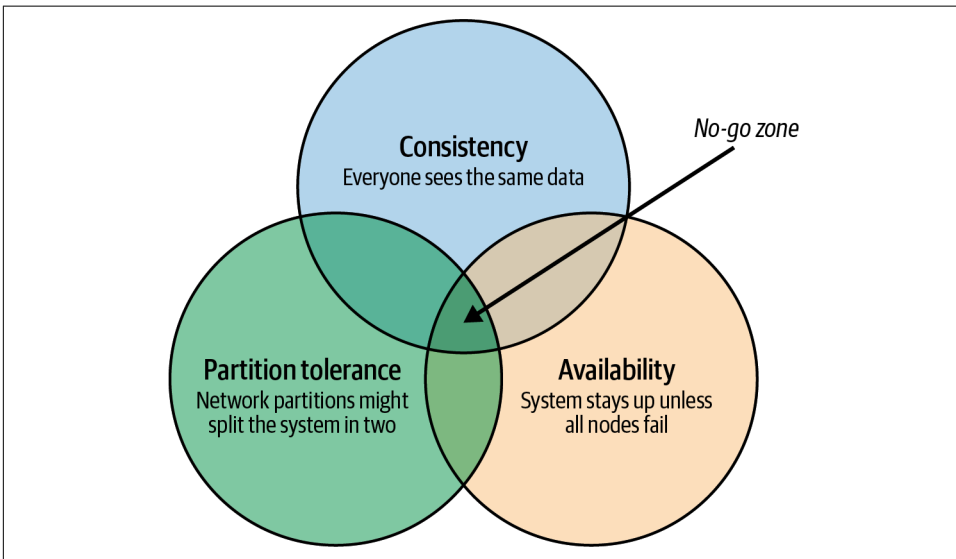


Figure 1-3. CAP theorem states that a system cannot support all three properties of consistency, availability, and partition tolerance

For instance, consider the case of a global ecommerce system with users in North America and Europe. If the network between the two continents fails (a network partition), then you must choose one of the following outcomes:

- Users in Europe and North America may see different versions of the database: *sacrificing consistency*.
- One of the two regions needs to shut down (or go read-only): *sacrificing availability*.

Clustered RDBMSs at that time would generally sacrifice availability. For instance, in Oracle’s Real Application Clusters (RAC) clustered database, a network partition between nodes would cause all nodes in one of the partitions to shut down.

Internet pioneers such as Amazon, however, believed that availability was more important than strict consistency. Amazon developed a database system—*Dynamo*—that implemented “*eventual consistency*.” In the event of a partition, all zones would continue to have access to the system, but when the partition was resolved, inconsistencies would be reconciled—possibly losing data in the process.

The NoSQL Movement

Between 2008 and 2010, dozens of new database systems emerged, all of which abandoned the three pillars of the RDBMS: the relational data model, SQL language, and ACID transactions. Some of these new systems—Cassandra, Riak, Project Voldemort, and HBase, for example—were directly influenced by nonrelational technologies developed at Amazon and Google.

Many of these systems were essentially “schema-free”—supporting or even requiring no specific structure for the data they stored. In particular, in key-value (KV) databases, an arbitrary key provides programmatic access to an arbitrary structured “value.” The database knows nothing about what is in this value. From the database’s view, the value is just a set of unstructured bits. Other nonrelational systems represented data in semi-tabular formats or as JSON (JavaScript Object Notation) documents. However, none of these new databases implemented the principles of the relational model.

These systems were initially referred to as distributed nonrelational database management systems (DNRDBMSs), but—because they did not include the SQL language—rapidly became known by the far catchier term “NoSQL” databases.

NoSQL was always a questionable term. It defined what the class of systems discarded, rather than their unique distinguishing features. Nevertheless, “NoSQL” stuck, and in the following decade, NoSQL databases such as Cassandra, DynamoDB, and MongoDB became established as a distinct and important segment of the database landscape.

The Emergence of Distributed SQL

The challenges of implementing distributed transactions at a web scale, more than anything else, led to the schism in modern DBMSs. With the rise of global applications with extremely high uptime requirements, it became unthinkable to sacrifice availability for perfect consistency. Almost in unison, the leading web 2.0 companies such as Amazon, Google, and Facebook introduced new database services that were only “eventually” or “weakly” consistent but globally and highly available, and the open source community responded with databases based on these principles.

However, NoSQL databases had their own severe limitations. The SQL language was widely understood and was the basis for almost all business intelligence tools. NoSQL databases found that they had to offer some SQL compatibility, so many added some SQL-like dialect—leading to the redefinition of NoSQL as “not only SQL.” In many cases, these SQL implementations were query-only and intended only to support business intelligence features. In other cases, a SQL-like language supported transactional processing but provided only the most limited subset of SQL functionality.

The problems caused by weakened consistency were harder to ignore. Consistency and correctness in data are often nonnegotiable for mission-critical applications. While in some circumstances—social media, for instance—it might be acceptable for different users to see slightly different views of the same topic, in other contexts—such as logistics—any inconsistency is unacceptable. Advanced nonrelational databases adopted tunable consistency and sophisticated conflict resolution algorithms to mitigate data inconsistency. However, any database that abandons strict consistency must accept scenarios in which data can be lost or corrupted during the reconciliation of network partitions or from ambiguously timed competing transactions.

Google pioneered many of the technologies behind important open source NoSQL systems. For instance, the Google File System and MapReduce technologies led directly to Apache Hadoop, and Google Bigtable led to Apache HBase. As such, Google was well aware of the limitations of these new data stores.

The Spanner project was initiated as an attempt to build a distributed database, similar to Google’s existing Bigtable system, that could support both strong consistency and high availability.

Spanner benefited from Google’s highly redundant network, which reduced the probability of network-based availability issues, but the really novel feature of Spanner was its *TrueTime* system. TrueTime explicitly models the uncertainty of time measurement in a distributed system so that it can be incorporated into the transaction protocol. Distributed databases go to a lot of effort to return consistent information from replicas maintained across the system. Locks are the primary mechanism to prevent inconsistent information from being created in the database, while snapshots are the primary mechanism for returning consistent information. Queries don’t see changes to data that occur while they are executing because they read from a consistent “snapshot” of data. Maintaining snapshots in distributed databases can be tricky: usually, there’s a large amount of inter-node communication required to create agreement on the ordering of transactions and queries. Clock information provided by TrueTime enables the use of snapshots with minimal communication between nodes.

Google Spanner further optimizes the snapshot mechanism by using GPS receivers and atomic clocks installed in each data center. GPS provides an externally validated timestamp while the atomic clock provides high-resolution time between GPS “fixes.” The result is that every Spanner server across the world has almost the same clock time. This allows Spanner to order transactions and queries precisely without requiring excessive inter-node communication or delays due to excessive clock uncertainty.



Spanner is highly dependent on Google’s redundant network and specialized server hardware. Spanner can’t operate independently of the Google network.

The initial version of Spanner pushed the boundaries of the CAP theorem as far as technology allowed. It represented a distributed database system in which consistency was guaranteed, availability maximized, and network partitions avoided as much as possible. Over time, Google added relational features to the data model of Spanner as well as SQL language support. By 2017, Spanner had evolved to a distributed database that supported all three pillars of the RDBMS: the SQL language, relational data model, and ACID transactions.

The Advent of CockroachDB

With Spanner, Google persuasively demonstrated the utility of a highly consistent distributed database. However, Spanner was tightly coupled to the Google Cloud Platform (GCP) and—at least initially—not publicly available.

There was an obvious need for the technologies pioneered by Spanner to be made more widely available. In 2015, a trio of Google alumni—Spencer Kimball, Peter Mattis, and Ben Darnell—founded Cockroach Labs with the intention of creating an open source, geo-scalable, ACID-compliant database.

Spencer, Peter, and Ben chose the name “CockroachDB” in honor of the humble cockroach, which, it is told, is so resilient that it would survive even a nuclear war (Figure 1-4).



Figure 1-4. The original CockroachDB logo

CockroachDB Design Goals

CockroachDB was designed to support the following attributes:

Scalability

The CockroachDB distributed architecture allows a cluster to scale seamlessly as workload increases or decreases. Nodes can be added to a cluster without any manual rebalancing, and performance will scale predictably as the number of nodes increases.

High availability

A CockroachDB cluster has no single point of failure. CockroachDB can continue operating if a node, zone, or region fails without compromising availability.

Consistency

CockroachDB provides the highest practical level of transactional isolation and consistency. Transactions operate independently of each other and, once committed, transactions are guaranteed to be durable and visible to all sessions.

Performance

The CockroachDB architecture is designed to support low-latency and high-throughput transactional workloads. Every effort has been made to adopt database best practices with regard to indexing, caching, and other database optimization strategies.

Geo-partitioning

CockroachDB allows data to be physically located in specific localities to enhance performance for “localized” applications and to respect data sovereignty requirements.

SQL compatibility

CockroachDB implements ANSI-standard SQL and is wire-protocol compatible with PostgreSQL. This means that the majority of database drivers and frameworks that work with PostgreSQL will also work with CockroachDB. Many PostgreSQL applications can be ported to CockroachDB without requiring significant coding changes.

Portability

CockroachDB is offered as a fully managed database service, which in many cases is the easiest and most cost-effective deployment mode. But it’s also capable of running on pretty much any platform you can imagine, from a developer’s laptop to a massive cloud deployment. The CockroachDB architecture is well aligned with containerized deployment options, and in particular, with Kubernetes. CockroachDB provides a Kubernetes operator that eliminates much of the complexity involved in a Kubernetes deployment.

You may be thinking, “This thing can do everything!” However, it’s worth pointing out that CockroachDB was not intended to be all things to all people. In particular:

CockroachDB prioritizes consistency over availability.

We saw earlier how the CAP theorem states that you have to choose either consistency or availability when faced with a network partition. Unlike “eventually” consistent databases such as DynamoDB or Cassandra, CockroachDB guarantees consistency at all costs. This means that there are circumstances in which a CockroachDB node will refuse to service requests if it is cut off from its peers. A Cassandra node in similar circumstances might accept a request even if there is a chance that the data in the request will later have to be discarded.

The CockroachDB architecture prioritizes transactional workloads.

CockroachDB includes the SQL constructs for issuing aggregations and the SQL 2003 analytic “windowing” functions, and CockroachDB is certainly capable of integrating with popular business intelligence tools such as Tableau. There’s no specific reason why CockroachDB could not be used for analytic applications. However, the unique features of CockroachDB are targeted more at transactional workloads. For analytic-only workloads that do not require transactions, other database platforms might provide better performance.

It’s important to remember that while CockroachDB was inspired by Spanner, it is in no way a “Spanner clone.” The CockroachDB team has leveraged many of the Spanner team’s concepts but has diverged from Spanner in several important ways.

First, Spanner was designed to run on very specific hardware. Spanner nodes have access to an atomic clock and GPS device, allowing incredibly accurate timestamps. CockroachDB is designed to run well on commodity hardware and within containerized environments (such as Kubernetes) and therefore cannot rely on atomic clock synchronization. As we will see in [Chapter 2](#), CockroachDB does rely on decent clock synchronization between nodes but is far more tolerant of clock skew than Spanner; as a result, CockroachDB can run anywhere, including any cloud provider or on-premises data center (and one CockroachDB cluster can even span multiple cloud environments).

Second, while the distributed storage engine of CockroachDB is inspired by Spanner, the SQL engine and APIs are designed to be PostgreSQL compatible. PostgreSQL is one of the most implemented RDBMSs today and is supported by an extensive ecosystem of drivers and frameworks. The “wire protocol” of CockroachDB is completely compatible with PostgreSQL, which means that any driver that works with PostgreSQL will work with CockroachDB. At the SQL language layer, there will always be differences between PostgreSQL and CockroachDB because of differences in the underlying storage and transaction models. However, most commonly used SQL syntax is shared between the two databases.

Third, CockroachDB has evolved to satisfy the needs of its community and has introduced many features never envisaged by the Spanner project. Today, CockroachDB is a thriving database platform whose connection to Spanner is only of historical interest.

CockroachDB Releases

The first production release of CockroachDB appeared in May 2017. This release introduced the core capabilities of the distributed transactional SQL databases, albeit with some limitations of performance and scale. Version 2.0—released in 2018—included new partitioning features for geographically distributed deployments, support for JSON data, and massive improvements in performance.

In 2019, CockroachDB courageously leaped from version 2 to version 19! This was not because of 17 failed versions between 2 and 19 but instead reflected a change in numbering strategy to associate each release with its release year rather than designating releases as “major” or “minor.”

Some highlights of past releases include:

- Version 19.1 (April 2019) introduced security features such as encryption at rest and LDAP (Lightweight Directory Access Protocol) integration, the change data capture facility described in [Chapter 7](#), and multiregion optimizations.
- Version 19.2 (November 2019) introduced the Parallel Commits transaction protocol and other performance improvements.
- Version 20.1 (May 2020) introduced many SQL features, including ALTER PRIMARY KEY, SELECT FOR UPDATE, nested transactions, and temporary tables.
- Version 20.2 (November 2020) added support for spatial data types, new transaction detail pages in the DB console, and made the distributed BACKUP and RESTORE functionality available for free.
- Version 21.1 (May 2021) simplified the use of multiregion functionality and expanded logging configuration options.
- Version 21.2 (November 2021) introduced bounded staleness reads and numerous stability and performance improvements, including an admission control system to prevent overloading the cluster.
- Version 22.1 (May 2022) added support for super regions, row-level time-to-live (TTL), and index recommendations.
- Version 22.2 (December 2022) introduced user-defined functions, the MOLT (Migrate Off Legacy Technology) Schema Conversion Tool, trigram indexes, and the insights page.

- Version 23.1 (May 2023) added support for full-text search, user-defined composite types, redaction of PII (Personally Identifiable Information) during EXPLAIN queries, and a new shell editor with tab completion.
- Version 23.2 (February 2024) introduced the READ COMMITTED isolation level, support for PL/pgSQL, and column-level encryption—additionally, it includes the capability to visualize network partitions in the DB Console, migrate to CockroachDB via Oracle GoldenGate and Debezium, utilize the MOLT Live Migration Service, and perform physical cluster replication.
- Version 24.1 (May 2024) hit a 99.999% (five nines) availability target for multiregion managed service clusters. It also introduced the capability to emit change-feed events to Azure Event Hub, connect managed service clusters to Google Private Service Connect, and included MOLT Fetch.
- Version 24.2 (August 2024) introduced support for the VECTOR data type and performance improvements to the cost-based optimizer for prepared statements via generic query plans.

CockroachDB in Action

CockroachDB has gained strong and growing traction in a crowded database market. Users who have been constrained by the scalability of traditional relational databases such as PostgreSQL and MySQL are attracted by the greater scalability of CockroachDB. Those who have been using distributed NoSQL solutions such as Cassandra are attracted by the greater transactional consistency and SQL compatibility offered by CockroachDB. And those who are transforming toward modern containerized and cloud native architectures appreciate the cloud and container readiness of the platform.

Today, CockroachDB can boast of significant adoption at scale across multiple industries. Let's look at a few of these case studies.²

CockroachDB at Netflix

Netflix has been using CockroachDB in production since 2020, and in that time has deployed more than 380 large-scale CockroachDB clusters (of which, more than 160 are production clusters and more than 60 of *those* are multiregion clusters).

Their adoption of CockroachDB is advanced, offering internal teams access to CockroachDB-as-a-service. This allows them to create clusters on-demand, enhance their development speed and agility, and standardize on best practices.

² Cockroach Labs maintains a growing list of [CockroachDB case studies](#).

CockroachDB at Devsisters

Devsisters is a South Korea–based game development company responsible for games such as the mobile phone game *Cookie Run: Kingdom*. Originally, Devsisters used Couchbase for its persistence layer but was challenged by issues relating to transactional integrity and scalability. When looking for a new database solution, Devsisters’ requirements included scalability, transactional consistency, and support for very high throughput.

Devsisters considered Amazon Aurora and DynamoDB as well as CockroachDB, but in the end, chose CockroachDB. Sungyoon Jeong from the DevOps team says, “It would have been impossible to scale this game on MySQL or Aurora. We experienced more than six times the workload size we anticipated, and CockroachDB was able to scale with us throughout this journey.”

CockroachDB at DoorDash

DoorDash is a local commerce platform that connects consumers with their favorite businesses across the United States, Canada, Australia, Japan, and Germany. Today, DoorDash has created more than 350 CockroachDB clusters for its developers for various customer-facing, backend analytics, and internal workloads.

The DoorDash team likes that CockroachDB scales horizontally, speaks SQL and has Postgres wire compatibility, and handles heavy reads/writes without impacting performance. CockroachDB’s resilient architecture and live schema changes are also a huge bonus for the team. “DoorDash has been able to use CockroachDB to forklift-migrate and scale numerous workloads without having to rewrite applications—only small index or schema changes,” says Sean Chittenden, engineering lead for the Core Infrastructure team at DoorDash.

CockroachDB at Bose

Bose is a world-leading consumer technology company particularly well known as a provider of high-fidelity audio equipment.

Bose’s customer base spans the globe, and Bose aims to provide those customers with best-in-class cloud-based support solutions.

Bose has embraced modern microservices-based software architecture. The backbone of the Bose platform is Kubernetes, which allows applications access to low-level services—containerized computation—and to higher-level services such as Elasticsearch, Kafka, and Redis. CockroachDB became the foundation of the database platform for this containerized microservice platform.

Aside from the resiliency and scalability of CockroachDB, CockroachDB's capability to be hosted within a Kubernetes environment was decisive.

By running CockroachDB in a Kubernetes environment, Bose has empowered developers by providing a self-service database-on-demand capability. Developers can spin up CockroachDB clusters for development or testing simply and quickly within a Kubernetes environment. In production, CockroachDB running with Kubernetes provides full-stack scalability, redundancy, and high availability.

CockroachDB at Form3

Form3 is revolutionizing how payments work. Powering some of the largest financial institutions in the world, CockroachDB provides a resilient backbone to a tier-0, critical national payments infrastructure.

Form3 regularly tests CockroachDB to its limits with chaos engineering, allowing its built-in self-healing to recover from anything they throw at it. Running CockroachDB across not only multiple cloud regions but also multiple cloud providers makes Form3 one of the most robust and resilient payment providers in the world.

CockroachDB at Hard Rock Digital

Answering to the US Federal Wire Act (1961), Hard Rock Digital runs a CockroachDB cluster that spans cloud regions and on-prem data centers.

The Federal Wire Act requires that all data and processing for a bet remains within the state it was placed. To reach customers where hyperscalers don't have regions, Hard Rock Digital runs CockroachDB on AWS Outposts—an AWS service that runs on their own hardware. This allows Hard Rock Digital to run CockroachDB in every state they serve, while remaining compliant with regulations.

CockroachDB at Spreedly

Spreedly is a global payments orchestration platform that gives customers access to payment services in more than 100 countries, all from a single API. Following their adoption of CockroachDB, they've drastically simplified their architecture by:

- Removing multiple legacy databases
- Eliminating unnecessary extract, transform, and load (ETL) pipeline components
- Solving a complex split-brain scenario and ensuring a consistent view of data to all consumers

CockroachDB at Route

Route is a leading post-purchase platform, and CockroachDB is the beating heart of their platform’s always-on data model—processing billions of orders for millions of customers.

Given the seasonal nature of the ecommerce businesses they support, the ease of scaling both up and down afforded by CockroachDB allows them to serve their customers efficiently—all with zero downtime.

CockroachDB offers the biggest ROI I’ve seen in terms of paid support from any vendor.

—Brian Call, senior principal engineer at Route

Summary

In this chapter, we’ve placed CockroachDB in a historical context and introduced the goals and capabilities of the CockroachDB database.

The RDBMSs that emerged in the 1970s and 1980s were a triumph of software engineering that powered software applications from client/server through to the early internet. But the demands of globally scalable, always available internet applications were inconsistent with the monolithic, strictly consistent RDBMS architectures of the day. Consequently, a variety of NoSQL distributed, “eventually consistent” systems emerged around 2010 to support the needs of a new generation of internal applications.

While these NoSQL solutions have their advantages, they are a step backward for many or most applications. The inability to guarantee data correctness and the loss of the highly familiar and productive SQL language was a regression in many respects. CockroachDB was designed as a highly consistent and highly available SQL-based transactional database that provides a better compromise between availability and consistency—prioritizing consistency above all but providing very high availability.

CockroachDB is a highly available, transactionally consistent SQL database compatible with existing development frameworks and with increasingly important containerized deployment models and cloud architectures. CockroachDB has been deployed at scale across a wide range of verticals and circumstances.

In the next chapter, we’ll examine the architecture of CockroachDB and see exactly how it achieves its ambitious design goals.

CockroachDB Architecture

The architecture of a software system defines the high-level design decisions that enable the goals of that system. As you may recall from [Chapter 1](#), the goals of CockroachDB are to provide a scalable, highly available, highly performant, strongly consistent, geo-distributed, SQL-powered relational database system capable of running across a wide variety of hardware platforms. The architecture of CockroachDB is aligned to those objectives.

Feel Free to Skip Ahead!

The CockroachDB architecture is sophisticated: it incorporates decades of database engineering best practice designs together with several unique innovations. However, CockroachDB doesn't require that you understand its internals to get things done. If you're in a hurry to get started with CockroachDB, you can skip forward to the next chapter and return to this chapter later as necessary. We will, however, assume you are broadly familiar with the key concepts in this chapter when we consider advanced topics later in the book. Those key concepts are summarized over the next few pages and elaborated on in the remainder of the chapter.

There are multiple ways of looking at the CockroachDB architecture. At the cluster level, a CockroachDB deployment consists of one or more shared-nothing, leaderless nodes that collaborate to present a single logical view of the distributed database system. Within each node, we can observe the CockroachDB architecture as a series of layers that provide essential database services, including SQL processing, transaction processing, replication, distribution, and storage.

In this chapter, we'll endeavor to give you a comprehensive overview of the CockroachDB architecture. The aim of the chapter is to provide you with the

fundamental concepts that will help you make sensible decisions regarding schema design, performance optimization, cluster deployment, and other topics.

The CockroachDB Cluster Architecture

From a distance, a CockroachDB deployment consists of one or more database server processes. Each server has its own dedicated storage—the familiar “shared-nothing” database cluster pattern. The nodes in a CockroachDB cluster are symmetrical—there are no “special” or “primary” nodes. This storage is often directly attached to the machine on which the CockroachDB server runs, though it’s also possible for that data to be physically located on a shared storage subsystem. Data is distributed across the cluster based on *key ranges*. Each range is replicated to at least three members of the cluster.

Database clients—applications, administrative consoles, the CockroachDB shell, and so on—connect to a CockroachDB server within the cluster.

The communications between a database server and database client occur over the PostgreSQL *wire protocol* format. This protocol describes how SQL requests and responses are transmitted between a PostgreSQL client and a PostgreSQL server. Because CockroachDB uses the PostgreSQL wire protocol, any PostgreSQL driver can be used to communicate with a CockroachDB server. In a more complex deployment, one or more *load balancer* processes will be responsible for ensuring that these connections are evenly and sensibly distributed across nodes. The load balancer will connect the client with one of the nodes within the cluster, which will become the *gateway server* for the connection.

The client request might involve reading and writing data to a single node or to multiple nodes within the cluster. For any given range of KVs, a *leaseholder node* will be responsible for controlling reads and writes to that range. The leaseholder is also usually the *Raft leader*, which has the responsibility to make sure that replicas of the data are maintained correctly.

Figure 2-1 illustrates some of these concepts. A database client connects to a load balancer (1) that serves as a proxy for the CockroachDB cluster. The load balancer directs requests to an available CockroachDB node (2). This node becomes the gateway node for this connection. The request requires data in range 4, so the gateway node communicates with the leaseholder node for this range (3), which returns data to the gateway, which in turn returns the required data to the database client (4).

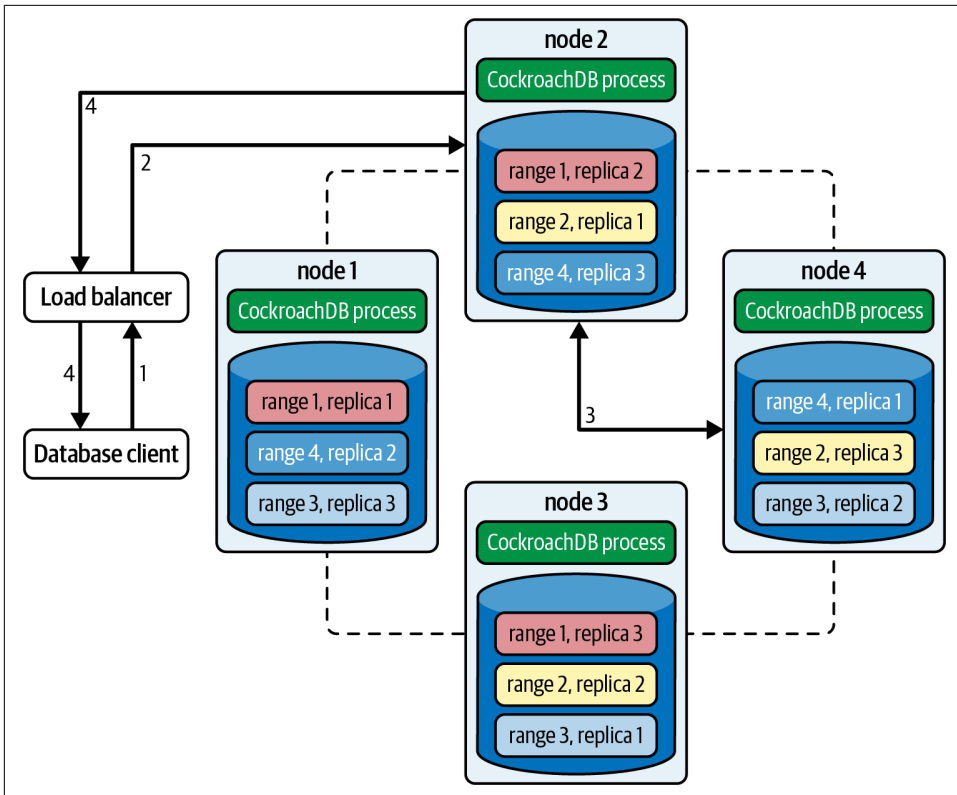


Figure 2-1. CockroachDB cluster architecture

This architecture distributes load evenly across the nodes of the cluster. Gateway duties are distributed evenly across the nodes of the cluster by the load balancer; leaseholder duties are similarly distributed by ranges across all the nodes.

If a query requires data from multiple ranges or where data must be changed (and therefore replicated), the workflow involves more steps. We'll examine the nuances of CockroachDB distribution and replication later in this chapter, but for now there are a few concepts we need to understand.

Under the hood, data in a CockroachDB table is organized in a KV storage system. The key for the KV store is the table's primary key. The value in the KV store is a binary representation of the values for all the columns in that row.

Indexes are also stored in the KV system. In the case of a nonunique index, the key is the index key concatenated to the table's primary key. In the case of a unique index, the key is the index key, with the primary key appearing as the corresponding value for that key.

Ranges store contiguous spans of KVs. Ranges are analogous to *shards* or *shard chunks* in other databases. **Figure 2-2** illustrates how a “dogs” table might be segmented into ranges.

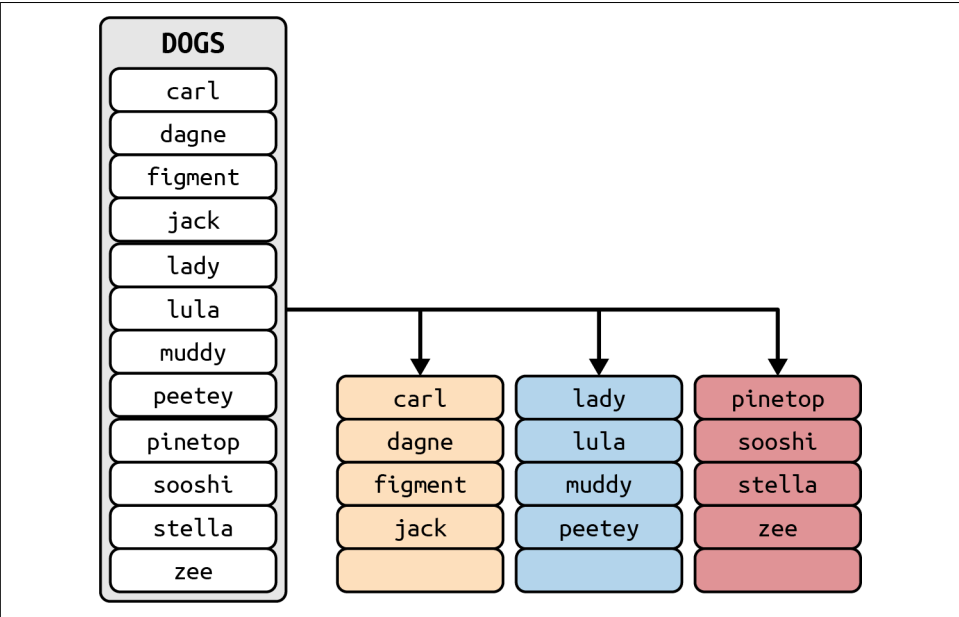


Figure 2-2. Ranges

As mentioned earlier, *leases* are granted to a node giving it responsibility for managing reads and writes to a range. The node holding the lease is known as the *leaseholder*. The same node is generally also the *Raft leader*, which is responsible for ensuring that replicas of the node are correctly maintained across multiple nodes.

The CockroachDB Software Stack

Each CockroachDB node runs a copy of the CockroachDB software, which is a single multithreaded process. From the OS perspective, the CockroachDB process might seem like a closed box, but internally it is organized into multiple logical layers, as shown in **Figure 2-3**.

We’ll discuss each of these layers in turn as we proceed through the chapter.

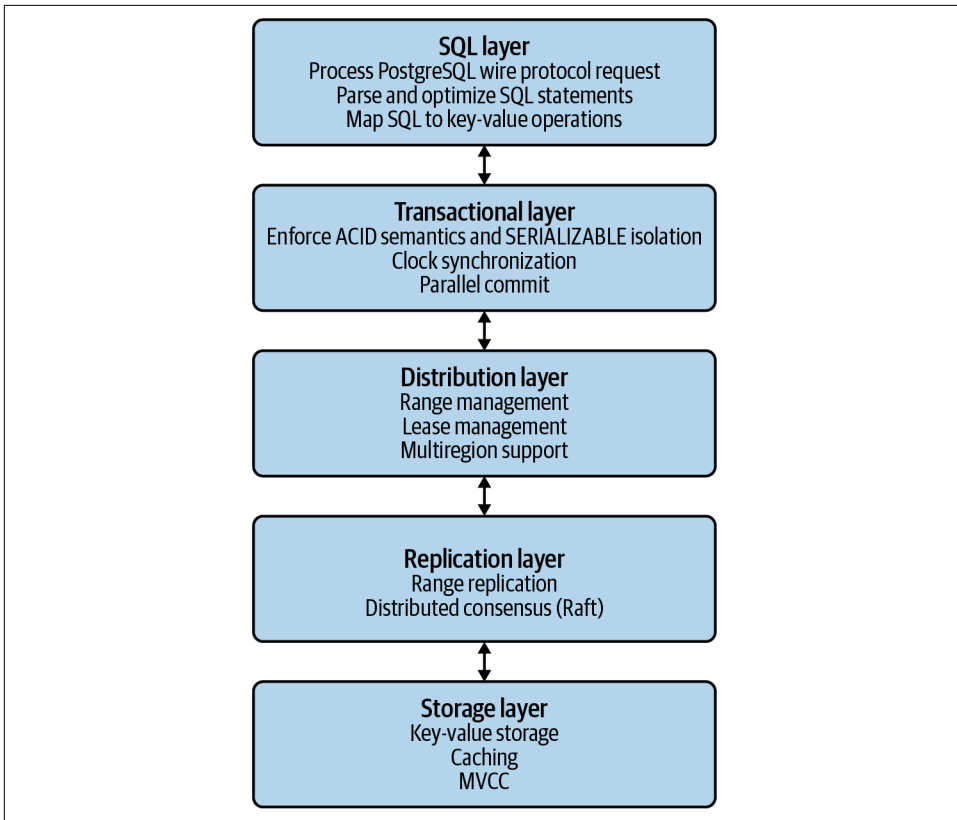


Figure 2-3. CockroachDB software layers

The CockroachDB SQL Layer

The SQL layer is the part of the CockroachDB software stack that is responsible for handling SQL requests. Because CockroachDB is a SQL database, you would be forgiven for thinking that the SQL layer does pretty much everything. However, the core responsibility of the SQL layer is actually to turn SQL requests into KV operations. Other layers handle transactions, distribution, and replication of ranges and physical storage to disk.

The SQL layer receives requests from database clients over the PostgreSQL wire protocol. A database client is any program that uses a database driver to communicate with the server. It includes the CockroachDB command-line SQL processor, GUI tools such as DBeaver or Tableau, or applications written in Java, Go, Node.js, Python, or any other language that has a compatible driver.

The PostgreSQL wire protocol describes the format of network packets that are used to send requests and receive results from a database client and server. The wire protocol lays on top of a transport medium such as TCP/IP or Unix-style sockets. The use of the PostgreSQL wire protocol allows CockroachDB to take advantage of the large ecosystem of compatible language drivers and tools that support the PostgreSQL database.

The SQL layer parses the SQL request, checking it for syntactical accuracy and ensuring that the connection has privileges to perform the requested task.

CockroachDB then creates an execution plan for the SQL statement and proceeds to *optimize* that plan.

SQL is a declarative language: you define the data you want, not how to get it. Although the nonprocedural nature of SQL results in improvements in programmer productivity, the database server must support a set of sophisticated algorithms to determine the optimal method of executing the SQL. These algorithms are collectively referred to as *the optimizer*.

For almost all SQL statements, there will be more than one way for CockroachDB to retrieve the rows required. For instance, given a SQL statement with JOIN and WHERE clauses, there may be multiple join orders and multiple access paths (table scans, index lookups, etc.) available to retrieve data. It's the goal of the optimizer to determine the best access path. CockroachDB's SQL optimizer has some unique features relating to its distributed architecture, but broadly speaking, the cost-based optimizer is similar to that found in other SQL databases such as Oracle or PostgreSQL.

The optimizer uses both heuristics—rules—and cost-based algorithms to perform its work.

The first stage of the SQL optimization process is to transform the SQL into a normalized form suitable for further optimization. This transformation removes any redundancies in the SQL statement and performs rule-based transformations to improve performance. The transformation takes into account the distribution of data for the table, adding predicates to direct parts of the queries to specific ranges or adding predicates that allow the use of indexed retrieval paths.

The optimization of the SQL statement proceeds in two stages: expansion and ranking. The SQL statement is transformed into an initial plan. Then the optimizer expands that plan into a set of equivalent candidate plans that involve alternative execution paths such as join orders or indexes. The optimizer then ranks the plans by calculating the relative cost of each operation, leveraging statistics that supply the size and distribution of data within each table. The plan with the lowest cost is then selected.

CockroachDB also supports a *vectorized execution* engine that can speed up the processing of batches of data. This engine translates data from a row-oriented format (where sets of data contain data from the same row) to a column-oriented format (where every set of data contains information from the same column).

We'll return to the optimizer in [Chapter 8](#) when we look in detail at SQL tuning.

From SQL to Key-Values

As we mentioned earlier, CockroachDB data is stored in a KV storage system that is distributed across multiple nodes in ranges. We'll look at the details of this storage system toward the end of the chapter, but since the outputs of the SQL layer are, in fact, KV operations, the mapping of data from tables and indexes to KV representation is part of the SQL layer. The output of the SQL layer are KV operations.

This translation means that only the SQL layer needs to be concerned with SQL syntax—all the subsequent layers are blissfully unaware of the SQL language.

Tables as Represented in the KV Store

Each entry in the KV store has a key based on the following structure:

`/<tableID>/<indexID>/<IndexKeyValues>/<ColumnFamily>`

We'll discuss column families in the next section. By default, all columns are included in a single default column family.

For a base table, the default `indexID` is "primary."

[Figure 2-4](#) shows a simplified version of this mapping, omitting the column family identifier.

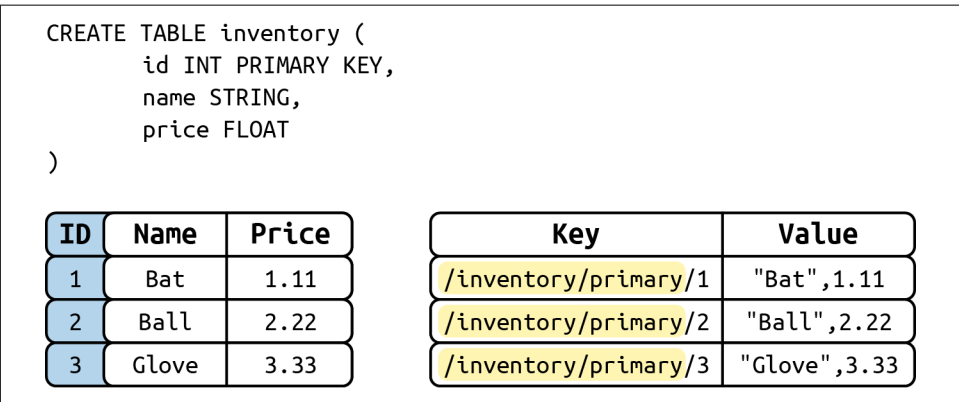


Figure 2-4. KV to column mappings

Figure 2-4 illustrates the table name and index name (“primary”) as text, but within the KV store, these are represented as compact table and index identifiers.

Column Families

In the preceding example, all the columns for a table are aggregated in the value section of a single KV entry. However, it’s possible to direct CockroachDB to store groups of columns in separate KV entries using column families. Each column family in a table will be allocated its own KV entry. Figure 2-5 depicts this concept—if a table has two column families, then each row in the table will be represented by two KV entries.

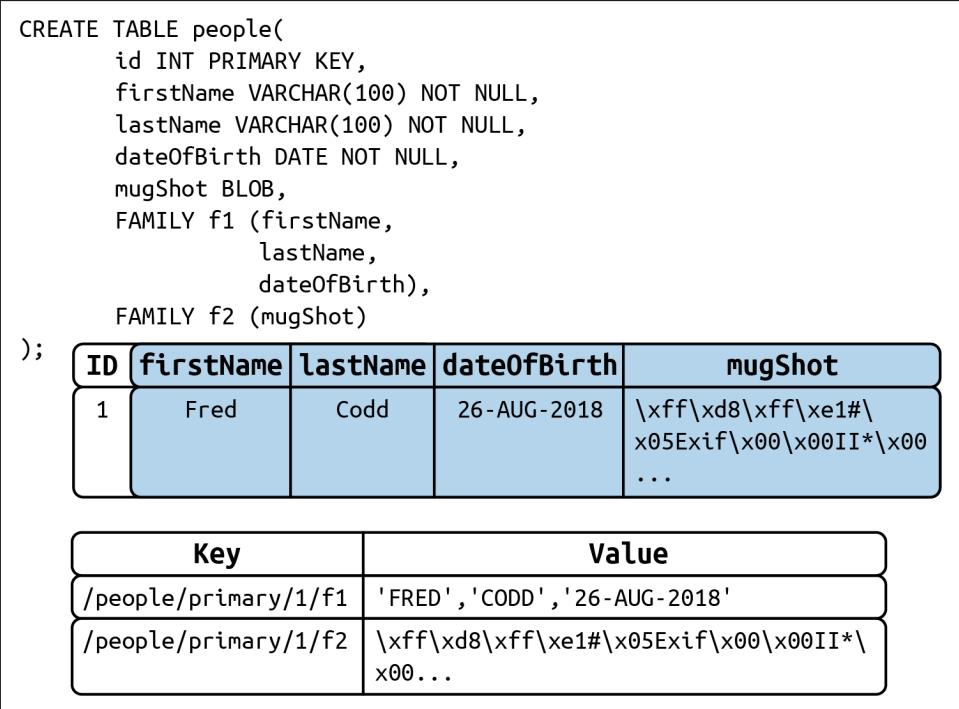


Figure 2-5. Column families in the KV store

Column families can have a number of advantages. If infrequently accessed large columns are separated, then they will not be retrieved during row lookups, which can improve the efficiency of the KV store cache. Furthermore, concurrent operations on columns in separate column families will not interfere with each other.

Indexes in the KV Store

Indexes are represented by a similar KV structure. For instance, the representation of a nonunique index is shown in [Figure 2-6](#).

```
CREATE TABLE inventory (  
    id INT PRIMARY KEY,  
    name STRING,  
    price FLOAT,  
    INDEX name_idx (name)  
)
```

ID	Name	Price	Key	Value
1	Bat	1.11	/inventory/name_idx/"Bat"/1	∅
2	Ball	2.22	/inventory/name_idx/"Ball"/2	∅
3	Glove	3.33	/inventory/name_idx/"Glove"/3	∅
4	Bat	4.44	/inventory/name_idx/"Bat"/4	∅

Figure 2-6. Nonunique index KV store representation

The key for a nonunique index includes the table and index name, the KV, and the primary KV. For a nonunique index, there is no “value” by default.

For a unique index, the KV value defaults to the value of the primary key. So, if name was unique in the inventory table used in previous examples, a unique index on name is represented as in [Figure 2-7](#).

```
CREATE TABLE inventory (  
    id INT PRIMARY KEY,  
    name STRING,  
    price FLOAT,  
    UNIQUE INDEX name_idx (name)  
)
```

ID	Name	Price	Key	Value
1	Bat Acme	1.11	/inventory/name_idx/"Bat Acme"	/1
2	Ball	2.22	/inventory/name_idx/"Ball"	/2
3	Glove	3.33	/inventory/name_idx/"Glove"	/3
4	Bat Sears	4.44	/inventory/name_idx/"Bat Sears"	/4

Figure 2-7. Unique index KV store representation

Inverted Indexes

CockroachDB columns can be defined as arrays or JSON documents. We'll discuss this in detail in [Chapter 4](#).

Inverted indexes allow indexed searches into values included in these arrays or JSON documents. In this case, the KVs include the JSON path and value together with the primary key, as shown in [Figure 2-8](#).

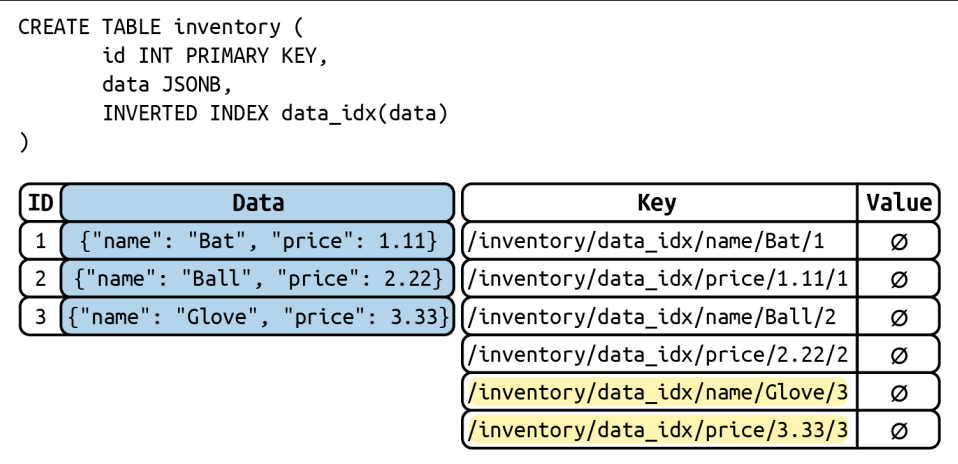


Figure 2-8. Inverted index KV representation

Inverted indexes are also used on spatial data.

Inverted indexes can be larger and more expensive to maintain than other indexes because a single JSON document in a row will generate one index entry for each unique attribute. For complex JSON documents, this might result in dozens of index entries for each document. We'll also discuss this further—and consider some alternatives—in [Chapter 8](#).

The STORING Clause

The `STORING` clause of `CREATE INDEX` allows us to add additional columns to the value portion of the KV index structure. These additional columns can streamline a query that contains a projection (e.g., a `SELECT` list) that includes only those columns and the index keys. For instance, in [Figure 2-9](#), we see a nonunique index on `name` and `dateOfBirth` that uses the `STORING` clause to add the phone number to the KV value. Queries that seek to find the phone number using name and date of birth can now be resolved by the index alone without reference to the base table.

```
CREATE TABLE people(
  id INT PRIMARY KEY,
  firstName VARCHAR(100) NOT NULL,
  lastName VARCHAR(100) NOT NULL,
  dateOfBirth DATE NOT NULL,
  phoneNumber int not null,
  otherColumns blob,
  INDEX (firstName,lastName,dateOfBirth) STORING (phoneNumber)
);
```

id	firstName	lastName	dateOfBirth	phoneNumber	otherColumns
1	Fred	Codd	26-AUG-1918	+1-033-333-3333	...

Key	Value
/people/indexName/Fred/Codd/26-Aug-1918/1	+1-033-333-3333

Figure 2-9. *STORING* clause of *CREATE INDEX*

Table Definitions and Schema Changes

The schema definitions for tables (and their associated indexes) are stored in a special keyspace called a *table descriptor*. For performance reasons, table descriptors are replicated on every node. The table descriptor is used to parse and optimize SQL and to correctly construct KV operations for a table.

CockroachDB supports online schema changes using `ALTER TABLE`, `CREATE INDEX`, and other commands. The schema is changed in discrete stages that allow the new schema to be rolled out while the previous version is still in use. Schema changes run as background tasks.

The node initiating the schema change will acquire a write lease on the relevant table descriptor. Nodes that are performing data manipulation language (DML) on a table will have a lease on the relevant table descriptor. When the node holding the write lease modifies the definition, it is broadcast to all nodes in the cluster that will—when it becomes possible—release their lease on the old schema.

The schema change may involve changes to table data (removing or adding columns) and/or creating new index structures. When all of the instances of the table are stored according to the requirements of the new schema, then all nodes will switch over to the new schema and will allow reads and writes of the table using the new schema.

The CockroachDB Transaction Layer

The transaction layer is responsible for maintaining the atomicity of transactions by ensuring that all operations in a transaction are committed or aborted.

Additionally, the transaction layer maintains—by default—serializable isolation between transactions; this means that transactions are completely isolated from the effects of other transactions. Although multiple transactions may be in progress at the same time, the experience of each transaction is as if the transactions were run one at a time—the *serializable* isolation level.

Isolation Levels

Transaction “isolation levels” define to what extent transactions are isolated from the effects of other transactions. ANSI SQL defines four isolation levels that are, from weakest to strongest: `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, and `SERIALIZABLE`. Additionally, an isolation level of `SNAPSHOT` is used by many databases as an alternative “strong” isolation level.

The majority of relational databases use a default isolation level of `READ COMMITTED`, allowing for improved concurrency at the expense of consistency. CockroachDB supports both `SERIALIZABLE` and `READ COMMITTED`, with `SERIALIZABLE` being the default isolation level.

When using the default `SERIALIZABLE` isolation level, CockroachDB transactions must exhibit absolute independence from all other transactions. The results of a set of concurrent transactions must be the same as if they had all been performed one after the other.

For applications designed for `READ COMMITTED` isolation, CockroachDB allows operators to lower the default isolation to `READ COMMITTED` for a more streamlined migration to CockroachDB.

The transaction layer processes KV operations generated by the SQL layer. A transaction consists of multiple KV operations, some of which may be the result of a single SQL statement. In addition to updating table entries, index entries must also be updated. Maintaining perfect consistency under all circumstances involves multiple sophisticated algorithms, not all of which can be covered in this chapter. For comprehensive information, you may wish to consult the [CockroachDB 2020 SIGMOD paper](#), which covers many of these principles in more detail.

MVCC Principles

Like most transactional database systems, CockroachDB implements the multiversion concurrency control (MVCC) pattern. MVCC allows readers to obtain a consistent view of information, even while that information is being modified. Without MVCC, consistent reads of a data item need to block (typically using a “read lock”) simultaneous writes of that item and vice versa. With MVCC, readers can obtain a consistent view of information even while the information is being modified by a concurrent transaction.

Figure 2-10 illustrates the basic principles of MVCC. At time t_1 , session s_1 reads from row r_2 and accesses version v_1 of that row (1). At timestamp t_2 , another database session, s_2 , updates the row (2), creating version v_2 of that row (3). At t_3 , session s_1 reads the row again, but—because s_2 has not yet committed its change—continues to read from version v_1 (4). After s_2 commits (5), session s_1 issues another select and now reads from the new v_2 version of the row (6).

The CockroachDB implementation limits the ability of transactions to read from previous versions. For instance, if a read transaction commences after a write transaction has begun, it may not be able to read the original version of the row because it might be inconsistent with other data already read or that will be read later in the transaction. This may result in the read transaction “blocking” until the write transaction commits or aborts.

We’ll see later on how the storage engine implements MVCC, but for now, the important concept is that multiple versions of any row are maintained by the system, and transactions can determine which version of the row to read depending on their timestamp and the timestamp of any concurrent transactions.

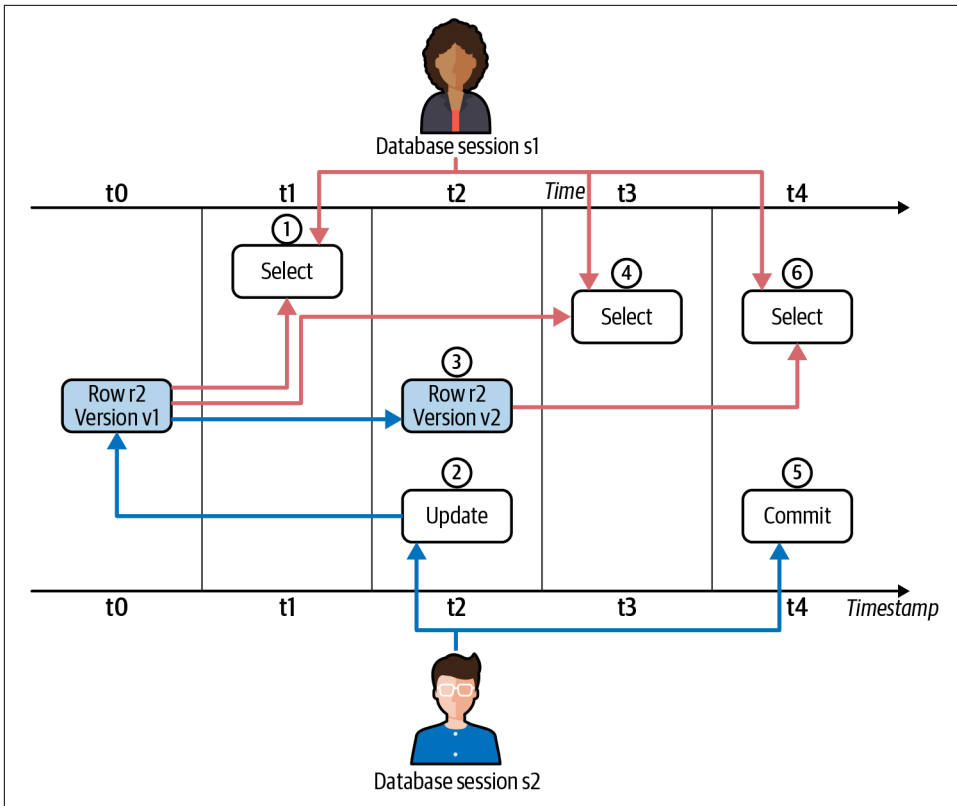


Figure 2-10. MVCC

Transaction Workflow

Distributed transactions must proceed in multiple stages. Simplistically, each node in the distributed system must lay the groundwork for the transaction and the transaction will be finalized only if all nodes report that the transaction can be performed.

Figure 2-11 illustrates a highly simplified flow of transaction preparation. In this case, a two-statement transaction is sent to the CockroachDB gateway node (1). The first statement involves a change to range 2, so that request is sent to the leaseholder for that range (2), which creates a new tentative version of the row and propagates changes to replica nodes (3 and 4). The second statement affects range 4, so the transaction coordinator sends that request to the appropriate leaseholder (5), which is also propagated (6 and 7). When all changes have correctly propagated, the transaction completes, and the client is notified of success (8).

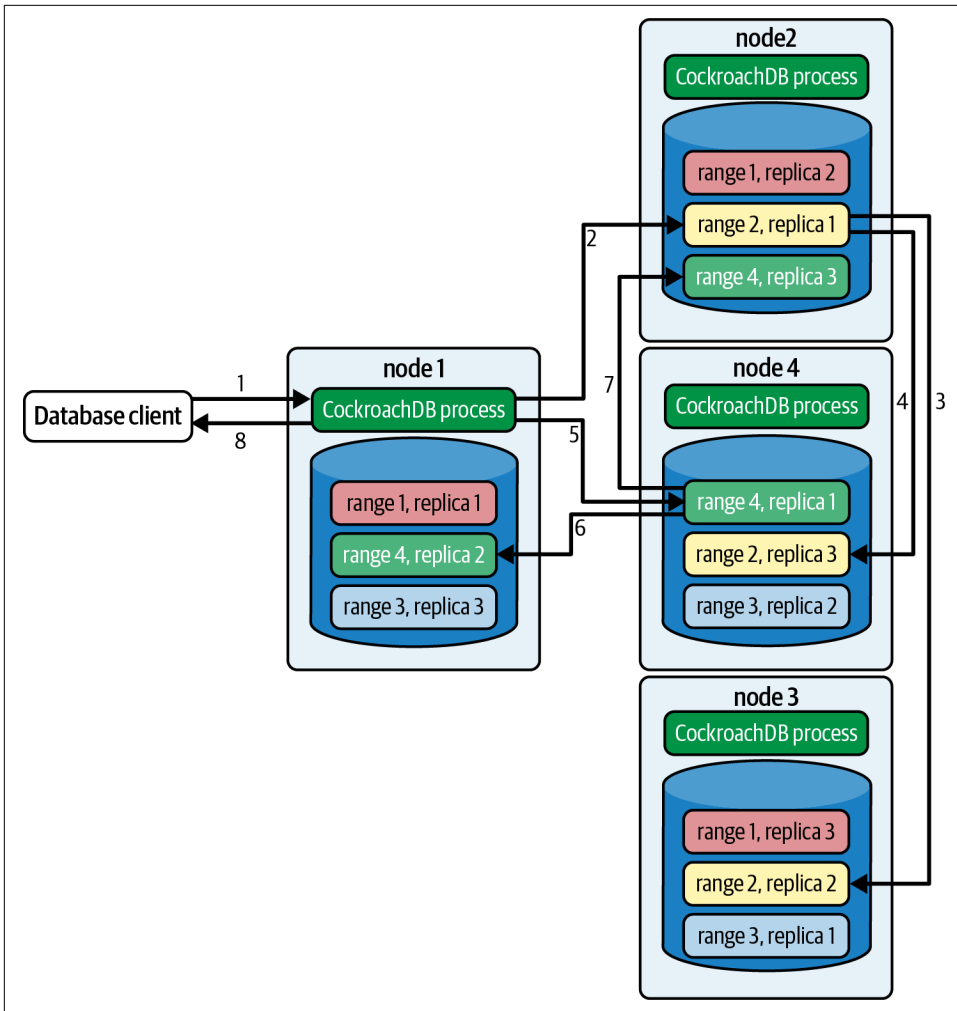


Figure 2-11. Basic transaction flow

Write Intents

During the initial stages of transaction processing, when it is not yet known whether the transaction will succeed, the leaseholder writes tentative modifications to modified values known as *write intents*. Write intents are specially constructed MVCC-compliant versions of the records, which are marked as provisional. They serve both as tentative transaction outcomes and as locks that prevent any concurrent attempts to update the same record.

Inside the first key range to be modified by the transaction, CockroachDB writes a special *transaction record*. This records the definitive status of the transaction. In the example shown in [Figure 2-11](#), this transaction record would be stored in range 2 because that is the first range to be modified in the transaction.

This transaction record will record the transaction state as one of the following:

PENDING

Indicates that the write intent's transaction is still in progress.

STAGING

All transaction writes have been performed, but the transaction is not yet guaranteed to commit.

COMMITTED

The transaction has been successfully completed.

ABORTED

Indicates that the transaction was aborted and its values should be discarded.

Parallel Commits

In a distributed database, the number of network round trips is often the dominant factor in latency. In general, committing a distributed transaction requires at least two round trips (indeed, one of the classic algorithms for this is called Two-Phase Commit). CockroachDB uses an innovative protocol called *Parallel Commits* to hide one of these round trips from the latency as perceived by the client.

The key insight behind Parallel Commits is that the gateway can return success to the client as soon as it becomes impossible for the transaction to abort, even if it is not yet fully committed. The remaining work can be done after returning as long as its outcome is certain. This is done by transitioning the transaction to the STAGING state in parallel with the transaction's last round of writes. The keys of all of these writes are recorded in the transaction record. A STAGING transaction must be committed if and only if all of those writes succeed.

Usually, the gateway learns the status of these writes as soon as they are complete and returns control to the client before beginning the final resolution of the transaction in the background. If the gateway fails, the next node to encounter the STAGING transaction record is responsible for querying the status of each write and determining whether the transaction must be committed or aborted (but because the transaction record and each write intent have been written durably, the outcome is guaranteed to be the same whether the transaction is resolved by its original gateway or by another node).

Note that any locks held by the transaction are not released until after this resolution process has been completed. Therefore, the duration of a transaction from the perspective of another transaction waiting for its locks is still at least two round trips (just as in Two-Phase Commit). However, from the point of view of the session issuing the transaction, the elapsed time is significantly reduced.

Transaction Cleanup

As discussed in the previous section, a COMMIT operation “flips a switch” in the transaction record to mark the transaction as committed, minimizing any delays that would otherwise occur when a transaction is committed. After the transaction has reached the COMMIT stage, then it will asynchronously resolve the write intents by modifying them into normal MVCC records representing the new record values.

However, as with any asynchronous operation, there may be a delay in performing this cleanup. Furthermore, since a committed write intent looks the same as a pending write intent, transactions that encounter a write intent record when reading a key will need to determine if the write intent is committed.

If another transaction encounters a write intent that has not yet been cleaned up by the transaction coordinator, then it can perform the write intent cleanup by checking the transaction record. The write intent contains a pointer to the transaction records, which can reveal if the transaction is committed.

Overview of Transaction Flow

Figure 2-12 illustrates the flow of a successful two-statement transaction. A client issues an UPDATE statement (1). This creates a transaction coordinator that maintains a transaction record in PENDING state. Write intent commands are issued to the leaseholder for the range concerned (2). The leaseholder writes the intent markers to its copy of the data. It returns success to the transaction coordinator without waiting for the replica’s intents to be acknowledged.

Subsequent modifications in the transaction are processed in the same manner.

The client issues a COMMIT (3). The transaction coordinator marks the transaction status as STAGING. When all write intents are confirmed, the initiating client is advised of success, and then the transaction status is set to COMMITTED (4).

After a successful commit, the transaction coordinator resolves the write intents in affected ranges, which become normal MVCC records (5). At this point, the transaction has released all its locks, and other transactions on the same records are free to proceed.

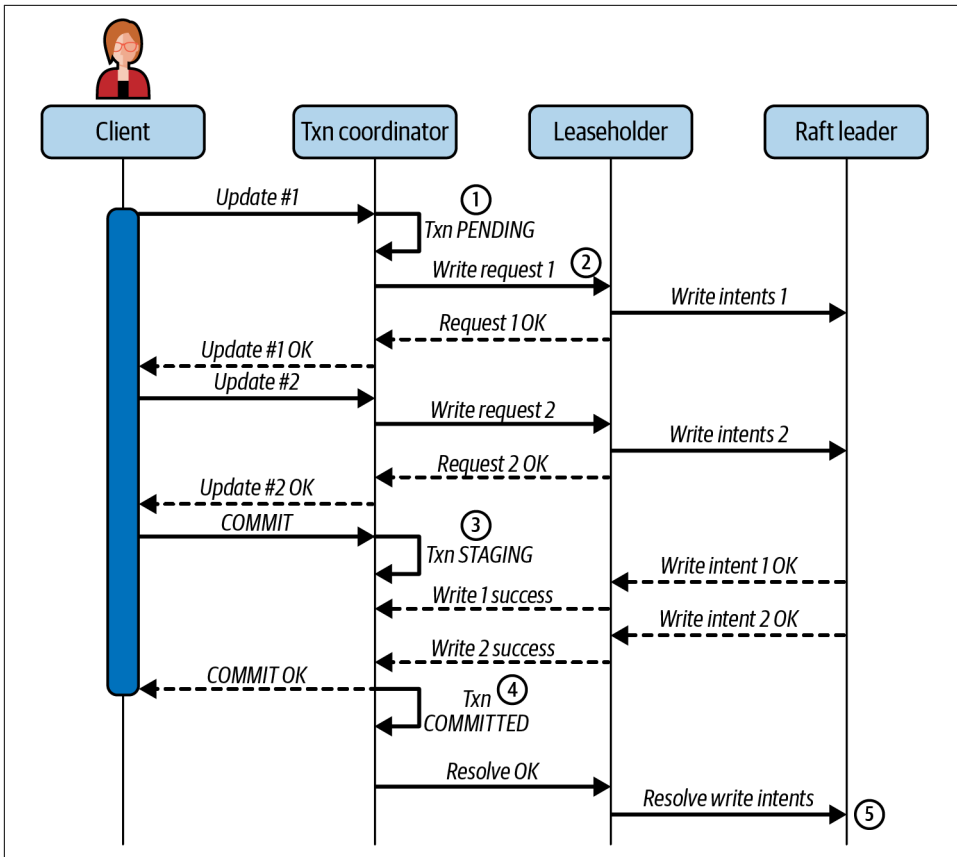


Figure 2-12. Transaction sequence

Figure 2-12 is highly simplified but can still be a little hard to unpack. There are two main takeaways from the diagram:

- Most operations respond in two stages; we can proceed to the next step after the first response and need to resolve everything only at the end of the commit.
- The latency for the client doesn't include all of the cleanup operations. The UPDATE operations return before all the write intents are propagated, and the COMMIT returns before all the write intents are resolved. Hopefully, this removes a lot of the overhead of distributed database management from application response time.

Read/Write Conflicts

So far, we've looked at the processing of successful transactions. It would be great if all transactions succeeded, but in all but the most trivial scenarios, concurrent transactions create conflicts that must be resolved.

The most obvious case is when two transactions attempt to update the same record. There cannot be two write intents active against the same key, so one of the transactions will wait for the other to complete, or one of the transactions will be aborted. If the transactions are of the same priority, then the second transaction—the one that has not yet created a write intent—will wait. However, if the second transaction has a higher priority, then the original transaction will be aborted and will have to retry.

Transaction priorities can be adjusted with the `SET TRANSACTION` statement—see [Chapter 6](#).

Transaction isolation levels can be set on a per-transaction basis with the `BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED` statement or by setting defaults at the database or role level as follows:

```
-- Database level.  
ALTER DATABASE your_db SET default_transaction_isolation = 'read committed';  
  
-- Role level.  
ALTER ROLE your_role SET default_transaction_isolation = 'read committed';
```

The `TxnWaitQueue` object tracks the transactions that are waiting and the transactions that they are waiting on. This structure is maintained within the Raft leader of the range associated with the transaction. When a transaction commits or aborts, the `TxnWaitQueue` is updated, and any waiting transactions are notified.

A *deadlock* can occur if two transactions are both waiting on write intents created by the other transaction. In this case, one of the transactions will be randomly aborted. We'll discuss this in more detail in [Chapter 6](#).

Transaction conflicts can also occur between readers and writers. If a reader encounters an uncommitted write intent that has a lower (i.e., earlier) timestamp than the consistent read timestamp for the read, then a consistent read cannot be completed. This can happen if a modification occurs between the time a read transaction starts and the time it attempts to read the key concerned. In this case, the read will need to wait until the write either commits or aborts.

These “blocked reads” can be avoided in the following circumstances:

- If the read has a high priority, CockroachDB may “push” the lower-priority write’s timestamp to a higher value, allowing the read to complete. The “pushed” transaction may need to restart if the push invalidates any previous work in the transaction.
- Stale reads that use `AS OF SYSTEM TIME` will not block (as long as the transaction does not exceed the specified staleness). We’ll discuss `AS OF SYSTEM TIME` a bit later in this chapter.
- In multiregion configurations—which we’ll describe in detail in [Chapter 11](#)—GLOBAL tables use a modified transaction protocol in which reads are not blocked by writes.

Many transaction conflicts are managed automatically, and while these have performance implications, they don’t impact functionality or code design. However, there are multiple scenarios in which an application may need to handle an aborted transaction. We’ll look at these scenarios and discuss best practices for transaction retries in [Chapter 6](#).

Clock Synchronization and Clock Skew

You may have noticed in previous sections that CockroachDB must compare timestamps of operations frequently to determine if a transaction is in conflict. Simplistically, we might imagine that every node in the system can agree on the time of each operation and make these comparisons easily. In reality, every system is likely to have a slightly different system clock time, and this discrepancy is likely to be greater the more geographically distributed a system is. The difference in clock times is referred to as *clock skew*. Consequently, in widely distributed systems with very high transaction rates, getting nodes to agree on the exact sequence of transactions is problematic. As you might remember, Spanner attacked this problem by using specialized hardware—atomic clocks and GPS—to reduce the inconsistency between system clocks. As a result, Spanner can keep the clock skew within 7 ms and simply adds a 7 ms sleep to every transaction to ensure that no transactions complete out of order.

Since CockroachDB must run reliably on generic hardware, it synchronizes time using the venerable and ubiquitous internet Network Time Protocol (NTP). NTP produces accurate timestamps but nowhere near as accurate as Spanner’s GPS and atomic clocks.

By default, CockroachDB will tolerate a clock skew as high as 500 ms. Adding half a second to every transaction in the Spanner manner would be untenable, so CockroachDB takes a different approach for dealing with transactions that appear

within the 500 ms uncertainty interval. Put simply, while Spanner always waits after writes, CockroachDB sometimes retries reads.

If a reader can't say for certain whether a value being read was committed before the read transaction started, then it pushes its own provisional timestamp just above the timestamp of the uncertain value. Transactions reading constantly updated data from many nodes may be forced to restart multiple times, though never for longer than the uncertainty interval, nor more than once per node.

The CockroachDB time synchronization strategy allows CockroachDB to deliver true serializable consistency. However, there are still some anomalies that can occur. Two transactions that operate on unrelated KVs that still have some real-world sequencing dependency might appear to be committed in reverse order—the *causal reverse* anomaly. This is not a violation of serializable isolation because the transactions are not actually logically dependent. Nevertheless, it is possible in CockroachDB for transactions to have timestamps that do not reflect their real-world ordering.

The CockroachDB Distribution Layer

Logically, a table is represented in CockroachDB as a monolithic KV structure, in which the key is a concatenation of the primary keys of the table, and the value is a concatenation of all of the remaining columns in the table. We introduced this structure back in [Figure 2-2](#).

The distribution layer breaks this monolithic structure into contiguous chunks of approximately 512 MB. The 512 MB chunk is sized to keep the number of ranges per node manageable. The distribution layer keeps data distributed evenly across the cluster while simultaneously presenting a unified and consolidated view of that data to the applications that need it.

Meta Ranges

The distribution of ranges is stored in global keyspaces `meta1` and `meta2`. `meta1` can be thought of as a “range of ranges” lookup, which then allows a node to find the location of the node holding the `meta2` record, which in turn points to the nodes holding copies of every range within the “range of ranges.” [Figure 2-13](#) illustrates this two-level lookup structure.

Node 1 needs to get data for the key “HarrisonGuy.” It looks in its copy of `meta1`, which tells it that node2 contains the `meta2` information for the range G–M. It accesses the `meta2` data concerned from node 2, which indicates that node4 is the leaseholder for the range G–I, and therefore the leaseholder for the range concerned.

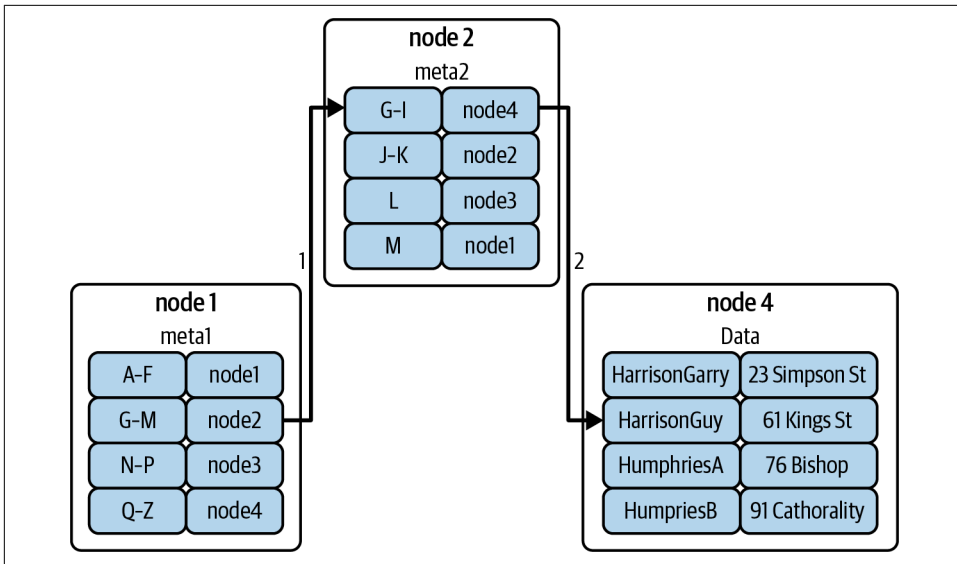


Figure 2-13. Meta ranges

Gossip

CockroachDB uses the *gossip* protocol to share ephemeral information between nodes. Gossip is a widely used protocol in distributed systems in which nodes propagate information virally through the network.

Gossip maintains an eventually consistent KV map on all the CockroachDB nodes. It is used primarily for bootstrapping: it contains a “meta0” record that tells the cluster where the meta1 range can be found, as well as mappings from the node IDs stored in meta records to network addresses. Gossip is also used for certain operations that do not require strong consistency, such as maintaining information about the available storage space on each node for rebalancing purposes.

Leaseholders

The leaseholder is the CockroachDB node responsible for serving reads and coordinating writes for a specific range of keys. We discussed some of the responsibilities of the leaseholder in “[The CockroachDB Transaction Layer](#)” on page 32. When a transaction coordinator or gateway node wants to initiate a read or write against a range, it finds that range’s leaseholder (using the meta ranges structure discussed in the previous section) and forwards the request to the leaseholder.

Leaseholders are assigned using the Raft protocol, which we will discuss in “[The CockroachDB Replication Layer](#)” on page 45.

Ranges can be merged as well. If DELETE statements remove data from ranges and the range falls below a size threshold, CockroachDB may merge the range with a neighboring range.

Multiregion Distribution

Geo-partitioning allows data to be located within a specific geographic region. This might be desirable from a performance point of view—reducing latencies for queries from a region about that region—or from a data sovereignty perspective—keeping data within a specific geographic region for legal or regulatory reasons. CockroachDB supports a multiregion configuration that controls how data should be distributed across regions. The following core concepts are relevant:

- *Cluster regions* are geographic regions that a user specifies at node start time.
- *Regions* may have multiple zones.
- *Super Regions* allow for data domiciling and contain one or more regions.
- Databases within the cluster are assigned to one or more regions: one of these regions is the *primary* region.
- Tables within a database may have specific *locality rules* (global, regional by table, regional by row), which determine how its data will be distributed across zones.
- *Survival goals* dictate how many simultaneous failures a database can survive.

With the *zone-level survival goal*, the database will remain fully available for reads and writes, even if a zone goes down. However, the database may not remain fully available if multiple zones fail in the same region. Surviving zone failures is the default setting for multiregion databases.

The *region-level survival goal* has the property that the database will remain fully available for reads and writes, even if an entire region goes down. This, of course, means that copies of data will need to be maintained in other regions, magnifying write time.

By default, all tables in a multiregion database are *regional tables*—that is, CockroachDB optimizes access to the table’s data from a single region (by default, the database’s primary region). *Regional by row* tables provide low-latency reads and writes for one or more rows of a table from a single region. Different rows in the table can be optimized for access from different regions.

Global tables are optimized for low-latency reads from all regions.

The CockroachDB Replication Layer

High availability requires that data not be lost or made unavailable should a node fail. This, of course, requires that multiple copies of data be maintained.

The two most commonly used high-availability designs are:

Active-passive

A single node is a “primary” or “active” node whose changes are propagated to passive “secondary” or “passive” nodes.

Active-active

All nodes run identical services. Typically, active-active database systems are of the “eventually consistent” variety. Since there is no “primary,” conflicting updates can be processed by different nodes. These will need to be resolved, possibly by discarding one of the conflicting updates.

CockroachDB implements a *distributed consensus* mechanism that is called multi-active. Like active-active, all replicas can handle traffic, but for an update to be accepted, it must be confirmed by a majority of voting replicas.

Not all replicas necessarily get a vote. Nonvoting replicas are useful in globally distributed systems since they allow for low latency reads in remote regions without requiring that region to participate in consensus during writes. This concept is discussed in more detail in [Chapter 11](#).

This architecture ensures that there is no data loss in the event of a node failure, and the system remains available, providing at least a majority of nodes remain active.

CockroachDB implements replication at the range level: each range is replicated independently of other ranges. At any given moment, a single node is responsible for changes to a single range, but there is no overall “primary” node within the cluster.

Raft

CockroachDB employs the widely used *Raft protocol* as its distributed consensus mechanism. In CockroachDB, each range is a distinct Raft group—the consensus for each range is determined independently of other ranges.

In Raft and in most distributed consensus mechanisms, we need a minimum of three nodes. This is because a majority of nodes (a quorum) must always agree on the state. In the event of a network partition, only the side of the partition with the majority of nodes can continue.

In a Raft group, one of the nodes is elected as leader by a majority of nodes in the group. The other nodes are known as followers. The Raft leader controls changes to the Raft group.

Changes sent to the Raft leader are written to its *Raft log* and propagated to the followers. When a majority of nodes accept the change, then the change is committed by the leader. Note that in CockroachDB, each range has its own Raft log because every range is replicated separately.

Leader elections occur regularly or may be triggered when a node fails to receive a heartbeat message from the leader. In the latter case, a follower who cannot communicate with the leader will declare itself a candidate and initiate an election. Raft includes a set of safety rules that prevent any data loss during the election process. In particular, a candidate cannot win an election unless its log contains all committed entries.

Nodes that are temporarily disconnected from the cluster can be sent to relevant sections of the Raft log to resynchronize or—if necessary—a point-in-time snapshot of the state followed by a catch-up via Raft logs.

Raft and Leaseholders

The CockroachDB leaseholder and the Raft leader responsibilities serve similar purposes. The leaseholder controls access to a range for the purposes of transactional integrity and isolation, while the Raft leader controls access to a range for the purposes of replication and data safety.

The leaseholder is the only node that can propose writes to the Raft leader. CockroachDB will attempt to elect a leaseholder who is also the Raft leader so that these communications can be streamlined. The leaseholder serves all writes and most reads, so it is able to maintain the in-memory data structures necessary to mediate read/write conflicts for the transaction layer.

Closed Timestamps and Follower Reads

Periodically the leaseholder will “close” a timestamp in the recent past, which guarantees that no new writes with lower timestamps will be accepted.

This mechanism also allows for *follower reads*. Normally, reads have to be serviced by a replica’s leaseholder. This can be slow since the leaseholder may be geographically distant from the gateway node that is issuing the query. A follower read is a read taken from the closest replica, regardless of the replica’s leaseholder status. This can result in much better latency in geo-distributed, multiregion deployments.

If a query uses the `AS OF SYSTEM TIME` clause, then the gatekeeper forwards the request to the closest node that contains a replica of the data—whether it be a follower or the leaseholder. The timestamp provided in the query (i.e., the `AS OF SYSTEM TIME` value) must be less than or equal to the node’s closed timestamp. This allows followers to service consistent reads in the recent past (i.e., several seconds ago).

Global tables in a multiregion database use a special variation of the transaction protocol called *nonblocking transactions* that is optimized for reads (from any replica) at the expense of writes. Writes to tables in this mode are assigned timestamps in the future, and timestamps in the future may be closed. This makes it possible for followers to serve consistent reads at the present time.

The CockroachDB Storage Layer

We touched upon the logical structure of the KV store earlier in the chapter when we discussed the store. However, we have not yet looked at the physical implementation of the KV storage engine.

Since CockroachDB v20.2, CockroachDB has used the Pebble storage engine—an open source KV store inspired by the LevelDB and RocksDB storage engines. Pebble is primarily maintained by the CockroachDB team and is optimized specifically for CockroachDB use cases. Older versions of CockroachDB use the RocksDB storage engine.

Let’s look under the hood of the Pebble storage engine so that we can fully appreciate how CockroachDB stores and manipulates data at its foundational layer.

Log-Structured Merge Trees

Pebble implements the log-structured merge (LSM) tree architecture. LSM is a widely implemented and battle-tested architecture that seeks to optimize storage and support extremely high insert rates, while still supporting efficient random read access.

The simplest possible LSM tree consists of two indexed “trees:”

- An in-memory tree that is the recipient of all new record inserts—the *MemTable*.
- A number of on-disk trees represent copies of in-memory trees that have been flushed to disk. These are referred to as *sorted strings tables* (SSTables).

SSTables exist at multiple levels, numbered L0 to L6 (L6 is also called the base level). L0 contains an unordered set of SSTables, each of which is simply a copy of an in-memory MemTable that has been flushed to disk. Periodically, SSTables are compacted into larger consolidated stores in the lower levels. In levels other than L0, SSTables are ordered and nonoverlapping so that only one SSTable per level could possibly hold a given key.

SSTables are internally sorted and indexed, so lookups within an SSTable are fast.

The basic LSM architecture ensures that writes are always fast since they primarily operate at memory speed, although there is often also a sequential *write-ahead log* (WAL) on disk. The transfer to on-disk SSTables is also fast since it occurs

in append-only batches using fast sequential writes. Reads occur either from the in-memory tree or from the disk tree; in either case, reads are facilitated by an index and are relatively swift.

Of course, if a node fails while data is in the in-memory store, then it could be lost. For this reason, database implementations of the LSM pattern include a WAL that persists transactions to disk. The WAL is written via fast sequential writes.

Figure 2-15 illustrates LSM writes. Writes from higher CockroachDB layers are first applied to the WAL (1) and then to the MemTable (2). Once the MemTable reaches a certain size, it is flushed to disk to create a new SSTable (3). Once the flush completes, WAL records may be purged (4). Multiple SSTables are routinely merged (compacted) into larger SSTables (5).

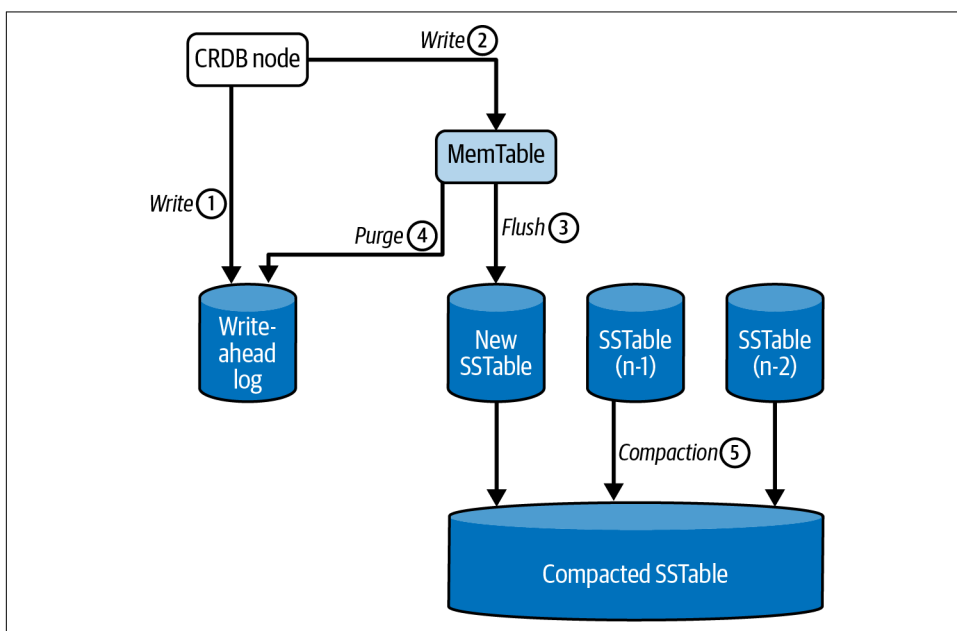


Figure 2-15. LSM writes

The compaction process results in multiple “levels”—Level 0 (L0) contains the uncompact data. Each compaction creates a file at a deeper level—up to 7 levels (L0–L6) are typical.

SSTables and Bloom Filters

Each SSTable is indexed. However, there may be many SSTables on disk, and this creates a multiplier effect on index lookups because we might theoretically have to examine every index for every SSTable to find our desired row.

To reduce the overhead of multiple index lookups, *Bloom filters* are used to reduce the number of lookups that must be performed. A Bloom filter is a compact and quick-to-maintain structure that can quickly tell you if a given SSTable “might” contain a value. CockroachDB uses Bloom filters to quickly determine which SSTables have a version of a key. Bloom filters are compact enough to fit in memory and are quick to navigate. However, to achieve this compression, Bloom filters are “fuzzy” and may return false positives. If you get a positive result from a Bloom filter, it means only that the file *may* contain the value. However, the Bloom filter will never incorrectly advise you that a value is not present. So, if a Bloom filter tells us that a key is not included in a specific SSTable, then we can safely omit that SSTable from our lookup.

Figure 2-16 shows the read pattern for an LSM. A database request first reads from the MemTable (1). If the required value is not found, it will consult the Bloom filters for all SSTables in L0 (2). If the Bloom filter indicates that no matching value is present, it will examine the SSTable in each subsequent level that covers the given key (3). If the Bloom filter indicates a matching KV may be present in the SSTable, then the process will use the SSTable index (4) to search for the value within the SSTable (5). Once a matching value is found, no older SSTables need to be examined.

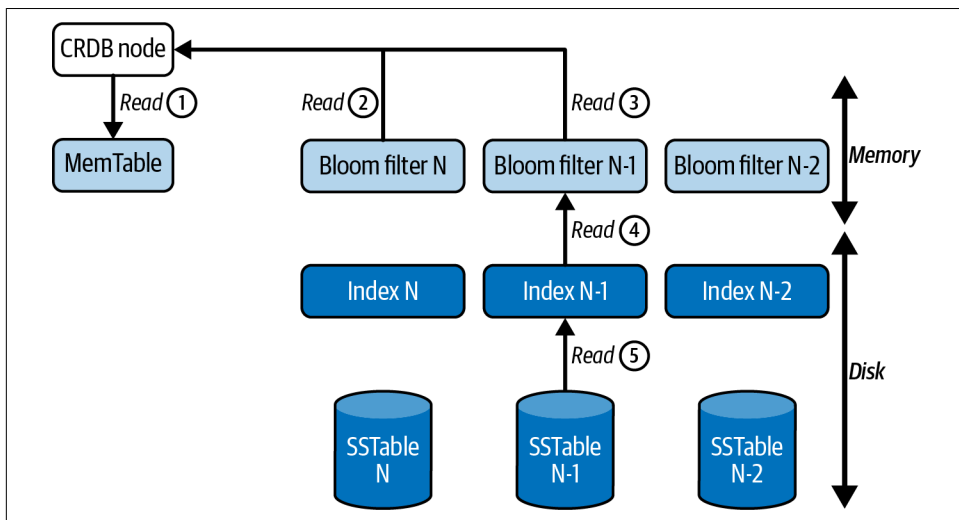


Figure 2-16. LSM reads

Deletes and Updates

SSTables are immutable—once the MemTable is flushed to disk and becomes an SSTable, no further modifications to the SSTable can be performed. If a value is modified repeatedly over a period of time, the modifications will build up across multiple SSTables. When retrieving a value, the system will read SSTables from youngest to oldest to find the most recent value for a key. Therefore, to update a value, we only

need to insert the new value since the older values will not be examined when a newer version exists.

Deletions are implemented by writing tombstone markers into the MemTable, which eventually propagate to SSTables. Once a tombstone marker for a row is encountered, the system stops examining older entries and reports “not found” to the application.

As SSTables multiply, read performance and storage will degrade as the number of Bloom filters, indexes, and obsolete values increases. During compaction, rows that are fragmented across multiple SSTables will be consolidated and deleted rows removed. Tombstones are retained until they are compacted to the base level, L6.

Multiversion Concurrency Control

We introduced MVCC as a logical element of the transaction layer in “[MVCC Principles](#)” on [page 33](#). CockroachDB encodes the MVCC timestamp into each key so that multiple MVCC versions of a key are stored as distinct keys within Pebble. However, the Bloom filters that we introduced previously exclude the MVCC timestamp so that a query does not need to know the exact timestamp to look up a record.

CockroachDB removes records older than the configuration variable `gc.ttlseconds`, but will not remove any records covered by *protected timestamps*. Protected timestamps are created by long-running jobs such as backups, which need to be able to obtain a consistent view of data.

The Block Cache

Pebble implements a block cache providing fast access to frequently accessed data items. This block cache is separate from the in-memory indexes, Bloom filters, and MemTables. The block cache operates on a least recently used (LRU) basis—when a new data entry is added to the cache, the entry that was least recently accessed will be evicted from the cache.

Reading from the block cache bypasses the need to scan multiple SSTables and associated Bloom filters. We’ll speak more about the cache in [Chapter 14](#) when we discuss cluster optimization.

Summary

In this chapter, we’ve given you an overview of the essential architectural elements of CockroachDB. Although having a strong grasp of the CockroachDB architecture is advantageous when performing advanced systems optimization or configuration, it’s by no means a prerequisite for working with a CockroachDB system. CockroachDB includes many sophisticated design elements, but its internal complexity is not

reflected in its UI—you can happily develop a CockroachDB application without mastering the architectural concepts in this chapter.

At a cluster level, a CockroachDB deployment consists of three or more symmetrical nodes, each of which carries a complete copy of the CockroachDB software stack and each of which can service any database client requests. Data in a CockroachDB table is broken up into ranges of 512 MB in size and distributed across the nodes of the cluster. Each range is replicated at least three times.

The CockroachDB software stack consists of five major layers:

- The SQL layer accepts SQL requests in the PostgreSQL wire protocol. It parses and optimizes the SQL requests and translates the requests into KV operations that can be processed by lower layers.
- The transaction layer is responsible for ensuring ACID transactions and transaction isolation. It ensures that transactions see a consistent view of data and that modifications occur as if they had been executed one at a time.
- The distribution layer is responsible for the partitioning of data into ranges and the distribution of those ranges across the cluster. It is responsible for managing Range leases and assigning leaseholders.
- The replication layer ensures that data is correctly replicated across the cluster to allow high availability in the event of a node failure. It implements a distributed consensus mechanism to ensure that all nodes agree on the current state of any data item.
- The storage layer is responsible for the persistence of data to local disk and the processing of low-level queries and updates on that data.

In the next chapter, we'll gleefully abandon the complexities and sophisticated CockroachDB architecture and focus on the far simpler task of getting started with the CockroachDB system.

Getting Started

CockroachDB has a sophisticated and modern architecture and is designed for global scale. However, that complexity and scalability don't imply a steep learning curve or barrier to entry. In this chapter, we'll help you get started with a CockroachDB installation and introduce you to the basics of working with a CockroachDB system.

Installation

CockroachDB can be installed on virtually any flavor of desktop OS within a few minutes. Alternatively, you can create a free CockroachDB Cloud Basic database or run CockroachDB within a Docker container or Kubernetes cluster.

Installing CockroachDB Software

In most scenarios, you'll want to have the CockroachDB software installed on your desktop computer, so let's start with that. You'll find a full list of binaries in the [CockroachDB docs](#). From there, you can pick your OS and download the most recent version or pick a previous version.

The instructions that follow worked as of the time of writing, but installation can change with each release, so make sure you consult the CockroachDB website for the most up-to-date instructions.

Installation on macOS

If you have the Homebrew package manager installed, then that is probably the easiest way to get started installing CockroachDB on Mac. In fact, even if you don't have Homebrew installed, it's probably easier to install it and then install CockroachDB than to install CockroachDB manually.

To install Homebrew, issue the following command from a terminal window:

```
/bin/bash -c \  
"$ (curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh
```

Once Homebrew is installed, you can install CockroachDB with this command:

```
% brew install cockroachdb/tap/cockroach  
==> Tapping cockroachdb/tap  
Cloning into '/usr/local/Homebrew/Library/Taps/cockroachdb/homebrew-tap'...  
...  
==> Installing cockroach from cockroachdb/tap  
...  
To have launchd start cockroachdb/tap/cockroach now and restart at login:  
  brew services start cockroachdb/tap/cockroach  
Or, if you don't want/need a background service you can just run:  
  cockroach start-single-node --insecure  
==> Summary  
/usr/local/Cellar/cockroach/24.2.0: 195 files, 289.2MB, built in 7 seconds
```

One of the great things about Homebrew is that it sets up CockroachDB as a service, so you can issue `brew services start cockroach` to start a background instance of CockroachDB. However, if you don't want to use Homebrew, then you can download the CockroachDB binary directly and copy the binary into your path. Review the [releases documentation](#) to determine the path for the release you want, then use `curl` or `wget` to copy and decompress that release:

```
% curl https://binaries.cockroachdb.com/cockroach-v24.2.0.darwin-11.0-arm64.tgz \  
| tar -xJ  
% Total      % Received % Xferd  Average Speed   Time    Time     Time  Current  
             Dload  Upload   Total   Spent    Left  Speed  
100 120M  100 120M    0     0  8196k      0  0:00:15  0:00:15 --:--:-- 7955k
```

You can then copy the binary into your path so you can execute Cockroach commands from any directory:

```
sudo cp -R cockroach-v24.2.0.darwin-11.0-arm64/* /usr/local/bin
```

Once you've installed CockroachDB either manually or via Homebrew, run the `cockroach demo` command to start a demo instance and confirm that it is running:

```
% cockroach demo  
#  
# Welcome to the CockroachDB demo database!  
#  
# You are connected to a temporary, in-memory CockroachDB cluster of 1 node.  
#  
# Enter \? for a brief introduction.  
#
```

```
demo@127.0.0.1:26257/movr> show databases;
database_name | owner | primary_region | regions | survival_goal
-----+-----+-----+-----+-----
defaultdb    | root | NULL           | {}      | NULL
movr         | demo | NULL           | {}      | NULL
postgres     | root | NULL           | {}      | NULL
system       | node | NULL           | {}      | NULL
(4 rows)
```

Time: 2ms total (execution 1ms / network 0ms)

Installation on Linux

To perform a basic installation on Linux, find the **latest release** (or a specific version you're interested in), and download and unpack it. Of course, you can use `curl` or `wget` to obtain the tarball once you have determined its path:

```
$ wget https://binaries.cockroachdb.com/cockroach-v24.2.0.linux-arm64.tgz

2024-09-09 08:01:44 (7.71 MB/s) - 'cockroach-v24.2.0.linux-arm64.tgz' saved

$ tar zxvf cockroach-v24.2.0.linux-arm64.tgz
cockroach-v24.2.0.linux-arm64/cockroach
cockroach-v24.2.0.linux-arm64/lib/libgeos.so
cockroach-v24.2.0.linux-arm64/lib/libgeos_c.so

$ sudo cp -r cockroach-v24.2.0.linux-arm64/* /usr/local/bin
```

Once installed, run the `cockroach demo` command to start a temporary local instance of CockroachDB and verify the installation:

```
$ cockroach demo
#
# Welcome to the CockroachDB demo database!
#
# You are connected to a temporary, in-memory CockroachDB cluster of 1 node.
#
...
#
# Enter \? for a brief introduction.
#
root@127.0.0.1:44913/movr> show databases;
database_name | owner
-----+-----
defaultdb    | root
movr         | root
postgres     | root
system       | node
(4 rows)
```

Time: 1ms total (execution 1ms / network 0ms)

```
root@127.0.0.1:44913/movr>
```

For a completely manual installation like this, you may have to install geo-spatial libraries manually if you want to use geo-spatial features. See the [CockroachDB documentation](#) for more details.

Installation on Microsoft Windows

Microsoft Windows is not a fully supported platform for running a CockroachDB server. However, Windows is completely supported for CockroachDB clients and the server runs well enough for experimentation and most development.

Find the [link for the release](#) you'd like to download. Once downloaded, unzip the archive into a directory and add the subdirectory containing the *cockroach.exe* file to your path. Alternatively, you can download the file directly from a PowerShell prompt. Instructions for doing so can be found on the [CockroachDB documentation site](#).

CockroachDB URLs

When connecting to a CockroachDB cluster, we need to identify the location and credentials with which we wish to connect. When connecting to a local server using `cockroach demo` or `cockroach sql`, the CockroachDB client will default to a local server on the default port, but as we will see, more complex installations require quite a bit more information.

The most common way to connect is to use a PostgreSQL-compatible URL. This URL is of the following format:

```
postgres://[user[:passwd]]@[host][:port]/[db][?parameters...]
```

The simplest possible URL for a local cluster running without authentication would look something like this:

```
$ cockroach sql --url 'postgres://root@localhost:26257?sslmode=disable'
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: 072189bb-3970-4f37-afe4-55bc37cdf76e
#
# Enter \? for a brief introduction.
#
root@localhost:26257/defaultdb>
```

This is equivalent to running the command `cockroach sql -insecure`.

The beauty of the URL is that it can be accepted by most PostgreSQL-compatible programs or drivers. For instance, if we have the PostgreSQL client installed, we can use it to connect to CockroachDB:

```
$ psql 'postgres://root@localhost:26257?sslmode=disable'
psql (13.2, server 9.5.0)
Type "help" for help.

root=#
```

Creating a CockroachDB Cloud Basic Instance

The `cockroach demo` command is a handy way to play around with the CockroachDB server, but the easiest way to get a fully functional CockroachDB server with persistent storage is to take advantage of the free CockroachDB Cloud Basic database service. This service grants you access to a fully functional multitenant cloud service with 10 GiB of storage.

CockroachDB Cloud Basic has a number of advantages compared with a desktop deployment:

- It's automatically configured for high availability and backup. You don't have to worry about losing your data in the event of a hard drive failure on your desktop.
- It's fully secured using encryption at rest and in transit.
- It's available from anywhere, so it can be used for team development purposes.
- It's well-suited for starter projects and evaluating CockroachDB. To create a CockroachDB Cloud Basic cluster, navigate to the [signup page](#) and select the Basic plan.

After entering your details and validating your email address, you'll be given the option to create your free cluster.

Once the cluster is created, you'll be given instructions on how to connect. These instructions will be similar to users of every operating system, but for a Mac user, they'll look something like [Figure 3-1](#).

Download CA Cert (Required only once)
^

Select operating system
Mac (detected)

Run the following command to download the CA cert for server certificate verification

```
curl --create-dirs -o $HOME/.postgresql/root.crt
'https://cockroachlabs.cloud/clusters/614eb05c-0493-4947-962a-b07e3b282ef2/cert'
```

Copy

[Learn more about CA certs](#)

General connection string
^

Use the following connection string to connect to your cluster

```
postgresql://rob:REVEAL_PASSWORD@cloudy-jackal-5535.j77.aws-eu-west-1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-full
```

Copy

Forgot Password? [Regenerate password](#)

Figure 3-1. Connection instructions

Assuming you have already downloaded the CockroachDB binary, issue the `curl` command from the instructions to copy the necessary certificates to the desktop, then the `cockroach sql` command to connect to the database:

```
$ curl -create-dirs -o $HOME/.postgresql/root.crt -O \
https://cockroachlabs.cloud/clusters/614eb05c-0493-4947-962a-b07e3b282ef2/cert

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 2728    0 2728    0    0   5250      0 --:--:-- --:--:-- --:--:-- 5317

$ cockroach sql \
  -url "postgresql://rob:xxxx@cloudy-jackal-5535.j77.aws-eu-west-1.cockroachlabs
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
```

```
# Cluster ID: 614eb05c-0493-4947-962a-b07e3b282ef2
warning: cannot show server execution timings: unexpected column found
#
# Enter \? For a brief introduction.
#
rob@cloudy-jackal.../defaultdb> SELECT
                                database_name,
                                primary_region,
                                regions,
                                survival_goal
                                FROM [SHOW DATABASES];
```

database_name	primary_region	regions	survival_goal
defaultdb	NULL	{}	NULL
postgres	NULL	{}	NULL
system	aws-eu-west-1	{aws-eu-west-1}	zone

(3 rows)

Time: 32ms total (execution 7ms / network 26ms)

CockroachDB Cloud Basic Passwords

Note that the password in the connection string is *not* the password you provided to connect to your CockroachDB Cloud Basic account. Your CockroachDB Cloud Basic account might be associated with many databases, each of which has its own password. The password shown in the connection dialog will be shown only if you hover over the `REVEAL_PASSWORD` link and will be shown only at this point in the database creation. It's up to you to save that password and keep it safe. If you forget your password, you can reset it by going to the SQL Users page.

Starting a Local Single-Node Server

As we've seen previously, you can use the `cockroach demo` command to start a temporary demo cluster, and we can quickly create a free CockroachDB Cloud Basic cluster. If you want to start a single-node CockroachDB server with persistent storage on your own hardware, you can use the `cockroach start-single-node` command:

```
$ cockroach start-single-node --insecure --listen-addr=localhost
*
* WARNING: ALL SECURITY CONTROLS HAVE BEEN DISABLED!
*
* This mode is intended for non-production testing only.
*
* In this mode:
* - Your cluster is open to any client that can access any
  of your IP addresses.
* - Intruders with access to your machine or network can observe
```

```

    client/server traffic.
* - Intruders can log in without password and read or write any
    data in the cluster.
* - Intruders can consume all your server's resources and cause unavailability.
*
*
* INFO: To start a secure server without mandating TLS for clients,
* consider --accept-sql-without-tls instead. For other options, see:
*
* - https://go.crdb.dev/issue-v/53404/v24.2
* - https://www.cockroachlabs.com/docs/v24.2/secure-a-cluster.html
*

```

This will start a single-node CockroachDB cluster with no security. To connect to this server, we can use the `cockroach sql` command with the default connection string:

```

$ cockroach sql --insecure
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: 848d8b85-4000-484a-b4ad-8f2c76c68221
#
# Enter \? for a brief introduction.
#
root@:26257/defaultdb> show databases;
  database_name | owner | primary_region | regions | survival_goal
-----+-----+-----+-----+-----
  defaultdb     | root  | NULL           | {}      | NULL
  postgres      | root  | NULL           | {}      | NULL
  system        | node  | NULL           | {}      | NULL
(3 rows)

Time: 3ms total (execution 3ms / network 0ms)

root@:26257/defaultdb>

```

Insecure Mode

The use of the `--insecure` flag is convenient for quickly starting a CockroachDB cluster, but it's absolutely not appropriate for a production system. Please see [Chapter 10](#) for instructions on setting up a properly secured production system.

Starting Up CockroachDB in a Docker Container

If you have Docker, you can quickly start a CockroachDB single-node instance inside a Docker container. You'll need a persistent volume for data, so let's create that first:

```
$ docker volume create crdb1
```


Then, we invoke `docker run` to pull and start the latest CockroachDB Docker image and start the server in single-node, insecure mode:

```
$ docker run -d \
> --name=crdb1 \
> --hostname=crdb1 \
> -p 26257:26257 -p 8080:8080 \
> -v "crdb1:/cockroach/cockroach-data" \
> cockroachdb/cockroach:latest start-single-node \
> --insecure \
>
Unable to find image 'cockroachdb/cockroach:latest' locally
latest: Pulling from cockroachdb/cockroach
a591faa84ab0: Pull complete
...
6913e7a5719...914b1aafe8
```

The output of the `docker run` command is the container identifier for the CockroachDB container. Using that containerId, we can connect to that container using the `cockroach sql` command:

```
$ docker exec -it 6913e7a5719...914b1aafe8 \
    cockroach sql --insecure
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: 8fcbb9bb-ec7c-40dc-afe0-90306c87f5d7
#
# Enter \? for a brief introduction.
#
root@26257/defaultdb> show databases;
  database_name | owner | primary_region | regions | survival_goal
-----+-----+-----+-----+-----
    defaultdb   | root  | NULL            | {}      | NULL
    postgres    | root  | NULL            | {}      | NULL
    system       | node  | NULL            | {}      | NULL
(3 rows)

Time: 4ms total (execution 3ms / network 1ms)
```

We don't need to have the CockroachDB software installed on our local host to connect using the preceding method because we're using the CockroachDB client installed within the Docker container. However, since we've forwarded port 26257 from the Docker container, we can attach from the desktop using the default connection:

```
$ ~ cockroach sql --insecure
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
```

```
# To exit, type: \q.
#
# Cluster ID: d070609f-58a7-4aea-aa27-92bc4a1e5406
#
# Enter \? for a brief introduction.
#
root@:26257/defaultdb>
```

Note that this port forwarding can work only if there's not already a CockroachDB server listening on that port.

Starting Up a Secure Server

In the previous examples, we've used the `--insecure` flag to start the server without needing to configure secure communications. This is a quick way to set up a test server but is catastrophically dangerous for anything that contains valuable data.

We'll cover CockroachDB security in depth within [Chapter 13](#), but for now, to set up a secure server, we need to create security certificates to encrypt the communications channel and authenticate the client and server.

The following commands create the certificates. The certificate authority key will be held in `my-safe-directory`; the certificates themselves will be held in the `certs` directory:

```
$ mkdir certs my-safe-directory

$ # CA certificate and keypair

$ cockroach cert create-ca \
>   --certs-dir=certs \
>   --ca-key=my-safe-directory/ca.key

$ # certificate and keypair for localhost
$ cockroach cert create-node localhost 'hostname' --certs-dir=certs \
>   --ca-key=my-safe-directory/ca.key

$ # certificate for the root user
$ cockroach cert create-client root \
>   --certs-dir=certs \
>   --ca-key=my-safe-directory/ca.key
```

We can now start the server and specify the directory containing the certificates:

```
$ # start single node
$ cockroach start-single-node --certs-dir=certs \
  --listen-addr=localhost
```

Now, when connecting, we must specify the certificates directory. If we're connecting from a remote host, then we would need to copy the certificates to that host.

```
$ cockroach sql --certs-dir=certs
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: f908d29e-1fb6-40b8-9e1f-a2a0a3763603
#
# Enter \? for a brief introduction.
#
root@:26257/defaultdb>
```

Certificates Directory

On Linux or macOS systems, CockroachDB will look for certificates in the `~/.cockroach-certs` directory. If your certificates are placed there, then you won't need to specify the `--certs-dir` argument. However, if you have multiple CockroachDB servers, then you may need to maintain distinct certificates for each, possibly in their own directories.

Shutting Down the Server

If the server has been started with the `--background` flag, then we shut down the server using a kill signal. For instance, `killall` can be used to issue a kill command to all cockroach commands currently running:

```
$ killall cockroach
```

```
initiating graceful shutdown of server
server drained and shutdown completed
```

To kill a specific server, identify its process ID and then issue the `kill` command specifying the process ID:

```
$ ps -ef |grep cockroach
ubuntu      13911      1 10 10:16 pts/0    00:00:43 cockroach
             start-single-node --insecure --listen-addr=localhost
```

```
$ kill 13911
$ initiating graceful shutdown of server
server drained and shutdown completed
```

Remote Connection

In the previous examples, we've connected to a server running on the same host as our client. This is pretty unusual in the real world, where we would normally be connecting to a server on another machine. Typically, we'd specify the URL parameter

to identify the server concerned. For instance, to connect to a server on the mubuntu server on the default port, we could issue the following command:

```
$ cockroach sql --certs-dir=certs --url postgresql://root@mubuntu:26257/defaultdb
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: f908d29e-1fb6-40b8-9e1f-a2a0a3763603
#
# Enter \? for a brief introduction.
#
root@mubuntu:26257/defaultdb>
```

Creating a Kubernetes Cluster

In earlier examples, we've created single-node clusters and connected to a free CockroachDB Cloud Basic database that is a shared region of a multitenant cluster. If you want to start with a dedicated multinode cluster, then the easiest way is to install a CockroachDB cluster in a Kubernetes environment using the CockroachDB Kubernetes operator.

Kubernetes is an increasingly ubiquitous framework that coordinates—orchestrates—the management of the components of a distributed system. The CockroachDB Kubernetes operator contains the configuration and utilities that allow CockroachDB to be deployed in Kubernetes.

We'll come back to production deployment options for Kubernetes later in the book. For now, we will deploy CockroachDB in a Kubernetes minikube cluster, which implements a local Kubernetes cluster on a desktop system.

For this example, we are using a minikube cluster running on macOS with 6 CPUs and 12 GB of memory. You can start such a cluster with the following command:

```
~ minikube start --memory=12G --cpus=6
🍏 minikube v1.18.1 on Darwin 12.1
```

The first step is to deploy the operator and its manifest:

```
$ kubectl apply -f https://cockroa.ch/crdbclusters_yaml

customresourcedefinition.apiextensions.k8s.io/crdbclusters.crdp.cockroachlabs.com
created

$ kubectl apply -f https://cockroa.ch/operator_yaml

clusterrole.rbac.authorization.k8s.io/cockroach-database-role created
serviceaccount/cockroach-database-sa created
clusterrolebinding.rbac.authorization.k8s.io/cockroach-database-rolebinding
created
```

```

role.rbac.authorization.k8s.io/cockroach-operator-role created
clusterrolebinding.rbac.authorization.k8s.io/cockroach-operator-rolebinding
created
clusterrole.rbac.authorization.k8s.io/cockroach-operator-role created
serviceaccount/cockroach-operator-sa created
rolebinding.rbac.authorization.k8s.io/cockroach-operator-default created
deployment.apps/cockroach-operator created

```

Once this is done, a `kubectl get pods` command should show the CockroachDB Kubernetes operator running inside the cluster:

```

$ kubectl config set-context --current --namespace=cockroach-operator-system
$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
cockroach-operator-84bf588dbb-65m8k 0/1     ContainerCreating   0           9s

```

We then retrieve the example configuration file that is included in the operator's repository:

```

$ curl -O https://cockroa.ch/example_yaml -o example.yaml

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left    Speed
100 1098 100 1098    0     0   3399      0  --:--:-- --:--:-- --:--:-- 3399

```

This file contains definitions for the cluster to be configured, such as the number of nodes to be created and the memory and CPU required by each node. The configuration is tilted toward a production deployment, so you might want to trim down the requirements. For instance, in the following code we see that the default configuration file specifies a 60 GB storage requirement. We might want to change this to a lower value for a simple test system (or increase it for a bigger deployment):

```

apiVersion: crdb.cockroachlabs.com/v1alpha1
kind: CrdbCluster
metadata:
  name: cockroachdb
spec:
  dataStore:
    pvc:
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: "60Gi"
        volumeMode: Filesystem

```

You could edit other elements of the configuration file, such as the number of nodes to be created or the version of CockroachDB to be used.

We now apply the configuration file to the operator, which will perform the necessary tasks to create the cluster:

```
$ kubectl apply -f example.yaml
crdbcluster.crd.cockroachlabs.com/cockroachdb created
```

The cluster creation process can take some time. We'll know it's complete when a `kubectl get pods` command shows all nodes in Running state:

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
cockroach-operator-84bf588dbb-65m8k 1/1     Running   0           6m59s
cockroachdb-0                        1/1     Running   0           87s
cockroachdb-1                        1/1     Running   0           71s
cockroachdb-2                        1/1     Running   0           57s
```

We can connect to the cluster by invoking the `cockroach sql` command from within any of the CockroachDB nodes. For instance, here we connect to `cockroachdb-2` and connect to the cluster:

```
$ kubectl exec -it cockroachdb-2 -- ./cockroach sql --certs-dir cockroach-certs
#
# Welcome to the CockroachDB SQL shell.
# All statements must be terminated by a semicolon.
# To exit, type: \q.
#
# Cluster ID: cb78255b-befa-4447-9fa8-c06b7a353564
#
# Enter \? for a brief introduction.
#
  database_name | owner | primary_region | regions | survival_goal
-----+-----+-----+-----+-----
  defaultdb    | root  | NULL           | {}      | NULL
  postgres     | root  | NULL           | {}      | NULL
  system        | node  | NULL           | {}      | NULL
(3 rows)

Time: 7ms total (execution 6ms / network 1ms)
```

Connecting to the cluster using this method requires a high level of access to the cluster. In a production environment, we would set up a load balancer to securely handle incoming requests to the cluster. We'll look at these sorts of configurations in [Chapter 10](#).

Creating a Cluster with Terraform

CockroachDB Cloud clusters can be created and configured using Terraform, a popular infrastructure as code (IaC) platform from HashiCorp. In this section, we'll create a CockroachDB Cloud Basic, Standard, and Advanced cluster using Terraform.

We'll start by initializing the **CockroachDB Terraform provider**:

```
terraform {
  required_providers {
    cockroach = {
      source = "cockroachdb/cockroach"
      version = "2.0.0"
    }
  }
}
```

Next, we'll create a folder. While optional, folders can help customers organize their CockroachDB Cloud clusters and can be nested up to four levels deep. On the **Billing dashboard**, folders allow customers to view the aggregate cost of any clusters within them.

```
resource "cockroach_folder" "top" {
  name      = "level_1"
  parent_id = "root"
}

resource "cockroach_folder" "nested" {
  name      = "level_2"
  parent_id = cockroach_folder.top.id
}
```

Next we'll create the clusters, starting with a CockroachDB Cloud Basic cluster in AWS. The following Terraform resource creates a Basic cluster across three regions, specifying a cap of 1,000,000 Request Units (RUs) per month and a storage limit of 1 GiB. The `parent_id` references the ID of the nested folder we defined previously:

```
resource "cockroach_cluster" "basic" {
  cloud_provider = "AWS"
  name           = "aws-basic"
  regions = [
    { name : "us-east-1", primary = true },
    { name : "eu-central-1" },
    { name : "ap-southeast-1" }
  ]

  serverless = {
    usage_limits = {
      request_unit_limit = 1000000
      storage_mib_limit  = 1024
    }
  }

  parent_id = cockroach_folder.nested.id
}
```

Next, we'll create a CockroachDB Cloud Standard cluster in GCP. As with the Basic cluster, the Standard cluster will run across three regions, but this time we're limiting it by vCPUs, guaranteeing the cluster access to 2 vCPUs:

```
resource "cockroach_cluster" "standard" {
  cloud_provider = "GCP"
  name           = "gcp-standard"
  regions = [
    { name : "us-east1", primary = true },
    { name : "europe-west1" },
    { name : "asia-southeast1" }
  ]

  serverless = {
    usage_limits = {
      provisioned_virtual_cpus = 2
    }
  }

  parent_id = cockroach_folder.nested.id
}
```

Finally, we'll create a CockroachDB Cloud Advanced cluster in Azure. As with the Basic and Standard clusters, the Advanced cluster will run across three regions. Unlike the Basic and Standard clusters, however, Advanced clusters run with dedicated hardware, so we specify a node count per region, a disk size in GiB, and a vCPU count for each node:

```
resource "cockroach_cluster" "advanced" {
  cloud_provider = "AZURE"
  name           = "azure-advanced"
  regions = [
    {
      name : "eastus",
      node_count = 3
    },
    {
      name : "germanywestcentral",
      node_count = 3
    },
    {
      name : "southeastasia",
      node_count = 3
    }
  ]

  dedicated = {
    storage_gib      = 16
    num_virtual_cpus = 4
  }
}
```



```

    parent_id = cockroach_folder.nested.id
  }

```

For the Standard cluster, let's provide some more resources to better understand the options available. This will not be exhaustive, and the full list of available resources can be found in the [CockroachDB Terraform Provider documentation](#).

First, let's define an IP allow list, granting access to all IPs globally. Typically, you'd want to expose your cluster privately via technologies like VPC Peering, AWS Private-Link, and GCP Private Service Connect, but for this example we'll keep things simple:

```

resource "cockroach_allow_list" "standard" {
  name      = "Unrestricted access"
  cidr_ip   = "0.0.0.0"
  cidr_mask = 0
  sql       = true
  ui        = false
  cluster_id = cockroach_cluster.standard.id
}

```

Next, we'll create a SQL user with a password. We'll generate the password, so we'll need to use an output block to make it available once applied:

```

resource "random_password" "standard" {
  special = false
  length  = 50
}

resource "cockroach_sql_user" "standard" {
  cluster_id = cockroach_cluster.standard.id
  name       = "rob"
  password   = random_password.standard.result
}

output "password" {
  value     = cockroach_sql_user.standard.password
  sensitive = true
}

```

Post apply, we can access the password as follows:

```

terraform output -raw password
rk7B6QncmN2jRvU2...

```

Databases can also be defined in Terraform. Let's create an example database for the Standard cluster:

```

resource "cockroach_database" "standard" {
  name      = "accounts"
  cluster_id = cockroach_cluster.standard.id
}

```

Finally, we'll create a service account with minimal access to the cluster. This account could be used for things like accessing the Metrics Export endpoint. As with the user password, we'll need to output the API key for it to be available outside of Terraform:

```
resource "cockroach_service_account" "standard_monitor" {
  name      = "standard-cluster-monitor"
  description = "A service account for use in monitoring the standard cluster."
}

resource "cockroach_user_role_grant" "standard_monitor" {
  user_id = cockroach_service_account.standard_monitor.id
  role = {
    role_name      = "CLUSTER_OPERATOR_WRITER",
    resource_type  = "CLUSTER",
    resource_id    = cockroach_cluster.standard.id
  }
}

resource "cockroach_api_key" "standard_monitor" {
  name      = "standard-cluster-monitor-key-v1"
  service_account_id = cockroach_service_account.standard_monitor.id
}

output "standard_monitor_api_key" {
  value      = cockroach_api_key.standard_monitor.secret
  description = "The standard-cluster-monitor-key-v1 API key"
  sensitive  = true
}
```

Post apply, we can access the API key as follows:

```
terraform output -raw standard_monitor_api_key
CCDB1_X363xP8T6Y...
```

The configuration can be applied as follows:

```
terraform apply
```

And destroyed as follows:

```
terraform destroy
```

Using a GUI Client

While some are more than happy to use only a command-line client to interact with a database, some of us prefer a GUI. Many GUI applications for PostgreSQL exist, and most of these will work with CockroachDB. However, **DBeaver Community edition** is a free database GUI that has dedicated support for CockroachDB.

The CockroachDB documentation has some further information about using **DBeaver**.

Exploring CockroachDB

Now that we've got access to a CockroachDB cluster and have the client ready to connect, let's take CockroachDB for a drive! In the following examples, we are using a local cluster—the connection strings may be different if you are using a Cloud Basic instance.

Adding Some Data

As folks say in Australia, “A database without data is like a pub with no beer!” Let's get some data into the database so that we have something to look at.

The CockroachDB software includes a number of demonstration databases that you can quickly add to your CockroachDB installation. In some cases, these databases are prepopulated with data; in other cases, you create the schemas and then add data afterward.

To initialize the schemas, we use the `cockroach workload init [schema]` command. To run a workload against the schema, we use the `cockroach workload run [schema]` command. The schemas include:

bank

Models a set of accounts with currency balances. After initializing the schema, use `workload run` to generate a workload against the database.

intro

A simple single-table database.

kv

A simple KV schema. After initializing the schema, use `run` to generate a workload that will be evenly distributed across the cluster.

movr

A schema for a fictional ride-sharing application. This schema can be used with the `workload run` command to generate load against the databases.

startrek

A *Star Trek* database, with two tables, episodes and quotes.

tpcc

A transaction processing schema for the TPC-C standard benchmark. This schema can be used with the `workload run` command to generate load against the databases.

yycsb

The Yahoo Cloud Serving Benchmark schema. This schema can be used with the workload run command to generate load against the databases.

For the intro and startrek databases, we create the tables and data using the workload init command. For instance, in the following example, we create the startrek schema and look at some data:

```
$ cockroach workload init startrek \  
  'postgres://root@localhost:26257?sslmode=disable'  
I210501 04:29:29.694340 1   imported episodes (0s, 79 rows)  
I210501 04:29:29.898945 1   imported quotes (0s, 200 rows)  
$ cockroach sql --insecure  
#  
# Welcome to the CockroachDB SQL shell.  
# All statements must be terminated by a semicolon.  
# To exit, type: \q.  
#  
#  
# Enter \? for a brief introduction.  
#  
root@:26257/defaultdb> show databases;  
  
  database_name | owner | primary_region | regions | survival_goal  
-----+-----+-----+-----+-----  
  defaultdb    | root  | NULL           | {}      | NULL  
  postgres     | root  | NULL           | {}      | NULL  
  startrek      | root  | NULL           | {}      | NULL  
  system       | node  | NULL           | {}      | NULL  
(4 rows)  
  
Time: 3ms total (execution 3ms / network 1ms)  
  
root@:26257/defaultdb> use startrek;  
SET  
  
Time: 1ms total (execution 0ms / network 0ms)  
  
root@:26257/startrek> show tables;  
  
  schema_name | table_name | type  | owner | estimated_row_count | locality  
-----+-----+-----+-----+-----+-----  
  public      | episodes  | table | root  | 0                   | NULL  
  public      | quotes    | table | root  | 0                   | NULL  
(2 rows)  
  
Time: 24ms total (execution 24ms / network 0ms)
```

```

root@:26257/startrek> select * from episodes limit 1;
 id | season | num | title | stardate
-----+-----+-----+-----+-----
  1 |      1 |   1 | The Man Trap | 1531.1
(1 row)

```

Time: 1ms total (execution 1ms / network 0ms)

In this example, we create the bank schema:

```

$ cockroach workload init bank \
  'postgres://root@localhost:26257?sslmode=disable'
I210501 04:31:41.214008 1 imported bank (0s, 1000 rows)
I210501 04:31:41.221478 1 starting 9 splits

```

And then run a workload simulation for 60 seconds:

```

$ cockroach workload run bank 'postgres://root@localhost:26257?sslmode=disable' \
  --duration 60s
I210501 04:33:52.340852 1 creating load generator...
I210501 04:33:52.344074 1 creating load generator... done (took 3.220303ms)
_elapsed_ops/sec(inst)___ops/sec(cum)___p50(ms)___p95(ms)___pMax(ms)
  1.0s          187.3          187.9      16.8      48.2     121.6 transfer
  2.0s          295.0          241.5      11.0      31.5      79.7 transfer
  3.0s          260.9          248.0      13.1      37.7      83.9 transfer
  4.0s          203.1          236.7      17.8      39.8      79.7 <snip>

_elapsed___ops(total)___ops/sec(cum)___avg(ms)___p50(ms)___p99(ms)___pMax(ms)
 60.0s          14230          237.2      16.9      13.6      65.0     192.9

```

The run command is primarily meant to generate data for load testing purposes but is useful to generate data for query purposes as well.

Databases and Tables

As we've seen already, data in a CockroachDB deployment is organized into specific namespaces called databases. Database is a fairly loosely used and overloaded term—it's quite common for a CockroachDB cluster to be referred to as a database or for a database within a cluster to be referred to as a schema. However, in CockroachDB, as in most other SQL databases, a database cluster contains one or more databases. Within a database, one or more schemas may be defined, though it's common for each database to contain only one schema.

We can list the databases in the cluster using the `show databases` command:

```

root@:26257/defaultdb> show databases;
 database_name | owner | primary_region | regions | survival_goal
-----+-----+-----+-----+-----
 bank          | root | NULL           | {}      | NULL
 defaultdb     | root | NULL           | {}      | NULL
 postgres      | root | NULL           | {}      | NULL

```

```

startrek      | root | NULL          | {}      | NULL
system       | node | NULL          | {}      | NULL
(5 rows)

```

We can set our current database with the use command:

```

root@:26257/defaultdb> use startrek;
SET

```

Time: 1ms total (execution 0ms / network 0ms)

We list tables within a database with the show tables command:

```

root@:26257/startrek> show tables;
 schema_name | table_name | type | owner | estimated_row_count | locality
-----+-----+-----+-----+-----+-----
 public      | episodes  | table | root  | 79 | NULL
 public      | quotes    | table | root  | 200 | NULL
(2 rows)

```

Time: 16ms total (execution 16ms / network 0ms)

We can describe a table using the \d command:

```

root@:26257/startrek> \d quotes;
 column_name | data_type | is_nullable | column_default | indices
-----+-----+-----+-----+-----
 quote       | STRING    | true        | NULL           | {primary}
 characters   | STRING    | true        | NULL           | {primary}
 stardate     | DECIMAL   | true        | NULL           | {primary}
 episode      | INT8      | true        | NULL           | {primary,quotes_epis
 rowid        | INT8      | false       | unique_rowid() | {primary,quotes_epis
(5 rows)

```

Time: 13ms total (execution 12ms / network 1ms)

Issuing SQL

From the CockroachDB client, we can issue any SQL commands for which we are authorized.

Here we connect to the *Star Trek* sample database and issue a query to find the episodes with the most quotes:

```

root@localhost:26257/defaultdb> USE startrek;

SELECT id,
       title,
       count(*) AS quote_count
FROM episodes AS e
LEFT OUTER JOIN quotes AS q ON (e.id = q.episode)
GROUP BY id,
         title
ORDER BY 3 DESC

```

```
LIMIT 10;  
SET
```

```
Time: 1ms total (execution 0ms / network 1ms)
```

id	title	quote_count
53	The Ultimate Computer	11
77	The Savage Curtain	9
11	The Menagerie, Part I	7
38	Metamorphosis	7
16	The Galileo Seven	7
28	The City on the Edge of Forever	6
26	Errand of Mercy	6
24	This Side of Paradise	5
23	A Taste of Armageddon	5
37	I, Mudd	5

(10 rows)

```
Time: 5ms total (execution 3ms / network 1ms)
```

The DB Console

The CockroachDB server exposes a web-based client that shows the status of the cluster and useful performance metrics. The web server is usually exposed on port 8080, though this can be changed using the `--http-addr` setting when starting the server.

The full DB Console is available for self-hosted and CockroachDB Cloud Advanced clusters. CockroachDB Cloud Basic and Standard clusters use a simplified version of the console.

Working with Programming Languages

Working with the CockroachDB shell is useful for experimentation, but eventually, most databases interact with application code written in languages such as JavaScript, Java, Go, or Python.

Because CockroachDB is wire-compatible with Postgres, most Postgres-compatible drivers will work with CockroachDB. Indeed, there are no CockroachDB-specific drivers on the market because the Postgres drivers work so well. In this section, we'll get you up to speed with “hello world” programs in Java, Go, Python, and JavaScript that connect to and query a CockroachDB cluster. A full list of supported languages can be found in the [CockroachDB online documentation](#).

Connecting to CockroachDB from Node.js

Server-side JavaScript using the Node.js platform is an increasingly popular choice for application development because it allows the same JavaScript language to be used for both frontend web presentation code and server-side application logic.

Assuming that you have Node.js and the Node package manager (npm) installed, we'll use the `node-postgres` driver to connect to CockroachDB. We can install this driver with the following command:

```
npm install pg
```

Once `pg` is installed, then the following example should connect to any CockroachDB database using a connection URI:

```
// Example of connecting to CockroachDB using Node.js

const CrClient = require('pg').Client; //load pg client

async function main() {
  try {
    // Check parameters
    if (process.argv.length !== 3) {
      console.log('Usage: node helloWorld.js CONNECTION_URI');
      process.exit(1);
    }
    // Establish a connection using the command-line URI
    const connectionString = process.argv[2];
    const crClient = new CrClient(connectionString);
    await crClient.connect();

    // Issue a SELECT
    const data = await crClient.query(
      `SELECT CONCAT('Hello from CockroachDB at ',
                     CAST (NOW() as STRING)) as hello`
    );
    // Print out the error message
    console.log(data.rows[0].hello);
  } catch (error) {
    console.log(error.stack);
  }
  // Exit
  process.exit(0);
}

main();
```

This program expects the connection string to be provided as the first argument to the program. The `process.argv` array contains the full command line including "node" and "helloWorld.js", so the URI actually shows up as the third element in the array.

We then attempt to establish a connection using that connection string, then issue a SELECT statement that retrieves the time as known to the server.

Here we connect to a CockroachDB Cloud Basic cluster:

```
$ node helloWorld.js "postgresql://jesse:xxxxxxxxxxxxx
aws-us-west-2.cockroachlabs.cloud:26257/defaultdb?
sslmode=verify-full&sslrootcert=$HOME/.postgresql/root.crt&
options=-ccluster%3Dalert-dingo-2030"
```

```
Hello from CockroachDB at 2024-05-02 00:17:40.835834+00:00
```

And here we connect to a local CockroachDB running in insecure mode:

```
$ node helloWorld.js 'postgres://root@localhost:26257?sslmode=disable'
Hello from CockroachDB at 2024-05-02 00:32:39.125419+00:00
```

Connecting to CockroachDB from Java

Java is the workhorse of millions of applications across all industries and contexts. In this example, we will use the official PostgreSQL Java Database Connectivity (JDBC) driver to connect to a CockroachDB server. Download the **JDBC driver** and place it in your class path or configure it as a dependency in your IDE.

The following program accepts a URL, username, and password as arguments on the command line, connects to the CockroachDB cluster concerned, and issues a SELECT statement:

```
package helloCRDB;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class HelloCRDB {
    public static void main(String[] args) {
        Connection cdb = null;
        try {
            Class.forName("org.postgresql.Driver");
            String connectionURL="jdbc:"+args[0];
            String userName=args[1];
            String passWord=args[2];

            cdb = DriverManager.getConnection(connectionURL,userName,passWord);
            Statement stmt = cdb.createStatement();
            ResultSet rs = stmt
                .executeQuery("SELECT CONCAT('Hello from CockroachDB at',"
                    + "CAST (NOW() as STRING)) AS hello");

            rs.next();
            System.out.println(rs.getString("hello"));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

    } catch (Exception e) {
        e.printStackTrace();
        System.err.println(e.getClass().getName() + ": " + e.getMessage());
        System.exit(0);
    }
}
}

```

If we want to connect to the CockroachDB Cloud Basic cluster we created earlier, we issue the following command:¹

```

$ java -m helloCRDB/helloCRDB.HelloCRDB
postgres://gcp-asia-southeast1.cockroachlabs.cloud:26257/defaultdb
?sslmode=verify-full&sslrootcert=/Users/guyharrison/CockroachDBCockroachDBKeys/
cc-ca.crt&options=-cluster=grumpy-orca-56 \
guy xxxxxxxxxxxx

```

```
Hello from CockroachDB at 2024-05-05 15:39:07.667438+10:00
```

And here we connect to a local CockroachDB cluster in insecure mode:

```

$ java -m helloCRDB/helloCRDB.HelloCRDB postgres://localhost:26257/
?sslmode=disable root ''

```

```
Hello from CockroachDB at 2024-05-05 15:38:56.691009+10:00
```

Connecting to CockroachDB from Python

Python is a widely used scripting language as well as the tool of choice for many data scientists and data wranglers. In this example, we'll use the `psycopg` python-`postgres` package to connect to CockroachDB.

To install the `psycopg` package, issue the following command:

```
$ pip3 install psycopg2
```

```

Collecting psycopg2
  Using cached psycopg2-2.8.6.tar.gz (383 kB)
Building wheels for collected packages: psycopg2
  Building wheel for psycopg2 (setup.py) ... done
  Created wheel for psycopg2: filename=psycopg2-2.8.6-cp39-...
  Stored in directory: /Users/guyharrison/Li...
Successfully built psycopg2
Installing collected packages: psycopg2
Successfully installed psycopg2-2.8.6

```

¹ Note that the URL is slightly different from the ones used for other languages. The Postgres JDBC driver does not support embedding the username and password in the URL, so we need to pass them separately.

Now the following short program will connect to CockroachDB using a URL provided on the command line and issue a SELECT statement:

```
#!/usr/bin/env python3

import psycopg2
import sys

def main():

    if ((len(sys.argv)) !=2):
        sys.exit("Error:No URL provided on command line")
    uri=sys.argv[1]

    conn = psycopg2.connect(uri)
    with conn.cursor() as cur:
        cur.execute("""SELECT CONCAT('Hello from CockroachDB at ',
                                   CAST (NOW() as STRING))""")
        data=cur.fetchone()
        print("%s" % data[0])

main()
```

Here we connect to the CockroachDB Cloud Basic database we established earlier in the chapter:²

```
$ python helloCockroachDB.py \
'postgres://guy:xxxxxx@gcp-asia-
southeast1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-
full&sslrootcert=/Users/guyharrison/CockroachDBCockroachDBKeys/
cc-ca.crt&options=-cluster%3dgrumpy-orca-56'
```

```
Hello from CockroachDB at 2024-05-02 02:39:55.859734+00:00
```

And here we connect to a local CockroachDB cluster running in insecure mode:

```
$ python helloCockroachDB.py 'postgres://root@localhost:26257?sslmode=disable'
Hello from CockroachDB at 2024-05-02 02:33:00.755359+00:00
```

Connecting to CockroachDB from Go

The Go language is one of the fastest-growing programming languages, and it offers high performance, modern programming paradigms, and a low footprint. Much of the CockroachDB database platform is written in Go, so Go is a great choice for CockroachDB development.

² Note that because of limitations in the psycopg2 driver, we need to replace the final “=” in the URL with “%3d.” Instead of `cluster=grumpy-orca-56`, we use `cluster%3dgrumpy-orca-56`.

In this example, we're going to use the `pgx` PostgreSQL driver for Go to connect to the CockroachDB Cloud Basic cluster we created earlier. First, we need to install the driver:

```
$ go get github.com/jackc/pgx/v5
```

This short program connects to CockroachDB using the URL provided on the command line and issues a `SELECT` statement:

```
package main

import (
    "context"
    "fmt"
    "os"
    "github.com/jackc/pgx/v5"
)

func main() {
    uri := "postgres://root@localhost:26257/bank?ssl=disabled"
    conn, err := pgx.Connect(context.Background(), uri)
    if err != nil {
        fmt.Fprintf(os.Stderr,
            "Unable to connect to database: %v\n", err)
        os.Exit(1)
    }
    var text string
    err = conn.QueryRow(context.Background(),
        `SELECT CONCAT('Hello from CockroachDB at ',
            CAST (NOW() as STRING))`).Scan(&text)
    if err != nil {
        fmt.Fprintf(os.Stderr, "QueryRow failed: %v\n", err)
        os.Exit(1)
    }

    fmt.Println(text)
}
```

We connect to a CockroachDB Cloud Basic cluster:

```
$ go run helloCockroachDB.go \
    "postgres://guy:xxxxxxx@gcp-asia-
southeast1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-
full&sslrootcert=$HOME/CockroachDBCockroachDBKeys/
cc-ca.crt&options=--cluster=grumpy-orca-56"
```

```
Hello from CockroachDB at 2024-05-02 02:24:13.930662+00:00
```

And then we run the program to connect to a local CockroachDB cluster in insecure mode:

```
$ go run helloCockroach.go 'postgres://root@localhost:26257?sslmode=disable'
Hello from CockroachDB at 2024-05-02 02:21:59.179171+00:00
```

Summary

In this chapter, we've shown you how to install CockroachDB software on a local computer, how to create a CockroachDB cluster in a variety of configurations, and how to work with CockroachDB from the command line or a programming language.

It's easy to install CockroachDB software on a desktop and, in most cases, necessary if you want to work with a CockroachDB server from the command line. You can also install CockroachDB software using Docker or Kubernetes.

While a single-node test server can be a useful tool for learning CockroachDB, the CockroachDB Cloud Basic plan offers a free 10 GiB server that provides backup and security. You can also install CockroachDB in a Kubernetes cluster to experiment with a full cluster in a local environment.

Because CockroachDB is PostgreSQL-compatible, you can use any PostgreSQL-compatible driver to connect to CockroachDB. We also provided simple examples of connecting to CockroachDB using the PostgreSQL drivers for Java, Python, Go, and Node.js.

In the next chapter, we'll dive into SQL, covering the most common SQL statements you'll want to use when interacting with CockroachDB.

CockroachDB SQL

The language of CockroachDB is SQL. While there are some command-line utilities, all interactions between an application and the database are mediated by SQL language commands.

SQL is a rich language with a long history—we touched upon some of that history in [Chapter 1](#). A full definition of all SQL language features would require a book in its own right and would be almost instantly out of date because the SQL language evolves with each release.

Therefore, this chapter aims to provide you with a broad overview of the SQL language used in CockroachDB without attempting to be a complete reference. We'll take a task-oriented approach to SQL, covering the most common SQL language tasks with particular reference to unique features of the CockroachDB SQL implementation.

As we described in [Chapter 1](#), SQL is a *declarative* language. SQL statements represent logical requests for queries and data manipulation operations without specifying how the database should implement those requests.

A complete reference for the CockroachDB SQL language can be found in the [CockroachDB documentation set](#). A broader review of the SQL language can be found in the O'Reilly book *SQL in a Nutshell*.

Some of the examples in this chapter use the MovR sample data set to illustrate various SQL language features. We showed how to install sample data in [Chapter 3](#).

SQL Language Compatibility

CockroachDB is broadly compatible with the PostgreSQL implementation of the SQL:2016 standard. The SQL:2016 standard contains a number of independent

modules, and no major database implements all of the standards. However, the PostgreSQL implementation of SQL is arguably as close to “standard” as exists in the database community.

CockroachDB varies from PostgreSQL in a couple of areas (e.g., CockroachDB does not support PostgreSQL XML functions). For a list of these differences, see [PostgreSQL Compatibility](#).

Querying Data with SELECT

Although we need to create and populate tables before querying them, it’s logical to start with the SELECT statement since many features of the SELECT statement appear in other types of SQL—subqueries in UPDATES, for instance—and for data scientists and analysts, the SELECT statement is often the only SQL statement they ever need to learn.

The SELECT statement ([Figure 4-1](#)) is the workhorse of relational queries and has a complex and rich syntax. The CockroachDB SELECT statement implements the typical features of the standard SELECT, with just a few CockroachDB-specific features.

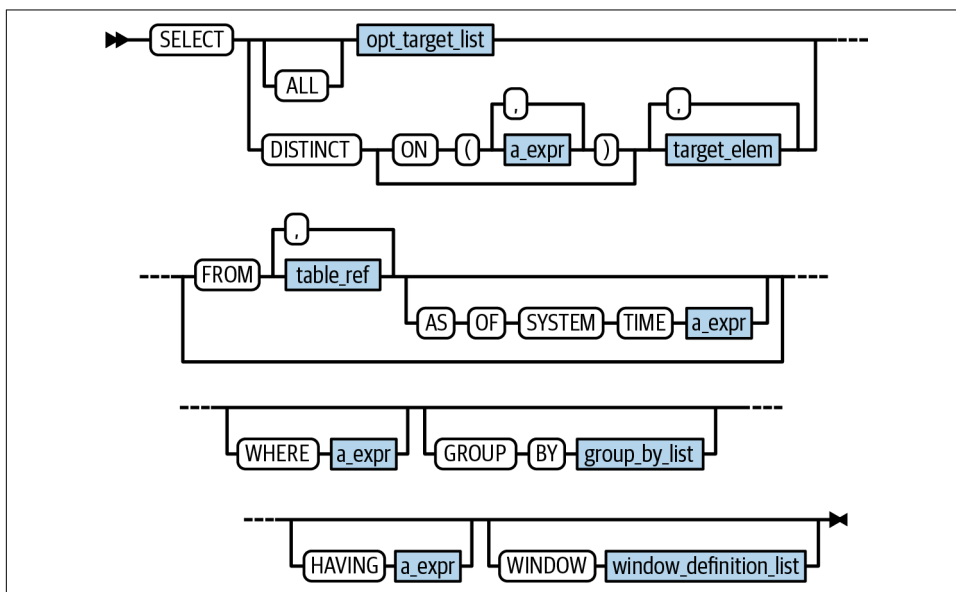


Figure 4-1. SELECT statement

In the following sections, we’ll examine each of the major elements of the SELECT statement as well as the functions and operators that can be included in it.

The SELECT List

A simple SQL statement consists of nothing but a SELECT statement together with scalar expressions (i.e., expressions that return a single value). For instance:

```
SELECT CONCAT('Hello from CockroachDB at ',
              CAST (NOW() as STRING)) as hello;
```

The SELECT list includes a comma-separated list of expressions that can contain combinations of constants, functions, and operators. The CockroachDB SQL language supports all the familiar SQL operators. A complete list of functions and operators can be found in the [CockroachDB documentation set](#).

The FROM Clause

The FROM clause is the primary method of attaching table data to the SELECT statement. In its most simple incarnation, all rows and columns from a table can be fetched via a full table scan:

```
SELECT * FROM rides;
```

Table names may be aliased using the AS clause or simply by following the table name with an alias. That alias can then be used anywhere in the query to refer to the table. Column names can also be aliased. For instance, the following are all equivalent:

```
SELECT name FROM users;
SELECT u.name FROM users u;
SELECT users.name FROM users;
SELECT users.name AS user_name FROM users;
SELECT u.name FROM users AS u;
```

Joins

Joins allow the results from two or more tables to be merged based on some common column values.

The INNER JOIN is the default JOIN operation. In this join, rows from one table are joined to rows from another table based on some common (“key”) values. Rows that have no match in both tables are not included in the results. For instance, the following query links vehicle and ride information in the movr database:

```
SELECT v.id,v.ext,r.start_time, r.start_address
FROM vehicles v
INNER JOIN rides r
ON (r.vehicle_id=v.id);
```

Note that a vehicle that had not been involved in a ride would not be included in the result set.

The ON clause specifies the conditions that join the two tables—in the previous query, the columns `vehicle_id` in the `rider` table were matched with the `id` column in the `vehicles` table. If the JOIN is on an identically named column in both tables, then the USING clause provides a handy shortcut. Here we join `users` and `user_ride_counts` using the common `name` column:¹

```
SELECT *
FROM users u
JOIN user_ride_counts urc
USING (name);
```

The OUTER JOIN allows rows to be included even if they have no match in the other table. Rows that are not found in the OUTER JOIN table are represented by NULL values. LEFT and RIGHT determine which table may have missing values. For instance, the following query prints all the users in the `users` table, even if some are not associated with a promo code:

```
SELECT u.name , upc.code
FROM users u
LEFT OUTER JOIN user_promo_codes upc
ON (u.id=upc.user_id);
```

The RIGHT OUTER JOIN reverses the default (LEFT) OUTER JOIN. So this query is identical to the previous query because the `users` table is now the “right” table in the join:

```
SELECT DISTINCT u.name , upc.code
FROM user_promo_codes upc
RIGHT OUTER JOIN users u
ON (u.id=upc.user_id);
```

Anti-Joins

It is often required to select all rows from a table that do not have a matching row in some other result set. This is called an anti-join, and while there is no SQL syntax for this concept, it is typically implemented using a subquery and the IN or EXISTS clause. The following example illustrates an anti-join using the EXISTS and IN operators.

Each example selects users who are not also employees:

```
SELECT *
FROM users
```

¹ Note that `user_ride_counts` is not defined in the default `rides` schema. It is defined as `SELECT u.name, COUNT(u.name) AS rides FROM "users" AS u JOIN "rides" AS r ON (u.id=r.rider_id) GROUP BY u.name.`

```
WHERE id NOT IN
      (SELECT id FROM employees);
```

This query returns the same results but using a correlated subquery (we'll discuss subqueries in more detail in an upcoming section):

```
SELECT *
FROM users u
WHERE NOT EXISTS
      (SELECT id
       FROM employees e
       WHERE e.id=u.id);
```

Cross Joins

CROSS JOIN indicates that every row in the left table should be joined to every row in the right table. Usually this is a recipe for disaster, unless one of the tables has only one row or is a laterally correlated subquery (see “[Correlated Subquery](#)” on page 89).

Set Operations

SQL implements a number of operations that deal directly with result sets. These operations, collectively referred to as “set operations,” allow result sets to be concatenated, subtracted, or overlaid.

The most common of these operations is the UNION operator, which returns the sum of two result sets. By default, duplicates in each result set are eliminated. By contrast, the UNION ALL operation will return the sum of the two result sets, including any duplicates. The following example returns a list of customers and employees. Employees who are also customers will be listed only once:

```
SELECT name, address
FROM customers
UNION
SELECT name,address
FROM employees;
```

INTERSECT returns those rows that are in both result sets. This query returns customers who are also employees:

```
SELECT name, address
FROM customers
INTERSECT
SELECT name,address
FROM employees;
```

EXCEPT returns rows in the first result set that are not present in the second. This query returns customers who are not also employees:

```
SELECT name, address
FROM customers
```

```

EXCEPT
SELECT name,address
FROM employees;

```

All set operations require that the component queries return the same number of columns and that those columns are of a compatible data type.

Group Operations

Aggregate operations allow for summary information to be generated, typically upon groupings of rows. Rows can be grouped using the `GROUP BY` operator. If this is done, the select list must consist only of columns contained within the `GROUP BY` clause and aggregate functions.

The most common aggregate functions are shown in [Table 4-1](#).

Table 4-1. Aggregate functions

Function	Description
AVG	Calculate the average value for the group.
COUNT	Return the number of rows in the group.
MAX	Return the maximum value in the group.
MIN	Return the minimum value in the group.
STDDEV	Return the standard deviation for the group.
SUM	Return the total of all values for the group.

The following example generates summary ride information for each city:

```

SELECT u.city,SUM(urc.rides),AVG(urc.rides),max(urc.rides)
FROM users u
JOIN user_ride_counts urc
USING (name)
GROUP BY u.city;

```

Subqueries

A subquery is a `SELECT` statement that occurs within another SQL statement. Such a “nested” `SELECT` statement can be used in a wide variety of SQL contexts, including `SELECT`, `DELETE`, `UPDATE`, and `INSERT` statements.

The following statement uses a subquery to count the number of rides that share the maximum ride length:

```

SELECT COUNT(*) FROM rides
WHERE (end_time-start_time)=
(SELECT MAX(end_time-start_time) FROM rides );

```

Subqueries may also be used in the FROM clause wherever a table or view definition could appear. This query generates a result that compares each ride with the average ride duration for the city:

```
SELECT id, city, (end_time-start_time) ride_duration, avg_ride_duration
FROM rides
JOIN (SELECT city,
      AVG(end_time-start_time) avg_ride_duration
      FROM rides
      GROUP BY city)
USING(city) ;
```

Correlated Subquery

A correlated subquery is one in which the subquery refers to values in the parent query or operation. The subquery returns a potentially different result for each row in the parent result set. We saw an example of a correlated subquery when performing an “anti-join” earlier in the chapter.

```
SELECT *
FROM users u
WHERE NOT EXISTS
  (SELECT id
   FROM employees e
   WHERE e.id=u.id);
```

Subqueries can often be used to perform an operation that is functionally equivalent to a join. In many cases, the query optimizer will transform these statements to joins to streamline the optimization process.

Lateral Subquery

When a subquery is used in a join, the LATERAL keyword indicates that the subquery may access columns generated in preceding FROM table expressions. For instance, in the following query, the LATERAL keyword allows the subquery to access columns from the users table:

```
SELECT name, address, start_time
FROM users CROSS JOIN
  LATERAL (SELECT *
          FROM rides
          WHERE rides.start_address = users.address ) r;
```

This example is a bit contrived, and clearly, we could construct a simple JOIN that performed this query more naturally. Where LATERAL joins really shine is in allowing subqueries to access computed columns in other subqueries within a FROM clause. Andy Woods’s [CockroachDB blog post](#) describes a more serious example of lateral subqueries.

The WHERE Clause

The WHERE clause is common to SELECT, UPDATE, and DELETE statements. It specifies a set of logical conditions that must evaluate to true for all rows to be returned or processed by the SQL statement concerned.

Common Table Expressions

SQL statements with a lot of subqueries can be hard to read and maintain, especially if the same subquery is needed in multiple contexts within the query. For this reason, SQL supports *Common Table Expressions* using the WITH clause. Figure 4-2 shows the syntax of a Common Table Expression.

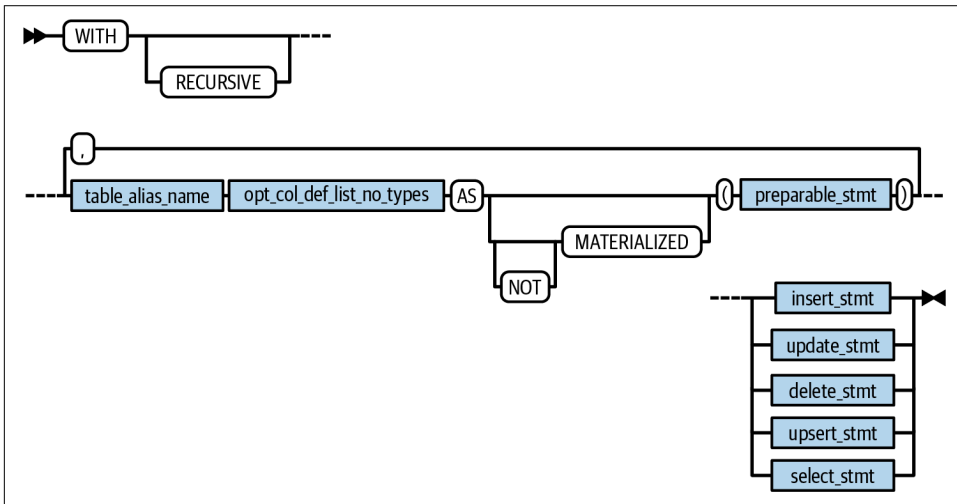


Figure 4-2. Common Table Expression

In its simplest form, a Common Table Expression is simply a named query block that can be applied wherever a table expression can be used. For instance, here we use the WITH clause to create a Common Table Expression, `riderRevenue`, then refer to it in the FROM clause of the main query:

```
WITH riderRevenue AS (
    SELECT u.id, SUM(r.revenue) AS sumRevenue
    FROM rides r JOIN "users" u
    ON (r.rider_id=u.id)
    GROUP BY u.id)
SELECT * FROM "users" u2
JOIN riderRevenue rr USING (id)
ORDER BY sumrevenue DESC;
```

The `RECURSIVE` clause allows the Common Table Expression to refer to itself, potentially allowing for a query to return an arbitrarily high (or even infinite) set of results. For instance, if the `employees` table contained a `manager_id` column that referred to the manager's row in the same table, then we could print a hierarchy of employees and managers as follows:

```
WITH RECURSIVE employeeMgr AS (  
  SELECT id,manager_id, name , NULL AS manager_name, 1 AS level  
    FROM employees managers  
   WHERE manager_id IS NULL  
 UNION ALL  
  SELECT subordinates.id,subordinates.manager_id,  
         subordinates.name, managers.name ,managers.LEVEL+1  
    FROM employeeMgr managers  
   JOIN employees subordinates  
     ON (subordinates.manager_id=managers.id)  
)  
SELECT * FROM employeeMgr;
```

The `MATERIALIZED` clause forces CockroachDB to store the results of the Common Table Expression as a temporary table rather than reexecuting it on each occurrence. This can be useful if the Common Table Expression is referenced multiple times in the query.

ORDER BY

The `ORDER BY` clause allows query results to be returned in sorted order. [Figure 4-3](#) shows the `ORDER BY` syntax. Note that `sortby_index` is expanded in [Figure 4-4](#).

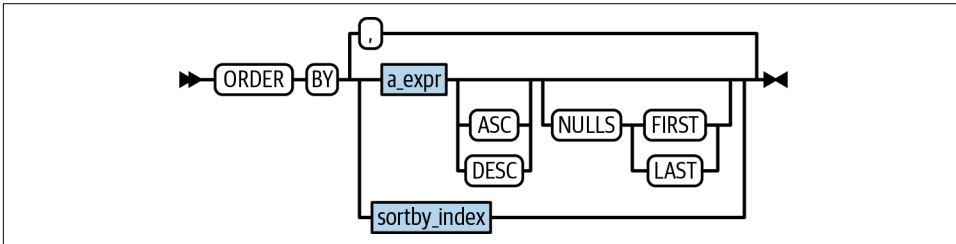


Figure 4-3. `ORDER BY`

In the simplest form, `ORDER BY` takes one or more column expressions or column numbers from the `SELECT` list.

In this example, we sort by column numbers:

```
SELECT city,start_time, (end_time-start_time) duration  
FROM rides r  
ORDER BY 1,3 DESC;
```

And in this case, by column expressions:

```
SELECT city,start_time, (end_time-start_time) duration
FROM rides r
ORDER BY city,(end_time-start_time) DESC;
```

As shown in [Figure 4-4](#), you can also order by an index.

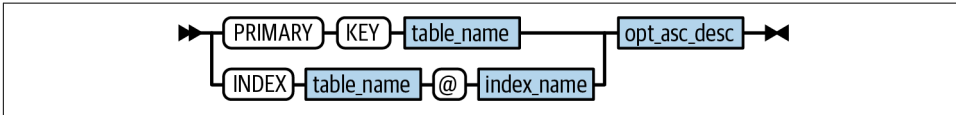


Figure 4-4. SORT BY INDEX

In the following example, rows will be ordered by city and start_time, since those are the columns specified in the index:

```
CREATE INDEX rides_start_time ON rides (city ,start_time);

SELECT city,start_time, (end_time-start_time) duration
FROM rides
ORDER BY INDEX rides@rides_start_time;
```

The use of ORDER BY INDEX guarantees that the index will be used to directly return rows in sorted order, rather than having to perform a sort operation on the rows after they are retrieved. See [Chapter 8](#) for more advice on optimizing statements that contain an ORDER BY.

Window Functions

Window functions are functions that operate over a subset—a “window” of the complete set of the results. [Figure 4-5](#) shows the syntax of a window function.

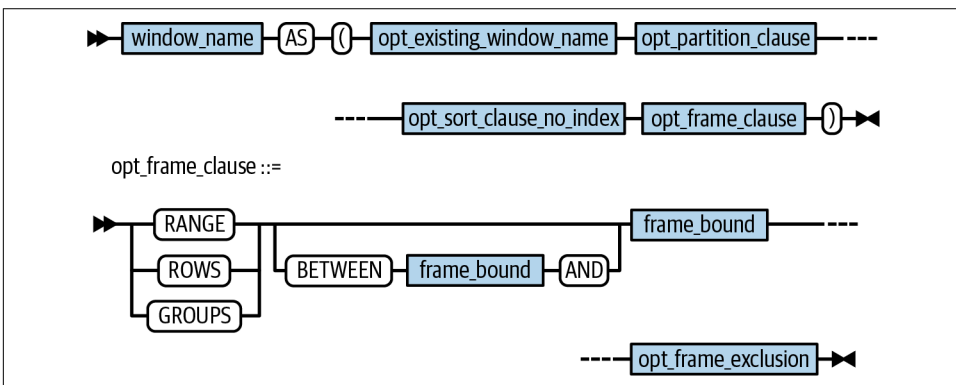


Figure 4-5. Window function syntax

PARTITION BY and ORDER BY create a sort of “virtual table” that the function works with. For instance, this query lists the top 10 rides in terms of revenue, with the percentage of the total revenue and city revenue displayed:

```
SELECT city, r.start_time ,revenue,
       revenue*100/SUM(revenue) OVER () AS pct_total_revenue,
       revenue*100/SUM(revenue) OVER (PARTITION BY city) AS pct_city_revenue
FROM rides r
ORDER BY 5 DESC
LIMIT 10;
```

There are some aggregation functions that are specific to windowing functions. RANK() ranks the existing row within the relevant window, and DENSE_RANK() does the same while allowing no “missing” ranks. LEAD and LAG provide access to functions in adjacent partitions.

For instance, this query returns the top 10 rides, with each ride’s overall rank and rank within the city displayed:

```
SELECT city, r.start_time ,revenue,
       RANK() OVER
         (ORDER BY revenue DESC) AS total_revenue_rank,
       RANK() OVER
         (PARTITION BY city ORDER BY revenue DESC) AS city_revenue_rank
FROM rides r
ORDER BY revenue DESC
LIMIT 10;
```

Other SELECT Clauses

The LIMIT clause limits the number of rows returned by a SELECT while the OFFSET clause “jumps ahead” a certain number of rows. This can be handy to paginate through a result set, though it is almost always more efficient to use a filter condition to navigate to the next subset of results—otherwise, each request will need to reread and discard an increasing number of rows.

CockroachDB Arrays

The ARRAY type allows a column to be defined as a one-dimensional array of elements, each of which shares a common data type. We’ll talk about arrays in the context of data modeling in the next chapter. Although they can be useful, strictly speaking they are a violation of the relational model and should be used carefully.

An ARRAY variable is defined by adding [] or the word ARRAY to the data type of a column. For instance:

```
CREATE TABLE arrayTable (arrayColumn STRING[]);
CREATE TABLE anotherTable (integerArray INT ARRAY);
```

The ARRAY function allows us to insert multiple items into the ARRAY:

```
INSERT INTO arrayTable VALUES (ARRAY['sky', 'road', 'car']);
SELECT * FROM arrayTable;
```

```
arraycolumn
-----
{sky,road,car}
```

We can access an individual element of an array with the following familiar array element notation:

```
SELECT arrayColumn[2] FROM arrayTable;
```

```
arraycolumn
-----
road
```

The @> operator can be used to find arrays that contain one or more elements:

```
SELECT * FROM arrayTable WHERE arrayColumn @>ARRAY['road'];
```

```
arraycolumn
-----
{sky,road,car}
```

We can add elements to an existing array using the array_append function and remove elements using array_remove:

```
UPDATE arrayTable
SET arrayColumn=array_append(arrayColumn,'cat')
WHERE arrayColumn @>ARRAY['car']
RETURNING arrayColumn;
```

```
arraycolumn
-----
{sky,road,car,cat}
```

```
UPDATE arrayTable
SET arrayColumn=array_remove(arrayColumn,'car')
WHERE arrayColumn @>ARRAY['car']
RETURNING arrayColumn;
```

```
arraycolumn
-----
{sky,road,cat}
```

Finally, the unnest function transforms an array into a tabular result—one row for each element of the array. This can be used to “join” the contents of an array with data held in relational form elsewhere in the database. We show an example of this in the next chapter:

```
SELECT unnest(arrayColumn)
FROM arrayTable;
```

```

unnest
-----
sky
road
cat

```

Working with JSON

The JSONB data type allows us to store JSON documents into a column, and CockroachDB provides operators and functions to help us work with JSON.

For these examples, we've created a table with a primary key `customerid` and all data in a JSONB column, `jsondata`. We can use the `jsonb_pretty` function to retrieve the JSON in a nicely formatted manner:

```

SELECT jsonb_pretty(jsondata)
FROM customersjson WHERE customerid=1;

      jsonb_pretty
-----
{
  "Address": "1913 Hanoi Way",
  "City": "Sasebo",
  "Country": "Japan",
  "District": "Nagasaki",
  "FirstName": "MARY",
  "LastName": "Smith",
  "Phone": 886780309,
  "_id": "5a0518aa5a4e1c8bf9a53761",
  "dateOfBirth": "1982-02-20T13:00:00.000Z",
  "dob": "1982-02-20T13:00:00.000Z",
  "randValue": 0.47025846594884335,
  "views": [
    {
      "filmId": 611,
      "title": "MUSKETEERS WAIT",
      "viewDate": "2013-03-02T05:26:17.645Z"
    },
    {
      "filmId": 308,
      "title": "FERRIS MOTHER",
      "viewDate": "2015-07-05T20:06:58.891Z"
    },
    {
      "filmId": 159,
      "title": "CLOSER BANG",
      "viewDate": "2012-08-04T19:31:51.698Z"
    },
    /* Some data removed */
  ]
}

```

Each JSON document contains some top-level attributes and a nested array of documents that contains details of films that they have streamed.

We can reference specific JSON attributes in the SELECT clause using the -> operator:

```
SELECT jsondata->'City' AS City
FROM customersjson WHERE customerid=1;

city
-----
"Sasebo"
```

The ->> operator is similar but returns the data formatted as text, not JSON.

If we want to search inside a JSONB column, we can use the @> operator:

```
SELECT COUNT(*) FROM customersjson
WHERE jsondata @> '{"City": "London"}';

count
-----
3
```

We can get the same result using the ->> operator:

```
SELECT COUNT(*) FROM customersjson
WHERE jsondata->>'City' = 'London';

count
-----
3
```

The ->> and @> operators can have different performance characteristics. In particular, ->> might exploit an inverted index where @> would use a table scan.

We can interrogate the structure of the JSON document using the jsonb_each and jsonb_object_keys functions. jsonb_each returns one row per attribute in the JSON document, while jsonb_object_keys returns just the attribute keys. This is useful if you don't know what is stored inside the JSONB column.

jsonb_array_elements returns one row for each element in a JSON array. For instance, here we expand out the views array for a specific customer, counting the number of movies that they have seen:

```
SELECT COUNT(jsonb_array_elements(jsondata->'views'))
FROM customersjson
WHERE customerid =1;

count
-----
      37
(1 row)
```

Summary of SELECT

The SELECT statement is probably the most widely used statement in database programming and offers a wide range of functionality. Even after decades of working in the field, the four of us don't know every nuance of SELECT functionality. However, here we've tried to provide you with the most important aspects of the language. For more depth, view the [CockroachDB documentation set](#).

Although some database professionals use SELECT almost exclusively, the majority will be creating and manipulating data as well. In the following sections, we'll look at the language features that support those activities.

Creating Tables and Indexes

In a relational database, data can be added only to predefined tables. These tables are created by the CREATE TABLE statement. Indexes can be created to enforce unique constraints or to provide a fast access path to the data. Indexes can be defined within the CREATE TABLE statement or by a separate CREATE INDEX statement.

The structure of a database schema forms a critical constraint on database performance and also on the maintainability and utility of the database. We'll discuss the key considerations for database design in [Chapter 5](#). For now, let's create a few simple tables.

We use CREATE TABLE to create a table within a database. [Figure 4-6](#) provides a simplified syntax for the CREATE TABLE statement.

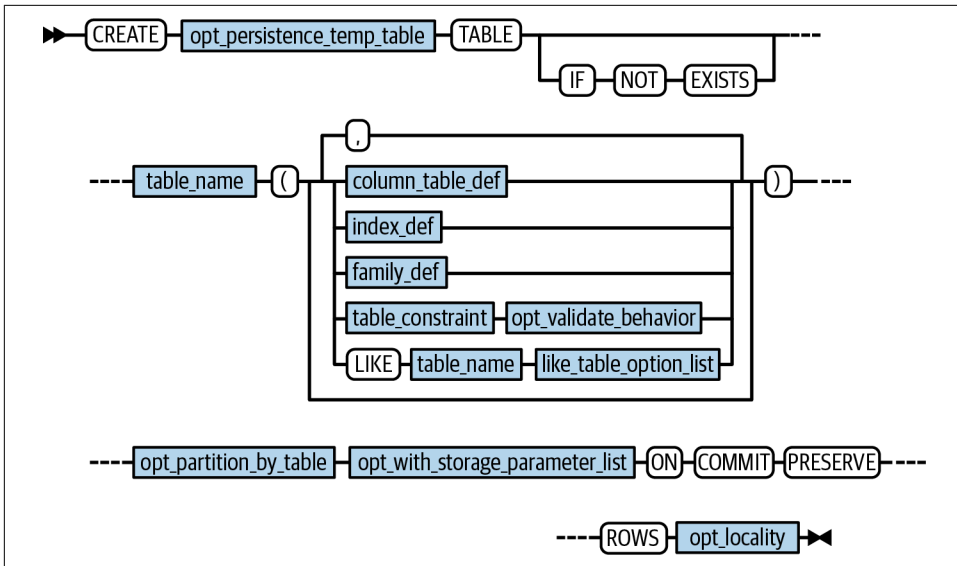


Figure 4-6. CREATE TABLE statement

A simple CREATE TABLE is shown in the next example. It creates a table, mytable, with a single column, mycolumn. The mycolumn column can store only integer values:

```
CREATE TABLE mytable
(
    mycolumn int
);
```

The CREATE TABLE statement must define the columns that occur within the table and can optionally define indexes, column families, constraints, and partitions associated with the table. For instance, the CREATE TABLE statement for the rides table in the movr database would look something like this:

```
CREATE TABLE public.rides (
    id UUID NOT NULL,
    city VARCHAR NOT NULL,
    vehicle_city VARCHAR NULL,
    rider_id UUID NULL,
    vehicle_id UUID NULL,
    start_address VARCHAR NULL,
    end_address VARCHAR NULL,
    start_time TIMESTAMP NULL,
    end_time TIMESTAMP NULL,
```

```

revenue DECIMAL(10,2) NULL,
CONSTRAINT "primary" PRIMARY KEY (city ASC, id ASC),
CONSTRAINT fk_city_ref_users
    FOREIGN KEY (city, rider_id)
    REFERENCES public.users(city, id),
CONSTRAINT fk_vehicle_city_ref_vehicles
    FOREIGN KEY (vehicle_city, vehicle_id)
    REFERENCES public.vehicles(city, id),
INDEX rides_auto_index_fk_city_ref_users
    (city ASC, rider_id ASC),
INDEX rides_auto_index_fk_vehicle_city_ref_vehicles
    (vehicle_city ASC, vehicle_id ASC),
CONSTRAINT check_vehicle_city_city
    CHECK (vehicle_city = city)
);

```

This CREATE TABLE statement specified additional columns, their nullability, primary and foreign keys, indexes, and constraints upon table values.

The relevant clauses in [Figure 4-6](#) are listed in [Table 4-2](#).

Table 4-2. CREATE TABLE options

Option	Description
column_def	The definition of a column. This includes the column name, data type, and nullability. Constraints specific to the column can also be included here, though it's better practice to list all constraints separately.
index_def	Definition of an index to be created on the table. Same as CREATE INDEX but without the leading CREATE verb.
table_constraint	A constraint on the table, such as PRIMARY KEY, FOREIGN KEY, or CHECK. See “Constraints” on page 103 for constraint syntax.
family_def	Assigns columns to a column family. See Chapter 2 for more information about column families.

Let's now look at each of these CREATE TABLE options.

Column Definitions

A column definition consists of a column name, data type, nullability status, default value, and possibly column-level constraint definitions. At a minimum, the name and data type must be specified. [Figure 4-7](#) shows the syntax for a column definition.

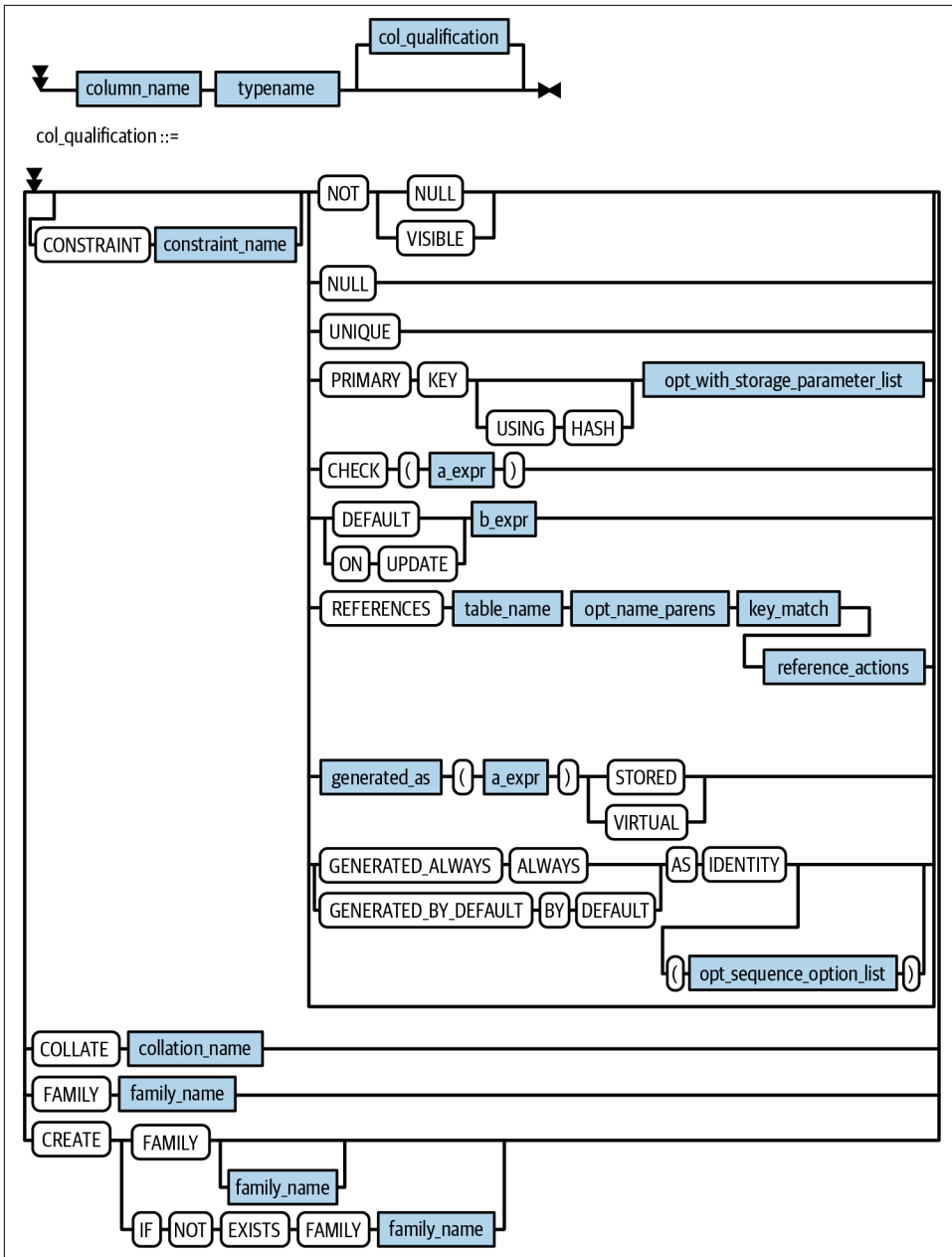


Figure 4-7. Column definition

Although constraints may be specified directly against column definitions, they may also be independently listed below the column definitions. Many practitioners prefer to list the constraints separately in this manner because it allows all constraints, including multicolumn constraints, to be located together.

Computed Columns

CockroachDB allows tables to include *computed columns* that in some other databases would require a view definition:

```
column_name AS expression [STORED|VIRTUAL]
```

A VIRTUAL computed column is evaluated whenever it is referenced. A STORED expression is stored in the database when created and need not always be recomputed.

For instance, this table definition has the `firstName` and `lastName` concatenated into a `fullName` column:

```
CREATE TABLE people
(
    id INT PRIMARY KEY,
    firstName VARCHAR NOT NULL,
    lastName VARCHAR NOT NULL,
    dateOfBirth DATE NOT NULL,
    fullName STRING AS (CONCAT(firstName, ' ', lastName) ) STORED,
    age int AS (now()-dateOfBirth) STORED
);
```

Computed columns cannot be context-dependent. That is, the computed value must not change over time or be otherwise nondeterministic. For instance, the computed column in the following example would not work since the `age` column would be static rather than recalculated every time. While it might be nice to stop aging in real life, we probably want the `age` column to increase as time goes on.

```
CREATE TABLE people
(
    id INT PRIMARY KEY,
    firstName VARCHAR NOT NULL,
    lastName VARCHAR NOT NULL,
    dateOfBirth timestamp NOT NULL,
    fullName STRING AS (CONCAT(firstName, ' ', lastName) ) STORED,
    age int AS (now()-dateOfBirth) STORED
);
```

Data Types

The base CockroachDB data types are shown in Table 4-3.

Table 4-3. CockroachDB data types

Type	Description	Example
ARRAY	A one-dimensional, one-indexed, homogeneous array of any nonarray data type.	<code>{"sky", "road", "car"}</code>
BIT	A string of binary digits (bits).	<code>B'10010101'</code>
BOOL	A Boolean value.	<code>true</code>
BYTES	A string of binary characters.	<code>b'\141\061\142\062\143\063'</code>
COLLATE	The COLLATE feature lets you sort STRING values according to language- and country-specific rules, known as collations.	<code>'a1b2c3' COLLATE en</code>
DATE	A date.	<code>DATE '2016-01-25'</code>
DECIMAL	An exact, fixed-point number.	<code>1.2345</code>
ENUM	New in v20.2: a user-defined data type comprised of a set of static values.	<code>ENUM ('club', 'diamond', 'heart', 'spade')</code>
FLOAT	A 64-bit, inexact, floating-point number.	<code>3.141592653589793</code>
INET	An IPv4 or IPv6 address.	<code>192.168.0.1</code>
INT	A signed integer, up to 64 bits.	<code>12345</code>
INTERVAL	A span of time.	<code>INTERVAL '2h30m30s'</code>
JSONB	JSON data.	<code>'{"first_name": "LoLa", "last_name": "Dog", "location": "NYC", "online": true, "friends": 547}'</code>
SERIAL	A pseudotype that creates unique ascending numbers.	<code>148591304110702593</code>
STRING	A string of Unicode characters.	<code>'a1b2c3'</code>
TIME TIMETZ	TIME stores a time of day in UTC. TIMETZ converts TIME values with a specified time zone offset from UTC.	<code>TIME '01:23:45.123456'</code> <code>TIMETZ '01:23:45.123456-5:00'</code>
TIMESTAMP TIMESTAMPZ	TIMESTAMP stores a date and time pairing in UTC. TIMESTAMPZ converts TIMESTAMP values with a specified time zone offset from UTC.	<code>TIMESTAMP '2016-01-25 10:10:10'</code> <code>TIMESTAMPZ '2016-01-25 10:10:10-05:00'</code>
TSQUERY	A list of lexemes to be used in a full-text search.	<code>'lazi' & 'dog'</code>
TSVECTOR	A list of lexemes with optional position information.	<code>'dog':2 'lazi':1</code>
UUID	A 128-bit hexadecimal value.	<code>7f9c24e8-3b12-4fef-91e0-56a2d5a246ec</code>
VECTOR	A fixed-length array of floating-point numbers representing points in an n-dimensional space.	<code>[0.1259227, 0.0318873, 0.4660917]</code>

Note that other data type names may be aliased against these CockroachDB base types. For instance, the PostgreSQL types `BIGINT` and `SMALLINT` are aliased against the CockroachDB type `INT`.

In CockroachDB, data types may be cast—or converted—by appending the data type to an expression using “::”. For instance:

```
SELECT revenue::int FROM rides;
```

The `CAST` function can also be used to convert data types and is more broadly compatible with other databases and SQL standards. For instance:

```
SELECT CAST(revenue AS int) FROM rides;
```

Primary Keys

As we know, a primary key uniquely defines a row within a table. In CockroachDB, a primary key is mandatory because all tables are distributed across the cluster based on the ranges of their primary key. If you don’t specify a primary key, a key will be automatically generated for you.

It’s common practice in other databases to define an autogenerating primary key using clauses such as `AUTOINCREMENT`. The generation of primary keys in distributed databases is a significant issue because it’s the primary key that is used to distribute data across nodes in the cluster. We’ll discuss the options for primary key generation in the next chapter, but for now, we’ll simply note that you can generate randomized primary KVs using the `UUID` data type with the `gen_random_uuid()` function as the default value:

```
CREATE TABLE people (  
    id UUID NOT NULL DEFAULT gen_random_uuid(),  
    firstName VARCHAR NOT NULL,  
    lastName VARCHAR NOT NULL,  
    dateOfBirth DATE NOT NULL  
);
```

This pattern is considered best practice to ensure even distribution of keys across the cluster. Other options for autogenerating primary keys will be discussed in [Chapter 5](#).

Constraints

The `CONSTRAINT` clause specifies conditions that must be satisfied by all rows within a table. In some circumstances, the `CONSTRAINT` keyword may be omitted, for instance, when defining a column constraint or specific constraint types such as `PRIMARY KEY` or `FOREIGN KEY`. [Figure 4-8](#) shows the general form of a constraint definition.

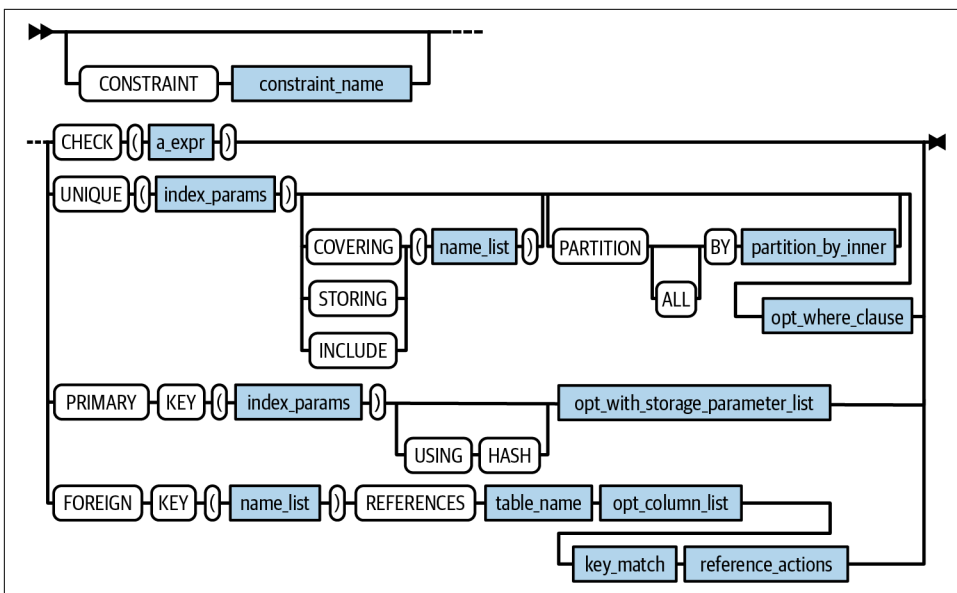


Figure 4-8. *CONSTRAINT* statement

A **UNIQUE** constraint requires that all values for the `column` or `column_list` be unique.

PRIMARY KEY implements a set of columns that must be unique and which can also be the subject of a **FOREIGN KEY** constraint in another table. Both **PRIMARY KEY** and **UNIQUE** constraints require the creation of an implicit index. If desired, the physical storage characteristics of the index can be specified in the **USING** clause. The options of the **USING INDEX** clause have the same usages as in the **CREATE INDEX** statement.

NOT NULL indicates that the column in question may not be **NULL**. This option is available for only column constraints, but the same effect can be obtained with a table **CHECK** constraint.

CHECK defines an expression that must evaluate to **true** for every row in the table. We'll discuss best practices for creating constraints in [Chapter 5](#).

Sensible use of constraints can help ensure data quality and can provide the database with a certain degree of self-documentation. However, some constraints have significant performance implications; we'll discuss these implications in [Chapter 5](#).

Indexes

Indexes can be created by the **CREATE INDEX** statement, or an **INDEX** definition can be included within the **CREATE TABLE** statement.

We talked a lot about indexes in [Chapter 2](#), and we’ll keep discussing indexes in the schema design and performance tuning chapters (Chapters [5](#) and [8](#), respectively). Effective indexing is one of the most important success factors for a performance CockroachDB implementation.

[Figure 4-9](#) illustrates a simplistic syntax for the CockroachDB `CREATE INDEX` statement.

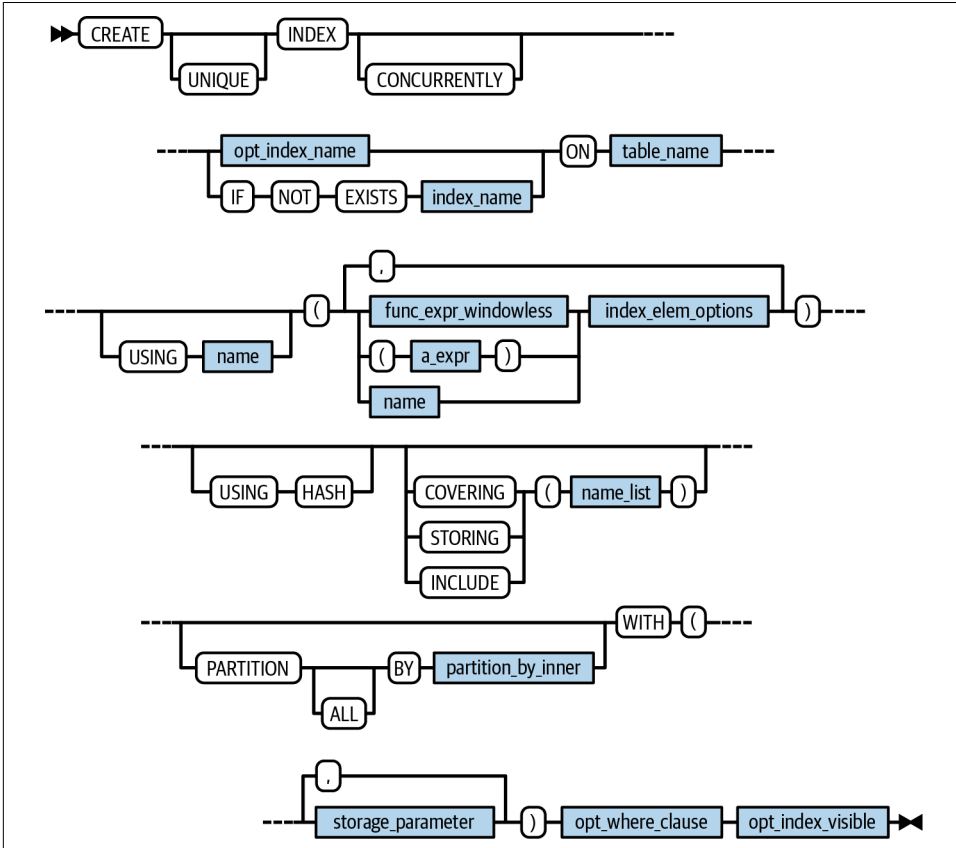


Figure 4-9. CREATE INDEX statement

We looked at the internals of CockroachDB indexes in [Chapter 2](#). From a performance point of view, CockroachDB indexes behave much as indexes in other databases—providing a fast access method for locating rows with a particular set of nonprimary KVs. For instance, if we simply want to locate a row with a specific name and date of birth, we might create the following multicolumn index:

```
CREATE INDEX people_namedob_ix ON people
(lastName,firstName,dateOfBirth);
```

If we wanted to further ensure that no two rows could have the same value for name and date of birth, we might create a unique index:

```
CREATE UNIQUE INDEX people_namedob_idx ON people
(lastName, firstName, dateOfBirth);
```

The `STORING` clause allows us to store additional data in the index, which can allow us to satisfy queries using the index alone. For instance, this index can satisfy queries that retrieve phone numbers for a given name and date of birth:

```
CREATE UNIQUE INDEX people_namedob_idx ON people
(lastName, firstName, dateOfBirth) STORING (phoneNumber);
```

Inverted indexes

An inverted index can be used to index the elements within an array or the attributes within a JSON document. We looked at the internals of inverted indexes in [Chapter 2](#). Inverted indexes can also be used for spatial data.

For example, suppose our `people` table used a JSON document to store the attributes for a person:

```
CREATE TABLE people
( id UUID NOT NULL DEFAULT gen_random_uuid(),
  personData JSONB );

INSERT INTO people (personData)
VALUES ('{
  "firstName": "Guy",
  "lastName": "Harrison",
  "dob": "21-Jun-1960",
  "phone": "0419533988",
  "photo": "eyJhbGciOiJIUzI1NiIsI..."}');
```

We might create an inverted index as follows:

```
CREATE INVERTED INDEX people_inv_idx ON
people(personData);
```

Which would support queries into the JSON document such as this:

```
SELECT *
FROM people
WHERE personData @> '{"phone": "0419533988"}';
```

Bear in mind that inverted indexes index every attribute in the JSON document, not just those that you want to search on. This potentially results in a very large index. Therefore, you might find it more useful to create a calculated column on the JSON attribute and then index on that computed column:

```
ALTER TABLE people ADD phone STRING AS (personData->>'phone') VIRTUAL;  
  
CREATE INDEX people_phone_idx ON people(phone);
```

Hash-sharded indexes

If you're working with a table that must be indexed on sequential keys, you should use hash-sharded indexes. Hash-sharded indexes distribute sequential traffic uniformly across ranges, eliminating single-range hotspots and improving write performance on sequentially keyed indexes at a small cost to read performance:

```
CREATE TABLE people  
( id INT PRIMARY KEY,  
  firstName VARCHAR NOT NULL,  
  lastName VARCHAR NOT NULL,  
  dateOfBirth timestamp NOT NULL,  
  phoneNumber VARCHAR NOT NULL,  
  serialNo SERIAL ,  
  INDEX serialNo_idx (serialNo) USING HASH WITH BUCKET_COUNT=4);
```

We'll discuss hash-sharded indexes—as well as other more advanced indexing topics—in more detail in the next section.

Vectors

CockroachDB's VECTOR data type is compatible with the popular **pgvector extension** for PostgreSQL and it allows you to embed CockroachDB as part of your large language model (LLM) or machine learning (ML) infrastructure.

Vectors enable efficient semantic similarity searches by representing features in an n-dimensional space, where proximity indicates conceptual relatedness. Let's look at some concrete examples to disambiguate things.

First, we'll create a table containing a VECTOR column. Let's assume we're an online retailer and want to provide product recommendations based on customer purchases:

```
CREATE TABLE customer (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email STRING NOT NULL UNIQUE,  
  vector VECTOR(7)  
);
```

The vector's dimensionality is defined by the number that's used when creating it. In **Table 4-4**, we've configured this vector to store seven dimensions. At first, this may seem arbitrary, but there's a reason for this number.

To represent a customer in a numerical format that can be used for distance searching, we need to store information about them that will be useful for distance searching in the vector.

Table 4-4. The seven features of our vector

Property	Example	Encoding type
Age	0.3780123	Min/max (a value between 0 and 1 representing where a customer's age falls between a minimum and maximum age range)
Gender	0, 0, 0, 1	One-hot (there is no ordinal relationship between genders, so one-hot encoding allows for genders to be encoded as unique floating-point arrays)
Location	0.0188633, 0.1944628	Min/max (similar to age but with a value for both latitude and longitude)

Let's insert some data into the table. We'll insert three customers, with vector values that might characterize the *a@example.com* customer as being, for example, a 35-year-old male living close to London:

```
INSERT INTO customer (email, vector) VALUES
  ('a@example.com', '[0.1259227, 0, 0, 0, 1, 0.0318873, 0.4660917]'),
  ('b@example.com', '[0.2331882, 0, 0, 0, 1, 0.0345919, 0.3901348]'),
  ('c@example.com', '[0.8177326, 0, 0, 1, 0, 0.7825610, 0.6627843]');
```

Now we'll run a distance query to return customers that are closely related to the *a@example.com* customer:

```
SELECT
  email,
  vector <-> '[0.1259227, 0, 0, 0, 1, 0.0318873, 0.4660917]' AS distance
FROM customer
ORDER BY distance;

a@example.com | 0
b@example.com | 0.13146349255472864
c@example.com | 1.7552207999159306
```

The lower the score, the more closely a customer's vector column matches the vector being tested. Hence, customer *a@example.com* (whose vector we're using for the comparison) scores 0; the closest possible distance. The *b@example.com* customer also has a low distance score, meaning their vector features are close to those being tested; perhaps the products they like might also be relevant to *a@example.com*?

Having support for vectors also means that CockroachDB can be used as a retrieval-augmented generation (RAG) backend. This allows your existing CockroachDB datasets to augment the output of LLMs, training the model and extracting more value from your data. See [CockroachDB for AI/ML: LLMs and RAG](#) for a demo.

CREATE TABLE AS SELECT

The `AS SELECT` clause of `CREATE TABLE` allows us to create a new table that has the data and attributes of a SQL `SELECT` statement. Columns, constraints, and indexes can be specified for an existing table but must align with the data types and number of columns returned by the `SELECT` statement. For instance, here we create a table based on a `JOIN` and aggregate of two tables in the `movr` database:

```
CREATE TABLE user_ride_counts AS
SELECT u.name, COUNT(u.name) AS rides
  FROM "users" AS u JOIN "rides" AS r
    ON (u.id=r.rider_id)
 GROUP BY u.name;
```

Note that while `CREATE TABLE AS SELECT` can be used to create summary tables and the like, `CREATE MATERIALIZED VIEW` offers a more functional alternative.

Altering Tables

The `ALTER TABLE` statement allows table columns or constraints to be added, modified, renamed, or removed, as well as allowing for constraint validation and partitioning. [Figure 4-10](#) shows the syntax.

Altering table structures online is not something to be undertaken lightly, although [CockroachDB provides highly advanced mechanisms for propagating such changes](#) without impacting availability and with minimal impact on performance. We'll discuss the procedures for online schema changes in later chapters.

The synopsis for the `ALTER TABLE` statement is large, so [Figure 4-10](#) shows a condensed representation that lists the available commands. For the complete synopsis, see the [ALTER TABLE documentation](#).

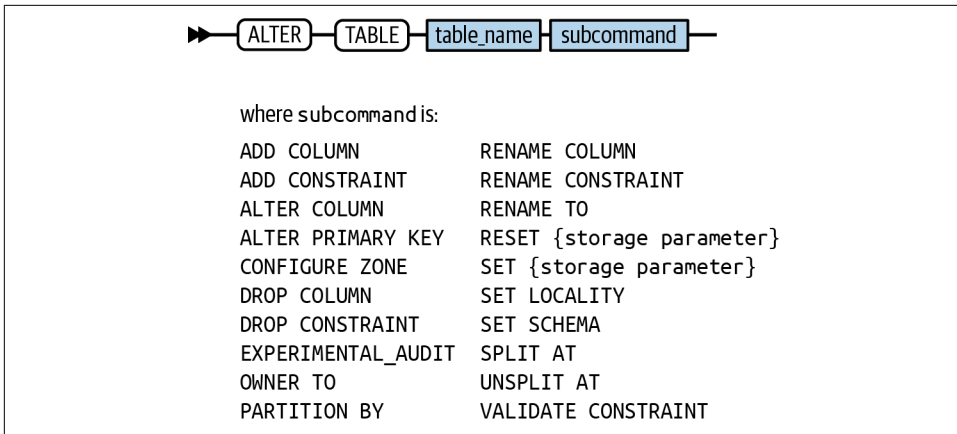


Figure 4-10. *ALTER TABLE* statement

Dropping Tables

Tables can be dropped using the `DROP TABLE` statement. Figure 4-11 shows the syntax.

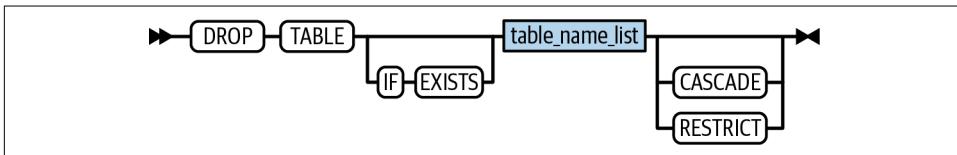


Figure 4-11. *DROP TABLE* statement

More than one table can be removed with a single `DROP TABLE` statement. The `CASCADE` keyword causes dependent objects such as views or foreign key constraints to be dropped as well. `RESTRICT`—the default—has the opposite effect; if there are any dependent objects, then the table will not be dropped.

DROP CASCADE and Foreign Keys

`DROP TABLE...CASCADE` will drop any foreign key constraints that reference the table but will not drop the tables or rows that contain those foreign keys. The end result will be “dangling” references in these tables.

Because of this incompleteness, and because it can be hard to be certain exactly what `CASCADE` will do, it’s usually better to manually remove all dependencies on a table before dropping it.

Views

A standard view is a query definition stored in the database that defines a virtual table. This virtual table can be referenced the same way as a regular table. Common Table Expressions can be thought of as a way of creating a temporary view for a single SQL statement. If you had a Common Table Expression that you wanted to share among SQL statements, then a view would be a logical solution.

A materialized view stores the results of the view definition in the database so that the view need not be reexecuted whenever encountered. This improves performance but may result in stale results. If you think of a view as a stored query, then a materialized view can be thought of as a stored result.

Figure 4-12 shows the syntax of the `CREATE VIEW` statement.

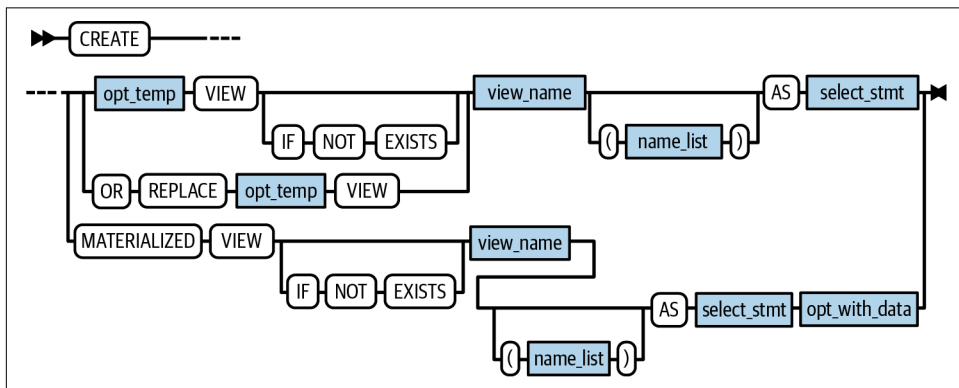


Figure 4-12. `CREATE VIEW` statement

The `REFRESH MATERIALIZED VIEW` statement can be used to refresh the data underlying a materialized view.

Functions

User-defined functions (UDFs) are named-functions within a database that can be invoked from `SELECT`, `FROM`, and `WHERE` clauses. They take zero or more arguments, return a value, and can be expressed in either SQL or PL/pgSQL.

Figure 4-13 shows the syntax of the `CREATE FUNCTION` statement.

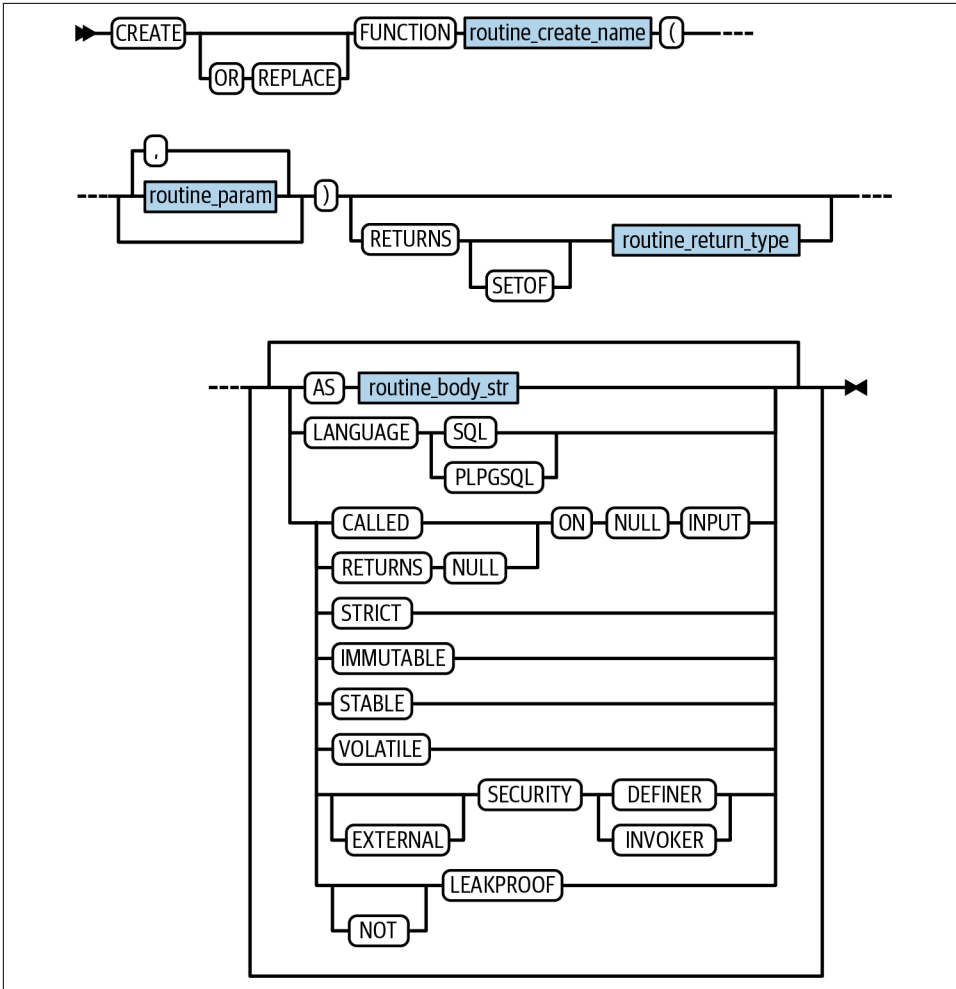


Figure 4-13. *CREATE FUNCTION* statement

In the following example, a function is created that takes zero arguments and returns an integer. Note that because the function does not mutate data and is guaranteed to return the same response on every invocation, we can decorate it with the `IMMUTABLE` and `LEAKPROOF` annotations:

```
CREATE FUNCTION add(a INT, b INT) RETURNS INT IMMUTABLE LEAKPROOF LANGUAGE SQL
AS 'SELECT $1 + $2';
```

The function is called with a `SELECT` statement as follows:

```
SELECT easy_as();
-- Returns 123
```

```
SELECT easy_as() * 2;
-- Returns 246
```

A function's volatility indicates whether you should expect side effects when calling it. The previous function simply returned an integer with zero side effects, allowing the cost-based optimizer to inline the function and improve its performance.

The following function is a little more involved and makes use of arguments and the PL/pgSQL language. Notice that the IMMUTABLE and LEAKPROOF annotations are no longer present, making this function volatile. Given the same arguments, this function may return different data between calls (e.g., products are added, edited, or removed):

```
CREATE OR REPLACE FUNCTION products_in_range(lo DECIMAL, hi DECIMAL)
RETURNS JSONB AS $$
DECLARE
    result JSONB;
BEGIN
    SELECT jsonb_agg(jsonb_build_object(
        'name', name,
        'price', price
    ))
    INTO result
    FROM product
    WHERE price BETWEEN lo AND hi;

    RETURN COALESCE(result, '[]'::JSONB);
END;
$$ LANGUAGE plpgsql;
```

Let's create and populate a table to show how this function works:

```
CREATE TABLE product (
    "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    "name" STRING NOT NULL,
    "price" DECIMAL NOT NULL
);

INSERT INTO product ("name", "price") VALUES
    ('a', 1.99),
    ('b', 2.99),
    ('c', 4.99),
    ('d', 8.99);
```

Calling the function with different values for the low and high price arguments yields the following results:

```
SELECT products_in_range(2, 5);
-- Returns [{"name": "b", "price": 2.99}, {"name": "c", "price": 4.99}]

SELECT products_in_range(10, 100);
-- Returns []
```

Procedures

Stored procedures are similar to functions in that they can accept zero or more arguments but they don't return a value, don't accept volatility annotations, and are invoked using `CALL` instead of `SELECT`. They're useful when you wish to implement complex logic within the database itself, rather than implementing that logic within your application.

Figure 4-14 shows the syntax of the `CREATE PROCEDURE` statement.

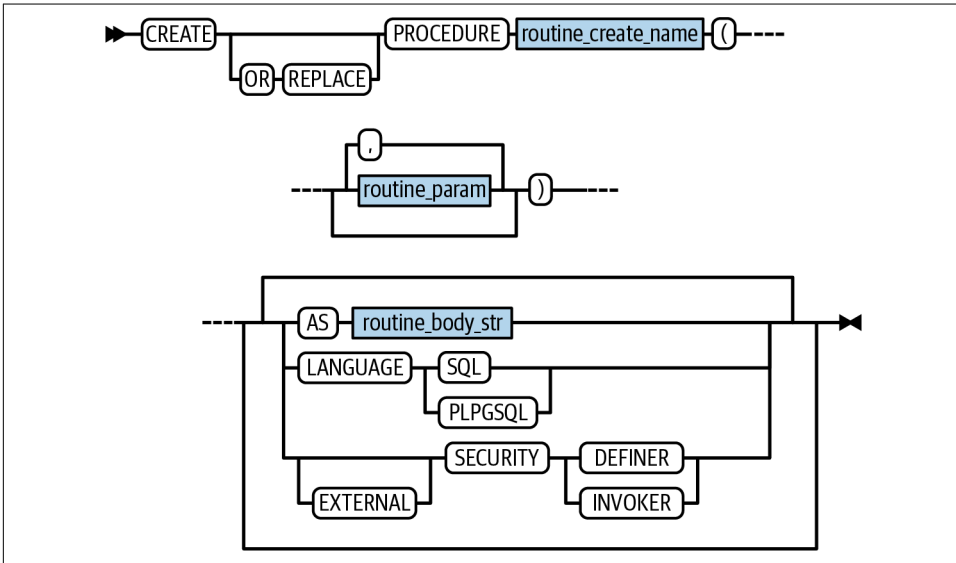


Figure 4-14. `CREATE PROCEDURE` statement

In the following example, a procedure is created that updates the prices in the product table by applying the amount specified in a new “tax” column. It will update the table in batches until all of the product prices have been updated, at which point the procedure completes.

First, we'll update the product table to add the new “tax” column:

```
ALTER TABLE product ADD tax DECIMAL NOT NULL DEFAULT 20;
```

Next, we'll create the procedure itself (you wouldn't want to update product prices like this in the real world; this procedure is just for demonstration purposes):

```
CREATE OR REPLACE PROCEDURE apply_tax(new_tax DECIMAL, batch_size INT DEFAULT 1)
LANGUAGE plpgsql AS $$
DECLARE
    updated_ids UUID;
```

```

BEGIN
  LOOP
    UPDATE product
    SET
      price = ROUND(price / (1 + tax / 100) * (1 + new_tax / 100), 2),
      tax = new_tax
    WHERE tax != new_tax
    LIMIT batch_size
    RETURNING id INTO updated_ids;

    EXIT WHEN updated_ids IS NULL;
  END LOOP;
END;
$$;

```

Finally, we'll call the procedure twice, once to increase the tax amount to 30%, then again to reduce it to 20%:

```

CALL apply_tax(30, 1000);
SELECT "name", "price", "tax" FROM product ORDER BY name;

```

name	price	tax
a	2.16	30
b	3.24	30
c	5.41	30
d	9.74	30

```

CALL apply_tax(20, 1000);
SELECT "name", "price", "tax" FROM product ORDER BY name;

```

name	price	tax
a	1.99	20
b	2.99	20
c	4.99	20
d	8.99	20

Inserting Data

We can load data into a new table using the INSERT statement inside a program or from the command-line shell, with the CREATE TABLE AS SELECT statement discussed earlier, or by loading external data using the IMPORT statement. There are also non-SQL utilities that insert data—we'll look at these in [Chapter 7](#).

The venerable INSERT statement adds data to an existing table. [Figure 4-15](#) illustrates a simplified syntax for the INSERT statement.

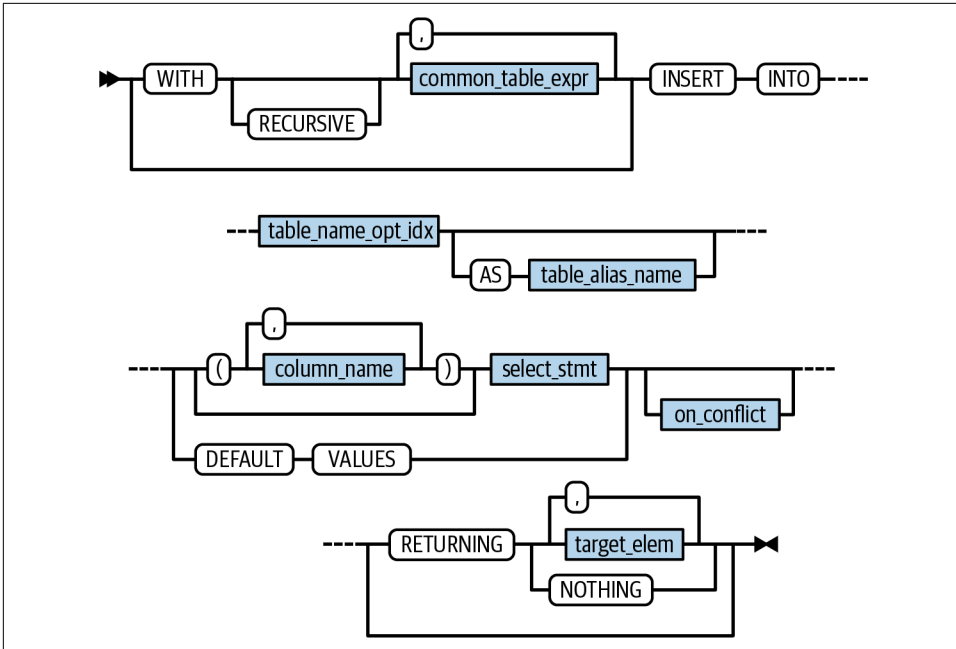


Figure 4-15. INSERT statement

INSERT takes either a set of values or a SELECT statement. For instance, in the following example, we insert a single row into the people table:

```
INSERT INTO people (firstName, lastName, dateOfBirth)
VALUES ('Guy', 'Harrison', '21-JUN-1960');
```

The VALUES clause of the INSERT statement can accept multiple values, resulting in multiple rows being inserted in a single execution:

```
INSERT INTO people (firstName, lastName, dateOfBirth)
VALUES ('Guy', 'Harrison', '21-JUN-1960'),
('Michael', 'Harrison', '19-APR-1994'),
('Oriana', 'Harrison', '18-JUN-2020');
```

There are alternative ways to insert batches in the various program language drivers, and we'll show some examples in [Chapter 7](#).

A SELECT statement can be specified as the source of the inserted data:

```
INSERT INTO people (firstName, lastName, dateOfBirth)
SELECT firstName, lastName, dateOfBirth
FROM peopleStagingData;
```


The RETURNING clause allows the data inserted to be returned to the user. The data returned will include not just the variables that were inserted but any autogenerated data. For instance, in this case, we INSERT data without specifying an ID value and have the ID values that were created returned to us:

```
INSERT INTO people (firstName, lastName, dateOfBirth)
VALUES ('Guy', 'Harrison', '21-JUN-1960'),
      ('Michael', 'Harrison', '19-APR-1994'),
      ('Oriana', 'Harrison', '18-JUN-2020')
RETURNING id;
```

The ON CONFLICT clause allows you to control what happens if an INSERT violates a uniqueness constraint. Figure 4-16 shows the syntax.

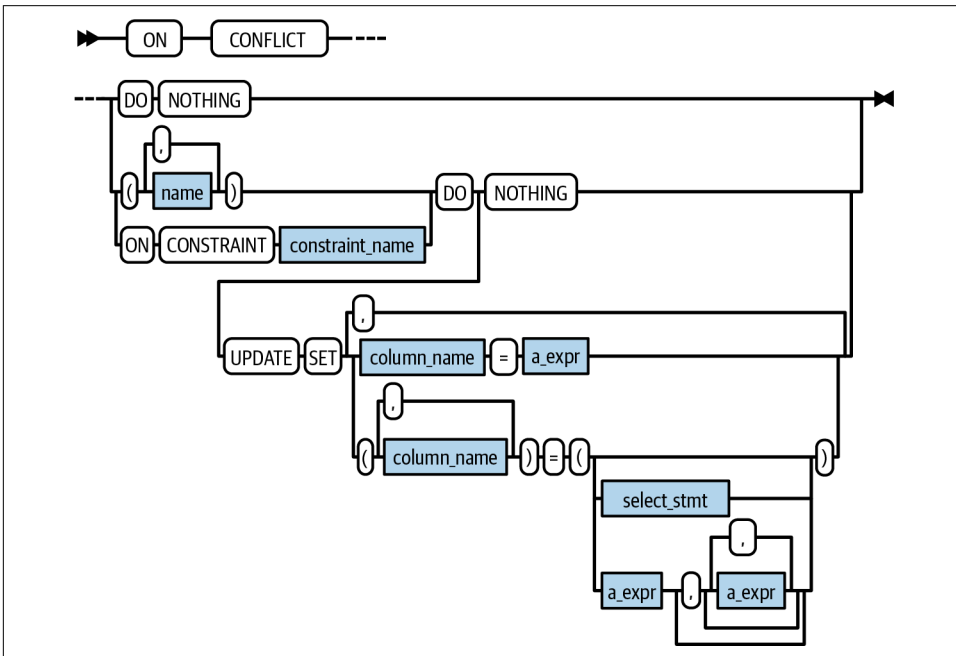


Figure 4-16. ON CONFLICT clause

Without an ON CONFLICT clause, a uniqueness constraint violation will cause the entire INSERT statement to abort. DO NOTHING allows the INSERT statement as a whole to succeed but ignores any inserts that violate the uniqueness clause. The DO UPDATE clause allows you to specify an UPDATE statement that executes instead of the INSERT. The DO UPDATE functionality is similar in functionality to the UPSERT statement discussed later in this chapter.

UPDATE

The UPDATE statement changes existing data in a table. [Figure 4-17](#) shows a simplified syntax for the UPDATE statement.

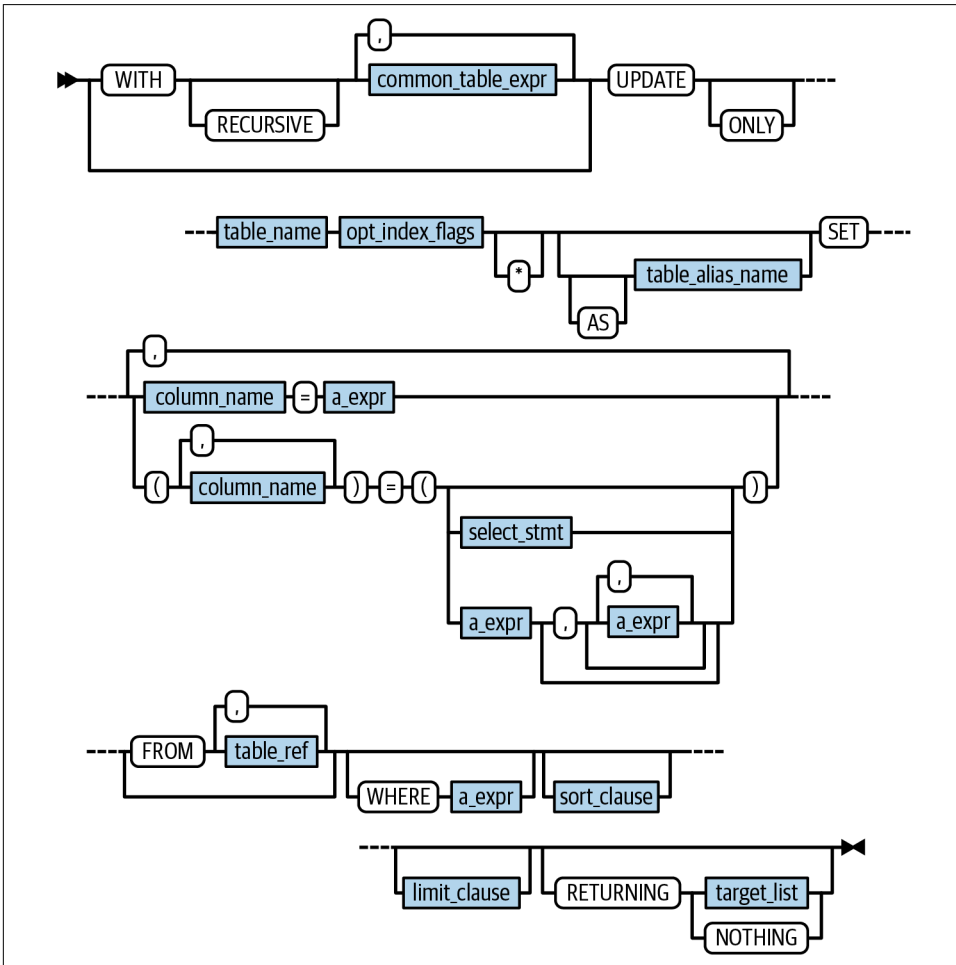


Figure 4-17. UPDATE statement

An UPDATE statement can specify static values as in the following example:

```
UPDATE users
SET address = '201 E Randolph St',
    city='amsterdam'
WHERE name='Maria Weber';
```

Alternatively, the values may be an expression referencing existing values:

```
UPDATE user_promo_codes
SET usage_count=usage_count+1
WHERE user_id='297fcb80-b67a-4c8b-bf9f-72c404f97fe8';
```

Or the UPDATE can use a subquery to obtain the values:

```
UPDATE rides SET (revenue, start_address) =
    (SELECT revenue, end_address FROM rides
     WHERE id = '94fdf3b6-45a1-4800-8000-000000000123')
WHERE id = '851eb851-eb85-4000-8000-000000000104';
```

The RETURNING clause can be used to view the modified columns. This is particularly useful if a column is being updated by a function and we want to return the modified value to the application:

```
UPDATE user_promo_codes
SET usage_count=usage_count+1
WHERE user_id='297fcb80-b67a-4c8b-bf9f-72c404f97fe8'
RETURNING (usage_count);
```

UPSERT

UPSERT can insert new data and update existing data in a table in a single operation. If the input data does not violate any uniqueness constraints, it is inserted. If an input matches an existing primary key, then the values of that row are updated.

In CockroachDB, the ON CONFLICT clause of INSERT provides a similar—though more flexible—mechanism. When this flexibility is not needed, UPSERT is likely to be faster than a similar INSERT...ON CONFLICT DO UPDATE statement.

Figure 4-18 shows the syntax of the UPSERT statement.

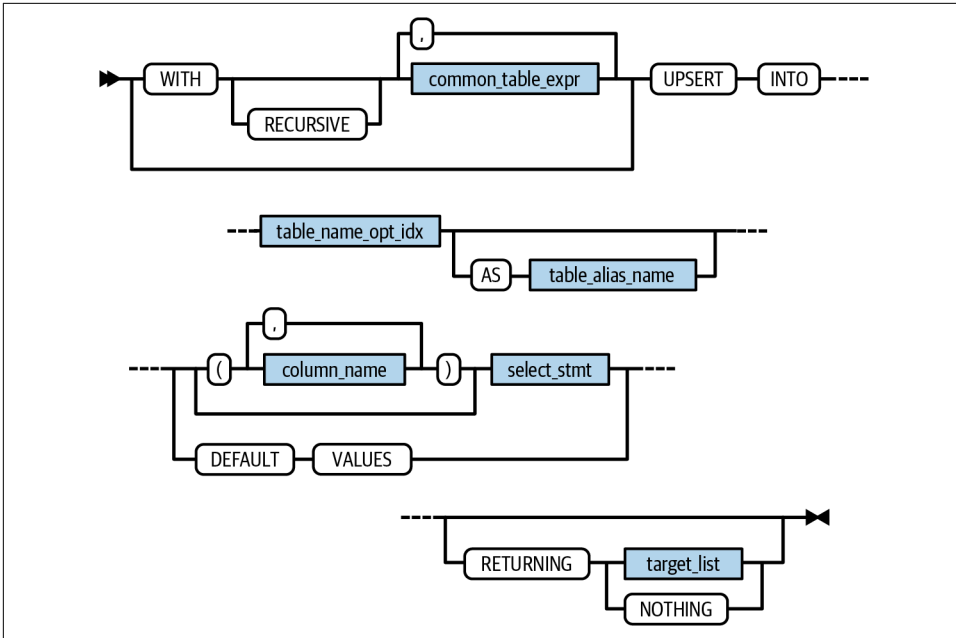


Figure 4-18. UPSERT statement

The UPSERT compares the primary KV of each row provided. If the primary key is not found in the existing table, then a new row is created. Otherwise, the existing row is updated with the new values provided.

The RETURNING clause can be used to return a list of updated or inserted rows.

In this example, the primary key of `user_promo_codes` is (`city`, `user_id`, `code`). If a user already has an entry for that combination in the table, then that row is updated with a `user_count` of 0. Otherwise, a new row with those values is created:

```
UPSERT INTO user_promo_codes
  (user_id,city,code,timestamp,usage_count)
SELECT id,city, 'NewPromo',now(),0
FROM "users";
```

DELETE

DELETE allows data to be removed from a table. Figure 4-19 shows a simplified syntax for the DELETE statement.

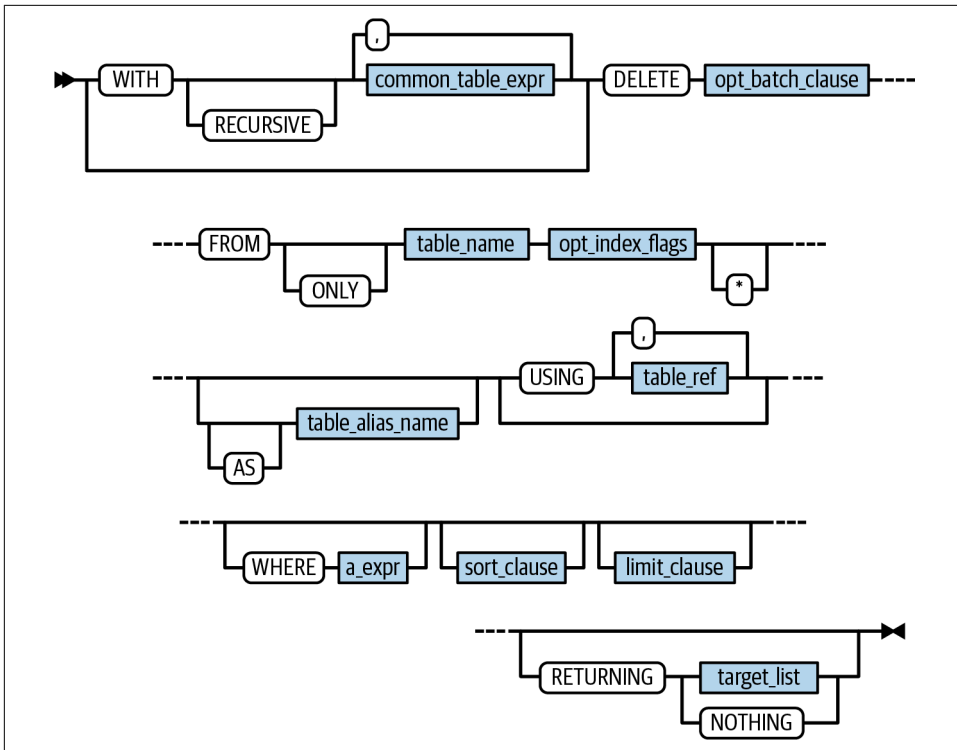


Figure 4-19. DELETE statement

Most of the time, a DELETE statement accepts a WHERE clause and not much else. For instance, here we delete a single row in the people table:

```
DELETE FROM people
WHERE firstName='Guy'
AND lastName='Harrison';
```

The RETURNING clause can return details of the rows that were removed. For instance:

```
DELETE FROM user_promo_codes
WHERE code='NewPromo'
RETURNING(user_id);
```

You can also include an `ORDER BY` and `LIMIT` clause to perform batch deletes in an incremental fashion. For instance, you can construct a `DELETE` statement to remove the oldest 1,000 rows. See the [CockroachDB documentation](#) for more information.

TRUNCATE

TRUNCATE provides a quick mechanism for removing all rows from a table. Internally, it is implemented as a DROP TABLE followed by a CREATE TABLE. TRUNCATE is not transactional—you cannot ROLLBACK a TRUNCATE.

IMPORT INTO

The IMPORT INTO statement imports the following types of data into an existing CockroachDB table:

- Avro
- Comma-separated values (CSV)/tab-separated values (TSV)

The files to be imported should exist either in a cloud storage bucket (Google Cloud Storage, Amazon S3, or Azure Blob storage), at an HTTP address, or on the local filesystem (“nodelocal”).

We’ll discuss the various options for loading data into CockroachDB in [Chapter 7](#). However, for now, let’s create a new table customers from a CSV file:

```
IMPORT INTO TABLE customers (  
    id INT PRIMARY KEY,  
    name STRING,  
    INDEX name_idx (name)  
);  
CSV DATA ('nodelocal://1/customers.csv');
```

job_id	status	fra	rows	index_entries	bytes
659162639684534273	succeeded	1	1	1	47

(1 row)

Time: 934ms total (execution 933ms / network 1ms)

For a single-node demo cluster, the nodelocal location will be somewhat dependent on your installation but will often be in an extern directory beneath the CockroachDB installation directory.

Transactional Statements

We talked a lot about CockroachDB transactions in [Chapter 2](#), so review that chapter if you need a refresher on how CockroachDB transactions work. From the SQL language point of view, CockroachDB supports the standard SQL transactional control statements.

BEGIN Transaction

The `BEGIN` statement commences a transaction and sets its properties. [Figure 4-20](#) shows the syntax.

PRIORITY sets the transaction priority. In the event of a conflict, HIGH priority transactions are less likely to be retried.

READ ONLY specifies that the transaction is read-only and will not modify data.

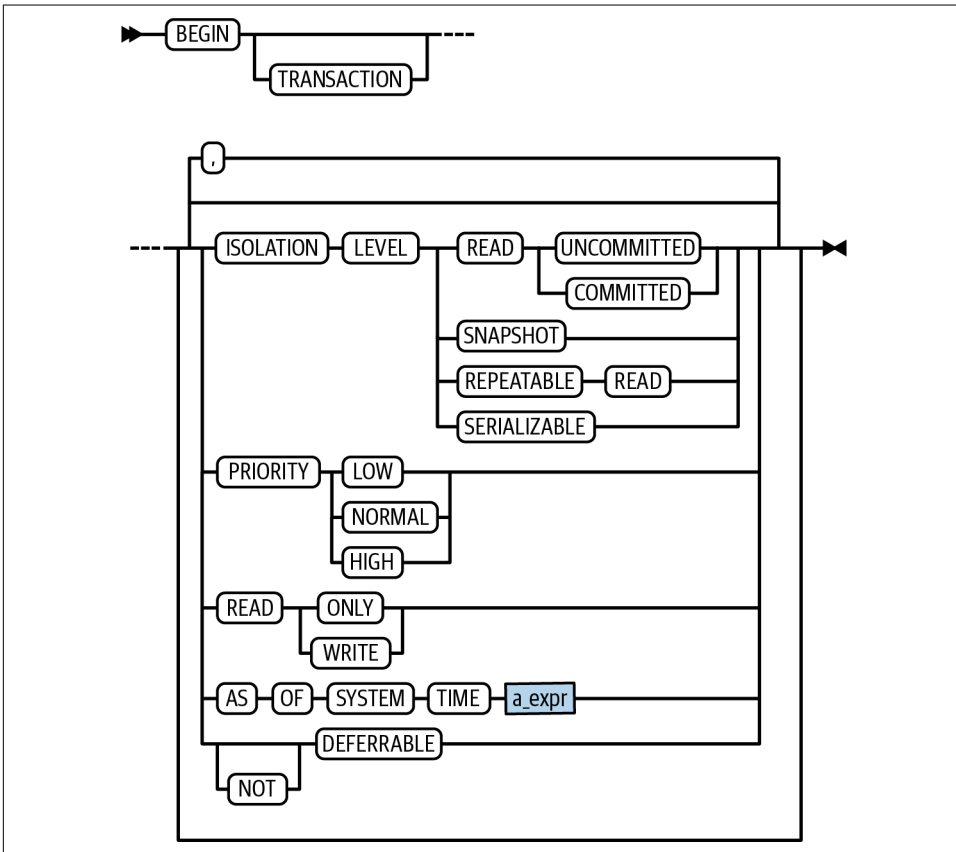


Figure 4-20. *BEGIN* transaction

AS OF SYSTEM TIME allows a READ ONLY transaction to view data from a snapshot of database history. We'll come back to this in a few pages.

SAVEPOINT

SAVEPOINT creates a named rollback point that can be used as the target of a ROLLBACK statement. This allows a portion of a transaction to be discarded without discarding all of the transaction's work. See the ROLLBACK section for more details.

COMMIT

The COMMIT statement commits the current transactions, making changes permanent.

Note that some transactions may require client-side intervention to handle retry scenarios. These patterns will be explored in [Chapter 6](#).

ROLLBACK

ROLLBACK aborts the current transaction. Optionally, we can ROLLBACK to a savepoint, which rolls back only the statements issued after the SAVEPOINT.

For instance, in the following example, the insert of the misspelled number *tree* is rolled back and corrected without abandoning the transaction as a whole:

```
BEGIN ;

INSERT INTO numbers VALUES(1, 'one');
INSERT INTO numbers VALUES(2, 'two');
SAVEPOINT two;

INSERT INTO numbers VALUES(3, 'tree');
ROLLBACK TO SAVEPOINT two;

INSERT INTO numbers VALUES(3, 'three');
COMMIT;
```

SELECT FOR UPDATE

The FOR UPDATE clause of a SELECT statement locks the rows returned by a query, ensuring that they cannot be modified by another transaction between the time they are read and when the transaction ends. This is typically used to implement the pessimistic locking pattern that we'll discuss in [Chapter 6](#).

A FOR UPDATE query should be executed within a transaction. Otherwise, the locks are released on completion of the SELECT statement.

A FOR UPDATE issued within a transaction will, by default, block other FOR UPDATE statements on the same rows or other transactions that seek to update those rows from completing until a COMMIT or ROLLBACK is issued. However, if a higher-priority transaction attempts to update the rows or attempts to issue a FOR UPDATE, then the lower-priority transaction will be aborted and will need to retry.

We'll discuss the mechanics of transaction retries in [Chapter 6](#).

Figure 4-21 illustrates two `FOR UPDATE` statements executing concurrently. The first `FOR UPDATE` holds locks on the affected rows, preventing the second session from obtaining those locks until the first session completes its transaction.

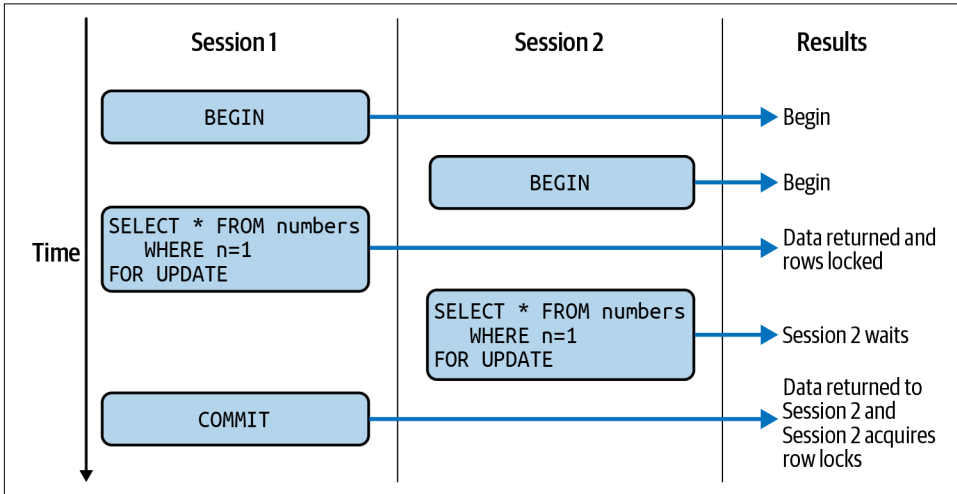


Figure 4-21. FOR UPDATE clause behavior

SELECT FOR SHARE

The `FOR SHARE` clause of a `SELECT` statement obtains a *shared* lock on the rows returned by a query, preventing writes and other exclusive locking reads from obtaining a lock until the transaction ends. If a row is locked by multiple shared locks, none of the lock holders will be able to write to the row.

A `FOR SHARE` query should be used when running in `READ COMMITTED` isolation, where you need up-to-date data but don't plan on making updates to that data after reading it.

A `FOR SHARE` issued within a transaction will not block other `FOR SHARE` statements on the same rows but will block other transactions that seek to update those rows from completing until a `COMMIT` or `ROLLBACK` is issued. However, if a higher-priority transaction attempts to update the rows, then lower-priority transactions will be aborted and will need to retry.

We'll discuss the mechanics of transaction retries in [Chapter 6](#).

Figure 4-22 illustrates two `FOR SHARE` statements executing concurrently. The first `FOR SHARE` creates a shared lock on the affected row, allowing the second session to

obtain a shared lock but preventing it from obtaining an exclusive lock until the first session completes its transaction.

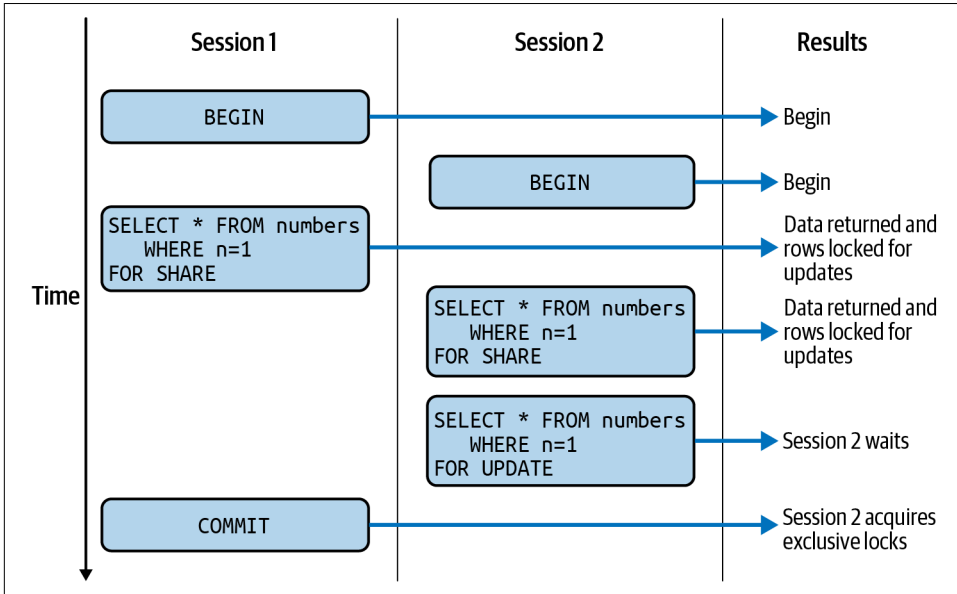


Figure 4-22. `FOR SHARE` clause behavior

AS OF SYSTEM TIME

The `AS OF SYSTEM TIME` clause can be applied to `SELECT` and `BEGIN TRANSACTION` statements as well as in `BACKUP` and `RESTORE` operations. `AS OF SYSTEM TIME` specifies that a `SELECT` statement or all the statements in a `READ ONLY` transaction should execute on a snapshot of the database at that system time. These snapshots are made available by the MVCC architecture described in [Chapter 2](#).

The time can be specified as an offset or an absolute timestamp, as in the following two examples:

```
SELECT * FROM rides r
  AS OF SYSTEM TIME '-1d';
```

```
SELECT * FROM rides r
  AS OF SYSTEM TIME '2024-5-22 18:02:52.0+00:00';
```

The time specified cannot be older in seconds than the replication zone configuration parameter `gc.ttlseconds`, which controls the maximum age of MVCC snapshots.

It is also possible to specify *bounded* stale reads using the `with_max_staleness` argument:

```
SELECT * FROM rides r
  AS OF SYSTEM TIME with_max_staleness('10s')
WHERE city='amsterdam'
AND id='aaaae297-396d-4800-8000-0000000208d6';
```

Bounded stale reads can be used to optimize performance in distributed deployments by allowing CockroachDB to satisfy the read from local replicas that may contain slightly stale data. We'll return to bounded stale reads in [Chapter 11](#).

Other Data Definition Language Targets

So far, we've looked at SQL to create, alter, and manipulate data in tables and indexes. These objects represent the core of database functionality in CockroachDB, as in other SQL databases. However, the CockroachDB data definition language (DDL) provides support for a large variety of other, less frequently utilized objects. A full reference for all these objects would take more space than we have available here—see the [CockroachDB documentation](#) for a complete list of CockroachDB SQL. [Table 4-5](#) lists some of the other objects that can be manipulated in CREATE, ALTER, and DROP statements.

Table 4-5. Other CockroachDB schema objects

Object	Description
Database	A database is a namespace within a CockroachDB cluster containing schemas, tables, indexes, and other objects. Databases are typically used to separate objects that have distinct application responsibilities or security policies.
Schema	A schema is a collection of tables and indexes that belong to the same relational model. In most databases, tables are created in the PUBLIC schema by default.
Sequence	Sequences are often used to create primary KVs; however, in CockroachDB there are often better alternatives. See Chapter 5 for more guidance on primary key generation.
Role	A role is used to group database and schema privileges, which can then be granted to users as a unit. See Chapter 12 for more details on CockroachDB security practices.
Type	In CockroachDB, a type is an enumerated set of values that can be applied to a column in a CREATE or ALTER TABLE statement.
User	A user is an account that can be used to log in to the database and can be assigned specific privileges. See Chapter 12 for more details on CockroachDB security practices.
Statistics	Statistics consist of information about the data within a specified table that the SQL optimizer uses to work out the best possible execution plan for a SQL statement. See Chapter 8 for more information on query tuning.
changefeed	A changefeed streams row-level changes for nominated tables to a client program. See Chapter 7 for more information on changefeed implementation.
Schedule	A schedule controls the periodic execution of backups. See Chapter 11 for guidance on backup policies.
Function	A user-defined function.
Stored Procedure	A user-defined stored procedure.

Administrative Commands

CockroachDB supports commands to maintain authentication of users and their authority to perform database operations. It also has a job scheduler that can be used to schedule backup and restore operations as well as scheduled schema changes. Other commands support the maintenance of the cluster topology.

These commands are generally tightly coupled with specific administrative operations, which we'll discuss in subsequent chapters, so we'll refrain from defining them in detail here. You can always see the definitions for them in the [CockroachDB documentation](#). [Table 4-6](#) summarizes the most significant of these commands.

Table 4-6. CockroachDB administrative commands

Command	Description
CANCEL JOB	Cancel long-running jobs such as backups, schema changes, or statistics collections.
CANCEL QUERY	Cancel a currently running query.
CANCEL SESSION	Cancel and disconnect a currently connected session.
CONFIGURE ZONE	CONFIGURE ZONE can be used to modify replication zones for tables, databases, ranges, or partitions. See Chapter 10 for more information on zone configuration.
SET CLUSTER SETTING	Change a cluster configuration parameter.
EXPLAIN	Show an execution plan for a SQL statement. We'll look at EXPLAIN in detail in Chapter 8 .
EXPORT	Dump SQL output to CSV files.
SHOW/CANCEL/PAUSE JOBS	Manage background jobs—imports, backups, schema changes, etc.—in the database.
SET LOCALITY	Change the locality of a table in a multiregion database. See Chapter 10 for more information.
SET TRACING	Enable tracing for a session. We'll discuss this in Chapter 8 .
SHOW RANGES	Show how a table, index, or database is segmented into ranges. See Chapter 2 for a discussion on how CockroachDB splits data into ranges.
SPLIT AT	Force a range split at the specified row in a table or index.
BACKUP	Create a consistent backup for a table or database. See Chapter 11 for guidance on backups and high availability.
SHOW STATISTICS	Show optimizer statistics for a table.
SHOW TRACE FOR SESSION	Show tracing information for a session as created by the SET TRACING command.
SHOW TRANSACTIONS	Show currently running transactions.
SHOW SESSION	Show sessions on the local node or across the cluster.

The Information Schema

The *information schema* is a special schema in each database that contains meta-data about the other objects in the database—it is named `INFORMATION_SCHEMA` in CockroachDB. You can use the information schema to discover the names and types of objects in the database. For instance, you can use the information schema to list all the objects in the `information_schema` schema:

```
SELECT * FROM information_schema."tables"  
WHERE table_schema='information_schema';
```

Or you can use `information_schema` to show the columns in a table:

```
SELECT column_name, data_type, is_nullable, column_default  
FROM information_schema.COLUMNS WHERE TABLE_NAME='customers';
```

The information schema is particularly useful when writing applications against an unknown data model. For instance, GUI tools such as DBeaver use the information schema to populate the database tree and display information about tables and indexes.

The information schema is defined by ANSI standards and implemented by many relational databases. CockroachDB also includes some internal tables specific to the CockroachDB system in the `crdb_internal` schema. Information about these tables can be found in the [CockroachDB documentation](#).

Summary

In this chapter, we've reviewed the basics of the SQL language for creating, querying, and modifying data within the CockroachDB database.

A full definition of all syntax elements of CockroachDB SQL would take an entire book, so we've focused primarily on the core features of the SQL language with some emphasis on CockroachDB-specific features. For detailed syntax and for details of CockroachDB administrative commands, see the CockroachDB online documentation.

SQL is the language of CockroachDB, so of course we'll continue to elaborate on the CockroachDB SQL language as we delve deeper into the world of CockroachDB.

In the next chapter, we'll explore schema design and the CockroachDB features that enable efficient and performant queries.

PART II

Developing Applications with CockroachDB

CockroachDB Schema Design

A sound data model is the foundation of a highly performant and maintainable application. In this chapter, we'll review the fundamentals of relational schema design, with a particular focus on aspects of schema design that bear on distributed database operations and on advanced CockroachDB features such as column families and JSON binary (JSONB) support. We'll cover the creation of tables, indexes, and other schema objects that support a well-designed CockroachDB application.

Although CockroachDB supports mechanisms for efficiently altering schemas online, schema changes to production applications are nevertheless high-impact changes, typically involving coordinated changes to application code and production database configuration. If done poorly, there's the risk of loss of application functionality, availability, or performance. Therefore, although it's quite possible to alter CockroachDB schemas in production, it's far better to get the schema right during application design.

Relational database design is a big topic and has been the subject of many books and continuing debate. We don't want to try to cover advanced design principles here, nor do we want to engage in any debates about the purity of various design patterns. Most database models are a compromise between the mathematical purity of the relational model and the practicalities imposed by the physical database system. Therefore, in this chapter, we'll attempt to briefly cover only the theoretical side of the relational model while diving quite deep into the practicalities of designing a model that will work well with a CockroachDB implementation.

Logical Data Modeling

Application data models are commonly created in two phases. Establishing the logical data model involves modeling the information that will be stored and processed by the application and ensuring that all necessary data is correctly, completely, and unambiguously represented. The logical data model is then mapped to a physical data model. The physical data model describes the tables, indexes, and views that are created in the DBMS.

The logical data model typically satisfies only the functional requirements of the application. The physical data model must also satisfy nonfunctional requirements, particularly performance requirements.

In practice, these two phases are often blurred together, especially in agile and other iterative development environments. Nevertheless, whether done explicitly or not, there is definitely a difference between the analysis required to determine *what* data an application might process and *how* that data is best represented in a specific database system.

We introduced some of the core concepts of the relational model in [Chapter 1](#). Theoretically, during logical data modeling, we deal with relations, tuples, and attributes, while in physical design, we deal with tables, rows, and columns. However, outside of academia, these distinctions are often ignored, and in practice, it's commonplace to develop a logical model using the language of tables and columns.

Mea Culpa

Relational data modeling has generated an immense volume of research and debate over the past four decades. It's almost impossible to say anything sensible about relational data modeling without oversimplification or misrepresentation.

This is a book about CockroachDB, not about relational theory, so we have tried to avoid getting bogged down in debates about the correct way to perform relational design. Our purpose here is to provide enough quick background on relational modeling to allow for us to sensibly talk about CockroachDB-specific physical design principles.

If you want to further explore relational theory and design, there are many excellent books dedicated to the topic, such as *Database Design and Relational Theory* by C.J. Date (O'Reilly).

Normalization

A normalized data model is one in which any data redundancy has been eliminated and in which all data is completely identifiable by primary and foreign keys. Although the normalized data model is rarely the final destination from a performance point of view, the normalized data model is almost always the best initial representation of a schema because it minimizes redundancy and ambiguity.

The relational theory defines multiple “levels” of normalization. The third normal form is the generally accepted standard of an adequately normalized data model.

In third normal form, every attribute (column) in a tuple (row) is dependent on the entire primary key of that tuple only and not on any other attribute or key. We sometimes remember this as, “The key, the whole key, and nothing but the key.” For example, consider the data shown in [Figure 5-1](#).

StudentTests	
123	rowid
ABC	studentname
ABC	studentaddress
🕒	studentdob
ABC	testname
🕒	testdate
ABC	question1
ABC	answer1
ABC	question2
ABC	answer2
ABC	question3
ABC	answer3
ABC	question4
ABC	answer4
ABC	question5
ABC	answer5
ABC	question6
ABC	answer6
ABC	question7
ABC	answer7
ABC	question8
ABC	answer8
ABC	question9
ABC	answer9
ABC	question10
ABC	answer10

Figure 5-1. Student test data (denormalized)

Even if we created a primary key on `studentname`, `testname`, and `testdate`, we would still be a long way from the third normal form. Attributes such as `studentdob` are dependent only on part of the key (`studentname`), and the repeating “answer” columns are dependent on a nonkey attribute (the corresponding question).

A normalized version of this data is shown in [Figure 5-2](#). Students take tests, and tests have questions, and students answer those questions. All attributes in each entity are now fully dependent on the primary keys for that relation.

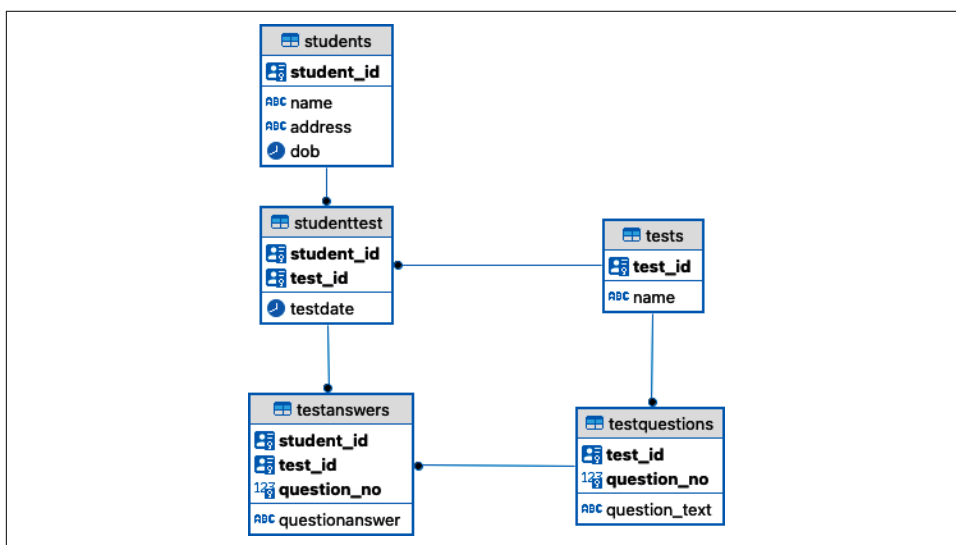


Figure 5-2. Student test data (normalized)

Don't Go Too Far

You'll generally recognize a well-normalized data model by the absence of any redundant information. For instance, in [Figure 5-2](#), you'll notice that student names, test names, question texts, etc., are never repeated across multiple entities. There is one and only one entry for each attribute. The only thing that is repeated in a well-normalized model should be foreign key references.

That being said, it's often a mistake to take the normalization process too far. In a real-world database, each new table in the model adds complexity to program code and overhead in joining information during data retrieval.

For example, from time to time we see addresses “normalized” as in [Figure 5-3](#). There's nothing theoretically wrong with this model. Two students could share an apartment, and the relationships between cities, states, and countries are very real. We could even throw continent, solar system, and galaxy entities into the model without violating the third normal form.

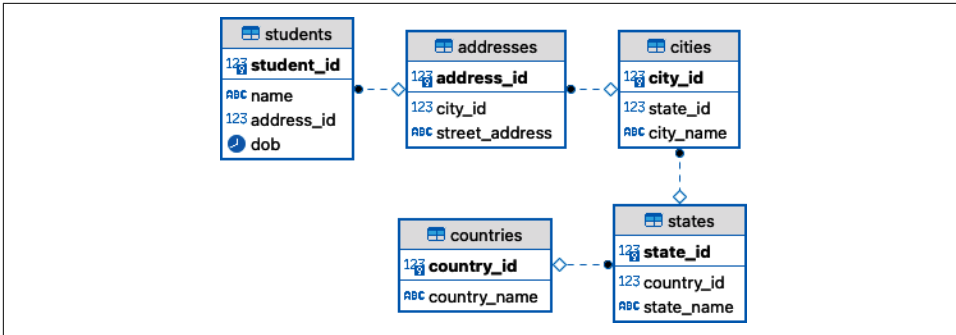


Figure 5-3. Student address model (normalized)

However, in practice, this model will require a five-way JOIN whenever a student's address needs to be retrieved. Because this is a pretty common operation, the cost of the JOIN across the life of the application will be high. It's probably better just to leave the address fully embedded in the students table, as shown in Figure 5-2.

Purists will definitely argue that this sort of denormalization should be performed in the physical data modeling stage. It's also worth noting that there may be good reasons for having a city or state table in a production system—each might be associated with specific attributes of relevance. But we'd suggest that you be pragmatic when contemplating such extended relationships. There's a lot of wasted effort in modeling logical relationships that are inevitably going to be collapsed in the physical design phase.

Primary Key Choices

In CockroachDB, the choice of primary keys is critical to performance because it is the primary key that will determine the distribution of data across the nodes in the system. We'll talk extensively about this in the Physical Design section.

However, even from the logical modeling point of view, there are some factors to consider.

The third normal form requires that each relation have a primary key. Yet, it does not specify whether that key should be natural or artificial. A *natural key* is one constructed from unique attributes that normally occur within the entity. An *artificial key* is one that is not constructed from the unique attributes of an entity and which exists only to uniquely identify the row. There is a continual debate within the database community regarding the merits of “artificial” primary keys versus the “natural” key.

CockroachDB will create an artificial key automatically if an explicit key is not provided.

In general, we're of the opinion that most fundamental entities should use an artificial key. Artificial keys are generally superior from a performance point of view and eliminate some of the overhead involved when a natural primary key is changed. Furthermore, in CockroachDB, the use of an artificial key provides us with methods for ensuring an even distribution of keys throughout the cluster.

For example, suppose we decided that a user's email address could serve as a primary key. The email address is unique to the user, so it's a perfectly valid key. However, users can change their email addresses. If that happens, then the update to the users' data will require not just an update in place to the row but also a relocation of the data within the cluster. Any foreign keys pointing to the email address key will also have to be updated and will also have to be relocated.

Special-Purpose Designs

For any given set of data, there usually exists more than one way to create a nearly correct relational model. Within the universe of possible models, there exist some patterns that are particularly applicable to certain workloads. Two of the most common are:

Data warehousing designs

These models, such as the *star* and *snowflake* schemas, have a large central “fact” table with foreign keys to multiple “dimension” tables. CockroachDB is not primarily intended as a data warehousing database, so these models are not typical of a CockroachDB deployment.

Time-series designs

In these models, the time of origin of data is part of each data element's key and data accumulates primarily as continual inserts. We'll briefly consider some of the considerations for time-series in the next section.

Physical Design

Physical design involves modifying the logical design to improve its performance, compatibility with the target database, or maintainability.

Not all “relational” databases implement relational features in the same way, and many have extensions to the relational model that can be useful. Therefore, a logical to physical mapping depends heavily on the characteristics of the target database.

Other changes are driven by workload considerations. For instance, if a table is only ever accessed in a JOIN with another table, we might replicate some columns from the second table into the first to avoid the join.

The other primary physical design drivers are the capabilities and performance characteristics of the database engine. For instance, in CockroachDB, ascending

primary keys cause hotspots on certain nodes and should be avoided, while in a nondistributed SQL database such as PostgreSQL, ascending keys are fine.

In the following sections, we'll discuss the various logical to physical translations that apply when implementing a model on CockroachDB.

Entities to Tables

The major output of the logical design process are entities, attributes, and keys. To convert the logical model to a physical model, we need to convert entities to tables and attributes to columns.

Depending on your logical model, this conversion may be close to a one-to-one mapping. However, be aware that in some cases, a single entity may map to multiple tables or vice versa. For instance, we might decide that the logical model shown in [Figure 5-3](#) should be collapsed to a single table, folding all address attributes into the student table. Or we might collapse the addresses entity into students and collapse states and countries into cities.

In some cases, a logical model may include *subtypes* in which an entity is defined that includes multiple “types” of tuples. For instance, a person entity might be defined as shown in [Figure 5-4](#).

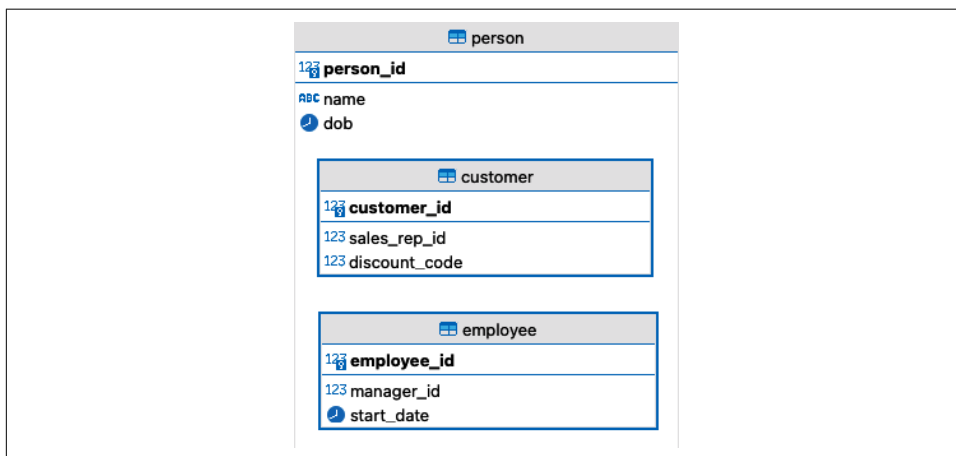


Figure 5-4. Logical model with subtypes

People can be customers or employees (or both). So should there be a single person table, a customer and an employee table, or even three tables: a person table with attributes common to customers and employees and a customer table and an employee table with attributes unique to each type?

The answer depends on your workload and performance requirements. Each of the previous solutions has a performance advantage for a certain class of query; you'll need to think through the operations that are most important to your application. However, the solution that we've seen most often is the two-table model (customers and employees).

Attributes to Columns

When mapping attributes to columns, we're mainly concerned with selecting the best data type for the column and defining its nullability correctly.

Null values are an important concept in relational databases—they distinguish between data that has a known value and data that is unknown or missing. Three-valued logic—TRUE/FALSE/NULL—is at the heart of SQL operations, such as WHERE.

In some systems, the use of NULLs in indexed columns is discouraged, and it is recommended to use NOT NULL with a DEFAULT value. This is because, in some databases (PostgreSQL, for instance), NULL values are not included in indexes. However, CockroachDB does store NULL values in indexes, and you can use an index to evaluate an IS NULL condition within a WHERE clause.

CockroachDB data types generally map easily to logical data types. Consider the following:

- All these CockroachDB string data types are equivalent: TEXT, CHAR, VARCHAR, CHARACTER VARYING, and STRING.
- All of the integer data types—INT, INT2, INT4, INT8, BIGINT, SMALLINT, etc.—are stored in the same manner in the database. A BIGINT and a SMALLINT consume the same storage (providing they hold the same value). The types serve to constrain only the ranges of values that can be stored. The INT type can hold any allowable integer value (a 64-bit signed integer).
- Similarly, FLOAT, FLOAT4, FLOAT8, and REAL data types all store 64-bit signed floating-point numbers.
- DECIMAL stores exact fixed-point numbers and should be used when it's important to preserve precision, such as for monetary values.
- BYTES, BYTEA, and BLOB store binary strings of variable length. The data is stored in line with other row data, and therefore, this data type is not suitable for very large objects (a maximum of 1 MB is suggested).
- TIME stores a time value in UTC, while TIMETZ stores a time value with a time zone offset from UTC. TIMESTAMP and TIMESTAMPTZ are similar but include both date and time in the value.

We'll discuss some of the other CockroachDB data types—such as ARRAYS and JSON—later in the chapter.

Primary Key Design

We've touched upon the importance of properly defining CockroachDB primary keys in preceding chapters; now it's time to get serious about this important topic.

The primary key of a table is used to distribute the ranges of that table's data across the cluster. If the primary key is monotonically increasing, then all new data will be appended to one range at a time (and hence, one node in the cluster). Most likely, this node will become a hotspot and limit the insert throughput for the cluster. This becomes particularly significant as your cluster grows—adding new nodes to a cluster may fail to result in higher throughput.

The same phenomenon can be encountered in a time-series database in which the primary key is prefixed with a timestamp. All “new” data will hit a single node, and your cluster scalability will be compromised.

For instance, consider this implementation of the ORDERS table:

```
CREATE SEQUENCE order_seq;

CREATE TABLE orders (
  salesorderid INT NOT NULL PRIMARY KEY DEFAULT nextval('order_seq'),
  orderdate DATE NOT NULL DEFAULT now() ,
  duedate DATE NOT NULL,
  shipdate DATE NULL,
  customerid INT NOT NULL,
  salespersonid INT NULL,
  totaldue DECIMAL NULL
);
```

The `orders_seq` sequence generator produces numbers that are guaranteed to be incrementing and—most of the time—without gaps.¹ Since every value of `ORDERID` is one higher than the preceding value, new orders will be inserted into a single range that will be located on a single node. Consequently, that node will bear the burden of all `INSERT` operations. As new ranges are created, the responsibility of handling inserts will shift to new nodes, but at any given point in time, just a single node will be handling all of the inserts.

¹ Sequence generators are provided mainly for compatibility with other databases and are not recommended for most CockroachDB applications.

Figure 5-5 illustrates this problem: because ranges are ordered by keys, workloads with sequential keys will cause all traffic to hit one of the boundaries of a range.

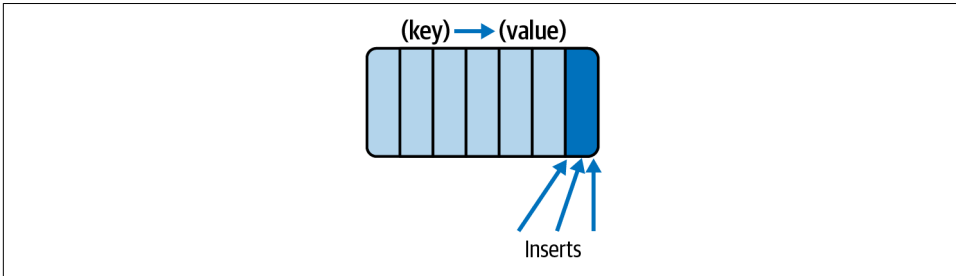


Figure 5-5. Sequential keys can create hotspots

In the next few sections, we’ll look at ways of avoiding this primary key “hotspot” antipattern.

UUID-based primary keys

If your application doesn’t need primary key values to be continuously increasing, then a universal unique identifier (UUID) primary key is the recommended solution.

A UUID is a value that is guaranteed to be unique across all systems. A UUID combines host-specific data, random numbers, and timestamp data to generate an identifier that will be unique across systems and times.

The `gen_random_uuid()` function generates UUIDs and can be used as the default value for a primary key, as in the following example:

```
CREATE TABLE orders (  
  orderid uuid NOT NULL PRIMARY KEY DEFAULT gen_random_uuid(),  
  orderdate DATE NOT NULL DEFAULT now() ,  
  duedate DATE NOT NULL,  
  shipdate DATE NULL,  
  customerid INT NOT NULL,  
  salespersonid INT NULL,  
  totaldue DECIMAL NULL  
);
```

UUIDs are unique, selective, and guaranteed to be evenly distributed across all nodes of a cluster. They are, therefore, the preferred mechanism for CockroachDB primary keys.

The SERIAL Data Type

In PostgreSQL, the SERIAL data type is typically used to create autoincrementing KVs. It's a handy alternative to creating a sequence, as shown in an earlier example.

However, in CockroachDB the SERIAL data type by default generates unique identifiers using the `unique_rowid()` function. `unique_rowid()` generates unique numbers that combine `nodeid` and `timestamp`. Although the numbers are generally ascending, the order is not absolutely guaranteed, so large gaps will occur and “hotspots” are still possible.

You can change the behavior of SERIAL to a more PostgreSQL-compatible behavior using the session variable `serial_normalization`. However, as with PostgreSQL, gaps in sequence numbers generated in this manner may still occur, and the performance overhead is significant. The CockroachDB team recommends against using SERIAL data types unless compatibility with PostgreSQL is required.

Avoiding hotspots with a composite key

It may be that your application really requires a monotonically increasing KV. One way to avoid a hotspot, in this case, is to create a composite primary key that leads with a nonmonotonically increasing value. For instance, in this implementation the `orderid` is prefixed with the `customerid` in the primary key:

```
CREATE TABLE orders (
  orderid INT NOT NULL DEFAULT nextval('order_seq'),
  orderdate DATE NOT NULL DEFAULT now() ,
  duedate DATE NOT NULL,
  shipdate DATE NULL,
  customerid INT NOT NULL,
  salespersonid INT NULL,
  totaldue DECIMAL NULL,
  PRIMARY KEY (customerid,orderid)
);
```

This implementation will send orders for a specific customer into the same ranges, but sequential orders from multiple customers should be distributed across the cluster.

There may be some upside in “clustering” customer data this way, but the clear downside is that we now need to know the customer ID when searching for an order. We’ve probably all experienced the irritation of having to provide both a customer identifier *and* an order identifier to a sales associate, so this downside is potentially significant. Of course, we could create a secondary index just on `orderid`, but then we’d create a secondary index with a hotspot.

What we need is a way to index monotonically increasing KVs without creating unscalable insert hotspots. The solution is *hash-sharded indexes*.

Hash-sharded primary keys

Hash-sharded indexes add a hashed value to the prefix of a primary key. Each hash value is unique and nonsequential. Consequently, if the primary key of a table is based on a hash-sharded index, then its values will be distributed evenly across all the ranges in the cluster. The result should be (statistically) a perfect distribution of writes across nodes.

To create a hash-sharded primary key, we would use this syntax:

```
CREATE TABLE orders (  
  orderid INT NOT NULL DEFAULT nextval('order_seq'),  
  orderdate DATE NOT NULL DEFAULT now() ,  
  due date DATE NOT NULL,  
  shipdate DATE NULL,  
  customerid INT NOT NULL,  
  salespersonid INT NULL,  
  totaldue DECIMAL NULL ,  
  PRIMARY KEY (orderid) USING HASH WITH BUCKET_COUNT=6  
);
```

The hash sharding is transparent to the application: you'll never see the hashed values, and all filters against existing primary keys will work as normal. However, the new index cannot be used to find ranges of primary keys or to sort the output by primary key. For instance, with a traditional primary key, the following query would be resolved efficiently by a range scan of the primary key index:

```
SELECT * FROM orders  
WHERE orderid > 0  
ORDER BY orderid;
```

With a hash-sharded index, scan and sort operations would be required, although the optimizer may be able to perform multiple short scans on hash buckets rather than one full scan.

The `WITH BUCKET_COUNT` clause determines how many “shards” of the index are created. Setting the number of buckets to twice the number of nodes in the cluster is a sensible default.

Gaps in Sequential Keys

Although sequences provide for guaranteed ascending KVs, they cannot guarantee that there will be no missing values in the ordered sequence. For performance reasons, sequence number increments are not within the scope of an application transaction. Therefore, if a transaction issues a `ROLLBACK` after a sequence number is consumed, that number is lost. To achieve anything like scalable distributed performance, you would use the `CACHE` option to give each node its own unique set of ranges—which will result in keys being inserted out of order across nodes. Furthermore, cached sequence numbers may be lost on a cluster restart.

If an application needs absolutely gap-free numbers (“no missing orders,” for instance), then the application will need to implement its own sequence-generating logic. Balancing performance and functionality, in this case, is not trivial—we’ll look at this more in [Chapter 6](#).

Ordering of primary key attributes

For a multicolumn primary key, the order of attributes has significant implications for performance. You should follow the guidelines for composite indexes, which we’ll outline later in the chapter. Generally, the more often a column is used independently of other columns, the more you’ll want to place that column first in the primary key. Likewise, the appearance of primary key columns in `ORDER BY` clauses should also influence the sequencing.

Summary of primary key performance

We’ve spent a fair bit of time on primary key mechanisms in CockroachDB for a good reason. The effect on scalability can be dramatic, and practices that worked fine in traditional monolithic SQL databases can backfire in CockroachDB.

[Figure 5-6](#) shows just how significant these effects can be. We see that insert throughput is severely diminished when `SERIAL` or sequence-generated keys are used. UUIDs are preferred, but if you need ascending primary keys, then a hash-sharded primary key index is recommended.

The data in [Figure 5-6](#) came from a nine-node CockroachDB Cloud cluster. The performance penalty from ascending primary keys is proportional to the size of the cluster; the more nodes in the cluster, the larger the relative penalty. So your mileage may vary depending on your cluster size.

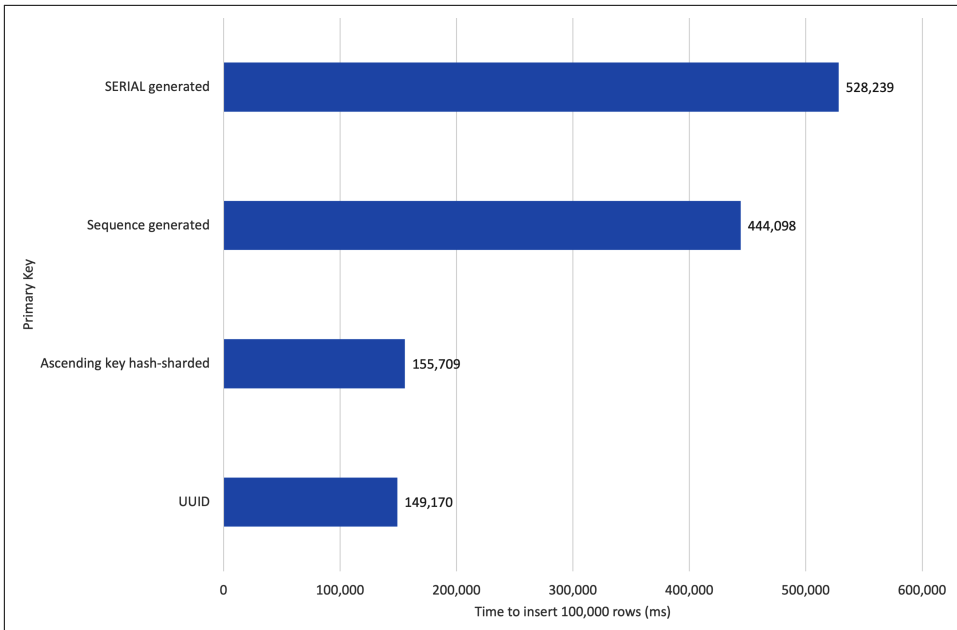


Figure 5-6. Insert performance with various primary key schemes

It's possible to greatly improve the performance of sequences by creating them with the `CACHE` option. This avoids the blocking wait involved in acquiring the “next” sequence number. However, in a distributed system like CockroachDB, using `CACHE` defeats the purpose of the sequence generator. Because each node in the cluster has its own cache, sequence numbers will be generated out of order across the cluster as a whole.

Foreign Key Constraints

Foreign key constraints help ensure data integrity and provide internal documentation of the data model, which can be leveraged by query generators and diagramming tools. However, during DML operations—particularly inserts—the validity of the foreign key must be checked by performing primary key lookups on the referenced table. These lookups can significantly increase the overhead of the operations and reduce throughput.

For the table that includes the foreign key constraint, this shows up mostly in insert performance since it is somewhat unusual for a foreign key to be updated, and the foreign key references do not need to be validated during deletes.

For the tables that are referenced within the foreign key constraint (e.g., the “parent” table), the overhead is felt most critically during deletes, where all child tables must be checked for “dangling” references.

The `ON DELETE CASCADE` clause of a `CONSTRAINT` definition will automatically delete any child rows during the delete of a parent row. `ON UPDATE CASCADE` has a similar effect when a primary key is updated (which in most applications is a rare event).

Because of the overhead of foreign key constraints, it is not unusual for them to be removed in a production system. They may be left enabled only in test and development environments to catch any data anomalies.

Denormalization

One of the outcomes in the development of a normalized data model is the removal of redundancies in data representation. In a well-normalized model, a data element is represented in just one place within the model. This eliminates the possibility of inconsistent information within the database.

Denormalization is the process of reintroducing redundant, repeating, or otherwise nonnormalized structures into the physical model—almost always with the intention of improving performance.

Denormalizing data is a common practice and one that you should not feel guilty about. However, do remember that denormalization has potential downsides:

Denormalized data can create inconsistencies.

These might be transitory (waiting for a materialized view to refresh) or permanent (a derived value is not updated due to a program error). You need to be sure that you have robust mechanisms in place to preserve data integrity.

Denormalization has a performance overhead.

Although denormalization exists to improve performance, most denormalizations have overhead. Typically, you improve query performance at the expense of DML performance. Make sure you understand and accept these trade-offs.

The best types of denormalizations are those that can be maintained by the database system automatically and transparently. For instance, in some databases, you might be tempted to vertically partition a table so that you can separate frequently accessed and rarely accessed columns. In CockroachDB, column families—introduced in [Chapter 2](#)—provide this capability without the need to change application code. We'll discuss column families in more detail in [“Vertical Partitioning” on page 149](#).

Replicating Columns to Avoid Joins

`JOIN` operations magnify the overhead of retrieving data. Over-enthusiastic normalization can often result in even the most trivial `SELECT` operations requiring multi-table joins. For instance, consider the partial schema shown in [Figure 5-7](#).

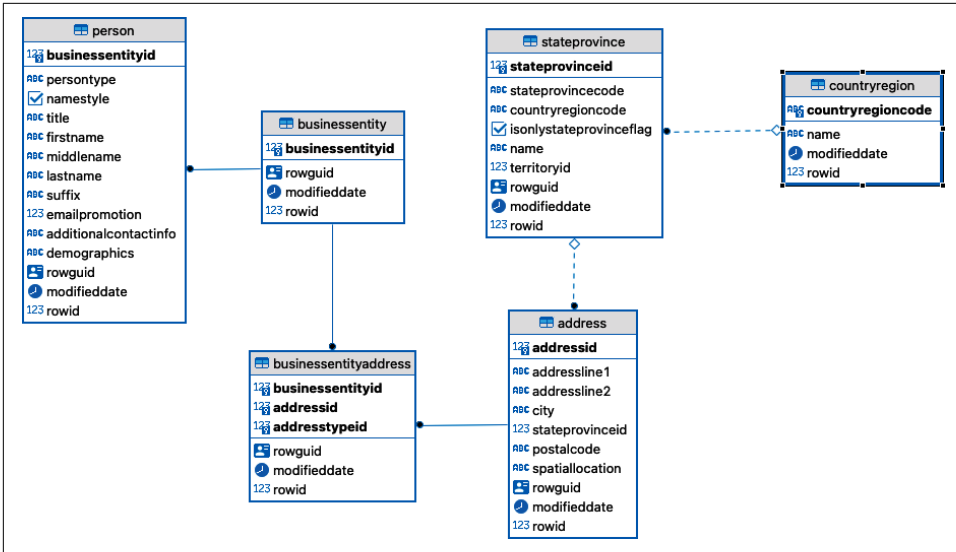


Figure 5-7. Overnormalized address data model

To retrieve the address for a person (something we presumably do a lot), we need a five-table join:

```
SELECT p.firstname, p.lastname, a.addressline1, a.city, s2.name, c2.name
FROM person p
JOIN businessentityaddress b3 ON (b3.businessentityid = p.businessentityid)
JOIN address a ON (b3.addressid = a.addressid)
JOIN stateprovince s2 ON (s2.stateprovinceid = a.stateprovinceid)
JOIN countryregion c2 ON (c2.countryregioncode = s2.countryregioncode)
WHERE p.businessentityid = 1
```

Because this JOIN follows primary KVs, it's going to be reasonably efficient, but it's still clearly going to involve five times as many lookup operations as would occur if all the columns were in the base table. So the solution is obvious: replicate the address directly into the person table. When a person's address changes, you may need to perform two UPDATES (one to address, one to person), but you will not have to perform a five-way JOIN every time you want an address.

As with many design decisions, there are many options between the two extremes. Suppose you want to preserve the multiple-address-per-person design of Figure 5-7. You could still consider collapsing the state and country tables into the address table to reduce the number of tables involved in the join.

Summary Tables

A summary table typically contains aggregated information that is expensive to compute on the fly. For instance, in the MovR application, we might have a dashboard that shows revenue trends by city based on the following query:

```
SELECT CAST(r.start_time AS date) AS ride_date, u.city, SUM(r.revenue)
FROM rides r
JOIN users u ON (u.id = r.rider_id)
GROUP BY ride_date, u.city;
```

Because revenue for previous days rarely changes, it's wasteful to continually reissue this expensive query every time the dashboard requests it. Instead, we create a summary table from this data and reload the data at regular intervals (perhaps once an hour).

We can create such a summary table manually, but *materialized views* exist for this very purpose. We'd create a materialized view as follows:

```
CREATE MATERIALIZED VIEW ride_revenue_by_date_city AS
SELECT CAST(r.start_time AS date) AS ride_date, u.city, SUM(r.revenue)
FROM rides r
JOIN users u ON (u.id=r.rider_id)
GROUP BY ride_date, u.city;
```

The resulting table is far smaller than the source tables, and we can manually refresh it whenever we like. One of the advantages of a materialized view is that we can also update it whenever we like with the REFRESH command:

```
REFRESH MATERIALIZED VIEW ride_revenue_by_date_city;
```

Vertical Partitioning

Vertical partitioning involves breaking up a table into multiple tables, each of which contains a different set of rows. This is typically done to reduce the amount of work that needs to be completed when updating a row and can also reduce the conflicts that occur when two columns are subject to high concurrent update activity.

For instance, consider an Internet of Things (IoT) application in which a city's current temperature and air pressure are updated multiple times a second by weather sensor devices across the city:

```
CREATE TABLE cityWeather (
    city_id uuid NOT NULL PRIMARY KEY DEFAULT gen_random_uuid(),
    city_name varchar NOT NULL,
    currentTemp float NOT NULL,
    currentAirPressure float NOT NULL
);
```

The temperature values and air pressure readings come from different systems, and we're concerned that they will cause transaction conflicts when they attempt to

change the same row simultaneously. We could partition the table into two tables to avoid this conflict:

```
CREATE TABLE cityTemp (
    city_id uuid NOT NULL PRIMARY KEY DEFAULT gen_random_uuid(),
    city_name varchar NOT NULL,
    currentTemp float NOT NULL
);

CREATE TABLE cityPressure (
    city_id uuid NOT NULL PRIMARY KEY DEFAULT gen_random_uuid(),
    city_name varchar NOT NULL,
    currentAirPressure float NOT NULL
);
```

However, CockroachDB column families provide a solution that does not require us to modify our data model. As discussed in [Chapter 2](#), column families allow groups of columns to be stored separately in the storage layer. We simply add each measurement to its own family:

```
CREATE TABLE cityWeather (
    city_id uuid NOT NULL PRIMARY KEY DEFAULT gen_random_uuid(),
    city_name varchar NOT NULL,
    currentTemp float NOT NULL,
    currentAirPressure float NOT NULL,
    FAMILY f1 (city_id, city_name),
    FAMILY f2 (currentTemp),
    FAMILY f3 (currentAirPressure)
);
```

Horizontal Partitioning

Horizontal partitioning (usually just referred to as partitioning) allows a table or index to be comprised of multiple segments. Some examples include the following:

- Queries can read only the partitions that contain relevant data, reducing the number of logical reads required for a particular query. This technique—known as partition elimination—is particularly suitable for queries that read too great a portion of the table to be able to leverage an index but still do not need to read the entire table.
- By splitting tables and indexes into multiple segments, parallel processing can be significantly improved because operations can be performed on partitions concurrently.

CockroachDB supports explicit table partitioning using syntax familiar to those who have used other enterprise databases, such as Oracle.

However, CockroachDB's multiregion capabilities eliminate many of the possible motivations for explicitly partitioning tables. *Regional by row tables* are transparently

partitioned in such a way as to optimize access to those rows from a particular region. In other databases, explicit partitioning might be required to realize this goal. We'll look at multiregion topologies in [Chapter 11](#).

Repeating Groups

The relational model abhors repeating groups, because in any such repeating group the attributes are not fully dependent on the primary key alone. For instance, an array element is identified by the primary key and the array index.

However, it can be extremely tedious to perform joins to retrieve small groups of data elements of the same type. For example, at the beginning of the chapter (see [Figure 5-2](#)), we defined a `testAnswers` entity that contains one row for each answer on a test. If a test has 100 questions, we need to access 100 rows to see all the results.

The array type provides an alternative mechanism. CockroachDB arrays are one-dimensional collections of data of the same data type. For instance, we could store the answers to the test in such an array:

```
CREATE TABLE studentTest (  
    student_id uuid NOT NULL,  
    test_id uuid NOT NULL,  
    testDate date NOT NULL,  
    testAnswers varchar[] NOT NULL  
);
```

We can set the results in a single update as follows:

```
UPDATE studentTest s  
SET testAnswers=array['a','b','c','d']  
WHERE student_id='2fdaadf5-ff3e-45c4-bc92-cc0d566e1ad9'  
AND test_id='dca69ac4-6c53-4efb-8c7e-bca9f412e2ee';
```

Now we need to access only a single row to get all the test results, which is a significant reduction in overhead.

Array data types do have some downsides—the query syntax is awkward, and it can be hard to perform analytic queries. For instance, finding the sum or average of all elements in an array is not directly supported.

Inverted indexes allow us to directly, efficiently retrieve data from an array data type; we'll elaborate on inverted indexes later in the chapter.

JSON Document Models

The biggest challenge to relational databases over the past decade has come from “document databases” such as MongoDB and Couchbase. These databases store all data in the form of JSON documents. JSON documents are self-describing, so there

doesn't need to be a formal implementation of a schema in the DBMS. One simply retrieves the JSON from the database and examines the JSON to decode the structure.

Without entering into any sort of religious debate about the obvious heresy involved in abandoning the relational model in favor of JSON documents, it's worth pointing out that document databases do offer significant conveniences for the developer:

- Modern object-oriented programming (OOP) practices involve the creation of complex “objects” that have an internal structure that allows for nesting and repeating groups. These program objects are typically highly denormalized and, when stored in an RDBMS, must be unpacked. Object-oriented programmers used to say, “A relational database is like a garage that forces you to take your car apart and store the pieces in little drawers.” In contrast, a document database allows the objects to be stored directly.
- JSON allows for the data model to evolve dynamically. For instance, an application can store responses from IoT devices over a REST interface without having a preconceived notion of how those responses are structured.
- Modern DevOps practices involve continuous integration in which the entire application can be built directly from code and tested upon any significant change. RDBMS makes this difficult because a code change and a database change need to be coordinated—ALTER TABLE statements and code commits need to be synchronously applied. Document databases avoid this issue.

If these document database advantages are attractive to you, then you'll probably be drawn to the idea of storing all or some of your data in a JSONB data type.

JSON objects are self-describing and can contain nested JSON objects and arrays. It's commonplace in document databases to embed child data within parent objects to avoid the need to perform joins. For instance, a house rental database might include all of the attributes of a property within a nested array:

```
{
  "_id": "10006546",
  "listing_url": "https://www.airbnb.com/rooms/10006546",
  "name": "Ribeira Charming Duplex",
  "summary": "Fantastic duplex apartment with three bedrooms,
located in the historic area of Porto, Ribeira (Cube)...",
  "amenities": [
    "TV",
    "Cable TV",
    "Wifi",
    "Kitchen",
    "Paid parking off premises",
    "Waterfront"
  ],
  "images": {
    "thumbnail_url": "",
```

```

    "medium_url": "",
    "picture_url": "https://a0.muscache.com/im/p/9b.jpg?aki_policy=large",
    "xl_picture_url": ""
  },
  "host": {
    "host_id": "51399391",
    "host_url": "https://www.airbnb.com/users/show/51399391",
  }
}

```

JSON Document Antipatterns

In CockroachDB, implementing *one-to-many relationships* in JSON documents is inadvisable. Because the JSON data is stored inline within the row data within the underlying KV store, CockroachDB recommends that you keep the size of the JSON documents fairly small—under 1 MB.

For instance, in the video streaming JSON model that we introduced in the previous chapter, we embedded all the films that a customer had viewed within a JSON array. Given all the video streaming that has been going on lately, it's quite likely that, at least for some users, the 1 MB limit would be exceeded.

Indexing JSON Attributes

As mentioned in previous chapters, you can create inverted indexes on JSONB columns. These inverted indexes allow you to search for attribute value matches within the JSON object. Inverted indexes are easy to create and guarantee that you'll have indexed access to attributes within the JSON object without having to anticipate what those attributes might be.

However, inverted indexes index every attribute in the JSON object and so can have many more entries in the index than rows in the table—with possible impacts on storage and index maintenance overhead. An alternative is to create computed columns on the JSONB attributes and create an index on those attributes. In this case, you'll need to know which attributes might need an index lookup, but your indexes will remain relatively compact.

So let's say we have decided to store our customer details as a JSONB document. The basic customer details look like this:

```

{
  "Address": "1913 Hanoi Way",
  "City": "Sasebo",
  "Country": "Japan",
  "District": "Nagasaki",
  "FirstName": "Mary",
  "LastName": "Smith",
  "Phone": "886780309",

```

```

    "dob": "1982-02-20T13:00:00Z",
    "likes": [
      "Dinosaurs",
      "Dogs",
      "People"
    ]
  }
}

```

We know that we want to search on LastName, FirstName, and we also know we want to have a foreign key out to an existing cities table. Our CREATE TABLE might look like this:

```

CREATE TABLE people (
  personId UUID PRIMARY KEY NOT NULL default gen_random_uuid(),
  cityId UUID ,
  personData JSONB,
  FirstName STRING AS (personData->>'FirstName') VIRTUAL,
  LastName STRING AS (personData->>'LastName') VIRTUAL,
  FOREIGN KEY (cityId) REFERENCES cities(cityid),
  INDEX (LastName, Firstname)
);

```

This design allows us to perform index searches on LastName and FirstName, and to retrieve those attributes from the JSONB document without the awkward JSON dereferencing syntax that we introduced in the last chapter. We can, however, add attributes without needing to issue an ALTER TABLE statement, and programmers can load the personData JSON data directly into a JSON object in their application code.

Using JSON or Arrays to Avoid Joins

We said before that one-to-many relationships should not be modeled in JSONB columns. The same is true of array columns. We want to avoid storing more data in these columns than the KV store can process in a single operation.

However, it can be quite effective to model *one-to-few relationships* in JSONB or arrays. For instance, consider the Student Tests schema we modeled way back in [Figure 5-2](#). We know that there can be at most only a couple of hundred questions in a test. In the normalized solution, we always have to JOIN tables to get the answers for a specific test:

```

SELECT s.student_id, s.test_id, question_no, questionanswer
FROM studentTest s
JOIN testAnswers t ON(t.student_id=s.student_id AND t.test_id = s.test_id)
WHERE s.student_id = ?
AND s.test_id = ?;

```

Being sure that the 1 MB limit will not be exceeded, we could collapse the test answers into a JSON document:

```

CREATE TABLE studentTest (
  student_id uuid NOT NULL ,
  test_id uuid NOT NULL,
  testDate date NOT NULL,
  answers JSONB
);

INSERT INTO studentTest (student_id, test_id, testDate, answers)
VALUES ( '2fdaadf5-ff3e-45c4-bc92-cc0d566e1ad9',
        'dca69ac4-6c53-4efb-8c7e-bca9f412e2ee',
        now(),
        '{"answers":[
          {"questionNumber":1,"Answer":5},
          {"questionNumber":2,"Answer":25},
          {"questionNumber":3,"Answer":58},
          {"questionNumber":4,"Answer":3425},
          {"questionNumber":5,"Answer":432},
          {"questionNumber":6,"Answer":0},
          {"questionNumber":7,"Answer":673}
        ]}');

```

We can also use arrays or JSON repeating groups to avoid joins where there is a *many-to-few relationship* between two tables. For instance, consider the relationship between students and classes as shown in [Figure 5-8](#).

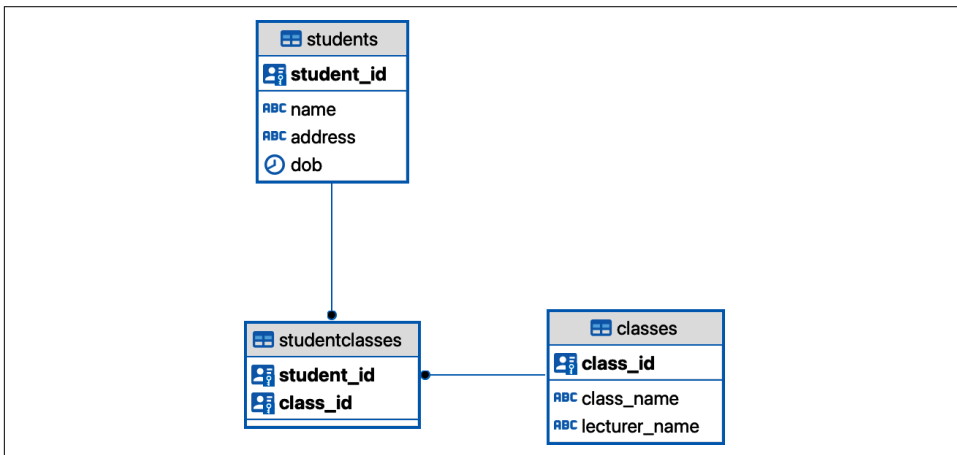


Figure 5-8. Students and classes

Whenever we want to get a list of a student's classes, we're forced to perform a three-way join:

```

SELECT class_name FROM students
  JOIN studentClasses USING(student_id)
  JOIN classes USING(class_id)
 WHERE student_id = '000390a6-4e1d-4bc1-aad7-66b645131d54';

```

The table `studentClasses` exists only to JOIN the `students` and `classes` tables—it contains no independent information.

We could instead store foreign keys for all the classes in an array type:

```
ALTER TABLE students ADD COLUMN classes UUID[];

UPDATE students s SET classes = (
    SELECT array_agg(class_id)
    FROM studentClasses sc
    WHERE s.student_id = sc.student_id);
```

The `array_agg` function takes all the columns in a result set and converts them to an array. In the prior example, we copied all the `class_id` values for each student into the `classes` array.

Now when we want to get the classes for a particular student, we can “unnest” the array and JOIN the resulting `class_id` values directly to the `classes` table:

```
WITH students_classes AS (
    SELECT student_id, UNNEST(classes) class_id
    FROM students
)
SELECT class_name FROM classes
JOIN students_classes USING(class_id)
WHERE student_id = '000390a6-4e1d-4bc1-aad7-66b645131d54';
```

This might all seem to be a bit convoluted, but in a high-performance workload, reducing a three-way JOIN to a two-way JOIN might be necessary to achieve performance objectives, even if it complicates application code a little.

Of course, we can avoid the joins altogether if we embed all the information about a student’s classes into a JSONB column, just as we did earlier for test answers. However, that array solution doesn’t duplicate any information from the `classes` table into the `students` table, so if the name of a class changes, we have only one update to perform.

Do bear in mind that by embedding foreign keys this way, we lose the capability of defining FOREIGN KEY constraints and create some opportunities for data inconsistencies. Furthermore, with this solution, it is difficult to find all students for a given class because we would have to unpack the array of classes for every student.

Indexes

An index is a database object that provides a fast path to specific data within a table.

We looked at the structure of indexes in [Chapter 2](#). You might recall that in CockroachDB, indexes and tables share a common fundamental storage structure. A base table is essentially a relation indexed by the primary key. Secondary indexes are

also relations but are indexed by the index key, with the column values representing the primary KVs associated with that secondary key.

Indexes exist to optimize performance and enforce uniqueness. Indexes can generally be added to a system without requiring any change to application code, so compared with other options for physical implementation, they are fairly easy to modify. Creating an optimal set of indexes is one of the most important factors in ensuring optimal database performance.

Index Selectivity

The *selectivity* of a column or group of columns is a common measure of the usefulness of an index on those columns. Columns or indexes are selective if they have a large number of unique values and few duplicate values. For instance, a `Date_of_birth` column will be quite selective, while a `Gender` column will not be at all selective.

Selective indexes are more efficient than nonselective indexes because they point more directly to specific values. The CockroachDB optimizer will determine the selectivity of the various indexes available to it and will generally try to use the most selective index.

Index Break-Even Point

When you want to look up just a few things in a textbook, you go to the index. When you want to assimilate all or most of the content, you bypass the index and go directly to the text. It's the same with database indexes—we generally want to use them only when we are retrieving a relatively small amount of a table's data.

A noncovering index—one that includes the filter conditions but not all the columns in the `SELECT` list—is generally effective only when we are retrieving a small percentage of a table's data. Beyond that, the overhead of going backward and forward from index to base table will be slower than simply reading all the rows in the table.

The optimizer will attempt to determine how much data is being accessed and choose an index or a table scan as appropriate. However, you don't want to create an index that will never be used, so it's important to understand the cut-off point between an index and a table scan.

However, when we create a covering index using the `STORING` clause, the situation is very different; in this case, the index can outperform the table access even if large proportions of data are accessed. It is true that adding columns to the index using the `STORING` clause will increase the overhead of index maintenance, but most of the time the improvement in query performance will be greater than the increase in write overhead.

For instance, let's say we have time-series data where a measurement (say a temperature) was recorded every minute over the past year. The application is often asked to determine the average measurement over some recent time period. The query looks something like this:

```
SELECT AVG(measurement)
FROM timeseries_data
WHERE measurement_timestamp >
  ((date '20220101') - INTERVAL '$dayFilter days');
```

The variable `$dayFilter` can take low or high values. We can create a noncovering index on the table as follows:

```
CREATE INDEX timeseries_timestamp_i1
ON timeseries_data(measurement_timestamp);
```

However, this index will be effective only when the number of days selected is very small—probably less than a week. Alternatively, we could create a covering index that includes the `measurement` column:

```
CREATE INDEX timeseries_covering
ON timeseries_data(measurement_timestamp) STORING (measurement);
```

This index can be used effectively for any span of data—from one day to the entire year's data.

Figure 5-9 compares the performance of an index scan with a table scan against the number of days of data being retrieved. The table scan must do the same amount of work regardless of the amount of data being processed, while an index scan increases in overhead the larger the amount of data being processed. For a noncovering index, a table scan is better if there's more than about a week's worth of data retrieved (about 2% of the total data). However, a covering index can perform well even if we are retrieving all of the table's data.

There are a few lessons to be drawn from Figure 5-9:

- The optimizer switches from an index scan to table scan when the amount of data hits about 10% to 15% of the total. The optimizer is a sophisticated piece of software, but it isn't magic, and it can't always work out which access path is better. In some circumstances, creating a noncovering index will actually degrade performance.
- A covering index is far superior in performance to a noncovering index and can be used effectively even if all or most of the table is being accessed. Whenever possible, use a covering index.
- Remember that in CockroachDB, indexes and tables have the same storage format: a covering index is not just a fast access mechanism—it's also a compact

representation of a subset of table columns that can be scanned far faster than the base table.

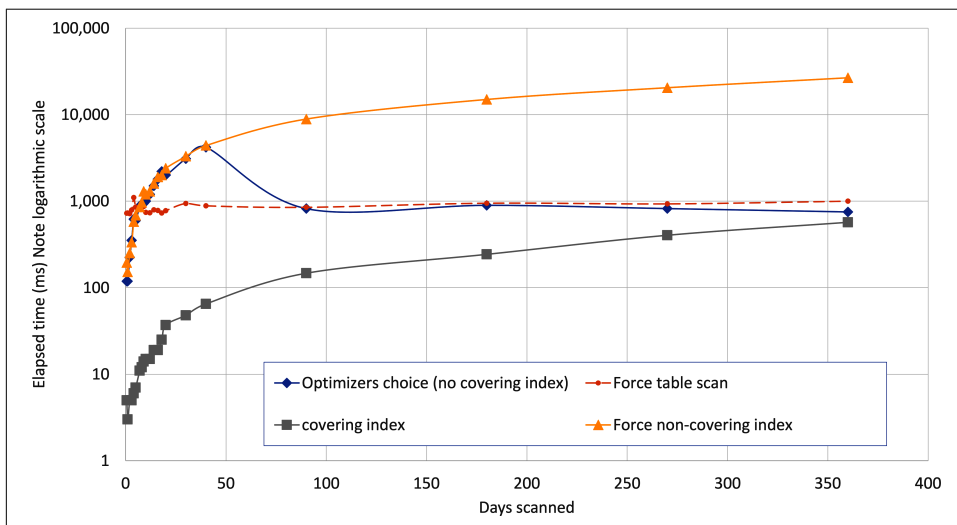


Figure 5-9. Comparison of table scan versus index scan performance

We'll come back to index performance and tuning queries in [Chapter 8](#).

Index Overhead

Although indexes can dramatically improve read performance, they do reduce the performance of write operations. All of a table's indexes must normally be updated when a row is inserted or deleted, and an index must also be amended when an update changes any column that appears in the index.

It is, therefore, important that all our indexes contribute to query performance, since these indexes will otherwise needlessly degrade write performance. In particular, you should be especially careful when creating indexes on frequently updated columns. A row can be inserted or deleted only once but may be updated many times. Indexes on heavily updated columns or on tables that have a very high insert/delete rate will therefore exact a particularly high cost. [Figure 5-10](#) illustrates the overhead on write performance that occurs as more indexes are added to a table.

Note that you can't improve performance by removing the primary key index. If an explicit primary key index does not exist, CockroachDB will create an artificial primary key for you.

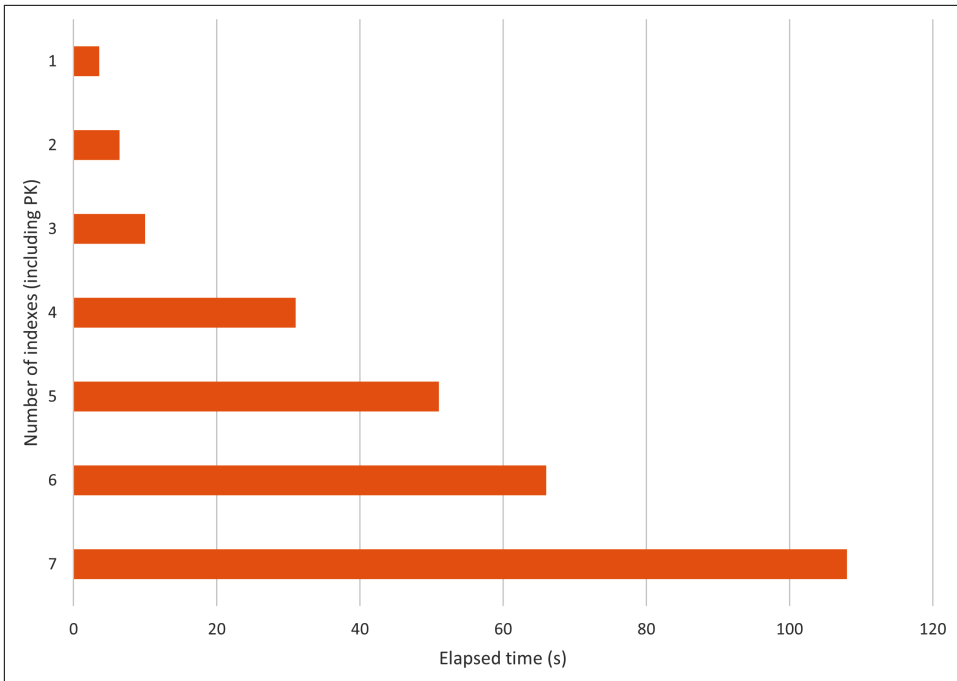


Figure 5-10. Index overhead—time to insert 120,000 rows

Composite Indexes

A composite index is simply an index created from more than one column. The advantage of a composite index is that it is often more selective than a single column index. The combination of columns will point to a smaller number of rows than separate indexes composed of the individual columns.

For instance, if we know that we frequently perform searches on `firstname` and `lastname`, then it makes sense to create an index on both of those columns:

```
CREATE INDEX flname_idx ON person (lastname,firstname);
```

Such an index will be far more effective than an index on `lastname` alone or separate indexes on `lastname` and `firstname`. We'll provide some performance comparisons for composite indexes a bit later in the chapter.

If a composite index could be used only when all of its keys appeared in the `WHERE` clause, then you would have to create a lot of composite indexes—as many as you had distinct combinations of columns in the `WHERE` clause. Luckily, a composite index can be used effectively, provided any of the initial or “leading” columns are used. Leading columns are those that are specified earliest in the index definition.

So, for instance, the index on (lastname, firstname) that we just created can optimize this query:

```
SELECT * FROM person WHERE lastname='Wood';
```

But not this query:

```
SELECT * FROM person WHERE firstname='John' ;
```

Covering Indexes

A covering index is one that is capable of satisfying a query without reference to the base table. For instance, consider the following query:

```
SELECT phonenumber
FROM people
WHERE lastname = 'Smith'
AND firstname = 'Samantha'
AND state = 'California' ;
```

Here, an index on lastname, firstname, state, and phonenumber would not only be able to *find* the data requested but would also be able to *return* the phonenumber. Only a single index access—and no base table read—would be needed.

In CockroachDB, we can use the `STORING` clause to store data elements that we might use in the `SELECT` clause but not in the `WHERE` clause. For the previous query, this index would be optimal:

```
CREATE INDEX people_lastfirststatephone_ix ON people
(lastname, firstname, state)
STORING (phonenumber);
```

Of course, we could also create a covering index without the `STORING` clause simply by adding the column to the index. For example:

```
CREATE INDEX people_lastfirststatephone_ix ON people
(lastname, firstname, state, phonenumber);
```

This index could then be used to satisfy queries that included the phone number in the `WHERE` clause and, therefore, will often be superior. However, `STORING` does have some advantages. Some data types (such as JSON and arrays) can't be indexed, but all data types can be stored. Other data types take up more space when indexed than when stored (this is most severe for collated strings). If you know you don't need to filter by a column, it may be more efficient to store it instead of indexing it.

Composite and Covering Index Performance

Figure 5-11 illustrates the performance advantages offered by composite and covering indexes. The chart shows the number of KV store options necessary to satisfy this query under various indexing scenarios:

```
SELECT phonenumber
FROM people
WHERE lastname = 'Smith'
      AND firstname = 'Samantha'
      AND state = 'California' ;
```

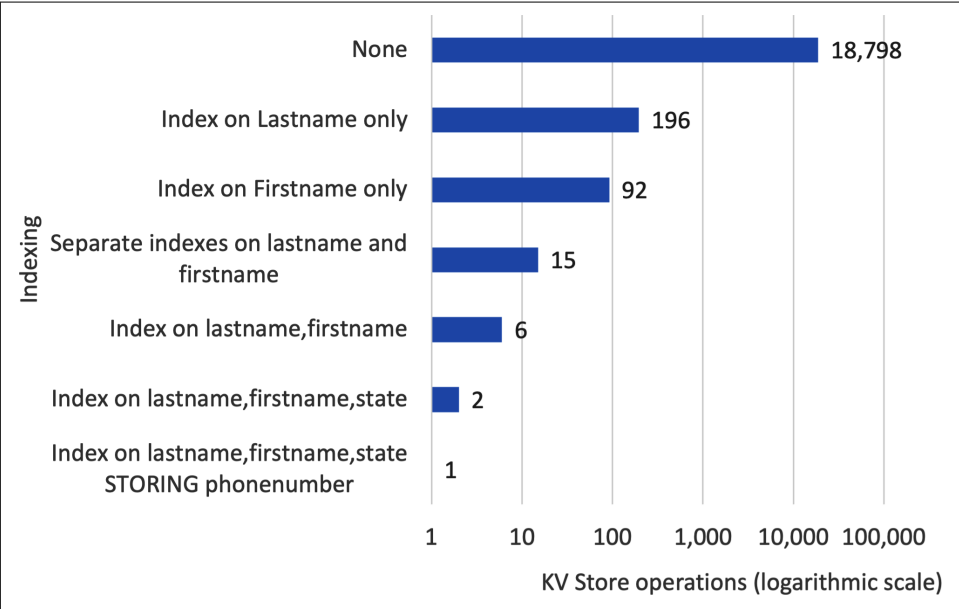


Figure 5-11. Composite index performance

Figure 5-11 shows that without indexing, the query requires 18,798 KV operations—we have to read every row in the table. Single indexes on lastname or firstname improve the performance somewhat, and having both a lastname and firstname index is better than just having either of the two indexes alone.

However, it's not until we use a composite index that we see truly efficient indexing. Only six KV reads are needed if we have an index on lastname and firstname, and only two KV operations are needed for an index on lastname, firstname, and state. If we STORE the phonenumber in the index, then only a single KV operation is needed.

Guidelines for Composite Indexes

As we saw earlier, the performance improvements from indexes don't come without a cost—each index adds overhead to DML operations, so we usually can't create every possible index that we might like.

The best strategy is to create composite indexes that cover the broadest possible ranges of queries. A composite index can be used if any of its leading columns are included in a `WHERE` clause, so the ordering of columns in composite indexes is important.

The following guidelines might help when deciding which indexes to create:

- Create composite indexes for columns that appear together in the `WHERE` clause.
- If columns sometimes appear on their own in a `WHERE` clause, place them at the start of the index.
- The more *selective* a column is, the more useful it will be at the leading end of the index.
- A composite index is more useful if it also supports queries where not all columns are specified. For instance, `lastname, firstname` is more useful than `firstname, lastname` because queries against only `lastname` are more likely to occur than queries against only `firstname`.

Indexes and Null Values

In many relational databases, null values are not included in indexes. Consequently, in these systems, it is often recommended not to use null values if you might want to search for those values. However, in CockroachDB, null values are included in indexes and can be found using an index in the normal way.

Inverted Indexes

We discussed inverted indexes in [Chapter 2](#) and earlier in this chapter in “[JSON Document Models](#)” on [page 151](#). An inverted index creates an index for all elements in an array and for all attributes in a JSONB column. As useful and flexible as these inverted indexes may be, they are expensive from a storage and maintenance point of view. We recommend, whenever possible, creating a computed column from the JSONB attribute concerned and indexing on that column.

Partial Indexes

A partial index can be created on only a subset of rows in the table. A partial index is created by adding a `WHERE` clause to the `CREATE INDEX` statement.

Partial indexes can have a lower maintenance overhead, require less storage in the database, and be more efficient for suitable queries. They are, therefore, a very useful type of index.

The key limitation with a partial index is that it can be used only when CockroachDB can be certain that the partial index contains all the necessary entries to satisfy the query. In practice, this means that a partial index is normally used to optimize queries that contain the same `WHERE` clause filter condition that was included in the index definition.

Sort-Optimizing Indexes

Indexes can be used to optimize `ORDER BY` operations in certain circumstances. When CockroachDB is asked to return data in sorted order, it must retrieve all the rows to be sorted and perform a sort operation on those rows before returning any of the data. However, if an index exists on the `ORDER BY` columns, then CockroachDB can read the index and retrieve the rows directly from the index in sorted order.

Using the index to retrieve data in sorted order is usually only worthwhile if you are optimizing for some small number of “top” rows. If you read the entire table in sorted order from the index, then you’ll be reading all the index entries as well as all the table entries, and the total number of I/O operations will be excessive. However, if you are just getting the first “page” of data or a “top 10,” then the index will be much faster since you never have to read the rest of the table rows at all.

However, if the index contains all the columns you need in the output, either because it indexes all those columns or is using `STORING` on the others, then you get the best of both worlds—you can retrieve all rows efficiently in sorted order.

Figure 5-12 illustrates the effect of an index to optimize a sort like this:

```
SELECT *  
FROM orderdetails  
ORDER BY modifieddate;
```

When a `LIMIT` clause was added to the query, the index reduced execution time from 123 ms to just 2 ms—a fantastic improvement. However, if we force CockroachDB to use the index to retrieve all rows (something it won’t do by default), then execution time will increase from 296 ms to 4,000 ms.

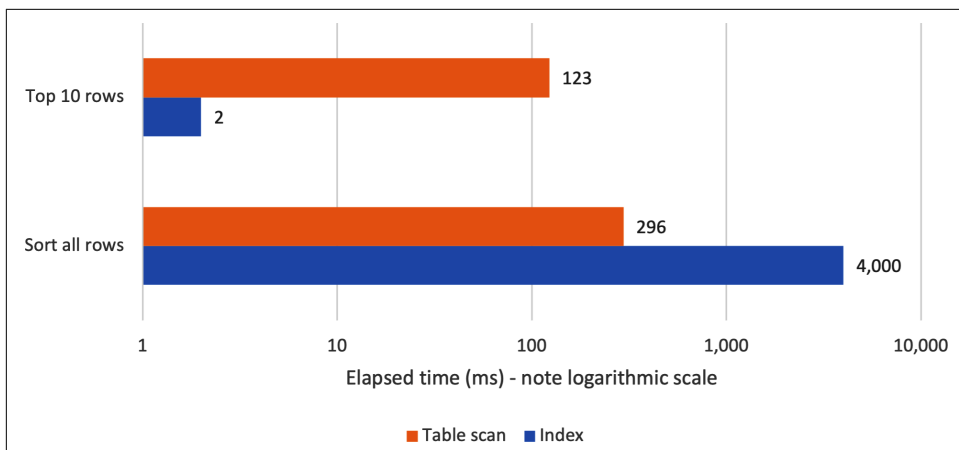


Figure 5-12. Indexes and sort performance

Expression Indexes

As the name suggests, expression indexes are indexes that are created on expressions! Perhaps you'd like to create an index on a column but will always be performing some kind of operation on that column before using it. Expression indexes are a good choice in these situations. Let's explore some examples where expression indexes can help.

For the examples that follow, we'll create a simple table and insert some data into it:

```
CREATE TABLE customer (
  "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "email" STRING NOT NULL,
  "address" JSON NOT NULL
);

INSERT INTO customer ("email", "address") VALUES
('person@acme.com', '{"zip": "10001"}'),
('person@example.com', '{"zip": "EC4M 9AF"}');
```

We'll build two expression indexes against the customer table. One will index the domain portion of a customer's email address (perhaps we'd like to filter for customers from a particular company) and another will index a field within their address JSON.

If we were to query for the domain portion of a customer's email address now, CockroachDB would have to perform a full table scan. It would have to read each of the email values, extract the domain from the email address, and test it against the given value we're filtering on:

```
EXPLAIN SELECT * FROM customer
WHERE REGEXP_EXTRACT("email", '@(.)+$') = 'acme.com';

distribution: local
vectorized: true

• filter
| filter: regexp_extract(email, '@(.)+$') = 'acme.com'
|
└─ • scan
    missing stats
    table: customer@customer_pkey
    spans: FULL SCAN
```

As shown in the following example, applying an expression index to the column allows CockroachDB to pinpoint an email's domain, not only preventing a regex extraction from occurring against every row during the search but also saving CockroachDB from having to scan the column to find the value (as the meta ranges would point to the exact location of the matching row or rows). The index usage can be seen in the last line of the EXPLAIN output as spans: `[/ 'acme.com' - / 'acme.com']`, which matches the value we were filtering for:

```
CREATE INDEX idx_customer_email_domain
ON customer ((REGEXP_EXTRACT("email", '@(.)+$')));

SELECT * FROM customer
WHERE REGEXP_EXTRACT("email", '@(.)+$') = 'acme.com';

-----+-----+-----
id      | email      | address
-----+-----+-----
e13ab911-61ac-42af-be0c-1abff74d598e | person@acme.com | {"zip": "10001"}

EXPLAIN SELECT * FROM customer
WHERE REGEXP_EXTRACT("email", '@(.)+$') = 'acme.com';

distribution: local
vectorized: true

• index join
| estimated row count: 1
| table: customer@customer_pkey
|
└─ • scan
    estimated row count: 2 (99% of the table; stats collected 4 minutes ago)
    table: customer@idx_customer_email_domain
    spans: [/ 'acme.com' - / 'acme.com']
```

Expression indexes can also be created against fields within JSON documents. The most flexible index for a JSON document is an inverted index, which indexes every field within the document. However, if you only care about one or two fields within the document, an inverted index might be overkill. Let's index the customer's zip code with an expression index to save having to index their entire address:

```

CREATE INDEX idx_customer_address_zip
ON customer ((("address"->'zip')));

SELECT * FROM customer
WHERE "address"->'zip' = 'EC4M 9AF';

```

id	email	address
07844da4-a417-45be-8ea5-b2c1767d6ebc	person@example.com	{"zip": "EC4M 9AF"}

```

EXPLAIN SELECT * FROM customer
WHERE "address"->'zip' = 'EC4M 9AF';

```

```

info
-----
distribution: local
vectorized: true

• index join
| estimated row count: 1
| table: customer@customer_pkey
|
└─ • scan
    estimated row count: 2 (99% of the table; stats collected 5 minutes ago)
    table: customer@idx_customer_address_zip
    spans: [/'EC4M 9AF' - /'EC4M 9AF']

```

The expression index has prevented a full table scan and has allowed CockroachDB to pinpoint the exact location of any rows containing our requested zip code.

Full-Text Indexes

CockroachDB supports natural-language full-text searching via a combination of the TSVECTOR and TSQUERY data types.

To prepare a string of text for full-text search, it must first be normalized. Use the `to_tsvector` function to normalize a string into a TSVECTOR comprising lexemes and positions as follows:

```

SELECT to_tsvector('the quick brown fox jumps over the lazy dog');
-- Returns: 'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2

```

Then prepare a query string via the `to_tsquery`, `plainto_tsquery`, or `phrase_to_tsquery` functions:

```

SELECT to_tsquery('over & the & lazy & dog');
-- Returns: 'lazi' & 'dog'

SELECT plainto_tsquery('over the lazy dog');
-- Returns: 'lazi' & 'dog'

SELECT phraseto_tsquery('over the lazy dog');
-- Returns: 'lazi' <-> 'dog'

```

Note how the word “lazy” becomes “lazi”. This is due to “stemming,” a process that reduces words to their root or base form. Note also that the `phraseto_tsquery` function results in a TSQUERY that enforces the ordinal relationship of the string provided (i.e., “lazy” must precede “dog” in this example).

Combining TSVECTOR and TSQUERY with the `ts_rank` function yields a numeric value indicating the frequency of matching lexemes, where a higher value indicates a better match:

```

SELECT ts_rank(
  to_tsvector('the quick brown fox jumps over the lazy dog'),
  plainto_tsquery('what did the quick brown fox jump over?')
);
-- Returns: 0.46362182

SELECT ts_rank(
  to_tsvector('the quick brown fox jumps over the lazy dog'),
  plainto_tsquery('i like coffee')
);
-- Returns: 1e-20 (or 0.00000000000000000001)

```

The `@@` matching operator can also be used to match vectors and queries. It returns a Boolean true or false value depending on whether a match has been found:

```

SELECT to_tsvector('the quick brown fox jumps over the lazy dog') @@
  plainto_tsquery('what did the quick brown fox jump over?');
-- Returns: t

SELECT to_tsvector('the quick brown fox jumps over the lazy dog') @@
  plainto_tsquery('i like coffee');
-- Returns: f

```

With what we’ve learned from the available data types and functions, let’s create a table with support for full-text searching:

```

CREATE TABLE article (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title STRING NOT NULL,
  title_vec TSVECTOR NOT NULL AS (to_tsvector('english', title)) STORED,

  INVERTED INDEX (title_vec)
);

```

```
INSERT INTO article (title) VALUES
  ('The rise and fall of North Korea - the sleeping giant of women's football'),
  ('Northern lights illuminate skies as far south as Cornwall'),
  ('The ''superfood'' taking over fields in northern India');
```

This table contains a “title” column and a TSVECTOR equivalent, which stores the title’s lexemes:

```
SELECT title_vec FROM article;

/* Returns:
'cornwal':9 'far':6 'illumin':3 'light':2 'northern':1 'sky':4 'south':7
'fall':4 'football':14 'giant':10 'korea':7 'north':6 'rise':2 'sleep':9 ...
'field':5 'india':8 'northern':7 'superfood':2 'take':3
*/
```

We can now search against the table with a combination of the `ts_rank` and `@@` operator:

```
SELECT title
FROM article, to_tsquery('northern') AS query
WHERE query @@ title_vec;

/* Returns:
Northern lights illuminate skies as far south as Cornwall
The 'superfood' taking over fields in northern India
*/
```

Spatial Indexes

A spatial index is a special type of inverted index that supports operations on the GEOMETRY and GEOGRAPHY two-dimensional spatial data types. Spatial indexing is a complex topic, and we aim only to introduce you to some key concepts here. For more details, consult the [CockroachDB documentation set](#).

To create a spatial index, we add the `USING GIST(geom)` clause:

```
CREATE INDEX geom_idx_1 ON some_spatial_table USING GIST(geom);
```

We can further fine-tune the index using various spatial [index tuning parameters](#):

```
CREATE INDEX geom_idx_1 ON geo_table1 USING GIST(geom)
WITH (s2_level_mod = 3);
CREATE INDEX geom_idx_2 ON geo_table2 USING GIST(geom)
WITH (geometry_min_x = 0, s2_max_level = 15)
CREATE INDEX geom_idx_3 ON geo_table3 USING GIST(geom)
WITH (s2_max_level = 10)
CREATE INDEX geom_idx_4 ON geo_table4 USING GIST(geom)
WITH (geometry_min_x = 0, s2_max_level = 15);
```

We don't recommend that you change these default tuning parameters; the default values will generally provide the best performance.

Hash-Sharded Indexes

Earlier in this chapter, we showed how in a distributed database, monotonically increasing primary keys can lead to “hotspots” in a distributed database. We recommended using hash-sharded indexes as a way of avoiding such an issue for monotonically increasing primary keys.

These sorts of hotspots don't just occur in primary keys. Any indexed column that is monotonically increasing might end up with all new values in a single range, thus creating a scalability and throughput issue.

If you have indexed columns where the value is constantly increasing (timestamps are a good example) and you want to avoid such an insert hotspot, then you should consider hash-sharding the index. The syntax is the same as for the primary key example we showed in “[Hash-sharded primary keys](#)” on [page 144](#). For instance, to create a hash-sharded index on the `modifieddate` column, we might do the following:

```
SET experimental_enable_hash_sharded_indexes=on;
```

```
CREATE INDEX orderdetails_hash_ix  
  ON orderdetails(modifieddate)  
  USING HASH WITH BUCKET_COUNT = 6;
```

Note that while CockroachDB might not optimize a sort with a hash-sharded index, it still might provide good enough performance for a “top 10” type of query. We can force the use of the hash-sharded index to perform an `ORDER BY` using an *index hint* (more on this in [Chapter 8](#)):

```
SELECT *  
  FROM orderdetails@orderdetails_hash_ix  
 ORDER BY modifieddate LIMIT 10;
```

CockroachDB will retrieve the top 10 from each “bucket” and amalgamate the results on the gateway node. The result might still be a marked improvement over a full scan.

Measuring Index Effectiveness

Having created an index, we'd like to be sure that it is being used to optimize our query and discover exactly how much benefit we have achieved. We can do this using the `EXPLAIN` and `EXPLAIN ANALYZE` commands.

`EXPLAIN` reveals to us the CockroachDB optimizer “plan” for a SQL statement. We'll dig into `EXPLAIN` in detail in [Chapter 8](#), but for now, let's just quickly see how `EXPLAIN` works to reveal your query's performance characteristics.

EXPLAIN reveals the optimizer's plan for resolving a query. For instance, if we created an index on the people table and wanted to see if the query would use it, we could issue the following command:

```
EXPLAIN
SELECT phonenumber
FROM people
WHERE lastname = 'Smith'
AND firstname = 'Samantha'
AND state = 'California';
```

info

```
distribution: local
vectorized: true

• filter
| estimated row count: 63
| filter: state = 'California'
└─
  • index join
  | estimated row count: 5
  | table: people@primary
  └─
    • scan
    | estimated row count: 5 (0.02% of the table;)
    | table: people@people_lastfirst_ix
    | spans: ['/Smith'/'Samantha'-'/'Smith'/'Samantha']
```

We can see that the `people_lastfirst_ix` will be used to resolve the query.

However, in some cases we might still not be sure if the index improved execution time. If we use `EXPLAIN analyze`, then CockroachDB will execute the operation and will report on the amount of I/O and other operations that occurred:

```
EXPLAIN analyze
SELECT phonenumber
FROM people
WHERE lastname='Smith'
AND firstname='Samantha'
AND state='California';
```

info

```
planning time: 2ms
execution time: 4ms
distribution: local
vectorized: true
rows read from KV: 6 (598 B)
cumulative time spent in KV: 3ms
maximum memory usage: 30 KiB
network usage: 0 B (0 messages)
```

```

• filter
  | cluster nodes: n1
  | actual row count: 1
  | estimated row count: 63
  | filter: state = 'California'
  └─
    • index join
      | cluster nodes: n1
      | actual row count: 3
      | KV rows read: 3
      | KV bytes read: 430 B
      | estimated row count: 5
      | table: people@primary
      └─
        • scan
          cluster nodes: n1
          actual row count: 3
          KV rows read: 3
          KV bytes read: 168 B
          estimated row count: 5 (0.02% of the table)
          table: people@people_lastfirst_ix
          spans: [/ 'Smith' / 'Samantha' - / 'Smith' / 'Samantha']

```

EXPLAIN has some additional advanced features that we'll learn about in [Chapter 8](#). However, you can see how easy it is to use EXPLAIN to simply determine index utilization and effectiveness.

Summary

In this chapter, we looked at design principles for a CockroachDB database schema. A sound data model is an essential foundation for a performant and maintainable CockroachDB database.

Database modeling typically proceeds in two stages: logical modeling followed by physical modeling. The aim of logical modeling is to identify the data required for application functionality. During physical modeling we aim to construct a data model that can meet functional requirements together with performance and availability requirements. The physical model should almost never be a direct copy of the logical model.

Database design for a distributed SQL database like CockroachDB creates some unique challenges when compared with a traditional monolithic database. In particular, primary keys should be constructed so that new rows are distributed equitably across the nodes in the cluster. The UUID data type can achieve this, but if an ascending primary key is required, then using hash-sharded primary key indexes is recommended.

We also looked at indexing choices for a CockroachDB database design. Creating the least number of composite indexes to support common filter conditions is our objective. We may also want to create some indexes to support sort operations.

Now that we've learned how to create a data model, we are in a position to start writing application code. In the next chapter, we'll see how to use CockroachDB SQL in application development frameworks.

Application Design and Implementation

Like all databases, CockroachDB responds to requests from application code. How an application requests and uses data has a huge bearing on application performance and scalability. In this chapter, we'll review how an application should work with CockroachDB, including best practices for coding CockroachDB requests and transactional models.

Because CockroachDB is PostgreSQL wire protocol-compatible, any programming language that supports PostgreSQL can be used with CockroachDB. And in general, the programming idioms and best practices of PostgreSQL apply to CockroachDB. However, because of the distributed nature of CockroachDB, there are some differences in programming styles between CockroachDB and PostgreSQL.

Although you can work with CockroachDB using pretty much any programming language in common use, in this chapter, we'll constrain our discussion to four languages: Go, Java, Python, and JavaScript.

Previously, we showed how to install language drivers for each of these languages. Please refer back to [Chapter 3](#) for instructions on driver installation, or refer to the [CockroachDB documentation](#) for more detailed guidelines, including guidance on how to install drivers for other languages or for alternative drivers.

CockroachDB Programming

CockroachDB is broadly compatible with the universe of SQL-relational databases and particularly compatible with PostgreSQL. However, there are a few unique programming idioms specific to CockroachDB as a result of its distributed nature and transactional consistency model.

In the following sections, we'll review both the general principles involved in coding an application against a CockroachDB server and look at some issues specific to CockroachDB.

Performing CRUD Operations

We provided basic “Hello world” examples for each language back in [Chapter 3](#). Let's extend those examples to perform some nontrivial CRUD operations—Create, Read, Update, Delete.

Programming drivers differ in terms of vocabulary, but they generally adopt a similar grammar. The fundamental operations in a database program are:

- The driver establishes a *connection* object representing a connection to the database server. In this chapter, we'll be creating individual connections, but applications will often use a *connection pool* to manage multiple reusable connections instead.
- The connection object is used to execute SQL statements.
- Some statements return *result sets* that can be used to iterate through tabular output returned by SELECT statements, DML statements that include a RETURNING clause, and some other statements that return results.

Here we see this basic pattern in Java:

```
package chapter06c;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class example1 {

    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            String connectionURL = "jdbc:" + args[0];
            String userName = args[1];
            String passWord = args[2];

            Connection connection = DriverManager.getConnection(
                connectionURL, userName, passWord);
            Statement stmt = connection.createStatement();
            stmt.execute("DROP TABLE IF EXISTS names");
            stmt.execute("CREATE TABLE names (name String PRIMARY KEY NOT NULL)");
            stmt.execute("INSERT INTO names (name) VALUES('Ben'),('Jesse'),('Guy')");

            ResultSet results = stmt.executeQuery("SELECT name FROM names");
```

```

        while (results.next()) {
            System.out.println(results.getString(1));
        }
        results.close();
        stmt.close();
        connection.close();
    } catch (Exception e) {
        e.printStackTrace();
        System.exit(0);
    }
}
}

```

We create a single *connection* object and a single *statement* object, then use the statement to execute multiple SQL commands. When we execute a query, we create a *ResultSet* object that we can use to iterate through results. Finally, we close all these objects.

Note that we can retrieve column values from the *ResultSet* object by position or by name—in the example, we provided the column position, but we could have also specified the column name.

What follows is similar logic for Python. The `cursor()` method of the connection object creates a cursor object that can be used to execute a statement or navigate through a result set:

```

import psycopg2
import sys

def main():

    if ((len(sys.argv)) !=2):
        sys.exit("Error:No URL provided on command line")
    uri=sys.argv[1]

    connection = psycopg2.connect(uri)
    cursor=connection.cursor()
    cursor.execute("DROP TABLE IF EXISTS names")
    cursor.execute("""CREATE TABLE names
                      (name String PRIMARY KEY NOT NULL)""")
    cursor.execute("""INSERT INTO names (name)
                      VALUES('Ben'),('Jesse'),('Guy)""")
    cursor.execute("SELECT name FROM names")
    for row in cursor:
        print(row[0])
    cursor.close()
    connection.close()

main()

```

Here we do the same thing in a Node.js program:

```
const CrClient = require('pg').Client;

async function main() {
  try {
    if (process.argv.length !== 3) {
      console.log(`Usage: node ${process.argv[1]} CONNECTION_URI`);
      process.exit(1);
    }

    const connection = new CrClient(process.argv[2]);
    await connection.connect();

    await connection.query('DROP TABLE IF EXISTS names');
    await connection.query(`CREATE TABLE names
      (name String PRIMARY KEY NOT NULL)`);
    await connection.query(`INSERT INTO names (name)
      VALUES('Ben'),('Jesse'),('Guy')`);

    const data = await connection.query('SELECT name from names');
    data.rows.forEach((row) => {
      console.log(row.name);
    });
  } catch (error) {
    console.error(error.stack);
  }
  process.exit(0);
}

main();
```

We've used the "async/await" style for handling asynchronous database requests. You can also use callbacks or promises if that is your programming style. The [node-postgres driver documentation](#) contains examples of using each of these programming styles.

Finally, let's look at how we'd perform the same task in Go:

```
package main

import (
    "context"
    "fmt"
    "os"

    "github.com/jackc/pgx"
)
```

```

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "Missing URL argument")
        os.Exit(1)
    }
    uri := os.Args[1]
    conn, err := pgx.Connect(context.Background(), uri)
    if err != nil {
        fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
    }
    execSQL(*conn, "DROP TABLE IF EXISTS names")
    execSQL(*conn, "CREATE TABLE names (name String PRIMARY KEY NOT NULL)")
    execSQL(*conn, "INSERT INTO names(name) VALUES('Ben'),('Jesse'),('Guy')")

    rows, err := conn.Query(context.Background(), "SELECT name FROM names")
    if err != nil {
        fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
    }
    defer rows.Close()
    for rows.Next() {
        var name string
        err = rows.Scan(&name)
        fmt.Println(name)
    }
}

func execSQL(conn pgx.Conn, sql string) {
    result, err := conn.Exec(context.Background(), sql)
    if err != nil {
        fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
        os.Exit(1)
    }
    fmt.Fprintf(os.Stdout, "%v rows affected\n", result.RowsAffected())
}

```

We created the `execSQL` function in the Go example to modularize the repetitive error checking involved in the initial SQL statements, though in production code, we would perform error checking independently for each query.

Connection Pools

It's often good practice to create small reusable routines to perform repetitive tasks. If the service requires database access, then it might seem natural to supply each of these routines with a dedicated connection. This has a clear advantage over a single shared connection because it allows for concurrent requests. For instance, imagine that we have a simple web service that we call whenever a new ride is commenced in our Uber-busting ride-sharing app.

We might code the database logic for it as follows:

```
async function newRide(city, riderId, vehicleId, startAddress) {  
  const connection = new pg.Client(connectionString);  
  await connection.connect();  
  const sql = `INSERT INTO movr.rides  
    (id, city,rider_id,vehicle_id,start_address,start_time)  
    VALUES(gen_random_uuid(), $1,$2,$3,$4,now())`;   
  await connection.query(sql, [city, riderId, vehicleId, startAddress]);  
  await connection.end();  
}
```

We don't want to single-thread these requests, so we've given each call its own connection. Unfortunately, creating a connection has a nontrivial overhead. When the database access is simple, the time taken to create and dispose of the connection might dominate overall response time. But we can't run every request through the same connection because that would restrict concurrent queries.

The solution is to use *connection pools*. A connection pool is a set of connections that the application can reuse. You avoid the overhead of constantly creating and destroying connections, and you can control the maximum amount of concurrency hitting the database.

In Node.js, we'd create the pool as follows:

```
const pool = new pg.Pool({  
  connectionString,  
  max: 40  
});
```

We now can change our routine so that it gets connections from the pool:

```
async function newRidePool(city, riderId, vehicleId, startAddress) {  
  const connection = await pool.connect();  
  const sql = `INSERT INTO movr.rides  
    (id, city,rider_id,vehicle_id,start_address,start_time)  
    VALUES(gen_random_uuid(), $1,$2,$3,$4,now())`;   
  await connection.query(sql, [city, riderId, vehicleId, startAddress]);  
  await connection.release();  
}
```

Figure 6-1 illustrates how the two approaches compare for performance. With 40 concurrent requests, a connection pool implementation outperformed the unique connection approach by about 700%. The benefit you get from connection pools will vary depending on the amount of work performed in each connection and the amount of concurrent activity that the application issues. However, it's almost always advisable to use a connection pool instead of a single connection used by all threads or allocating each thread with its own transitory connection.

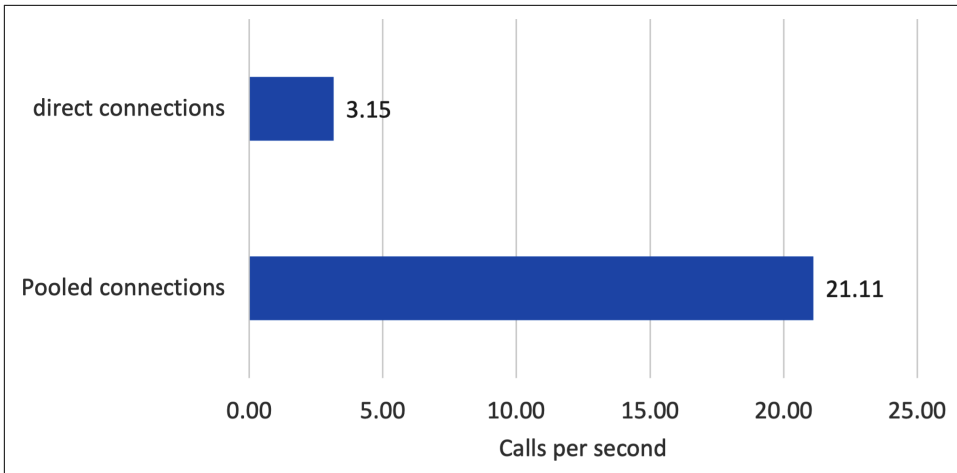


Figure 6-1. Using connection pools to improve concurrency

Connections in the pool can be broken by cluster topology changes or network interruptions. It's advisable to configure “keep alive” settings to periodically recheck the connections. See the [CockroachDB documentation](#) for further details.

Connection Pools and Blocked Connections

Most connection pool implementations will block requests for new connections if all the pooled connections are in use. Therefore, it's important to configure a sufficient number of connections in the pool for the anticipated concurrency. The [CockroachDB documentation](#) suggests configuring four connections for every core in the entire cluster. For instance, if you have a three-node cluster with eight cores in each node, you might configure $3 \times 8 \times 4 = 96$ connections. However, this is just a guideline—the optimal number will depend heavily on the duration of each connection and the amount of idle time each connection experiences as the application performs nondatabase tasks.

Bear in mind that the number of connections you determine should be shared across all of the connection pools that you have configured. So, for example, if you have calculated an ideal number of connections as 96, and you have 4 application servers, then each of these application servers should have 24 connections ($96 / 4$).

It's also critically important to release connections when not in use. For instance, in the Node.js example, the `connection.release()` statement at the end of our function is vital.

In Java, there are a variety of **connection pool options**. Here's an example using the **Hikari framework**:

```
import com.zaxxer.hikari.*;
import java.sql.*;

public class ConnectionPoolDemo {

    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            String connectionURL = "jdbc:" + args[0];
            String userName = args[1];
            String passWord = args[2];

            HikariConfig config = new HikariConfig();
            config.setJdbcUrl(connectionURL);
            config.setUsername(userName);
            config.setPassword(passWord);
            config.addDataSourceProperty("ssl", "true");
            config.addDataSourceProperty("sslMode", "require");
            config.addDataSourceProperty("rewriteBatchedInserts", "true");
            config.setAutoCommit(false);
            config.setMaximumPoolSize(40);
            config.setIdleTimeout(3000);

            HikariDataSource hikariPool = new HikariDataSource(config);
```

This example creates a connection pool with 40 connections using arguments passed in on the command line. Once the pool is created, a connection can be obtained from the pool as follows:

```
Connection connection = hikariPool.getConnection();
```

In the Go pgx driver, we can use the pgxpool package to create and use a connection pool:

```
ctx := context.Background()
config, err := pgxpool.ParseConfig(uri)
config.MaxConns = 40
pool, err := pgxpool.ConnectConfig(ctx, config)
defer pool.Close()
```

We can acquire a connection from the pool as follows:

```
connection, err := pool.Acquire(ctx)
```

The Python driver psycopg2 includes a built-in connection pool that we can easily configure as follows:

```
import psycopg2
from psycopg2 import pool
```

```
def main():

    if ((len(sys.argv)) != 2):
        sys.exit("Error:No URL provided on command line")

    uri = sys.argv[1]
    pool = psycpg2.pool.ThreadedConnectionPool(10, 40, uri)
    # min connections = 10, max connections = 40
```

And we can connect to the pool as follows:

```
connection = pool.getconn()
```

Prepared and Parameterized Statements

Most SQL operations are parameterized—the same statement is run multiple times with different input parameters. For instance, we might have a lookup program that retrieves rider names for a specified ride ID as follows:

```
SELECT u.name FROM movr.rides r
JOIN movr.users u ON (r.rider_id=u.id)
WHERE r.id='ffc3c373-63ec-43fe-98ff-311f29424d8b'
```

We would, of course, execute this SQL many times, each time specifying a different value for the ride ID.

When coding a generic lookup function, it seems natural enough to append the parameter to the SQL statement using string concatenation. For example, in Java, we might be tempted to do something like this:

```
private static String getRiderName(String riderId) throws SQLException {
    Statement stmt = connection.createStatement();
    String sql = " SELECT u.name FROM movr.rides r "
        + " JOIN movr.users u ON (r.rider_id=u.id) "
        + " WHERE r.id='"
        + riderId + "'";
    ResultSet rs = stmt.executeQuery(sql);
    rs.next();
    return (rs.getString("name"));
}
```

However, as natural as this might seem, it represents an extremely poor practice that has both performance and security downsides.

Most significantly, this code is vulnerable to *SQL injection*. For instance, imagine the application could somehow be persuaded to pass the following string to the function:

```
riderName = getRiderName(
    "ffc3c373-63ec-43fe-98ff-311f29424d8b' UNION
    select credit_card from movr.users order by 1,name 'n");
```

The resulting SQL statement would become:

```
SELECT u.name FROM movr.rides r
  JOIN movr.users u ON (r.rider_id=u.id)
 WHERE r.id='ffc3c373-63ec-43fe-98ff-311f29424d8b'
 UNION select credit_card from movr.users order by 1,name
```

And the function would now return credit card numbers as well as rider names.

Of course, the application should prevent such a string from being entered at the UI layer, but creating the vulnerability in the database code is poor practice. The solution is to use *prepared* or *parameterized* statements. As in the preceding Java example, we would declare a `PreparedStatement` as follows:

```
getRiderStmt = connection.prepareStatement(
    "SELECT u.name FROM movr.rides r "
    + " JOIN movr.users u ON (r.rider_id=u.id) "
    + " WHERE r.id=?");
```

The “?” indicates a placeholder for a parameter (sometimes called a *bind variable*). We can call the prepared statement by setting the parameter and executing the statement:

```
getRiderStmt.setString(1, riderId);
ResultSet rs = getRiderStmt.executeQuery();
rs.next();
return (rs.getString("name"));
```

As well as avoiding SQL injection, `prepareStatements` generally execute faster, because CockroachDB can more easily recognize the SQL as one that has already been parsed and can avoid some of the overhead involved with examining what would otherwise appear to be a brand-new statement.

Formally “preparing” statements is a Java practice. In other languages, it’s sufficient to simply call a SQL statement with placeholders and provide the values in the call. For instance, in JavaScript:

```
const sql = `SELECT u.name FROM movr.rides r
  JOIN movr.users u ON (r.rider_id=u.id)
 WHERE r.id=$1`;
const results = await connection.query(sql, ['ffc3c373-63ec-43fe-98ff...']);
console.log(results.rows[0].name);
```

In Python:

```
sql = """SELECT u.name FROM movr.rides r
  JOIN movr.users u ON (r.rider_id=u.id)
 WHERE r.id=%s"""
cursor.execute(sql,('ffc3c373-63ec-43fe-98ff-311f29424d8b',))
row=cursor.fetchone()
print(row[0])
```

And in Go:

```
sql := `SELECT u.name FROM movr.rides r
        JOIN movr.users u ON (r.rider_id=u.id)
        WHERE r.id=$1`
rows, err := conn.Query(ctx, sql, "ffc3c373-63ec-43fe-98ff-311f29424d8b")
rows.Next()
var name string
err = rows.Scan(&name)
fmt.Println(name)
```

Batch Inserts

It's common for an application to insert multiple rows of data in a single logical operation.

When you have an array of values to insert, it can seem natural to simply insert the values in a loop, as in this Python example:

```
for value in arrayValues:
    cursor.execute("INSERT INTO insertTestP1(id,x,y) VALUES ($1,$2,$3)",value)
```

It's inefficient to insert large amounts of data one row at a time—each insert will require a network round trip, and there may be transactional implications if we want all of the rows committed in a single transaction (because the single row inserts will take longer, the chance of a transaction conflict and subsequent retries will be magnified).

SQL allows multiple VALUES to be included in a single operation, such as:

```
INSERT INTO insertTest(id,x,y)
VALUES (3,'x',1) ,
       (4,'y',2) ,
       (5,'x',5)
```

So we could, if necessary, dynamically construct an INSERT statement to insert an array of data in a single operation. For instance, in Python, the following code will generate and execute an INSERT statement to insert an array of arbitrary length:

```
sql="INSERT INTO insertTestP(id,x,y) VALUES"
valueCount=0
for value in arrayValues:
    if valueCount>0:
        sql=sql+", "
    sql=sql+"(%d,'%s',%d)" % value
    valueCount+=1
cursor.execute(sql)
```

Note that this formulation is vulnerable to SQL injection. In the `psycpg2` extras package, there is an `execute_values` extras helper function that simplifies the coding required and reduces the chance of SQL injection:

```
from psycpg2 import extras

<snip>

extras.execute_values(cursor,
    "INSERT INTO insertTestP1(id,x,y) VALUES %s",
    arrayValues)
```

The performance improvements obtained with batch inserts are dramatic. [Figure 6-2](#) illustrates the improvement.

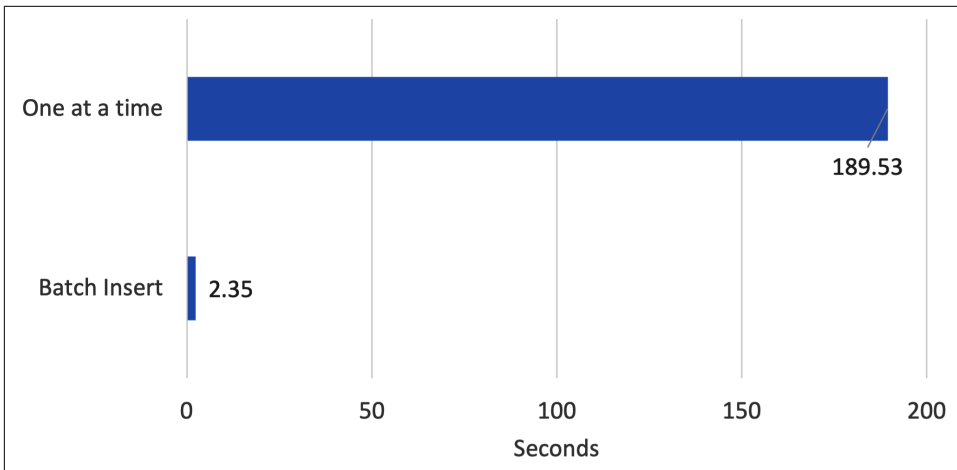


Figure 6-2. Improvement obtained by inserting rows in an array

JDBC includes `addBatch` and `executeBatch` methods that allow you to prepare inserts one at a time and then submit all the inserted values in a single operation. This avoids the need to concatenate a huge `VALUES` list and allows us to use formal parameters. Here's an example of the `addBatch` and `executeBatch` methods:

```
String sql="INSERT INTO insertTest(id,x,y) VALUES (?,?,?)";
PreparedStatement InsertStmt = connection.prepareStatement(sql);

for (int arrayIdx = 1; arrayIdx < arrayCount; arrayIdx++) {
    InsertStmt.setInt(1, idArray.get(arrayIdx));
    InsertStmt.setString(2, xArray.get(arrayIdx));
    InsertStmt.setInt(3, yArray.get(arrayIdx));
    InsertStmt.addBatch();
}

InsertStmt.executeBatch();
```

We use `setInt` and `setString` methods to supply values to the prepared statement as usual, but instead of executing, we use `addBatch` to add them to the batch of rows to be inserted. When we are ready, we call `executeBatch` to add all the rows in a single operation.

The JDBC `addBatch` method has minimal effect unless the `rewriteBatchedInserts` property is set to true. You can set `rewriteBatchedInserts` when establishing a connection:

```
Class.forName("org.postgresql.Driver");
String connectionURL = "jdbc:" + args[0];
String userName = args[1];
String passWord = args[2];
Properties props = new Properties();
props.setProperty("user", userName);
props.setProperty("password", passWord);
props.setProperty("rewriteBatchedInserts", "true");

Connection connection = DriverManager.getConnection(connectionURL, props);
```

If you're a Spring developer, consider using `JdbcTemplate`. It wraps JDBC, making your code easier to read and maintain.

The Node.js library does not include any direct support for batch inserts. However, we can use the `pg-format` package to create SQL statements that contain multiple VALUES from an array:

```
const pg = require('pg');
const format = require('pg-format');

async function main() {
  const connection = new pg.Client(connectionString);

  const sql = format('INSERT INTO insertTestP2(id,x,y) VALUES %L',
    arrayData);

  await connection.query(sql);
}
```

The Go `pgx` library provides a `Batch` type and a `SendBatch` method to use executing batch operations:

```
batch := &pgx.Batch{}

for _, v := range arrayValues {
  batch.Queue(
    "INSERT INTO insertTest(id, x, y) VALUES ($1, $2, $3)",
    v.id, v.x, v.y,
  )
}

results := db.SendBatch(context.Background(), batch)
defer results.Close()
```

Pagination of Results

Some applications need to return data in batches. For instance, an online application might want to return lists of information in pages—similar to the pages of results that you might get from a Google search.

From a syntactic point of view, most of the drivers allow you to scroll through rows one at a time. However, in many cases, you are still bringing the entire result set into program memory before retrieving the first row. For example, the Python `psycopg2` driver provides methods access to the entire result set (`fetchall()`), a selection of rows (`fetchmany()`), or a single row (`fetchone()`). However, regardless of the method called, the entire result set is always transferred from the database to the application.

The JDBC driver supports client-side cursors that allow for data to be efficiently pulled from the database in batches. The size of the batches is controlled by the `setFetchSize()` method of the `Statement` object. By default, `fetchSize` is set to 0, which results in *all* the rows being pulled into the application before the first row can be processed.

We can adjust the `fetchSize` if we want to pull only a few rows in each batch as follows:¹

```
Statement stmt = connection.createStatement();
stmt.setFetchSize(100);
results = stmt
    .executeQuery("SELECT post_timestamp, summary "
        + " FROM blog_posts "
        + " ORDER BY post_timestamp DESC ");
for (int ri = 0; ri < 10; ri++) {
    if (results.next())
        System.out.println(results.getString("SUMMARY"));
}
```

You don't have to change your loop logic when you change `setFetchSize()`, but under the hood the PostgreSQL driver will pull rows in batches of `setFetchSize()` size. **Figure 6-3** shows that this can be very effective if we want to optimize for fetching the first few rows.

In Java, the `fetchSize` parameter gives us an adequate solution for pagination. However, the other drivers either do not support effective client-side cursors or do so in a way that results in some unnatural coding styles.

¹ Note that the `setFetchSize()` call has no effect if `setAutoCommit` is set to `true`.

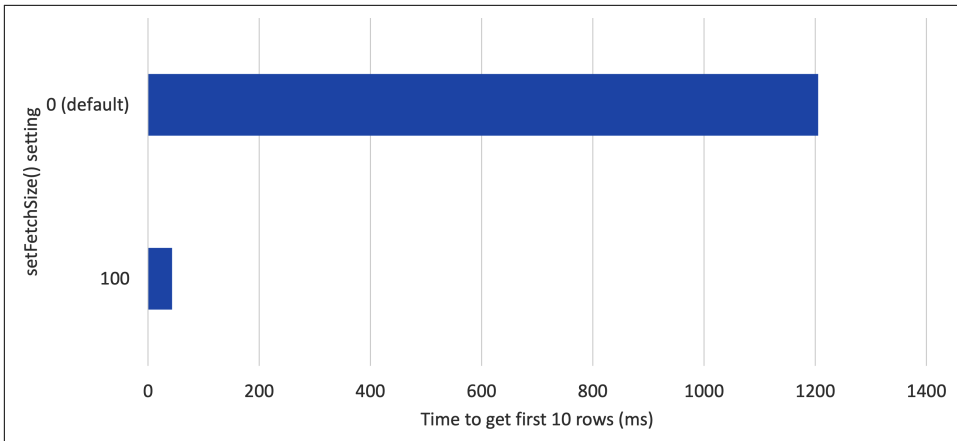


Figure 6-3. Reducing `setFetchSize` to improve fetch time for first rows

The recommended language-independent pattern for navigating pages of data is referred to as “KeySet pagination.” However, before considering this technique, let’s look at a “natural” solution that has very poor performance characteristics.

Let’s suppose that we are returning pages of blog posts from the following query:

```
SELECT post_timestamp, summary
FROM blog_posts ORDER BY post_timestamp DESC
```

We have a covering index on `POST_TIMESTAMP`, and this index stores the `summary` column, so we can retrieve rows efficiently in order. We want to create some code to display blog posts in order of posting, with a certain number of posts per “page.”

SQL supports `OFFSET` and `LIMIT` functions, allowing us to jump ahead and to limit the number of rows returned. This might seem an ideal solution for pagination; we can use the `OFFSET` to jump to the page we want and `LIMIT` to restrict the number of results to just that page. In Python, we might code this as follows:

```
def getPage(startIndex, nEntries):
    # Don't do this!
    cursor=connection.cursor()
    sql="""SELECT post_timestamp, summary
           FROM blog_posts ORDER BY post_timestamp DESC
           OFFSET %s LIMIT %s"""
    cursor.execute(sql,(startIndex,nEntries))
    return cursor.fetchall()
```

The problem with this approach is that OFFSET requires us to process all of the data up to and including the first row of interest. So, for instance, if we specify an offset of one million, we have to retrieve and discard all the data prior to the one-millionth row. Each page will take longer to retrieve than the last.

The correct method is to move through the result set in key order so that we can efficiently retrieve the rows using index ranges. Of course, this approach absolutely requires that we have an index on the WHERE clause condition, and ideally, this index should be a covering index that includes the SELECT list columns as well. So our Python method would look like this:

```
def getPageKeySet(startTimeStamp,nEntries):
    cursor=connection.cursor()
    sql="""SELECT post_timestamp, summary
           FROM blog_posts
           WHERE post_timestamp< %s
           ORDER BY post_timestamp DESC
           LIMIT %s"""
    cursor.execute(sql,(startTimeStamp,nEntries))
    return cursor.fetchall()
```

We would need to track the oldest blog timestamp from each page retrieved and forward that to the next invocation of the method.

In some circumstances, it might be important to ensure that the pages of data are consistent. Since each invocation of the method occurs at a different time, each “page” of data will reflect the database at a different time. If this is a concern, then AS OF SYSTEM TIME can be used to ensure that each page of data reflects the state of the database as of a specific time:

```
def getPageKeySetST(startTimeStamp,nEntries,systemTime):
    cursor=connection.cursor()
    sql="""SELECT post_timestamp, summary
           FROM blog_posts
           AS OF SYSTEM TIME %s
           WHERE post_timestamp< %s
           ORDER BY post_timestamp DESC
           LIMIT %s
           """
    cursor.execute(sql,(systemTime,startTimeStamp,nEntries))
    return cursor.fetchall()
```

Figure 6-4 illustrates how the use of OFFSET and LIMIT results in increasing overhead as we retrieve each page of information. In contrast, the KeySet pagination pattern returns each page in the same amount of time.

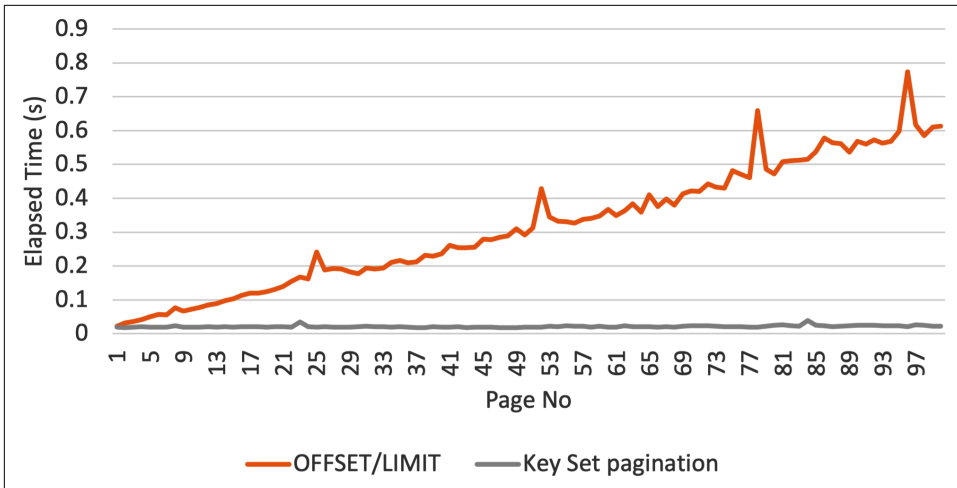


Figure 6-4. KeySet pagination provides better scalability than OFFSET/LIMIT

Projections

In relational database parlance, “projection” refers to the selection of a subset of columns from a table (or *attributes* from an *entity*). In practice, a projection is represented by the list of columns in a SELECT clause.

While SELECT accepts a wildcard projection (*), this should almost never be used in production code because it results in unnecessary transport of columns from the database to the application. Using * can seem like a handy programming shortcut, but it can have severe performance penalties when processing large result sets. Furthermore, it can cause errors if the structure of the table is changed.

For instance, let’s say we are retrieving a list of user IDs and blog post dates to populate a dashboard or to perform some other real-time diagnostic. The following code might seem acceptable:

```
ResultSet results = stmt.executeQuery(
    "SELECT * FROM blog_posts");
while (results.next()) {
    java.sql.Timestamp postTimestamp =
        results.getTimestamp("POST_TIMESTAMP");
    Integer userid = results.getInt("USERID");
    plotPost(userid, postTimestamp);
}
```

However, a couple of coding seconds saved in omitting the column names costs the application dearly. Every time this code is executed, it retrieves not only the user ID and timestamp but also the potentially very large blog post text. As a result,

each network packet can hold less data, and the number of network round trips is magnified. If we add a projection:

```
results = stmt.executeQuery("""
    SELECT userid, post_timestamp
    FROM blog_posts
    """)
);
```

then elapsed time is reduced dramatically. **Figure 6-5** illustrates the elapsed time savings for a 10-million row result set from a remote cluster.

Of course, the absolute time saved will depend on the total row size versus the size of the projection and the network latency between the application and the server. Furthermore, this degradation only kicks in when we pull more rows from the database than can fit in a single network packet. For single-row retrievals, the overhead is negligible.

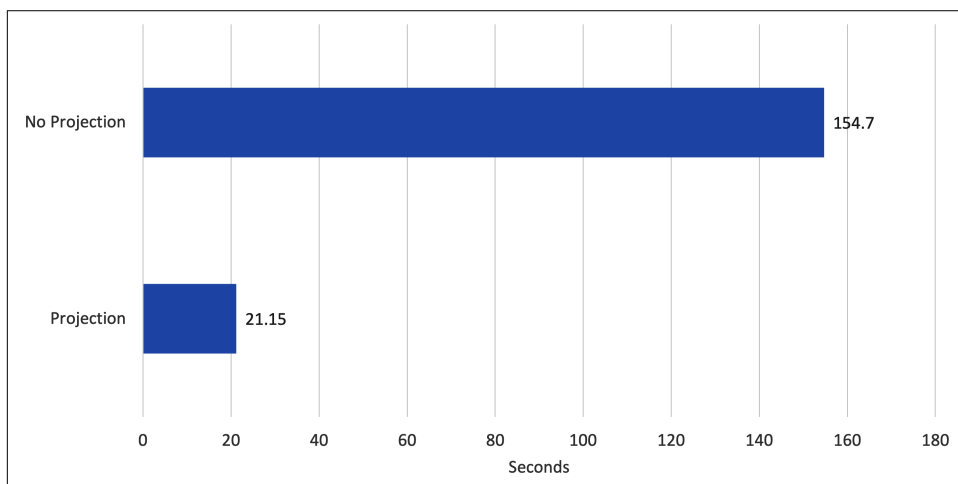


Figure 6-5. Improvement obtained by adding a projection to a query

Client-Side Caching

The best way to optimize a database request is to not send it at all. No matter how carefully we optimize the database—adding indexes, memory, fast disks, etc.—database requests are blocking operations that can never be made as fast as local computation. For most applications, database accesses are the slowest operations performed and the most critical component of application response time.

One of the most effective ways of avoiding unnecessary database calls is to cache frequently accessed static data in application code. Avoid asking the database repeatedly for the same data unless there's a chance that the data will change.

For instance, let's say that we have a function to return a user's name given a `userId`:

```
func getName(userId string) string {
    conn, err := pool.Acquire(context.Background())
    defer conn.Release()
    if err != nil {
        fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
    }
    sql := `SELECT name FROM movr.users WHERE id=$1`
    rows, err := conn.Query(context.Background(), sql, userId)
    defer rows.Close()
    if err != nil {
        fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
    }
    if !rows.Next() {
        return "Invalid userId"
    } else {
        var name string
        rows.Scan(&name)
        return (name)
    }
}
```

It's relatively simple to extend this function with a client-side cache. We just need to declare and initialize a map structure:

```
var userCache map[string]string

userCache = make(map[string]string)
```

Now in our function, we check this map to see if we can find the user's name. Only if the name does not exist in the cache do we go to the database:

```
func getCachedUserName(userId string) string {
    name, nameFound := userCache[userId]
    if !nameFound {
        conn, err := pool.Acquire(context.Background())
        defer conn.Release()
        if err != nil {
            fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
        }
        fmt.Println("cache miss")
        sql := `SELECT name FROM movr.users WHERE id=$1`
        rows, err := conn.Query(context.Background(), sql, userId)
        defer rows.Close()
        if err != nil {
            fmt.Fprintf(os.Stderr, "CockroachDB error: %v\n", err)
        }
        if !rows.Next() {
            return "Invalid userId"
        } else {
            rows.Scan(&name)
        }
    }
}
```

```

        userCache[userId] = name
    }
}
return (name)
}

```

The performance improvements obtained by not going to the database are greater than any tuning of the database accesses themselves, since we can never make a database access a zero-cost activity. However, bear in mind the following:

- Caches consume memory on the client program. In many environments, memory is abundant, and the tables considered for caching are relatively small. However, for large tables and memory-constrained environments, the implementation of a caching strategy could actually degrade performance by contributing to memory shortages in the application layer or client.
- If the table being cached is updated during program execution, then the changes may not be reflected in your cache unless you implement some sophisticated synchronization mechanism. For this reason, local caching is best performed on static tables.

Managing Transactions

Transactions provide an important mechanism for ensuring that related modifications succeed or fail as a unit. We discussed the internals of CockroachDB transactions back in [Chapter 2](#).

The basics of programming transactions are common across a wide variety of SQL databases and even some non-SQL systems. A transaction is commenced with a `BEGIN` statement. Multiple SQL statements are executed within the transaction scope, and then all the changes are made permanent with the `COMMIT` statement. If an error is encountered during the transaction, all of the transaction's work can be abandoned with a `ROLLBACK` statement.

Example 6-1 shows a relatively simple transaction—implemented in Node.js—that transfers money from one account to another, after first checking that there are sufficient funds.

Example 6-1. Example of a simple transaction

```

try {
  await connection.query('BEGIN TRANSACTION');
  // Check for sufficient funds
  const results = await connection.query(
    'SELECT balance FROM accounts WHERE id=$1',
    [fromId]
  );
}

```

```

const fromBalance = results.rows[0].balance;
if (fromBalance < transferAmt) {
  throw Error('Insufficient funds');
}
// Transfer the money
await connection.query(`UPDATE accounts SET balance=balance-$1
                        WHERE id=$2`,
  [transferAmt, fromId]);
await connection.query(`UPDATE accounts SET balance=balance+$1
                        WHERE id=$2`,
  [transferAmt, toId]);
await connection.query('COMMIT');
success = true;
} catch (error) {
  console.error(error.message);
  connection.query('ROLLBACK');
  success = false;
}

```

If you run this code in parallel, you'll find that some percentage of the transactions fail with an error something like this:

```

restart transaction: TransactionRetryWithProtoRefreshError:
WriteTooOldError: write at timestamp 16412

```

Transaction Retry Errors

In databases that default to lower levels of transaction isolation (PostgreSQL, for instance), this transaction would almost always succeed, perhaps failing only if there was a database outage. However, when using `SERIALIZABLE` transaction isolation (which is the default in CockroachDB and an option in other databases), there is a good chance of transaction failure. If a concurrent transaction modifies the same table row between the time our transaction commences and the time we attempt to modify that row, then we will encounter a `TransactionRetryWithProtoRefreshError: WriteTooOldError` (we'll call this a *transaction retry* error for the sake of brevity).

Types of Transaction Retry Errors

The `WriteTooOldError` type of transaction retry is one of a family of errors—including `RETRY_SERIALIZABLE` and others that indicate that a retry can and probably should be attempted. While the various errors have different—and sometimes quite complex—underlying causes,² they all issue the same 40001 error code.

² See the [Cockroach documentation](#) for a complete discussion.

Figure 6-6 illustrates a possible sequence of events in two concurrent transactions that would lead to a transaction retry error.

The chance of receiving a transaction retry error depends on the chance of two transactions colliding on the same row. In one test, the percentage of retries varied from fewer than 1% if there were 10,000 distinct accounts involved to more than 75% when there were just 10 accounts.³

However, whatever the probability of encountering a transaction retry error, the possibility exists, and your application code should be able to cope with these expected error scenarios.

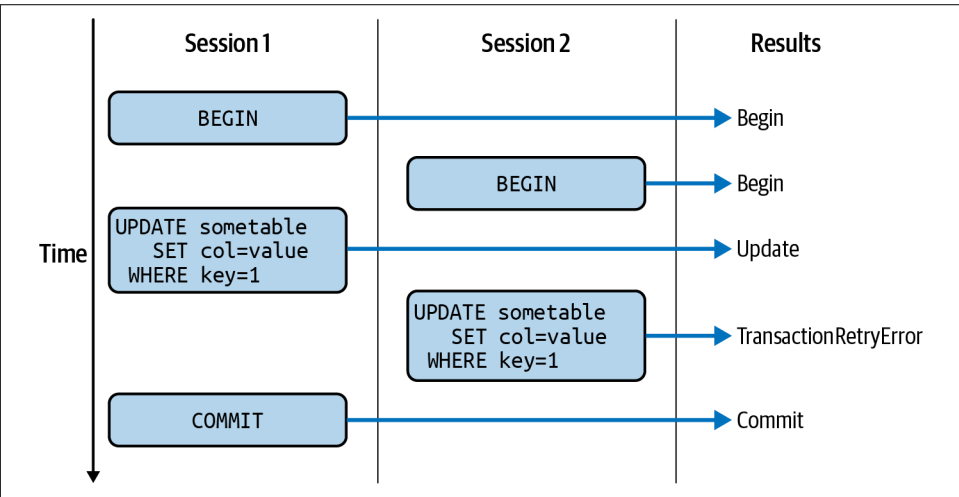


Figure 6-6. Transaction retry error scenario

When running CockroachDB with READ COMMITTED isolation, transaction retries resulting from serialization errors do not need to be handled by application code. Lock contention will still affect transactions vying for the same data, but rather than immediately receiving a retry error, a transaction will wait. This behavior can be seen in Figure 6-7, where two transactions seek to update the same row. Instead of encountering a transaction retry error as before, Session 2 will unblock once Session 1 commits.

³ The simulation ran 100 concurrent threads randomly executing a transaction 10 times per second.

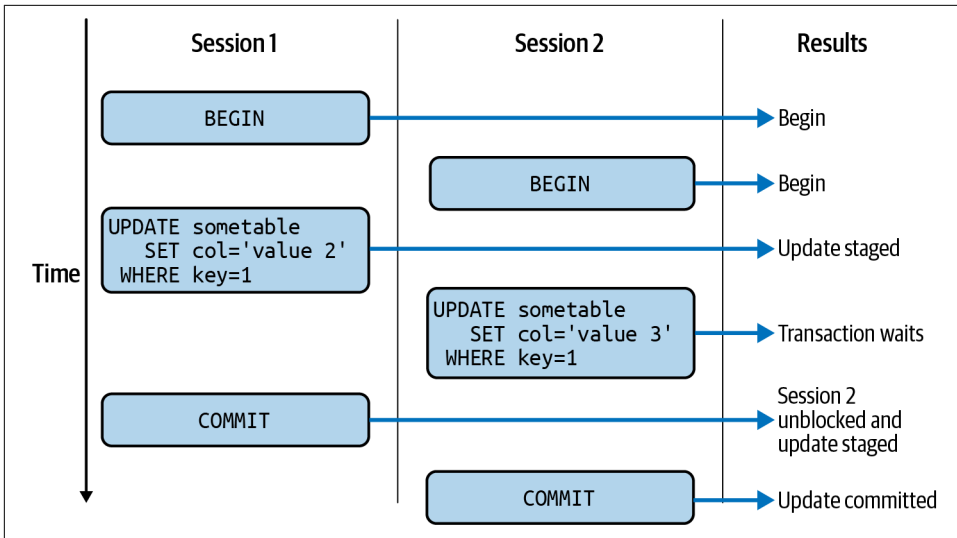


Figure 6-7. Transaction retry scenario under *READ COMMITTED* isolation

Implementing Transaction Retries

The relatively obvious way to handle retry errors is to do exactly what the error code suggests—retry the transaction. When the transaction retry error is encountered, issue a `ROLLBACK` command to discard the work done so far in the transaction and try the transaction again. In the following example, we add some logic to the Node.js method in [Example 6-1](#) to retry the transaction when necessary:

```
let retryCount = 0;
let transactionEnd = false;
while (!transactionEnd) {
  if (retryCount++ >= maxRetries) {
    throw Error('Maximum retry count exceeded');
  }
  try {
    await connection.query('BEGIN TRANSACTION');
    // Check for sufficient funds
    const results = await connection.query(
      'SELECT balance FROM accounts where id=$1',
      [fromId]
    );
    const fromBalance = results.rows[0].balance;
    if (fromBalance < transferAmt) {
      throw Error('Insufficient funds');
    }
    // Transfer the money
    await connection.query(
      `UPDATE accounts SET balance=balance-$1
      WHERE id=$2`,
```

```

        [transferAmt, fromId]
    );
    await connection.query(
        `UPDATE accounts SET balance=balance+$1
        WHERE id=$2`,
        [transferAmt, toId]
    );
    await connection.query('COMMIT');
    success = true;
    console.log('success');
} catch (error) {
    if (error.code == '40001') { // Transaction retry error
        console.log(error.code, retryCount);
        connection.query('ROLLBACK');
        // Exponential backoff
        const sleepTime = (2 ** retryCount) * 100
            + Math.ceil(Math.random() * 100);
        await sleep(sleepTime);
    } else {
        console.log('aborted ', error.message);
        transactionEnd = true;
    }
}
}

```

If this method encounters an error 40001—the retry transaction code—it issues a ROLLBACK, waits for a short time, and then tries the transaction again.

In this implementation, the sleep time increases exponentially as the number of retries increases. This is done to avoid a situation in which transactions “thrash” on a resource. This exponential backoff strategy tends to reduce the load on a busy system, but it can result in some high transaction waits for “unlucky” transactions. Furthermore, when we retry transactions, there is no guarantee that updates will succeed in the order in which they were originally submitted. Transactions that were submitted first may succeed only after transactions that were submitted at a later time commit.

Automatic Transaction Retries

The logic shown in the previous section can be implemented in any language.⁴ However, some drivers implement this logic for you transparently:

- If a single statement that returns less than 16 KB of output (a single UPDATE, for instance, with no RETURNS clause) encounters a 40001, then CockroachDB will automatically retry the statement with no intervention required on your part. This logic applies to both implicit transactions (without a BEGIN statement)

⁴ See the [Cockroach documentation](#) for a generic implementation.

and explicit transactions with only a single statement. The retries will continue indefinitely unless the session variable `statement_timeout` is specified.

- The Go DBTools library includes a transaction retry handler for Go transactions. You pass a set of operations to the transaction handler, which will automatically retry transactions with a **configurable retry limit and delay**. The **cockroach-go** project contains similar helper functions maintained by the CockroachDB team.
- Many object-relational mapping frameworks—SQLAlchemy for Python, for instance—will automatically retry transactions for you transparently. See the **CockroachDB documentation** for further details.

Why Can't CockroachDB Handle All Transaction Retries?

Coding for transaction retries can seem tedious. Since CockroachDB retries transactions automatically in some circumstances, why can't CockroachDB handle *all* retries automatically?

The short answer is that in many circumstances CockroachDB cannot determine the logical connection between different statements in a transaction. For instance, in **Example 6-1**, CockroachDB cannot know how the `SELECT` statement before the `UPDATES` might affect the `UPDATE` logic. Only when the transaction is completely unambiguous—which only really happens when there's just a single statement in the transaction—can CockroachDB safely perform a retry.

Using `FOR UPDATE` to Avoid Transaction Retry Errors

Performing transaction retries has some significant downsides. First, they are wasteful since work in the transaction that is done before the retry is discarded. Second, they introduce a delay in transaction processing that is unpredictable or even unnecessary. It's hard to know how long to sleep between transaction retries, and exponential backoffs can lead to some extreme waits. Finally, transaction retries result in nondeterministic behaviors. Transactions will not necessarily be applied to the database in the order in which they are submitted by the application; even under identical workloads, differences in outcomes will be observed.

The alternative to the transaction retry approach is to “lock” the rows required at the beginning of the transaction with a `FOR UPDATE` statement. `FOR UPDATE` is a blocking statement, and once it returns, your transaction has the update rights over the rows concerned.

Here's our sample code with the FOR UPDATE logic:

```
try {
  await connection.query('BEGIN TRANSACTION');
  // Check for sufficient funds (and lock row)
  const results = await connection.query(
    `SELECT balance FROM accounts where id=$1
     FOR UPDATE`,
    [fromId]
  );
  const fromBalance = results.rows[0].balance;
  if (fromBalance < transferAmt) {
    throw Error('Insufficient funds');
  }
  // Lock second row
  await connection.query(
    `SELECT balance FROM accounts where id=$1
     FOR UPDATE`,
    [toId]
  );
  // Transfer the money
  await connection.query(
    `UPDATE accounts SET balance=balance-$1
     WHERE id=$2`,
    [transferAmt, fromId]
  );
  await connection.query(
    `UPDATE accounts SET balance=balance+$1
     WHERE id=$2`,
    [transferAmt, toId]
  );
  await connection.query('COMMIT');
  success = true;
  console.log('success');
} catch (error) {
  console.error(error.message);
  connection.query('ROLLBACK');
  success = false;
}
```

By locking the ACCOUNTS rows with FOR UPDATE before actually issuing UPDATE statements, we avoid any chance of a transaction retry being issued. However, in a production implementation, it is probably advisable to include a transaction retry error handler in any transaction, even one that attempts to avoid a retry using FOR UPDATE, because retry errors can still occur due to clock synchronization or other issues. For instance, the preceding code is vulnerable to a deadlock condition if simultaneous transfers between two accounts in opposite directions collide—we'll look more at deadlocks in a couple of pages.

Optimistic Versus Pessimistic Transaction Design

The two patterns for transactions we've looked at here—retry handling versus FOR UPDATE locking—have historically been referred to as *optimistic* and *pessimistic* transaction models.

In the optimistic transaction model, we feel it is unlikely that there will be a conflicting update that will cause a transaction to abort. Therefore, we don't "pre-lock" data and rely on transaction retries to handle any conflicts that might occur.

In the pessimistic model, we are quite worried about transaction conflicts, so we preemptively lock rows that might come into conflict.

Neither model is superior—it really does depend on how likely row-level transaction conflicts are. Don't choose one or the other based on your emotional disposition. Think carefully about the likelihood of conflicts—benchmark if necessary—and act accordingly.

Reducing Contention by Eliminating Hot Rows

The most significant cause of transaction retries is contention for a small number of "hot" rows. Hot rows are those that are frequently changed by multiple database sessions. Hot rows often indicate design flaws in the data model. For instance, if we decided to maintain running totals of account transfers per day, we might end up updating a single row after every transaction.

The use of embedded arrays or JSON data types can also create these sorts of issues. For example, it might seem convenient to maintain an array of measurements in a JSON document:

```
SELECT * FROM latest_measurements;  
  
{  
  "measurements": [{  
    "locationid": "8a90ec6e-370a-4f90-bdc7-2f4bcdd381c2",  
    "measurement": "32.6933968058154"  
  }, {  
    "locationid": "ccc240a0-3322-4a02-9538-23e0d98a39e5",  
    "measurement": "1.1379426982748297"  
  }, {  
    "locationid": "15f41b26-f1a7-4d35-a88b-9f6bce022c7b",  
    "measurement": "39.21261847039683"  
  }, <snip>  
  {  
    "locationid": "f9b422d5-e9fd-44e7-8db3-35a243e45a95",  
    "measurement": "25.66958037632363"  
  }, {  
    "locationid": "abdd31e7-b553-4798-896d-be492b11dbf1",
```

```

        "measurement": "41.09557231178944"
    }]
}

```

This design might result in quick retrieval time but has now created a super hotspot. Keeping every location in its own row would be superior. Remember—denormalization should generally serve the goal of improving performance; beware of denormalizations that actually reduce throughput.

Reducing Transaction Elapsed Time

The longer a transaction runs, the greater the chance of contention with another transaction. Therefore, you should always move any time-consuming application logic—and certainly any human intervention—outside of the transaction. The code between the BEGIN and COMMIT (or ROLLBACK) statements should include only code critical to the transaction itself. For instance, the following is a variation on the transaction originally introduced in [Example 6-1](#):

```

await connection.query('BEGIN TRANSACTION');
// Check for sufficient funds (and lock row)
const results = await connection.query(
    `SELECT balance FROM accounts where id=$1
    FOR UPDATE`,
    [fromId]
);
const fromBalance = results.rows[0].balance;
if (fromBalance < transferAmt) {
    throw Error('Insufficient funds');
}
// Lock second row
await connection.query(
    `SELECT balance FROM accounts where id=$1
    FOR UPDATE`,
    [toId]
);

// Perform anti-money laundering check
await performAMLCheckViaRESTCall(txnDetails);

// Transfer the money
await connection.query(
    `UPDATE accounts SET balance=balance-$1
    WHERE id=$2`,
    [transferAmt, fromId]
);
await connection.query(
    `UPDATE accounts SET balance=balance+$1
    WHERE id=$2`,
    [transferAmt, toId]
);
await connection.query('COMMIT');

```

The `performAMLCheckViaRESTCall()` performs an anti-money-laundering (AML) check via a REST call—which might take a few seconds in the worst-case scenario. We issued this call after issuing a `FOR UPDATE` statement. While this might make sense from a logical point of view (not checking with the AML authorities until we are sure the transaction will go through), the extra duration of the `FOR UPDATE` locks will reduce throughput significantly. It would be better from a performance point of view to perform the AML check before commencing the transaction.

A similar effect can occur in retry logic. If unnecessary time-consuming statements occur within a transaction with retry logic, then the chance of a retry is increased with a consequent decrease in throughput.

Reordering Statements

The ordering of DML statements within a transaction can have a big impact on contention. Generally, the statement most likely to involve contention should be placed first in the transaction sequence. Placing the contentious statement first has several good implications:

- CockroachDB can automatically retry the first statement in a transaction transparently, without requiring explicit handling.
- If the transaction fails, it will fail before the execution of other statements. The execution and rollback of these other statements will involve overhead on the server.

Although the impact of the reordering statement will vary from case to case, in general, moving the most contentious statements earlier in the transaction sequence will be worthwhile.

Time Travel Queries

If a transaction attempts to read data that has a higher timestamp than the transaction start time, then a retry error will occur. For read operations, we can avoid these errors by using `AS OF SYSTEM TIME`.

This can be particularly important for read-only transactions in which you want consistent results from multiple `SELECT` statements but don't need these results to be completely up-to-date with respect to the system time. You can include the `AS OF SYSTEM TIME` in a `SELECT` statement or in a `BEGIN` statement. If included in the `BEGIN` statement, then the transaction is a read-only transaction with read consistency as of the timestamp provided.

For instance, here we read from `rides` and `user_ride_counts` consistently. Without the `AS OF SYSTEM TIME` clause, a concurrent write to `rides` or `user_ride_counts` might cause the transaction to fail:

```
cursor.execute("BEGIN AS OF SYSTEM TIME '-10s' ")
top10cities=cursor.execute('''SELECT city,count(*)
                             FROM movr.rides GROUP BY city
                             ORDER BY 2 DESC LIMIT 10''')
top10users=cursor.execute('''SELECT name, rides
                             FROM movr.user_ride_counts
                             ORDER BY rides DESC LIMIT 10''')
cursor.execute('COMMIT')
```

Ambiguous Transactions Errors

In a distributed system, some errors can have ambiguous results. For example, if you receive a connection closed error while processing a `COMMIT` statement, you cannot tell whether the transaction was successfully committed or not. These errors are possible in any database, but CockroachDB is somewhat more likely to produce them because ambiguous results can be caused by failures between cluster nodes. These errors are reported with the PostgreSQL error code `40003(statement_completion_unknown)`.

Ambiguous errors can be caused by nodes crashing, network failures, or timeouts. Note that ambiguity is possible for only the last statement of a transaction (`COMMIT` or `RELEASE SAVEPOINT`) or for statements outside a transaction. If a connection drops during a transaction that has not yet tried to commit, the transaction will definitely be aborted.

In general, you should handle ambiguous errors the same way as connection closed errors. If your transaction is idempotent—capable of being executed multiple times with the same result—it's safe to retry it on ambiguous errors. `UPSERT` operations are typically idempotent (providing there are no dynamically allocated column values), and other transactions can be written to be idempotent by verifying the expected state before performing any writes. Increment operations such as `UPDATE my_table SET x=x+1 WHERE id=$1` are typical examples of operations that cannot easily be made idempotent. For a detailed discussion of idempotency keys, see [@brandur's blog post](#).

If your transaction is not idempotent, then you should decide to retry based on whether it would be better for your application to apply the transaction twice or return an error to the user.

Deadlocks

Transaction retry errors can also occur with `FOR UPDATE` if two sessions lock resources required by the other. For instance, in [Figure 6-8](#), we see that two sessions have each locked resources that are required by the other. Session 1 has a lock on `id=1` and wants to lock `id=2`. Session 2 has a lock on `id=2` and wants to lock `id=1`. This situation can never resolve, so CockroachDB will terminate one of the sessions, and that session will have to retry the transaction.

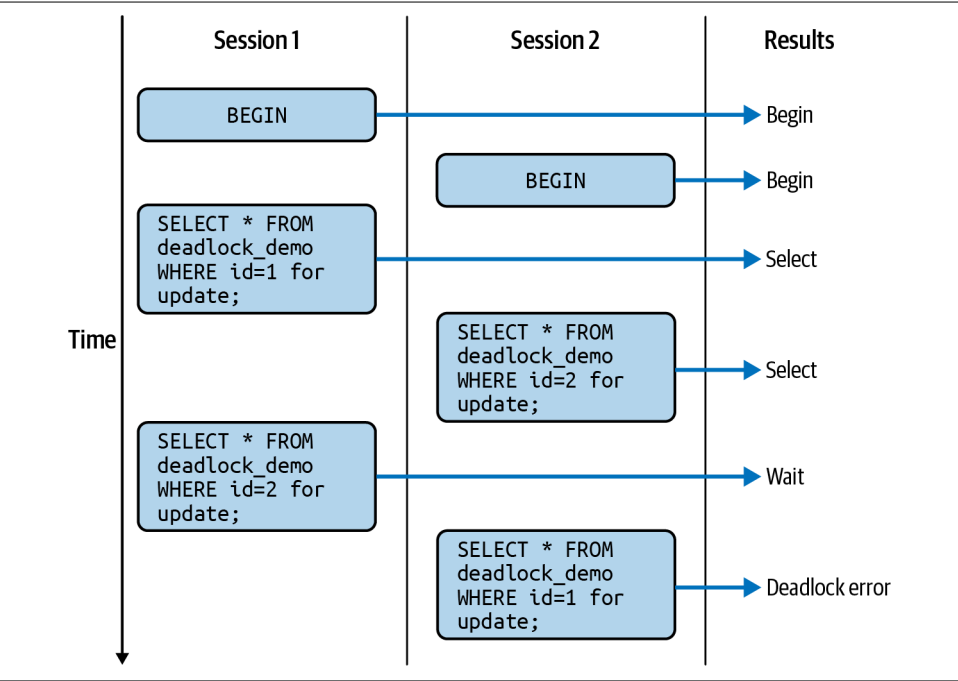


Figure 6-8. Deadlocks occur when two sessions each lock a resource required by the other

Deadlocks are less likely if transactions always lock resources in a specific order. However, we cannot be completely sure that a deadlock will never occur in a complex application. The solution is to ensure that all critical transactions have retry logic.

If the transactions in a deadlock scenario have different priorities, CockroachDB allows the transaction with higher priority to abort the other, which must then retry. If the transactions have the same priority, one is selected randomly to abort.

Transaction Priorities

Transactions can be associated with priorities, and—in the event of a conflict—a higher-priority transaction will be favored over a lower-priority transaction.

Adjusting transaction priorities can ensure that critical transactions don't get blocked by lower-priority work. However, these decisions should be made very deliberately. It's possible for a lower-priority transaction to be deferred indefinitely on a busy system, which might be more undesirable than a delay for the high-priority workload.

Transaction priorities can be set using the `SET TRANSACTION` command. By default, all transactions have the `NORMAL` priority.

Summary of Transaction Approaches

We've looked at a lot of transaction execution patterns, and you may be feeling that the "correct" way to program a CockroachDB transaction is not clear. Fair enough—there is indeed more than one way to do it. However, the following guidelines are generally applicable:

- Your critical transactions should include some form of retry logic. Even if you avoid retry errors using every technique we have explored, there's still a chance of a retry error due to contention on internal resources.
- Transactions should be kept as short in scope and duration as possible. Any statement that is not needed within the transaction should be moved out of scope.
- The DML most likely to cause a conflict should be placed first in the transaction.
- If preserving order in the queue is important, use the `FOR UPDATE` statement to lock resources before modifying them. This pessimistic locking pattern will not always be faster, but it will tend to ensure that transactions get processed in the order in which they are received.
- For read-only transactions, consider performing "time travel" queries with `AS OF SYSTEM TIME` to avoid transaction retries.
- When possible, batching all the SQL statements in the transaction into a single request can improve performance and simplify retry logic. Be mindful of the possibility of SQL injection in these batched routines.

Working with ORM Frameworks

Object-relational mapping (ORM) frameworks automate the mapping of program objects to relational structures and reduce or eliminate the need to use SQL language instructions in program code.

ORMs are popular because they reduce code complexity and relieve the developer from the need to manually determine the way in which object-oriented program artifacts map to relational tables. On the other hand, ORMs sometimes reduce

the flexibility provided by a relational database and can result in less-than-optimal performance.

Since the ORM layer is on top of the SQL layer and since the CockroachDB SQL layer is PostgreSQL-compatible, most PostgreSQL ORMs will work with CockroachDB without modification. In some cases, the CockroachDB team has worked with the ORM maintainer to ensure compatibility with CockroachDB. You can see a list of supported ORMs in the [CockroachDB documentation](#), together with instructions on installing any necessary CockroachDB prerequisites.

Table 6-1 summarizes some of the ORM framework options for CockroachDB.

Table 6-1. Object-relational mapping systems for CockroachDB

Go	Java	Python	JavaScript	Ruby
GORM	Hibernate	SQLAlchemy	Knex.js	ActiveRecord
go-pg	jOOQ	Django	Prisma	
upper/db	MyBatis	peewee	Sequelize	
			TypeORM	

We don't have space to explore all the options for using the various ORMs, and there are plenty of examples within the CockroachDB documentation. But let's review the essential workflow using one of the original ORMs: Hibernate for Java.

The code in this section is borrowed from the [CockroachDB Hibernate example on GitHub](#).

The Hibernate configuration is stored in an XML file and tells Hibernate how to connect to the backend database and which SQL language dialect to use. The following configuration file tells Hibernate to use the PostgreSQL driver and provides the connection URL, username, and password:

```
<hibernate-configuration>
  <session-factory>
    <!-- Database connection settings -->
    <property name="hibernate.connection.driver_class">
      org.postgresql.Driver</property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.CockroachDB201Dialect</property>
    <property name="hibernate.connection.url">
      jdbc:postgresql://localhost:26257/bank?ssl=true&sslmode=require</property>
    <property name="hibernate.connection.username">maxroach</property>
    <property name="hibernate.connection.password">password</property>
    <property name="hibernate.hbm2ddl.auto">create-drop</property>
  </session-factory>
</hibernate-configuration>
```

Note the property `hibernate.dialect` is set to `org.hibernate.dialect.CockroachDB201Dialect`; this should correspond to the version of CockroachDB being connected to.

In the user code, we create classes that map to database tables and methods within those classes that define operations that can be performed on those tables. For example, here we create an `Accounts` class. Annotations tell Hibernate that this class will map to the `accounts` table:

```
@Entity
@Table(name = "accounts")
public static class Account {
    @Id
    @Column(name = "id")
    public long id;
    public long getId() {
        return id;
    }
    @Column(name = "balance")
    public BigDecimal balance;
    public BigDecimal getBalance() {
        return balance;
    }
    public void setBalance(BigDecimal newBalance) {
        this.balance = newBalance;
    }
    // Convenience constructor.
    public Account(int id, int balance) {
        this.id = id;
        this.balance = BigDecimal.valueOf(balance);
    }
    // Hibernate needs a default constructor to create model objects.
    public Account() {
    }
}
```

We can now write functions that manipulate the database by using these Hibernate methods without writing SQL code:

```
private static Function<Session, BigDecimal> transferFunds
(long fromId, long toId, BigDecimal amount) throws JDBCException {
    Function<Session, BigDecimal> f = s -> {
        BigDecimal rv = new BigDecimal(0);
        try {
            Account fromAccount = (Account) s.get(Account.class, fromId);
            Account toAccount = (Account) s.get(Account.class, toId);
            if (!(amount.compareTo(fromAccount.getBalance()) > 0)) {
                fromAccount.balance = fromAccount.balance.subtract(amount);
                toAccount.balance = toAccount.balance.add(amount);
                s.save(fromAccount);
                s.save(toAccount);
                rv = amount;
            }
        }
    };
    return f;
}
```

```

        System.out.printf(
            "APP: transferFunds(%d, %d, %.2f) --> %.2f\n",
            fromId, toId, amount, rv);
    }
} catch (JDBCException e) {
    throw e;
}
return rv;
};
return f;
}

```

This code is fairly standard Hibernate code that would work on almost any SQL database. However, as we've seen, CockroachDB does have some unique transactional behaviors, and these may need to be accounted for in a nontrivial application. In the Java Hibernate example, the CockroachDB team defined a `runTransaction` method. It takes as its argument a function containing commands that might trigger a `retry` `Transaction` error. The method retries the transaction using the exponential backoff strategy shown in [Figure 6-6](#):

```

private static BigDecimal runTransaction(
    Session session,
    Function<Session, BigDecimal> fn) {
    BigDecimal rv = new BigDecimal(0);
    int attemptCount = 0;
    while (attemptCount < MAX_ATTEMPT_COUNT) {
        attemptCount++;
        if (attemptCount > 1) {
            System.out.printf(
                "APP: Entering retry loop again, iteration %d\n",
                attemptCount);
        }
        Transaction txn = session.beginTransaction();
        System.out.printf("APP: BEGIN;\n");
        if (attemptCount == MAX_ATTEMPT_COUNT) {
            String err = String.format("hit max of %s attempts, aborting",
                MAX_ATTEMPT_COUNT);
            throw new RuntimeException(err);
        }
        try {
            rv = fn.apply(session);
            if (!rv.equals(-1)) {
                txn.commit();
                System.out.printf("APP: COMMIT;\n");
                break;
            }
        } catch (JDBCException e) {
            if (RETRY_SQL_STATE.equals(e.getSQLState())) {
                // Exponential backoff
                System.out.printf("APP: retryable exception occurred:\n sql
state = [%s]\n    message = [%s]\n    retry counter = %s\n", e.getSQLState(),

```

```

e.getMessage(), attemptCount);
System.out.printf("APP: ROLLBACK;\n");
txn.rollback();
int sleepMillis = (int) (Math.pow(2, attemptCount) * 100) +
    RAND.nextInt(100);
System.out.printf("APP: Hit 40001 transaction retry error,
sleeping %s milliseconds\n", sleepMillis);
try {
    Thread.sleep(sleepMillis);
} catch (InterruptedException ignored) {
    // no-op
}
rv = BigDecimal.valueOf(-1);
} else {
    throw e;
}
}
}
return rv;
}
}

```

Each ORM framework has its own approach to configuration and coding, but the general practices are similar. [Table 6-2](#) summarizes your options.

Table 6-2. Options for retrying transactions in ORMs

Language/ORM	Retry transaction procedure
SQLAlchemy	Use the <code>sqlalchemy_cockroachdb.run_transaction()</code> method
Django	Define a transaction retry loop in the decorator function
GORM	Wrap the function call in <code>crdbgorm.ExecuteTx()</code>
pgx	Wrap the function call in <code>crdbpgx.ExecuteTx()</code>
Java	Use the <code>runTransaction()</code> method

Row-Level TTL

In many industries, companies are required to keep data for a period of time before it can be deleted. Unless you have a compelling reason to keep every row of data forever, it's likely that at some point, you'll want or need to archive or delete old data.

CockroachDB's built-in row-level TTL allows you to declaratively delete data from tables when certain criteria are met. In this section, we'll harness row-level TTL to expire data for us, eliminating the need for manual deletion or specialized code.

First, we'll create a cluster:

```
cockroach demo --insecure --no-example-database
```

Let's assume we're an ecommerce business and our customers add products to baskets before checking out. The statements that follow create a basket table, applying a row-level TTL expression that deletes rows based on the following logic:

- Delete pending baskets if they were last updated over one hour ago; this gives customers a chance to revisit their order before we clean up their basket.
- Delete purchased baskets immediately.

```
CREATE TYPE basket_status AS ENUM ('pending', 'purchased');

CREATE TABLE basket (
  "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "customer_id" UUID NOT NULL,
  "products" STRING[] NOT NULL,
  "status" basket_status NOT NULL DEFAULT 'pending',
  "last_update" TIMESTAMPTZ NOT NULL DEFAULT now()
) WITH (
  ttl_expiration_expression = '
    CASE
      WHEN status = 'purchased' THEN last_update

      WHEN status = 'pending' THEN
        ((last_update AT TIME ZONE 'UTC') + INTERVAL '1 hour')
        AT TIME ZONE 'UTC'

      ELSE NULL
    END',
  ttl_job_cron = '* * * * *' -- Run once every minute.
);
```

Next, we'll insert some baskets to simulate the various states they might be in:

```
-- An active pending basket.
INSERT INTO basket ("customer_id", "products", "status", "last_update")
VALUES (
  'a7d16771-09d7-460d-8264-cdbd6781b005',
  ARRAY['a', 'b', 'c'],
  'pending',
  now()
);

-- An inactive pending basket.
INSERT INTO basket ("customer_id", "products", "status", "last_update")
VALUES (
  'bcf370b8-c917-4629-b7cd-614b1cd37380',
  ARRAY['d', 'e', 'f'],
  'pending',
  now() - INTERVAL '1 hour'
);
```

```
-- A purchased basket
INSERT INTO basket ("customer_id", "products", "status", "last_update")
VALUES (
  'c6638bf0-cea9-4f10-8afa-7c1c0401a2b9',
  ARRAY['g', 'h', 'i'],
  'purchased',
  now()
);
```

After a short while (as defined by our `ttr_job_cron` parameter), the inactive pending basket and the purchased basket will be deleted, leaving just the active pending basket (which itself will be deleted after an hour of inactivity):

```
SELECT customer_id, status
FROM basket;
```

```

               customer_id                | status
-----+-----
a7d16771-09d7-460d-8264-cdbd6781b005 | pending
```

Summary

In this chapter, we covered topics relating to application development with CockroachDB.

Because CockroachDB is highly compatible with PostgreSQL and somewhat compatible with many other SQL databases, the basics of software development against CockroachDB are not unique. Best practices for software development include using connection pools, bulk processing, minimizing network traffic, and avoiding unnecessary database requests.

If you are migrating to CockroachDB from a database whose default isolation is `READ COMMITTED`, you may encounter retry errors in your application post migration. In the short term, you can switch to CockroachDB's `READ COMMITTED` isolation level, resulting in application behavior you're more accustomed to. In the long term, however, the integrity of your data will benefit from serializable isolation.

CockroachDB's default serializable consistency model is stricter than many other SQL databases. Together with the higher possibility of conflicts in a distributed database, this does lead to the possibility of transactional conflicts. There are two major patterns for dealing with these conflicts—retrying transactions (optimistic transactions) or locking data with `FOR UPDATE` before modification (pessimistic transactions). Both patterns are valid; regardless of which pattern you use, however, transaction retry logic is recommended.

In the next chapter, we'll explore the tools and techniques you can harness to get data into and out of CockroachDB. We'll also explore the CockroachDB features that allow it to integrate seamlessly into your wider architecture.

Application Migration and Integration

As a thoroughly modern database system, CockroachDB is a perfect choice for new application development. However, it's also a good choice for modernizing existing applications. Many applications built on the last generation of relational database systems are now facing limitations that can only be sensibly addressed by a migration to a distributed architecture. In many cases, a distributed SQL-based system such as CockroachDB is the most attractive migration target because—unlike distributed NoSQL systems—it does not require a complete rework of the application's data model or code base. In this chapter, we'll explain how to move data from existing databases into CockroachDB.

Moving data into CockroachDB is just one part of the story. While CockroachDB is a general-purpose database system, its sweet spot is undoubtedly operational transactional processing. In many environments, the data that is processed within CockroachDB will be leveraged for business intelligence or data science purposes in combination with data from other operational systems. In these cases, we will need a mechanism for moving data into external systems. The CockroachDB change data capture (CDC) facility is invaluable for this purpose—we'll see how CDC can be used to move data into external systems such as Snowflake or Kafka. We'll also see how data can be dumped from CockroachDB directly to external files. Moving data into CockroachDB is just one part of the story. While CockroachDB is a general-purpose database system, its sweet spot is undoubtedly operational transactional processing. In many environments, the data that is processed within CockroachDB will be leveraged for business intelligence or data science purposes in combination with data from other operational systems. In these cases, we will need a mechanism for moving data into external systems. The CockroachDB change data capture (CDC) facility is invaluable for this purpose—we'll see how CDC can be used to move data into external systems such as Snowflake or Kafka. We'll also see how data can be dumped from CockroachDB directly to external files.

Migration Objectives

When migrating from one system to another, it's important to consider how the new system will integrate into and complement your architecture and engineering organization.

As a famous Gondor-based warrior once said, “One does not simply replace a database with CockroachDB.” (He was ahead of his time.) Customers are realizing that once they’ve adopted CockroachDB, they are not only replacing legacy technology but also simplifying their architecture and—in many cases—their engineering organizations.

In this section, we’ll explore the various architectural simplifications enabled by CockroachDB. We’ve created a [YouTube series](#) to demonstrate these patterns before and after CockroachDB and created a [GitHub repo](#) with the code if you’d like to follow along.

Database Consolidation

There’s merit in choosing the right tool for the job. From a database perspective, however, this can lead engineering teams into implementing multiple specialized databases, with each performing different tasks (e.g., one for fast data ingestion and another for complex data querying) and needing to be integrated.

As a result of their migration to CockroachDB, Spreadly managed to:

- Replace Postgres and Riak
- Remove a cyclical data dependency in their architecture
- Remove three custom codebases that existed to get data out of Riak, through their architecture, and into Postgres
- Eliminate split-brain from their architecture

In an architecture with hyper-specialized databases, the following architectural patterns come into play and can be simplified by CockroachDB:

Dual writes

A situation whereby an application is required to write to multiple downstream integrations (e.g., two databases). In an architecture with dual writes, there’s no guarantee that both downstream integrations will be in sync. What happens if one write succeeds and another fails, or the application fails while making the second write? Architectures that rely on dual writes are brittle and come with higher application and operational complexity. Replacing multiple databases with CockroachDB (which can scale for both reads *and* writes) can eliminate the need for dual writes.

Read/write segregation

In any system where the database you're writing to is different from the database you're reading from, you're encountering read/write segregation. This is the realm of split-brain, where no single consistent view of data can be guaranteed across the architecture. By harnessing CockroachDB for both reads and writes in this scenario, you remove the risk of split-brain, reduce your architectural and operational cost and complexity, and gain one true source of data that's guaranteed to be consistent.

Failover Regions

The risks of high recovery point objective (RPO) and recovery time objective (RTO) are inherent in any system where primary and secondary/standby infrastructure is maintained. This is the world of disaster recovery and business continuity, and the process of recovering from outages can be long, complex, and risky.

The following architectural patterns and requirements are common among two-data center configurations:

Predictable failover latency

In any system where a primary node, region, or data center needs failing over to another following an outage, nonzero RPOs (data loss) and high RTOs (long downtimes) are a risk. The effort, architectural costs, and operation costs associated with maintaining separate primary and standby infrastructure are high. So too are the risks involved in failing over to the standby infrastructure and—post outage—failing back to the primary. Running CockroachDB across three or more nodes serves to eliminate the risks inherent in two-data center configurations. If a two-data center configuration is required, CockroachDB's built-in physical cluster replication simplifies the challenge of cluster replication, minimizes RPO and RTO, and makes the process of failover and failback safer and more straightforward.

Database maintenance

In a two-data center configuration, the process of maintaining (e.g., patching or upgrading) a database involves a number of complex steps. These steps are designed to ensure the availability of the database and dependent applications during the maintenance period. Performing a rolling upgrade of CockroachDB nodes in a cluster comprising three or more nodes with Raft replication results in zero downtime. If running across two data centers, CockroachDB's physical cluster replication and logical data replication will lower the architectural and operational cost and complexity of the process and reduce the amount of downtime required.

Horizontal scaling

The process of scaling a traditional database (or any predistributed computing system) is complex. The scaling options available for these systems include vertical scaling and sharding, neither of which are without their drawbacks. Vertical scaling is limited by the machine and involves downtime while resources are swapped out. Sharding, on the other hand, is extremely complex and introduces availability, consistency, and integrity risks to the platform. This is why most cloud databases scale for reads but not for writes and offer eventual rather than strong consistency. Scaling a CockroachDB cluster is as simple as adding and removing nodes. It incurs zero downtime and allows the cluster to scale for reads and writes, far beyond what is possible for traditional databases and even cloud databases—all while maintaining strong consistency.

Fragile Data Integrations

An architecture's complexity is a product of the components, integrations, and data flows within it. In a system with multiple services, the communication between those services needs to be carefully orchestrated. The following patterns speak to these complexities:

Queue coherence

In an architecture where applications write to both a database and a message queue, the issue of queue coherence becomes a risk. In the same way that dual-writes to multiple data stores can result in split-brain, dual-writes involving message queues can result in incorrect data to downstream consumers. By writing to CockroachDB and harnessing its built-in CDC, you enable the **Transactional Outbox pattern**, resulting in a stream of data for downstream consumers that's consistent with the database itself.

Extract, transform, and load (ETL)

This is the process where data is extracted from one data store, transformed into a new shape, then loaded into another data store. This process can be complex and involve a number of moving parts, especially if the data store you're extracting data from does not provide its own changefeeds. With CockroachDB, this process is as simple as configuring a CDC changefeed (see using "**CDC Queries**" on page 250). These changefeeds transform rows before sending them to downstream consumers, satisfying both the extract and transform phases of the ETL process.

Change data capture

With many databases, CDC is possible only with external tools like Debezium. This increases both architectural and operational complexity, and ongoing validation of system compatibility between the database and the CDC provider is

required. With CockroachDB, CDC is built-in, allowing you to build your CDC changefeeds from within CockroachDB itself.

Data purging

Unless there's a compelling reason to keep every row of data in your database forever, it usually makes sense to either archive or delete it when it ceases to be useful. Typically, this would involve writing a specialized service to scan your database tables for old data and perform the necessary archival or deletion manually. With CockroachDB, your data can be given a row-level expiry (see [“Row-Level TTL” on page 210](#)), a condition that, once met, will automatically delete affected data. By doing this from within CockroachDB itself, you're not only removing the need for custom logic, you're also benefitting from CockroachDB's built-in Admission Control (see [“Admission Control” on page 485](#)), ensuring that database performance is not impacted.

Polling clients

If your database has multiple consumers polling for the most up-to-date data, this can have a negative impact on performance. Rather than allowing consumers to continuously poll for data, consider pushing updates to them via CDC. This will result in both lower resource and network utilization and ensure consumers all receive a consistent view of the latest database state.

Data at the edge

To get data as close to users as possible, traditional SQL databases require asynchronous replication. Users near the replicas will enjoy lower read latencies but will be able to write to the database only via the primary region, resulting in increased write latency. CockroachDB solves the challenge of data locality using its powerful geo-partitioning topology patterns. GLOBAL tables provide low read latencies to all users, while REGIONAL BY ROW tables provide low read *and* write latencies. See [“Locality Rules” on page 363](#) for more information.

Unnecessary Caching Tier

With monolithic databases, your options for scaling were limited. You could upgrade your CPU, add memory, and fit a new network card, but ultimately there would be a ceiling. For this reason, developers might choose to take some of the read pressure off the database by implementing a cache.

With distributed SQL databases, your scaling options are less constrained, as the technology is *designed* to scale.

Consider read bottlenecks. In a read-heavy system, it may be tempting (or necessary) to offload read load to a cache to lessen the load on the database. The introduction of a cache results in higher architectural complexity, the dual-write problem, and the opportunity for split-brain. A horizontally scalable database like CockroachDB

obviates the need for a cache in all but the most extreme read-heavy scenarios (we’re talking about viral cat meme levels of read-heaviness). Generally, you should allow your database to handle read traffic, horizontally scaling as demand increases until it’s no longer feasible to avoid caching.

Unnecessary Data Warehouse Workloads

Implementing a separate relational and analytical database increases the complexity and operational overhead of an architecture, as both systems need to be maintained and integrated. For simple analytical workloads, users might benefit from running queries directly in CockroachDB.

For heavy analytical workloads, you may prefer to extract data out of CockroachDB and into a specialized Online Analytical Processing (OLAP) database like Snowflake, BigQuery, or RedShift. If, however, your analytical workload does not require a full OLAP database (and the complexities that come with extraction and loading), you may prefer to perform your analytical queries against CockroachDB. CockroachDB provides a number of tools to minimize the impact of analytical workloads, allowing you to gain analytical insights the moment they become available. See [“Ad Hoc or Analytic Queries” on page 473](#) for more information.

Application Silos

There are good reasons to isolate applications that serve different geographic markets (regulatory compliance, etc.). Isolation, however, doesn’t have to mean disparate.

As your user base grows, so too will your architecture. Unless this growth is carefully designed for, you may end up with multiple disparate instances of the application stack—each with their own databases, applications, dependencies, and vulnerabilities. In a fragmented architecture like this, config drift becomes an increasingly difficult challenge to prevent. Operational complexity increases, while velocity and resource utilization efficiency decrease. CockroachDB’s geo-partitioning allows you reach new markets by growing your existing cluster, giving you centralized control over a distributed system, rather than distributing your team to manage multiple non-distributed systems. See [“Multiregion Distribution” on page 44](#) for more information.

Loading Data

The first step when adopting CockroachDB is often to move data into the database from legacy systems or other sources. The most fundamental data load activity in CockroachDB is to load data from “flat files.” CockroachDB can load data from delimited files—such as the ubiquitous CSV file—or from the Avro format. Data is loaded using the `IMPORT INTO` command introduced in [Chapter 4](#).

File Locations

In monolithic databases, loading data from flat files is typically done directly from files on filesystems—typically staged to the database server first to avoid network overhead. However, as a distributed cloud native database, CockroachDB takes a slightly different tack.

The `IMPORT INTO` command requires that files are located on cloud storage (S3, GCP buckets, Azure containers), “userfiles” uploaded to the CockroachDB server, or HTTP file servers.

userfile storage

Let’s start with uploading a file to cluster userfile storage. *Userfile* storage is a sort of virtual filestore maintained by the CockroachDB cluster, allowing access to files from the SQL layer across the entire cluster.

The `cockroach userfile upload` command allows you to copy a file to cluster storage. The file will be accessible only to the user who uploaded the file:

```
$ cockroach userfile upload employees.csv employees.csv --url $CRDB_CLUSTER  
  
successfully uploaded to userfile://defaultdb.public.userfiles_guy/employees.csv
```

Nodelocal storage is similar to userfile storage but loads the file to just one of the nodes of the cluster. It is less secure than userfile storage—since it can be accessed from any CockroachDB database account—and less robust since an `IMPORT INTO` from userfile storage can survive a node failure. On the other hand, a *nodelocal* upload is somewhat faster to execute.

The `cockroach nodelocal upload` command uploads a file to *nodelocal* storage:

```
$ cockroach nodelocal upload employees.csv employees.csv --url $CRDB_LOCAL  
  
successfully uploaded to nodelocal://1/employees.csv
```

Both commands return the location for the file; you’ll need that to load the file into a table later.

HTTP storage

CockroachDB can also access files held on local filesystems by running an **HTTP file server** such as **nginx**.

Cloud storage

For large files, *nodelocal* and *userlocal* storage results in disk space and I/O being consumed on at least one of the cluster nodes, which may be less than ideal.

Therefore, CockroachDB supports cloud storage locations, including Google Cloud Storage, Amazon S3 buckets, and Azure Storage containers.

In this example, we create a Google Cloud Storage bucket and upload a CSV file to that location. The example assumes that we are already logged in to Google Cloud:

```
$ ~ gsutil mb gs://cockroachdefinitiveguide
Creating gs://cockroachdefinitiveguide/...
$ gsutil cp employees.csv gs://cockroachdefinitiveguide
Copying file://employees.csv [Content-Type=text/csv]...
/ [1 files][ 8.8 KiB/ 8.8 KiB]
Operation completed over 1 objects/8.8 KiB.
$ gsutil ls gs://cockroachdefinitiveguide/
gs://cockroachdefinitiveguide/employees.csv
```

Here we upload the same file to an Azure container with the URL:

```
$ az storage container create --name cockroachdbdefinitiveguide
--account-name crdbdg

{
  "created": true
}
$ az storage copy -s employees.csv -d
https://crdbdg.blob.core.windows.net/cockroachdbdefinitiveguide

<snip>
TotalBytesTransferred: 9011
Final Job Status: Completed
```

Finally, we upload this file to Amazon S3 storage. As with the other examples, we've already configured our AWS authentication:

```
$ aws s3 mb s3://cockroachdefinitiveguide
make_bucket: cockroachdefinitiveguide
$ aws s3 cp employees.csv s3://cockroachdefinitiveguide
upload: ./employees.csv to s3://cockroachdefinitiveguide/employees.csv
```

Importing from userfile Storage

Once you have your file in cloud, userlocal, or other accessible storage, you'll typically use the `IMPORT INTO` command to load the data. You'll need **appropriate privileges** on the target table.

Here's an example of `IMPORT INTO`, loading data into a new table and loading data from the userfile location that we created in the previous section.

First, we upload the CSV file:

```
$ cockroach userfile upload departments.csv departments.csv \
--url $CRDB_CLUSTER
uploaded to userfile://defaultdb.public.userfiles_guy/departments.csv
```


Now we create the table and import the data:

```
guy@cockroachlabs.cloud:26257/defaultdb>
CREATE TABLE departments (
    department_id integer PRIMARY KEY,
    department_name varchar,
    manager_id integer,
    location_id integer
);
CREATE TABLE
```

Time: 40ms total (execution 24ms / network 16ms)

```
guy@cockroachlabs.cloud:26257/defaultdb>
IMPORT INTO departments
    (department_id, department_name, manager_id, location_id)
    ("userfile://defaultdb.public.userfiles_guy/departments.csv")
    WITH skip = '1', nullif = '';
```

job_id	status	fraction_c	rows	index_entries	bytes
678724479937937414	succeeded	1	27	0	771

(1 row)

Time: 550ms total (execution 532ms / network 18ms)

In the previous example, we specified `WITH skip=1, nullif = ''`; this signified that we should skip the first line of the CSV file (which consisted of a header line) and treat blanks as null values. These two options are commonly used. Other options are shown in [Table 7-1](#).

Table 7-1. Common options for `IMPORT INTO`

Option	Usage	Default
<code>allow_quoted_null</code>	If provided, both quoted and unquoted CSV input values can match the nullif setting.	
<code>comment</code>	The Unicode character that identifies rows to skip.	
<code>data_as_binary_records</code>	Import Avro from a binary file. Use in conjunction with <code>schema</code> or <code>schema_uri</code> to reference the schema.	
<code>data_as_json_records</code>	Import Avro from a JSON file. Use in conjunction with <code>schema</code> or <code>schema_uri</code> to reference the schema.	
<code>decompress</code>	The decompression codec to be used: <code>gzip</code> , <code>bzip</code> , <code>auto</code> , or <code>none</code> . Default: <code>'auto'</code> , which guesses based on file extension (<code>.gz</code> , <code>.bz</code> , <code>.bz2</code>). <code>none</code> disables decompression.	
<code>delimiter</code>	The Unicode character that delimits columns in your rows.	,
<code>DETACHED</code>	Run the import job asynchronously. Job status will be available via <code>SHOW JOBS</code> .	
<code>fields_enclosed_by</code>	The Unicode character that encloses fields.	"

Option	Usage	Default
fields_escaped_by	The Unicode character, when preceding one of the above DELIMITED DATA options, to be interpreted literally. Often the “\” symbol serves the purpose.	
fields_terminated_by	The Unicode character used to separate fields in each input line.	\t
nullif	The string that should be converted to NULL.	
records_terminated_by	The Unicode character to indicate newlines in the input file.	\n
row_limit	The number of rows to import. This will import the first <i>n</i> rows from each table.	
rows_terminated_by	The Unicode character to indicate newlines in the input file.	\n
schema	The schema of the Avro records in the binary or JSON file (not required for Avro OCF).	
schema_uri	The URI schema file for the Avro records in the binary or JSON file (not required for Avro OCF).	
skip	The number of rows to be skipped while importing a file.	'0'
strict_validation	Reject Avro records that don't exactly match the CockroachDB schema.	

Importing from Cloud Storage

Specifying `userfile` or `nodelocal` file locations is relatively simple, but in many cases, cloud storage is a more suitable option. When specifying a cloud location, we need to also specify credentials to allow the connection to succeed. For instance, if we uploaded the `departments` data to Amazon S3 as follows:

```
$ aws s3 cp departments.csv s3://cockroachdefinitiveguide
upload: ./departments.csv to s3://cockroachdefinitiveguide/departments.csv
```

we need to specify our AWS access keys when referencing the file from within an `IMPORT INTO` statement:

```
defaultdb>
IMPORT INTO departments
  ( department_id,department_name,manager_id,location_id )
  CSV DATA (
    "s3://cockroachdefinitiveguide/departments.csv?
    AWS_ACCESS_KEY_ID=key
    &AWS_SECRET_ACCESS_KEY=key")
    WITH skip='1', nullif = '';

      job_id      | status      | fraction | rows | index_entries | bytes
-----+-----+-----+-----+-----+-----
  678728775096893441 | succeeded |          1 | 27 |              0 | 771
(1 row)

Time: 1.379s total (execution 1.363s / network 0.016s)
```

To avoid including access keys in SQL commands, you can specify authentication in environment variables. Similar authentication mechanisms are provided for Google Cloud and Azure Storage systems. See the [CockroachDB documentation](#) for more information.

To avoid the reuse and proliferation of sensitive credentials, CockroachDB provides [external connections](#). These allow operators to *reference* cloud storage endpoints without knowing their sensitive credentials. Here's how to create and use an external connection:

```
CREATE EXTERNAL CONNECTION your_s3_bucket AS
's3://your_bucket_name?AWS_ACCESS_KEY_ID=key&AWS_SECRET_ACCESS_KEY=key';

IMPORT INTO departments
(department_id, department_name, manager_id, location_id )
CSV DATA ('external://your_s3_bucket/optional_sub_dir/*')
WITH skip='1', nullif = '';
```

Publicly Accessible Cloud Storage

Specifying cloud storage authentication parameters can be awkward. You might be tempted to use publicly accessible buckets or file settings to avoid the pain. This is usually a Very Bad Idea—it potentially exposes sensitive data to the world and may even be illegal if confidential information protected by HIPAA (the US Health Insurance Portability and Accountability Act) or similar legislation is exposed.

Note that `IMPORT INTO` invalidates all foreign key constraints on the target table. These foreign keys need to be reenabled using the `VALIDATE CONSTRAINT` command.

We'll see some examples of using `IMPORT INTO` with PostgreSQL and MySQL dump files later in the chapter.

Import Performance

Import performance can vary markedly depending on the format of the data and the sequencing of DDL commands. It's often best to defer index and constraint creation until after the data load and to use compact data formats. The effectiveness of these strategies can depend on network bandwidth, memory constraints, and other factors. For a discussion of import performance, consult the [CockroachDB documentation](#). Generally, however, `IMPORT INTO` is the fastest way to get data into CockroachDB, as the table is taken offline during the import process. This makes it a good choice for bulk-loading data during periods where table downtime can be tolerated.

Migrating from Another Database

The easiest way to migrate to CockroachDB from another database is by using Cockroach Labs’ **MOLT tool suite**, outlined in **Table 7-2**.

Table 7-2. Available MOLT tools

Tool	Usage
Schema Conversion Tool	Convert a source database schema (e.g., Postgres, MySQL, Oracle, SQL Server) into a CockroachDB-compatible schema.
Fetch	Move data from a source database into CockroachDB. A good choice for an initial bulk-ingestion of data.
Verify	Checks for discrepancies between a source and target database. Run to ensure that the structure and content of your tables matches your expectations.
Live Migration Service	Sitting between your applications and databases, the Live Migration Service (or “LMS”) is a horizontally scaling proxy that provides different cutover strategies to suit your migration requirements.

We’ll explore each of the MOLT tools later in this chapter.

There are a variety of ways to migrate from one database system to another. The most straightforward way is as follows:

1. Extract DDL from the source system and convert that DDL to CockroachDB-compatible `CREATE TABLE`, `INDEX`, `VIEW`, and other statements.
2. Dump table data from the source system to CSV or another flat-file format.
3. Import the flat files using the `IMPORT INTO` statement.

This procedure copies static data only—for a live system, you may need to implement custom procedures to synchronize the state of the source and target system and perform a cutover with minimal downtime. While every software migration is unique, we will discuss several common approaches in the following sections.

Extracting and Converting DDL

Most database systems provide mechanisms for generating DDL statements for schema objects. Coming up, we’ll look at a few platform-specific techniques. However, some cross-platform IDEs can generate DDL for multiple types of database systems.

For example, we can generate DDL from the DBeaver object tree for most of the SQL database systems supported by DBeaver (which is practically all). In **Figure 7-1**, we generate a `CREATE TABLE` statement for the `departments` table in the AdventureWorks sample schema.

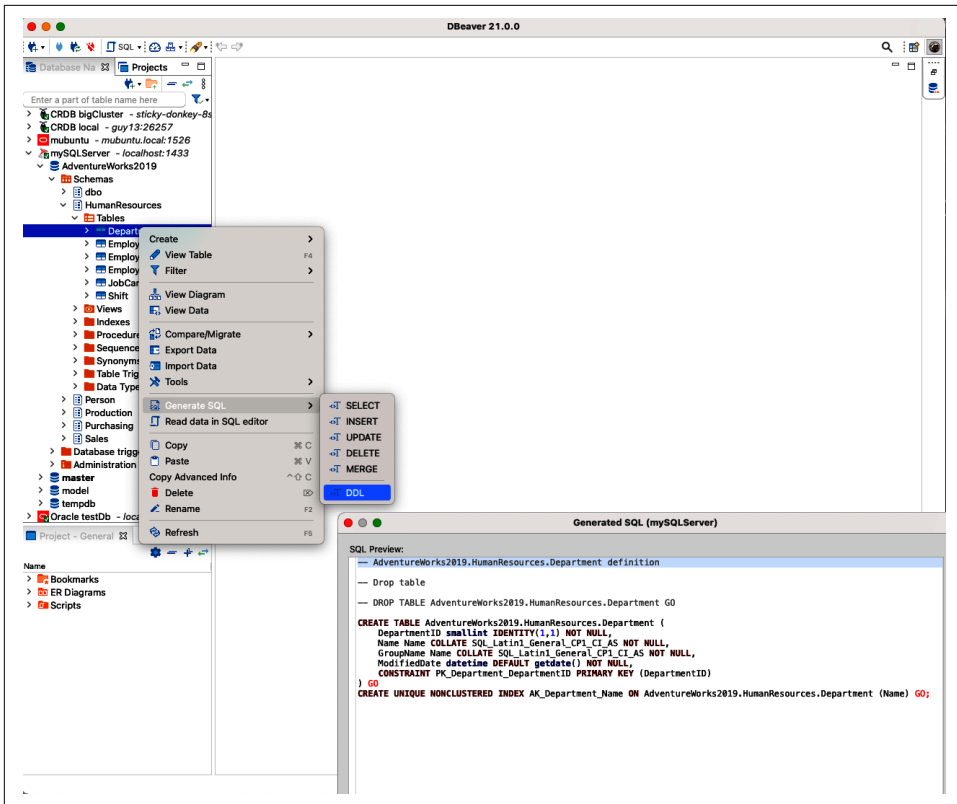


Figure 7-1. Generating DDL using DBeaver

Extracting Oracle DDL

Oracle includes the `dbms_metadata` package, which can generate DDL for Oracle objects. The package includes settings that suppress or allow various storage clauses and other Oracle-specific information, resulting in an output that is closer to CockroachDB compatibility than would otherwise be the case. [Example 7-1](#) shows a script that will emit DDL for all Oracle tables, indexes, and views in a specific Oracle account.

Example 7-1. Script to extract Oracle DDL

```

set long 100000
set head off
set echo off
set pagesize 0
set verify off
set feedback off
col DDL format a256

```

```

REM These statements limit Oracle-specific clauses generated
EXECUTE dbms_metadata.SET_TRANSFORM_PARAM(
dbms_metadata.SESSION_TRANSFORM, 'SEGMENT_CREATION', false);
EXECUTE dbms_metadata.SET_TRANSFORM_PARAM(
dbms_metadata.SESSION_TRANSFORM, 'CONSTRAINTS_AS_ALTER', true);
EXECUTE dbms_metadata.set_transform_param(
dbms_metadata.session_transform, 'TABLESPACE', false);
EXECUTE dbms_metadata.set_transform_param(
dbms_metadata.session_transform, 'STORAGE', false);
EXECUTE dbms_metadata.set_transform_param(
dbms_metadata.session_transform, 'SEGMENT_ATTRIBUTES', false);
EXECUTE dbms_metadata.set_transform_param (
dbms_metadata.session_transform, 'SQLTERMINATOR', true);
EXECUTE dbms_metadata.set_transform_param (
dbms_metadata.session_transform, 'PRETTY', true);

SELECT dbms_metadata.get_ddl('TABLE', table_name,user) AS ddl FROM user_tables;
SELECT dbms_metadata.get_ddl('INDEX', index_name,user) AS ddl FROM user_indexes;
SELECT dbms_metadata.get_ddl('VIEW',view_name,user) AS ddl FROM user_views;

EXIT

```

Example 7-1 generates the DDL shown in Example 7-2.

Example 7-2. Example of Oracle DDL

```

CREATE TABLE "HR"."EMPLOYEES"
(
    "EMPLOYEE_ID" NUMBER(6,0),
    "FIRST_NAME" VARCHAR2(20),
    "LAST_NAME" VARCHAR2(25) CONSTRAINT "EMP_LAST_NAME_NN" NOT NULL ENABLE,
    "EMAIL" VARCHAR2(25) CONSTRAINT "EMP_EMAIL_NN" NOT NULL ENABLE,
    "PHONE_NUMBER" VARCHAR2(20),
    "HIRE_DATE" DATE CONSTRAINT "EMP_HIRE_DATE_NN" NOT NULL ENABLE,
    "JOB_ID" VARCHAR2(10) CONSTRAINT "EMP_JOB_NN" NOT NULL ENABLE,
    "SALARY" NUMBER(8,2),
    "COMMISSION_PCT" NUMBER(2,2),
    "MANAGER_ID" NUMBER(6,0),
    "DEPARTMENT_ID" NUMBER(4,0)
);
CREATE UNIQUE INDEX "HR"."EMP_EMP_ID_PK" ON "HR"."EMPLOYEES" ("EMPLOYEE_ID")
;
ALTER TABLE "HR"."EMPLOYEES" ADD CONSTRAINT "EMP_EMP_ID_PK"
PRIMARY KEY("EMPLOYEE_ID")
USING INDEX "HR"."EMP_EMP_ID_PK" ENABLE;
ALTER TABLE "HR"."EMPLOYEES" ADD CONSTRAINT "EMP_DEPT_FK"
FOREIGN KEY ("DEPARTMENT_ID")
REFERENCES "HR"."DEPARTMENTS" ("DEPARTMENT_ID") ENABLE;

```

We still have some work to do to make this DDL CockroachDB-compatible:

- First, we need to convert data types to CockroachDB types. The Oracle `NUMBER` type encapsulates both `INTEGER` and `FLOAT` types, depending on the precision provided. `VARCHAR2` maps to `VARCHAR`. There are many esoteric Oracle data types, but in the majority of cases, date and number conversions will dominate.
- The `ENABLE` clauses are unknown to CockroachDB and need to be removed, as do the `USING INDEX` clauses.

The following `sed` commands will perform a lot of the edits required:

```
s/VARCHAR2(.*)/VARCHAR/g
s/NUMBER(.*,0)/INT/g
s/NUMBER(.*,.*)/DECIMAL(\1)/g
s/NUMBER\((.*)\.*\)/FLOAT(\1)/g
s/NUMBER/FLOAT/g
s/USING INDEX (.*) ENABLE//g
s/USING INDEX //g
s/ENABLE//g
s/\\"(.*)\\".//g
```

So, if we called the script in [Example 7-1](#) `getDDLOracle.sql`, then we could generate the DDL for a schema and perform some initial edits as follows:

```
sql -S hr/hr@local @getDDLOracle.sql |sed -f oracle.sed
```

```
CREATE TABLE "HR"."JOBS"
(
  "JOB_ID" VARCHAR,
  "JOB_TITLE" VARCHAR CONSTRAINT "JOB_TITLE_NN" NOT NULL,
  "MIN_SALARY" INT,
  "MAX_SALARY" INT
) ;
CREATE UNIQUE INDEX "HR"."JOB_ID_PK" ON "HR"."JOBS" ("JOB_ID")
;
ALTER TABLE "HR"."JOBS" ADD CONSTRAINT "JOB_ID_PK" PRIMARY KEY ("JOB_ID")
"HR"."JOB_ID_PK" ;
```

More changes to the SQL will usually be required. For instance, in the example DDL, the `JOB_ID` is initially created without the `NOT NULL` constraint—Oracle allows nullable primary keys. The primary key is added after the `CREATE TABLE` by an `ALTER TABLE` statement. However, in CockroachDB a primary key column must also be `NOT NULL`. Furthermore, it's best practice in CockroachDB to specify the `PRIMARY KEY` within the `CREATE TABLE` statement; otherwise, a hidden primary key field will be created.

Extracting DDL from SQL Server

There's no direct way to generate DDL from SQL Server T-SQL commands, but you can extract DDL from the Microsoft Management Studio (From Tasks → Generate Scripts). Some users have created **stored procedures to generate DDL**. You can also use DBeaver as described previously.

Extracting DDL from MySQL

MySQL supports a `SHOW CREATE` command, which can be used to extract DDL for a particular object. For instance, in the following code, we extract the DDL for the `customer` table in the `Sakila` schema:

```
$ mysql -uroot -D sakila -s -N
mysql> show create table customer;

customer      CREATE TABLE `customer` (\n
`customer_id` smallint unsigned NOT NULL AUTO_INCREMENT,\n
`store_id` tinyint unsigned NOT NULL,\n
`first_name` varchar(45) NOT NULL,\n
`last_name` varchar(45) NOT NULL,\n
`email` varchar(50) DEFAULT NULL,\n
`address_id` smallint unsigned NOT NULL,\n
`active` tinyint(1) NOT NULL DEFAULT '1',\n
`create_date` datetime NOT NULL,\n
`last_update` timestamp NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,\n
PRIMARY KEY (`customer_id`),\n
KEY `idx_fk_store_id` (`store_id`),\n
KEY `idx_fk_address_id` (`address_id`),\n
KEY `idx_last_name` (`last_name`),\n
CONSTRAINT `fk_customer_address` FOREIGN KEY (`address_id`)
REFERENCES `address` (`address_id`) ON DELETE RESTRICT ON UPDATE CASCADE,\n
CONSTRAINT `fk_customer_store` FOREIGN KEY (`store_id`)
REFERENCES `store` (`store_id`) ON DELETE RESTRICT ON UPDATE CASCADE\n)
ENGINE=InnoDB AUTO_INCREMENT=600 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci
```

You can also dump all the DDL for a schema using the `mysqldump` command with the `-d` option. `-d` suppresses data in the output:

```
$ sakila-db mysqldump -d -u root sakila
-- MySQL dump 10.13  Distrib 8.0.23, for osx10.16 (x86_64)
--

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
<snip>
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
```



```
--
-- Table structure for table `actor`
--

DROP TABLE IF EXISTS `actor`;
CREATE TABLE `actor` (
  `actor_id` smallint unsigned NOT NULL AUTO_INCREMENT,
  `first_name` varchar(45) NOT NULL,
  `last_name` varchar(45) NOT NULL,
  `last_update` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`actor_id`),
  KEY `idx_actor_last_name` (`last_name`)
) ENGINE=InnoDB AUTO_INCREMENT=201 DEFAULT CHARSET=utf8mb4
  COLLATE=utf8mb4_0900_ai_ci;
```

MySQL SQL is largely compatible with CockroachDB, but you will want to remove the `ENGINE` and `CHARSET` directives. CockroachDB does support the `COLLATE` keyword, but the syntax is not identical. Also, MySQL surrounds literal names with backticks (```)—these will need to be changed to double quotes for CockroachDB. In the example, you'll also need to remove the `ON UPDATE` and `AUTO_INCREMENT` flags.

Finally, secondary indexes in MySQL `CREATE TABLE` statements are shown as `KEY` elements in the `CREATE TABLE` statement. These will need to be removed and transformed to `CREATE INDEX` statements or to `INDEX` statements embedded within the `CREATE TABLE` statement.

In some cases, you might be able to directly import a MySQL dump file; we'll look at this option later in this chapter.

Extracting DDL from PostgreSQL

You can extract DDL from a PostgreSQL database by using the `pg_dump` command with the `-s` or `--schema-only` options. Here we dump the SQL from the `DVDRental` sample database.¹ You can also specify the `-t` option to extract DDL for a specific table in the `DVDRental` database:

```
$ pg_dump -s dvdrental -t customer
--
-- PostgreSQL database dump
--

-- Dumped from database version 13.3
-- Dumped by pg_dump version 13.3
<snip>
```

¹ A DVD is a sort of 3D-printed representation of a movie. In ancient times, primitive people used DVDs in areas where Netflix was not available.

```
--
-- Name: customer; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.customer (
  customer_id integer DEFAULT
  nextval('public.customer_customer_id_seq'::regclass) NOT NULL,
  store_id smallint NOT NULL,
  first_name character varying(45) NOT NULL,
  last_name character varying(45) NOT NULL,
  email character varying(50),
  address_id smallint NOT NULL,
  activebool boolean DEFAULT true NOT NULL,
  create_date date DEFAULT ('now'::text)::date NOT NULL,
  last_update timestamp without time zone DEFAULT now(),
  active integer
);
```

CockroachDB and PostgreSQL are highly compatible, but you'll still probably want to amend this SQL. In the preceding example, the primary key is based on a sequence generator, and as we discussed in [Chapter 5](#), there are better options for CockroachDB. Columns based on PostgreSQL domains will need amendment as well; we'll discuss this later when we look at importing PostgreSQL data directly.

General Considerations When Converting DDL

A lot of the drudgery involved in converting DDL to CockroachDB involves repetitively changing data types and removing syntax clauses that have no effect in CockroachDB. However, there are some more nuanced decisions that you'll need to make that are reflective of more substantial differences between CockroachDB and other SQL databases:

- In a distributed SQL system, the selection of the *primary keys* data type and population mechanisms are particularly significant. Review the section on primary keys in [Chapter 5](#) and make sure that you choose a primary key type that will work well for your application.
- *Triggers* may be implemented in other systems that implement business logic or refine referential integrity constraints. Carefully review the trigger code and determine if this logic needs to be implemented in application logic.
- Some databases allow the definition of user-defined data types or *domains*, which will be associated with their own constraints and data types. Most of these will need to be folded into your DDL (although CockroachDB does support ENUMs as user-defined types).
- The sequence of SQL statements is important. FOREIGN KEY references in CockroachDB can be created only if the referenced table exists. You may need to adjust the sequence of SQL statements to ensure that there are no broken

dependencies. If you can defer all foreign key constraint generation until after every table is created, then that would be ideal. You may also find that your overall migration is faster if all index and constraint creations are deferred until after table data has been loaded.

- Review the indexing scheme carefully. In particular, in CockroachDB, covering indexes are a more important optimization than in some other databases. See [Chapter 5](#) for more information on this topic.

Exporting Data

Having created the tables and other schema objects in the CockroachDB target, the next task is to dump the target data to CSV format. Each database system provides some mechanism to do this. You can sometimes use an IDE to perform this task. For instance, as shown in [Figure 7-2](#), DBeaver can export table data to CSV format from any of its supported database types.

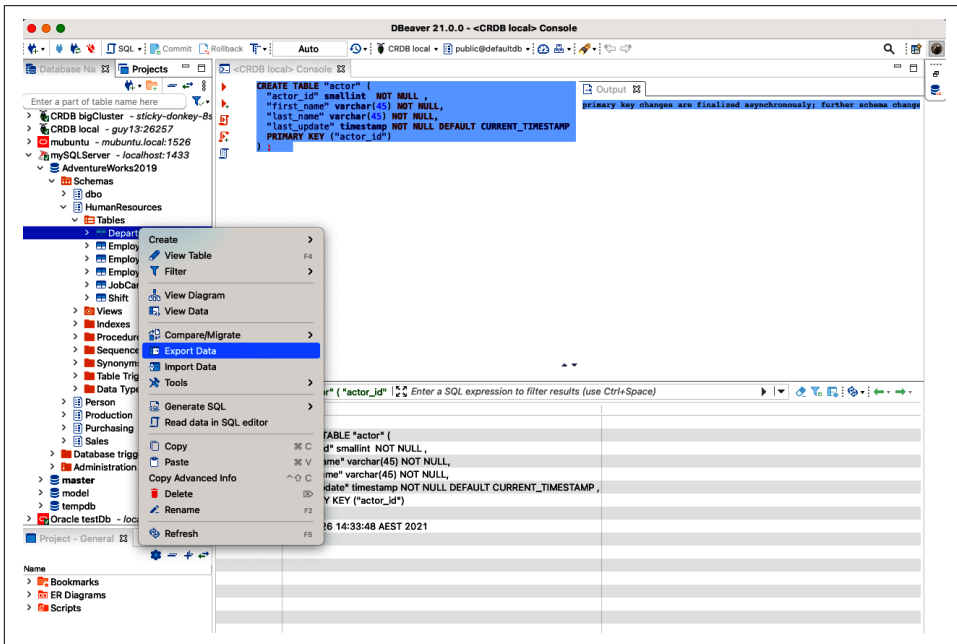


Figure 7-2. Exporting data from DBeaver

In Oracle, we can simply use the `set sqlformat csv` setting to cause the SQLcl command-line tool to write data out in CSV format. The following script can perform this task:

```
set head on
set echo off
set pagesize 0
set verify off
set feedback off
set sqlformat csv

select * from &1;

exit
```

In SQL Server, you can export data to CSV from the SQL Server Management Studio, or you can use the `sqlcmd` command line as follows:

```
sqlcmd -U SA -P **** -d AdventureWorks2019 \
-Q 'SELECT * FROM HumanResources.Employee' \
-W -w 1024 -s","
```

MySQL allows data to be dumped to CSV from the `mysqldump` command:²

```
$ mysqldump -u root --no-create-info --tab=/tmp Sakila customer \
--fields-terminated-by=,
```

In PostgreSQL, we can copy files to CSVs using the `COPY` command:

```
dvdrental=# COPY customer to '/tmp/customer.csv' DELIMITER ',' CSV HEADER;
COPY 599
```

Migrating Schemas to CockroachDB

Once a database schema has been exported from a source database (Postgres, MySQL, etc.), the easiest way to convert that schema into something CockroachDB can use is to run the MOLT Schema Conversion Tool (SCT). The SCT is available for CockroachDB Cloud deployments (Basic, Standard, and Advanced).

Before we can use the tool, we'll need to create a CockroachDB Cloud cluster. Head to the [CockroachDB Cloud console](#) and create a Basic, Standard, or Advanced cluster.

First, we'll need a schema to migrate. Let's start by creating a Postgres database using Docker:

```
docker run -d \
--name postgres \
-p 5432:5432 \
```

² This technique requires that `secure-file-priv` allows files to be written to the nominated directory and that the command is run from the MySQL host.

```
-e POSTGRES_PASSWORD=password \  
postgres:15.2-alpine
```

Next we'll hop into the terminal and create some database objects:

```
PGPASSWORD=password psql -h localhost -U postgres  
  
CREATE TABLE customers (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) NOT NULL UNIQUE  
);  
  
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  customer_id INT NOT NULL REFERENCES customers(id),  
  total DECIMAL NOT NULL,  
  ts TIMESTAMP NOT NULL DEFAULT now()  
);
```

Finally, we'll run `pg_dump` to export the database into a local file called *dump.sql*:

```
pg_dump -h localhost -U postgres -d postgres > dump.sql
```

Within this dump file will be instructions on how to re-create the database in Postgres, complete with `CREATE TABLE` and `CREATE SEQUENCE` statements as expected by Postgres.

Back in the CockroachDB Cloud console, click on “Migrations,” then “Add Schema.” Choose the PostgreSQL dialect and select between INT4 and INT8 to use as the default integer width. Lastly, upload the *dump.sql* file to begin the process.

The SCT will highlight any errors it encounters and walk you through the process of resolving them. It will also highlight database practices that may not translate well to distributed SQL, such as auto-incrementing identifiers, etc.

Loading Data into CockroachDB with MOLT Fetch

The easiest way to load data from a source database into CockroachDB is via **MOLT Fetch**. To keep things simple, we'll insecurely import data from Postgres using a direct copy (`COPY FROM` with no use of intermediate files). In reality, your imports would be secure and you should follow the MOLT Fetch **best practices**.

We'll start by running a local instance of both Postgres (our source database) and CockroachDB (our target database) using Docker:

```
# Postgres  
docker run -d \  
  --name postgres \  
  -p 5432:5432 \  
  -e POSTGRES_PASSWORD=password \  
  postgres:15.2-alpine
```

```
# CockroachDB
docker run -d \
  --name=cockroachdb \
  -p 26257:26257 \
  -p 8080:8080 \
  -v "data:/cockroach/cockroach-data" \
  cockroachdb/cockroach:v24.2.0 start-single-node \
  --insecure
```

Next, we'll insert some data into our Postgres source database:

```
PGPASSWORD=password psql -h localhost -U postgres

CREATE DATABASE eg;
\c eg;

CREATE TABLE postgres_table (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  value DECIMAL NOT NULL,
  ts TIMESTAMP NOT NULL
);

INSERT INTO postgres_table (value, ts)
SELECT
  random() * 100,
  t
FROM generate_series(
  '2022-01-01'::TIMESTAMP,
  '2023-01-01'::TIMESTAMP,
  '1 hour'::INTERVAL
) t;
```

Then, we'll prepare our CockroachDB target database:

```
cockroach sql --host localhost:26257 --insecure

CREATE DATABASE eg;
USE eg;

CREATE TABLE postgres_table (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  value DECIMAL NOT NULL,
  ts TIMESTAMP NOT NULL
);
```

Finally, we'll run MOLT Fetch, using our Postgres database as the `--source` and our CockroachDB database as the `--target`. The `--allow-tls-mode-disable` flag is required here, as the target CockroachDB cluster is running insecurely:

```
molt fetch \
  --source 'postgres://postgres:password@localhost:5432/eg?sslmode=disable' \
  --target 'postgres://root@localhost:26257/eg?sslmode=disable' \
  --table-filter 'postgres_table' \
  --direct-copy \
```

```
--cleanup \
--allow-tls-mode-disable
```

Running a SELECT query against the CockroachDB version of the table shows that the import has taken place:

```
SELECT COUNT(*) FROM postgres_table;
8761
```

However, to assert that our data has been migrated *correctly*, we can use MOLT Verify:

```
molt verify \
--source 'postgres://postgres:password@localhost:5432/eg?sslmode=disable' \
--target 'postgres://root@localhost:26257/eg?sslmode=disable' \
--table-filter 'postgres_table' \
--allow-tls-mode-disable

...
{"level":"info","net_duration_ms":161.29025,"net_duration":"000h 00m 00s",
"message":"verification complete"}
```

Our CockroachDB target database schema and data exactly matches our original Postgres schema and data; the migration has been successful.

Loading Data into CockroachDB with IMPORT INTO

If you would prefer to import data using the IMPORT INTO statement, once data is exported to CSV and we have valid CockroachDB DDL, we can import the data using the IMPORT INTO command. For instance, this statement will load the EMPLOYEES data from Oracle that we first looked at back in [Example 7-2](#), assuming that we've loaded the data into node local storage:

```
defaultdb>
IMPORT INTO "EMPLOYEES"
( "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "EMAIL",
  "PHONE_NUMBER", "HIRE_DATE",
  "JOB_ID", "SALARY", "COMMISSION_PCT", "MANAGER_ID",
  "DEPARTMENT_ID")
CSV DATA ("nodelocal://1/employees.csv") WITH skip='1', nullif = '';

      job_id      | status | fraction_c | rows | index_entries | bytes
-----+-----+-----+-----+-----+-----
678984786365906945| succeeded |          1 | 107 |              0 | 10955
(1 row)
```

Note that during an IMPORT INTO, all foreign key constraints are invalidated on the target table, so you have to use **VALIDATE CONSTRAINT** to revalidate the data.

Directly Importing PostgreSQL or MySQL Dumps

In some circumstances, you can directly import PostgreSQL or MySQL dump files in a single operation. The `IMPORT PGDUMP` and `IMPORT MYSQLDUMP` commands can read an entire dump file and attempt to create the tables and load the data directly.

For PostgreSQL, use `pg_dump` to dump the required data. In this case, we dump the DVD Rental databases:

```
$ pg_dump dvdrental >dvdrental.pgdump
```

From MySQL, use `mysqldump`. In this example, we dump the Sakila sample database:

```
$ mysqldump -u root Sakila >Sakila.dump
```

Now we load the files concerned into userfile storage:

```
$ cockroach userfile upload Sakila.dump Sakila.dump --url $CRDB_CLUSTER
uploaded to userfile://defaultdb.public.userfiles_guy/Sakila.dump
$ cockroach userfile upload dvdrental.pgdump --url $CRDB_CLUSTER
uploaded to userfile://defaultdb.public.userfiles_guy/dvdrental.pgdump
```

Then let's try to import the MySQL dump file:

```
guy@sticky-donkey-8sd:26257/sakila>
IMPORT MYSQLDUMP
'userfile://defaultdb.public.userfiles_guy/Sakila.dump' ;
ERROR: unimplemented: cannot import GEOMETRY columns at this time
SQLSTATE: 0A000
HINT: You have attempted to use a feature that is not yet implemented.
See: https://go.crdb.dev/issue-v/32559/v21.1
```

Whoops. Let's try the PostgreSQL dump:

```
guy@sticky-donkey-8sd:26257/dvdrental>
IMPORT PGDUMP
'userfile://defaultdb.public.userfiles_guy/dvdrental.pgdump'
WITH ignore_unsupported_statements;
ERROR: cannot add a SET NULL cascading action on column "payment.rental_id"
which has a NOT NULL constraint
SQLSTATE: 42830
guy@sticky-donkey-8sd:26257/dvdrental>
```

No, that didn't work either. Alas, this is the most common experience when importing dump files directly. Unless the source database uses only very vanilla schema objects, something in the dump file will not translate directly to CockroachDB. You are, therefore, probably better off extracting the DDL first—as shown in the previous sections—and hand-converting it to CockroachDB-compatible syntax and features.

Synchronizing and Switching Over

The extract DDL, export CSV, and import procedure is a good way to move over data for development and testing purposes. It's possible that you will be able to use the same technique for a production system, but it definitely requires downtime.

Because data from multiple tables will be dumped to CSV at different times, or at least within distinct transactional scopes, there will be no guarantee of consistency between tables. If you dump CSVs from an active transactional system, it's possible you will encounter a foreign key violation on import. Even worse, you may receive no error but still have inconsistent data. Furthermore, transactions that hit the production system during or after the dump will not be included. So if you plan on using a dump and load strategy, you'll need to ensure that the dumps occur during a period of inactivity.

Consequently, most production system cutovers involve sophisticated synchronization mechanisms between the source and the target system. In some cases, both systems are run concurrently, and the application layer switches from one system to another with no downtime or inconsistency. There's no one single correct way to do this and no off-the-shelf solution that can perform this sort of synchronization and seamless switchover. Each conversion usually involves some bespoke synchronization mechanism.

We can't cover every conceivable scenario here, but let's look at a simplified example in which we keep data in a CockroachDB database in sync with data in a PostgreSQL database before a seamless switchover.

Let's say we are migrating from the DVDrental sample database in PostgreSQL. Our DVD business is booming (who'd have thought?), and we need to move to a more scalable database platform.

The most important table in our application is the rental table. We can prevent updates to other tables for short intervals, but without rentals, we have no revenue. So we need to make sure that we can continue to add new rental records continuously until we are ready to switch over.

We can extract the PostgreSQL CREATE TABLE statement using the following command:

```
pg_dump -s dvdrental -t rental
```

A few edits to the DDL results in a CockroachDB-compatible table:

```
CREATE TABLE rental (  
  rental_id integer PRIMARY KEY,  
  rental_date timestamp without time zone NOT NULL,  
  inventory_id integer NOT NULL,  
  customer_id smallint NOT NULL,  
  return_date timestamp without time zone,
```

```

        staff_id smallint NOT NULL,
        last_update timestamp without time zone DEFAULT now() NOT NULL
    );

```

To ensure that we can add new rows to the PostgreSQL version of that table during the transition, we are going to set up a CDC stream between PostgreSQL and CockroachDB.

The following statement creates a CDC “slot” in PostgreSQL:³

```

dvdrental=#
SELECT * FROM
    pg_create_logical_replication_slot('cockroach_migration', 'wal2json');

 slot_name      | xlog_position
-----+-----
cockroach_migration | 0/1CF4F60

```

We’re using the **wal2json** plug-in to format the change records in an easy-to-parse JSON structure. This package might need to be installed on a default PostgreSQL deployment.

The first parameter to `pg_create_logical_replication_slot` is a unique name we are going to use to access the CDC stream. The second parameter specifies a plug-in that can be used to control how the change data feed is formatted and accessed. The `test_decoding` plug-in is included in the base PostgreSQL distribution; it translates the changefeed to text format, which we will use in our synchronization program.

Once the changefeed is created, we can use the `pg_logical_slot_peek_changes` function to pull change records from the stream. For instance:

```

dvdrental=#
SELECT *
FROM pg_logical_slot_get_changes('cockroach_migration',
                                NULL, NULL);

location | xid | data
-----+-----
0/1CFECC0 | 742 | [{"change":[{"kind":"insert","schema":"public",
"table":"rental",
"columnnames":["rental_id","rental_date","inventory_id","customer_id",
"return_date","staff_id","last_update"],
"columntypes":["integer",
"timestamp without time zone","integer","smallint",
"timestamp without time zone","smallint",
"timestamp without time zone"],
"columnvalues":["16051","2021-08-19 08:28:52.748047",
367,130,null,1,"2006-02-15 21:30:53"]}]]}

```

3 The PostgreSQL configuration parameter `wal_level` must be set to “logical” for this command to succeed.

You can see in the output a JSON document containing details about a new row in the rental table.

Our top-level migrate and synchronize logic might look something like this (in JavaScript):

```
await pgConnection.connect();
await crdbConnection.connect();

await startReplication();
await copyTable();
while (true) {
  await syncChanges();
}
```

We create a connection to PostgreSQL and to CockroachDB. We commence the PostgreSQL CDC stream, copy the table data from PostgreSQL to CockroachDB, then continuously synchronize any changes from PostgreSQL to CockroachDB.

The startReplication function is simple—it creates the replication slot that we’re going to use to capture the changefeed:

```
async function startReplication() {
  try {
    const replicationStatus = await pgConnection.query(
      `SELECT *
       FROM pg_create_logical_replication_slot(
         'cockroach_migration', 'wal2json')`
    );
    console.log(replicationStatus.rows[0]);
  } catch (error) {
    console.warn(
      'Warning ', error.message
    );
  }
}
```

The copyTable() function copies data from PostgreSQL to CockroachDB using SELECT and INSERT statements. You could substitute a more performant extract and load procedure using the principles outlined earlier in the chapter.

```
async function copyTable() {
  const pgData = await pgConnection.query('SELECT * from rental');
  for (let rowNo = 0; rowNo < pgData.rows.length; rowNo += 1) {
    const row = pgData.rows[rowNo];
    await crdbConnection.query(
      `INSERT INTO rental (rental_id,rental_date,inventory_id,
        customer_id,return_date,staff_id,last_update)
       VALUES ($1,$2,$3,$4,$5,$6,$7)`,
      [row.rental_id, row.rental_date, row.inventory_id, row.customer_id,
```

```

        row.return_date, row.staff_id, row.last_update]
    );
}
console.log(pgData.rows.length, ' rows copied');
}

```

Once the bulk of the data is migrated, we can keep the CockroachDB database in sync with PostgreSQL by monitoring the changefeed and issuing INSERTs or UPSERTs as appropriate. Each time we call `pg_logical_slot_get_changes`, we can retrieve one or more rows in the output:

```

async function syncChanges() {
    const changeSQL = await pgConnection.query(
        `SELECT * FROM pg_logical_slot_get_changes(
            'cockroach_migration', NULL, NULL)`
    );
    for (rowNo = 0; rowNo < changeSQL.rows.length; rowNo++) {
        const changePayload = changeSQL.rows[rowNo];
        await processSingleChange(changePayload);
    }
}

```

We pull change records using the `pg_logical_slot_get_changes` function. Note that this function removes change records once they are retrieved, so we never process the same record twice. However, we need to be sure that we definitely apply the change. There's no transactional consistency available to us here!

In the following example we process a single change record. This simplified implementation handles only INSERTs into the `rental` table but could be modified to dynamically process any changes to any table:

```

async function processSingleChange(rawPayload) {
    const jsonPayload = JSON.parse(rawPayload.data);
    if ('change' in jsonPayload) {
        for (let cindx = 0; cindx < jsonPayload.change.length; cindx++) {
            const changeData = jsonPayload.change[cindx];
            const columnCount = changeData.columnnames.length;
            if (changeData.kind === 'insert' && changeData.table === 'rental') {
                const newValue = {}; // Object containing CDC row values
                for (let colno = 0; colno < columnCount; colno++) {
                    const columnName = changeData.columnnames[colno];
                    newValue[columnName] = changeData.columnvalues[colno];
                }
                const insertSQL = `
                    UPSERT into rental
                        (rental_id,rental_date,inventory_id,
                         customer_id,return_date,staff_id,last_update)
                    VALUES ($1,$2,$3,$4,$5,$6,$7)`;
                const result = await crdbConnection.query(insertSQL,
                    [newValue.rental_id, newValue.rental_date,
                     newValue.inventory_id, newValue.customer_id,
                     newValue.return_date, newValue.staff_id,

```

```

        newValue.last_update ]);
    console.log(result.rowCount, 'rows', result.command + 'ed');
  }
}
}
}

```

This code is a little hard to read but essentially retrieves the JSON structure containing the new row from PostgreSQL, unpacks it, and inserts it into CockroachDB.

Now, both databases can run in parallel, and all new rentals will be automatically copied from PostgreSQL to CockroachDB. When we're ready to switch over, we can switch the application over from PostgreSQL to CockroachDB and shut down the PostgreSQL database.

This is a very simplified example of database synchronization. There's a lot more we would need to do in real life:

- We should implement code to synchronize other tables.
- We should implement a retry mechanism for retryable errors.
- We should handle UPDATES and DELETES as well as INSERTs.
- We may need to consider using bulk inserts and parallel threads of execution to ensure that the CockroachDB target does not fall behind the PostgreSQL source.
- If the application is distributed, there may be a need to prevent inserts from hitting the PostgreSQL database after inserts commence on the CockroachDB side. *All* inserts into PostgreSQL should cease before *any* inserts commence on CockroachDB.

When a CDC option is not available, you might be able to implement a similar functionality using database triggers. For example, if you wanted to synchronize with an Oracle database, you could write database triggers that capture changes to source tables and send these to a STAGING table.

Updating Application Code

Migrating the database schema and data is obviously only one step in migrating to CockroachDB. Application code will almost certainly have to change as well. The following considerations are important:

- Convert the application to use a PostgreSQL-compatible driver. For instance, if you're using the Node.js `oracledb` driver, you will want to convert your application to the `pg` driver. We discussed the range of compatible drivers available for each language in [Chapter 6](#). This won't be necessary if your application is currently using PostgreSQL.

- If your application uses an ORM, then it needs to move to an ORM that is PostgreSQL-compatible. Most ORMs support PostgreSQL already, so this might be only a minor concern. [Chapter 6](#) discusses ORMs that are compatible with CockroachDB.
- You should modify any SQL calls that use non-PostgreSQL syntax. For example, Oracle uses SYSDATE to refer to the current time, so references to SYSDATE should be changed to use NOW() or CURRENT_TIMESTAMP. Similarly, Oracle has a unique syntax for hierarchical queries and requires a “FROM dual” for SELECTs that operate on no data.
- It should go without saying that you should test your application thoroughly after any migration. Your testing should ensure that the migrated application is working correctly and that performance requirements are met.

The migration from one SQL database to another is certainly less involved than migrating to or from a SQL database to a non-SQL system such as MongoDB or Cassandra. Nevertheless, migrating from one database system to another is far from trivial and should be approached seriously.

Mainframe Migrations

Even though they are widely considered to be legacy, it’s believed that around **90% of the world’s global card transactions** are processed by mainframes. The overwhelming majority of mainframe users in the banking industry have plans to migrate their workloads out of the mainframe and into the cloud.

CockroachDB excels in mainframe migrations due to its compatibility with mainframe CPU architectures like S/390X and z/Architecture, its ability to operate across machines with different CPU architectures, and its capacity for horizontal scaling.

There are numerous options available for customers wishing to migrate off mainframe with CockroachDB, with the options outlined in [“Migrating from Another Database” on page 224](#) being equally suited to mainframe migrations.

A happy side effect of CockroachDB’s ability to horizontally scale and run across multiple CPU architectures is zero-downtime “stretch migrations.” A stretch migration from mainframe is simply a migration whereby CockroachDB is deployed across both your mainframe and your target environment (e.g., the cloud). The process might look as follows:

1. Create a CockroachDB cluster with nodes on your existing **mainframe architecture**.
2. Migrate data from your existing mainframe database (e.g., Db2) into CockroachDB.

3. Start CockroachDB nodes in your target environment and have them join your mainframe cluster.
4. One by one, decommission the nodes in your mainframe until only your cloud nodes remain.

Exporting CockroachDB Data

So far we've seen how to bring data into CockroachDB. However, if we have an application that wants to consume CockroachDB data, we have a number of options:

- We can simply write a program that reads data using `SELECT` statements and outputs it to files or feeds it to APIs.
- We can use the `EXPORT` command to dump data into flat files.
- We can use CDC to automatically capture changes to nominated tables.

We spent a lot of time looking at the CockroachDB `SELECT` command and CockroachDB-compatible drivers in Chapters 4 and 6. Let's look now at how we can use the `EXPORT` command to unload data.

We introduced the `EXPORT` command in Chapter 5. The syntax of `EXPORT` is shown in Figure 7-3.

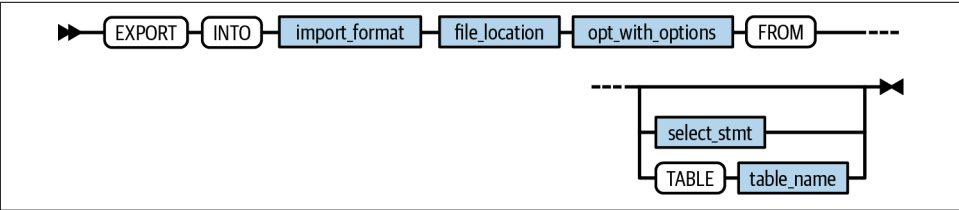


Figure 7-3. `EXPORT` statement syntax

`EXPORT` can write data to any of the file sources we discussed earlier in the context of `IMPORT INTO`. It can harness `EXTERNAL CONNECTION` URIs (e.g., `external://some_s3_bucket`) to prevent database operators from handling sensitive credentials.

Here's an export of the `rides` table to `nodeLocal` storage:

```
movr>
EXPORT into csv 'nodeLocal://self/rides'
  WITH nullas='' FROM TABLE rides;
```

filename	rows	bytes
export16976587190021a80000000000000001-n1.0.csv	13409	2374556

As discussed earlier, `node_local` storage is located in a deployment-specific location. On a single-node server, we might find it in the `extern` directory under the installation location:

```
$ ls -l /usr/local/var/cockroach/extern/rides
total 4640
-rw----- 1 guyharrison admin 2374556 2 Aug 15:17
  export16976587190021a80000000000000001-n1.0.csv
```

We can also export data to cloud storage locations, such as an Amazon S3 bucket:

```
movr> EXPORT INTO CSV
"s3://cockroachdefinitiveguide/?
AWS_ACCESS_KEY_ID=my-access-key&
AWS_SECRET_ACCESS_KEY=my-secret-key"
WITH nullas='' FROM TABLE rides;
```

filename	rows	bytes
export169765eed62b37b00000000000000001-n1.0.csv	13409	2374556

(1 row)

The `EXPORT` command can accept a `SELECT` statement that filters and projects the data to be exported:

```
movr> EXPORT INTO CSV
'userfile://defaultdb.public.userfiles_guy/'
WITH nullas='' FROM
SELECT rider_id,start_time,end_time
FROM rides WHERE city='amsterdam';
```

filename	rows	bytes
export169766268b2faad80000000000000001-n1.0.csv	154	10972

(1 row)

`EXPORT` provides some switches to control the formatting, compression, and chunking of output. See the [CockroachDB documentation](#) for information about these options.

There is also a quick-and-dirty way to export data from CockroachDB in CSV format. We can simply pass a `SELECT` statement to the CockroachDB shell and specify the `--format=csv` flag to request that the output be formatted as CSV:

```
$ cockroach sql -e "SELECT * from movr.rides;" \
--format=csv --url $CRDB_CLUSTER > rides.csv
```


Change Data Capture

CDC is an interface that tracks changes to nominated tables and allows you to respond to these changes or to forward them to an external system.

CockroachDB CDC comes in two forms:

- Core CDC allows you to stream changes to the host client. It's a simple method to both test CDC statements inside a SQL session and stream those changes to a downstream consumer.
- Enterprise CDC allows you to feed changes to downstream systems automatically. These downstream sources can be messaging systems such as Kafka, cloud storage such as S3, or a generic webhook.
- Core CDC allows you to stream changes to the host client. It's a simple method to both test CDC statements inside a SQL session and stream those changes to a downstream consumer.
- Enterprise CDC allows you to feed changes to downstream systems automatically. These downstream sources can be messaging systems such as Kafka, cloud storage such as S3, or a generic webhook.

Core Change Data Capture

Core CDC feeds changes to a client program that can then respond to these changes as it sees fit. Let's look at Core CDC in the CockroachDB shell.

The shell must be started with the `--format=csv` flag. Otherwise, the client keeps waiting for the data from the changefeed to end so it can construct a nice ANSI table display. Unfortunately, changefeeds stream indefinitely, so this backfires.

We also have to set the cluster setting `kv.rangefeed.enabled` to `true`.

CDC implies some performance overhead, estimated as between 5% and 10%. The actual overhead will depend heavily on the transaction rate of the system. Systems that are read-heavy might see less overhead:

```
$ cockroach sql --format csv --url $CRDB_CLUSTER
#
# Welcome to the CockroachDB SQL shell.

defaultdb>
use movr;
SET

Time: 29ms total (execution 11ms / network 17ms)
```

```
guy@sticky-donkey-8sd:26257/movr>
SET CLUSTER SETTING kv.rangefeed.enabled = true;
SET CLUSTER SETTING
```

Time: 35ms total (execution 18ms / network 17ms)

```
guy@sticky-donkey-8sd:26257/movr>
EXPERIMENTAL CHANGEFEED FOR rides;
```

```
table,key,value
rides,[""washington dc"", ""547ae147-ae14-4c00-8000-0000000000a5""],
{"after": {"city": "washington dc",
"end_address": "59500 Eddie Union Apt. 0",
"end_time": "2018-12-25T10:04:05",
"id": "547ae147-ae14-4c00-8000-0000000000a5",
"revenue": 14.00,
"rider_id": "428f5c28-f5c2-4000-8000-00000000000d",
"start_address": "81047 Perez Views",
"start_time": "2018-12-25T03:04:05",
"vehicle_city": "washington dc",
"vehicle_id": "44444444-4444-4400-8000-000000000004"}}}
rides,[""washington dc"", ""54fdf3b6-45a1-4c00-8000-0000000000a6""],
{"after": {"city": "washington dc",
"end_address": "78010 Woodard Plaza Apt. 9", "e
```

Initially, the changefeed displays all the current data in the table. It then will stop and wait for any changes to be made to the table concerned. For example, if we perform this transaction in a separate session:

```
guy@sticky-donkey-8sd:26257/movr>
update rides set revenue=revenue*1.1
  where id='c50d42a9-62ae-4501-97e0-38b9626502a1';
UPDATE 1
```

this new row of data will be emitted by the changefeed session:

```
rides,[""amsterdam"", ""c50d42a9-62ae-4501-97e0-38b9626502a1""],
{"after": {"city": "amsterdam", "end_address": null,
"end_time": null, "id": "c50d42a9-62ae-4501-97e0-38b9626502a1",
"revenue": 10.00, "rider_id": "ae147ae1-47ae-4800-8000-000000000022",
"start_address": "54948 Direct St Via Apt. 28",
"start_time": "2021-07-02T08:09:51.062777",
"vehicle_city": null,
"vehicle_id": "bbbbbbbbb-bbbb-4800-8000-00000000000b"}}}
```

The changefeed will continue indefinitely. To terminate the changefeed, you would hit Control-C from a SQL prompt or, in a programming language, simply stop fetching new rows from the cursor.

The output of the CDC stream contains three columns. The first is the table name; the second is the primary key of the row concerned; and the third is a JSON-formatted representation of the row's new value. We can add the timestamp—and push out timestamps periodically during idle periods with the following syntax:

```
guy@sticky-donkey-8sd:26257/movr>
  EXPERIMENTAL CHANGEFEED FOR rides with updated, resolved='10s';
<snip>
rides,[""amsterdam"", ""c5180421-4421-495f-ae7d-fa0ae08a7258""],
{"after": {"city": ""amsterdam"", "end_address": null,
"end_time": null, "id": ""c5180421-4421-495f-ae7d-fa0ae08a7258"",
"revenue": null, "rider_id": ""ae147ae1-47ae-4800-8000-000000000022"",
"start_address": ""2 Pool St Via Apt. 28"",
"start_time": ""2021-07-02T08:10:52.678899"",
"vehicle_city": null,
"vehicle_id": ""bbbbbbbbb-bbbb-4800-8000-00000000000b""},
"updated": ""1627285876946192391.0000000000""}
NULL,NULL,{"resolved": ""1627285883727456610.0000000000""}
NULL,NULL,{"resolved": ""1627285893772664924.0000000000""}
```

The resolved attribute tracks the timestamp that the changefeed has “caught up to.” So if you get a resolved message with timestamp *t*, you know that you have seen every message committed before timestamp *t*. This can be used for duplicate tracking or to downstream reorder messages. The resolved= flag specifies the minimum time between resolved messages.

If you’ve been tracking the timestamps emitted by the changefeed, you can use that timestamp to resume a changefeed at a later date. In the previous example, the most recent timestamp is 1627285893772664924.0000000000, and we use that to resume the feed:

```
guy@sticky-donkey-8sd:26257/movr>
  EXPERIMENTAL CHANGEFEED FOR rides with updated, resolved='10s',
  cursor='1627285913863014169.0000000000';
```

Using the Changefeed Programmatically

There's not much you can really do with a changefeed from the CockroachDB shell. More typically, we would consume the changefeed programmatically. The following code snippet shows an example of consuming the changefeed in Java:

```
Statement stmt = connection.createStatement();
stmt.setFetchSize(1);
stmt.execute("use movr");
ResultSet rs = stmt.executeQuery("EXPERIMENTAL CHANGEFEED
FOR rides WITH updated, resolved;");
while (rs.next()) {
    String tableName = rs.getString("table");
    String keyString="";
    byte[] keyBytes = rs.getBytes("key");
    if (!rs.wasNull()) { // Key is null for resolved messages
        keyString=new String(keyBytes, "UTF-8");
    }
    JsonObject valueJson = JsonParser.parseString(
        new String(rs.getBytes("value"),
"UTF-8")).getAsJsonObject();
    if (valueJson.has("resolved")) {
        System.out.println("Resolved timestamp: "+valueJson.get("resolved"));
    } else if (valueJson.has("updated")) {
        System.out.printf("Table %s new value for key %s at %s is\n\t%s\n",
            tableName, keyString, valueJson.get("updated"),
            valueJson.get("after"));
    }
}
```

The `setFetchSize` assignment is important. By default, `executeQuery` will pull all the rows from the result set before returning control to the program. Because changefeeds *never* return a last result, you need to set `fetchSize` to a value of 1 to prevent the program from hanging and to ensure that rows are emitted as they are received.

This program simply prints the contents of the changefeed in a human-readable format. However, in practice, we could use this code as the basis for something more ambitious. For example:

- The changefeed could be written to flat files that will be loaded into a downstream system such as Hadoop or Snowflake.
- The changefeed could be examined to perform application-specific operations. For instance, every time a ride is commenced, we might send an alert to drivers in the area or update a real-time map of rides in progress.
- The changefeed could be used to trigger a refresh or a materialized view or to manually update a summary table.

Changefeeds can be consumed by some—but not all—database drivers. Of the four drivers we’ve focused on in this book, only Java and Go `pgx` work seamlessly with changefeeds.

The default query interface in the Node.js `pg` package attempts to retrieve all query results before allowing the program to navigate through the results. The following example illustrates this approach:

```
async function main() {
  try {
    await connection.connect();
    const sql = 'EXPERIMENTAL CHANGEFEED FOR movr.rides WITH updated, resolved';
    const query = new pg.Query(sql);

    query.on('row', (row) => {
      console.log(row.value.toString());
    });
    query.on('error', (error) => console.error(error));
    query.on('end', () => {
      console.log('Changefeed terminated');
      process.exit(0);
    });
    await connection.query(query);
  } catch (error) {
    console.error(error.stack);
  }
}
```

In the Python `psycopg2` driver, all cursor calls—`fetchmany()`, `fetchone()`, `fetchall()`—can be applied only to a cursor that has retrieved all the rows from the query. As we’ve noted earlier, changefeeds *never* provide a “last” row. Consequently, changefeeds cannot currently be used with `psycopg2`. `Psycopg3` includes a **stream interface** that was experimental at the time of writing but suitable for change streams.

Enterprise Change Data Capture

Enterprise CDC has the following advantages over Core CDC:

- Once created, an Enterprise CDC is continuously running within the CockroachDB cluster—you are not responsible for running the job or tracking the last resolved timestamp.
- Enterprise CDC output can be directed to messaging systems like Kafka, cloud storage destinations like S3, or to webhook endpoints.
- Enterprise change streams operate in a distributed manner, doling out the workload across the cluster.

CDC Queries

CDC queries allow you to write changefeeds that transform rows before they're emitted. This is useful for scenarios where consumers are expecting events in a different format to how the data is stored.

In an ETL pipeline, CDC queries can be used to satisfy the extract and transform phases. The changefeed extracts rows as they are inserted, updated, or deleted and transforms them before publishing them.

In this section, we'll create a CDC queries changefeed to transform rows before publishing them to Kafka (and our downstream consumers).

First, we'll create a cluster and start a local Kafka instance. For brevity, we'll use Redpanda, which is Kafka-compatible but requires fewer components to run. We'll start Redpanda with Docker Compose, downloading the compose file from the Redpanda docs:

```
curl -L -s https://cockroach.ch/redpanda_yaml \
  -o docker-compose.yml

docker compose up -d

cockroach demo --insecure --no-example-database
```

Next, we'll create a table. This will be monitored by our CDC queries changefeed, and its rows will be transformed before being published:

```
CREATE TABLE temperature_sensor (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sensor_id UUID NOT NULL,
  temp_c DECIMAL NOT NULL,
  location GEOMETRY NOT NULL,
  enabled BOOL NOT NULL DEFAULT false,
  ts TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Then, we'll create a topic and start consuming messages from it. For this, we can harness Redpanda's own `rpk` CLI:

```
docker exec -it redpanda-0 rpk topic create temperature
docker exec -it redpanda-0 rpk topic consume temperature
```

It's now time to create the changefeed. Let's assume the following transformation rules for our CDC events:

- We only wish to publish events for enabled sensors.
- Some consumers prefer to see temperatures in Celsius; others prefer Fahrenheit.
- Consumers prefer to receive locations in Geohash format.

```
CREATE CHANGEFEED INTO 'kafka://localhost:19092?topic_name=temperature' AS
SELECT
    sensor_id,
    round(temp_c, 3) AS temp_c,
    round((temp_c * 9/5) + 32, 3) AS temp_f,
    st_geohash(location) AS location,
    ts
FROM temperature_sensor
WHERE enabled;
```

Finally, we'll insert some data to show CDC queries in action:

```
-- Data for a disabled sensor (won't be published).
INSERT INTO temperature_sensor (id, sensor_id, temp_c, location, enabled)
VALUES (
    '508424fa-0b5b-45f6-a907-d25831440256',
    '083845e8-de11-4209-bb4e-ad8a342d5103',
    12.401,
    'POINT(-0.1424448 51.5394466)',
    false
);

-- Data for an enabled sensor (will be published).
INSERT INTO temperature_sensor (id, sensor_id, temp_c, location, enabled)
VALUES (
    '12083e03-3ce5-411f-8f18-2e182f0aa391',
    '03370f5c-0620-4f9f-878e-3eb13441c8c1',
    12.485,
    'POINT(-0.0757618 51.5236428)',
    true
);

-- Enable the disabled sensor (publishing the original event).
UPDATE temperature_sensor
SET enabled = true
WHERE id = '508424fa-0b5b-45f6-a907-d25831440256';
```

Our rpk consumer will immediately receive the transformed rows from CockroachDB, printing something similar to the following (note that our requested transformations have been applied to the “value” field):

```
{
  "topic": "temperature",
  "key": "[\"12083e03-3ce5-411f-8f18-2e182f0aa391\"]",
  "value": "{\"location\": \"gcpvn78ffx5vj53...\", \"sensor_id\": \"0337...\"}",
  "timestamp": 1726136650509,
  "partition": 0,
  "offset": 0
}
```

```
{
  "topic": "temperature",
  "key": "[\"508424fa-0b5b-45f6-a907-d25831440256\"]",
  "value": "{\"location\": \"gcpvhy2r0c1qqwm...\", \"sensor_id\": \"0838...\"}",
  "timestamp": 1726136654514,
  "partition": 0,
  "offset": 1
}
```

Using cloud sinks

Cloud sinks are storage destinations offered by public cloud systems. These are defined using the same mechanisms that we discussed when using the `IMPORT INTO` command. See the [CockroachDB documentation](#) for details on specifying authentication and configuring these sinks.

Cloud sinks are a perfect mechanism for connecting two disparate systems that share only the ability to connect to the internet and, in particular, are the perfect medium for cloud-based applications. However, do bear in mind that there is a certain latency involved in writing to cloud sinks—changes are batched to ensure that the destination does not fill up with many tiny files, so some delay should be expected.

In this section, we'll configure an enterprise changefeed for tables in the `movr` sample database, which we will feed to an Amazon S3 bucket.

The syntax for creating a changefeed is relatively simple. You specify the Amazon S3 bucket name, optionally adding the AWS access key and secret access key. Access keys may also be [loaded from environment variables](#):

```
defaultdb>
CREATE CHANGEFEED FOR TABLE movr.rides,movr.users,movr.vehicles INTO
  's3://cockroachdefinitiveguide/movrFeed?
  AWS_ACCESS_KEY_ID=my_access_key&
  AWS_SECRET_ACCESS_KEY=my_secret_access_key
  WITH updated, resolved='10s';

      job_id
-----
680908602728218625
(1 row)
```

`CREATE CHANGEFEED` creates a background job that will continuously process the changefeed. We can examine the job with the `SHOW JOB` or `SHOW CHANGEFEED JOBS` command:

```
defaultdb> \set display_format=records;
defaultdb> show job 680908602728218625;
```



```

-[ RECORD 1 ]
job_id          | 680908602728218625
job_type        | CHANGEFEED
description     | CREATE CHANGEFEED FOR TABLE movr.rides, movr.users,
movr.vehicles INTO 's3://cockroachdefinitiveguide/movrFeed?
AWS_ACCESS_KEY_ID=my_access_key&
AWS_SECRET_ACCESS_KEY=redacted' WITH resolved = '10s', updated
statement       |
user_name       | root
status          | running
running_status  | running: resolved=1627867221.708330000,0
created         | 2021-08-02 01:20:14.797451+00:00:00
started        | 2021-08-02 01:20:15.014177+00:00:00
finished        | NULL
modified        | 2021-08-02 01:21:09.273081+00:00:00
fraction_completed | NULL
error           |
coordinator_id  | NULL

```

Time: 40ms total (execution 40ms / network 0ms)

We can also monitor jobs from the Jobs section of the admin console, as shown in [Figure 7-4](#).

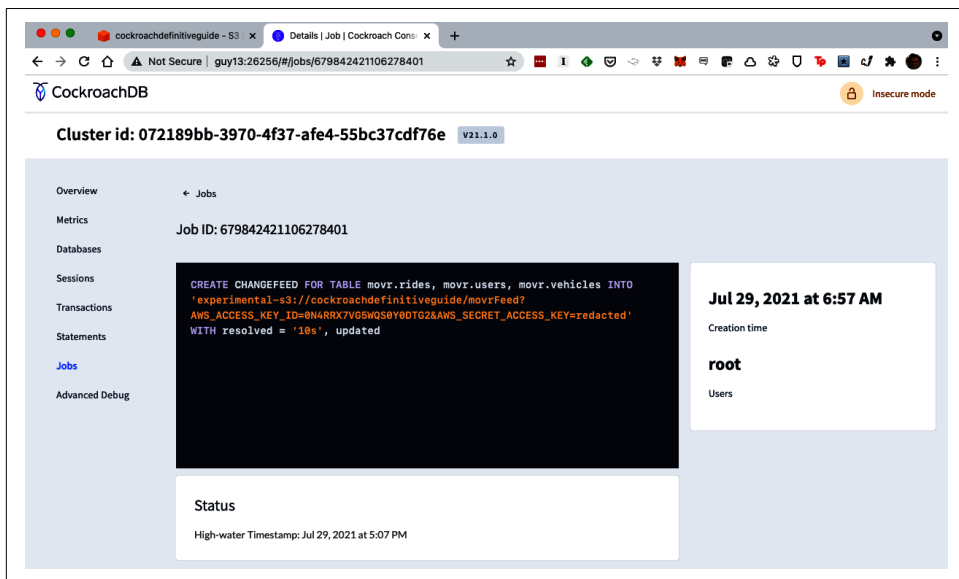


Figure 7-4. Showing job status in the Jobs console

We are able to examine the changefeed output from the Amazon S3 console, as shown in [Figure 7-5](#).

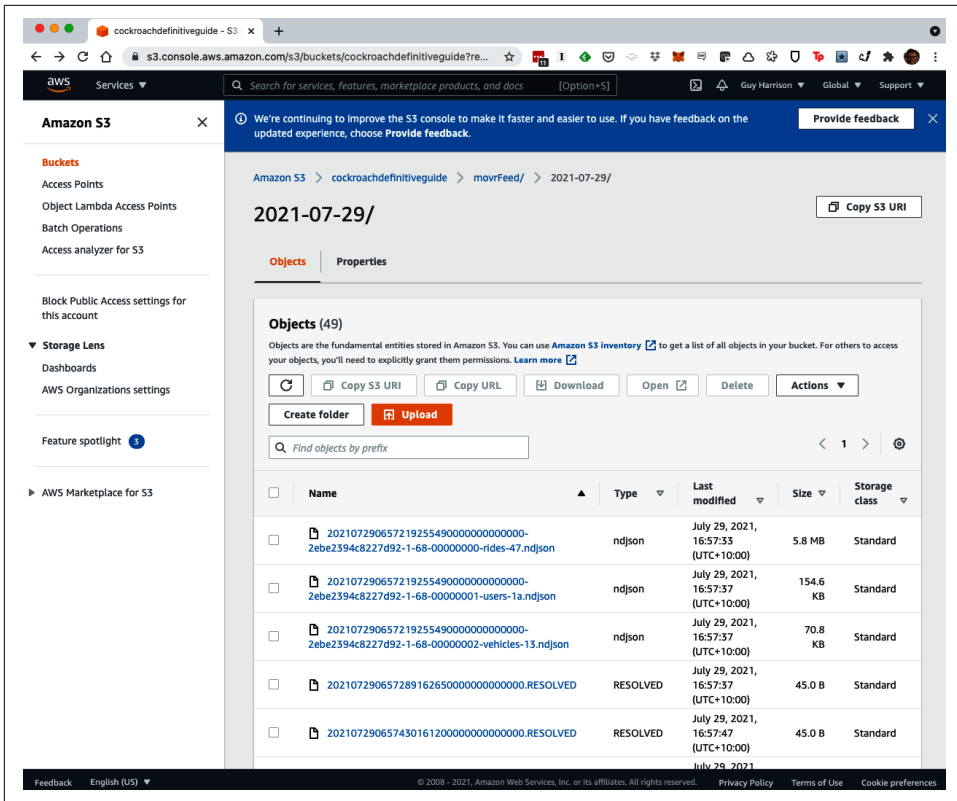


Figure 7-5. Changefeed files in the Amazon S3 console

Webhook sinks

CockroachDB's CDC webhook sink allows changefeed messages to be published to arbitrary HTTP endpoints. A webhook sink URL has the following format:

```
'webhook-https://{your-webhook-endpoint}?insecure_tls_skip_verify=true'
```

See the [CockroachDB documentation](#) for more information.

Managing jobs

Once commenced, Enterprise changefeeds continue indefinitely. You can examine the status and configuration of these jobs in the admin console, as shown in [Figure 7-4](#), or from using the `SHOW JOBS` command. Changefeeds can be paused or canceled using `PAUSE JOB` or `CANCEL JOB`:

```
movr>
SELECT job_id,description,running_status
FROM [show jobs]
WHERE job_type='CHANGEFEED';
```

job_id	description
680908602728218625	CREATE CHANGEFEED FOR TABLE movr.rides, movr.users, ...
680913149274030081	CREATE CHANGEFEED FOR TABLE movr.rides, movr.users, ...

(2 rows)

Time: 29ms total (execution 28ms / network 0ms)

```
movr> pause job 680913149274030081;
PAUSE JOBS 1
```

Time: 57ms total (execution 56ms / network 0ms)

```
movr> cancel job 680913149274030081;
CANCEL JOBS 1
```

Paused changefeeds can be resumed with the `RESUME JOB` command.

Errors in the changefeed will normally be revealed by the `SHOW JOB` command. For example, here we see a changefeed created with an invalid access key:

```
movr> \set display_format=records;
movr> SHOW JOB 680937397724741633;
-[ RECORD 1 ]
job_id          | 680937397724741633
job_type        | CHANGEFEED
description     | CREATE CHANGEFEED FOR TABLE movr.rides, movr.users,
movr.vehicles INTO
's3://cockroachdefinitiveguide/movrFeed?AWS_ACCESS_KEY_ID=notAValidKey
&AWS_SECRET_ACCESS_KEY=redacted' WITH resolved = '10s',
updated
statement       |
user_name       | root
status          | running
running_status  | retryable error: retryable changefeed error: failed
to put s3 object: InvalidAccessKeyId: The AWS Access Key Id
you provided does not exist in our records.+
                | status code: 403, request id: D1QT6FJS60SNX461,
host id: k7e18TvfVgN50dZJ3JJlywApEjU+a3++ =
created         | 2021-08-02 03:46:42.332999+00:00:00
started         | 2021-08-02 03:46:42.587753+00:00:00
finished        | NULL
modified        | 2021-08-02 03:48:02.009441+00:00:00
fraction_completed | 0
error           |
coordinator_id  | NULL
```

Change Data Capture to Kafka

Enterprise changefeeds can be sent directly to Kafka topics, to which any interested downstream system can subscribe.

The first step is to create a topic in Kafka matching the table names for which feeds will be created.⁴ This can be done using the `kafka-topics` command:

```
$ kafka-topics --create --topic users --bootstrap-server localhost:9092
Created topic users.
```

You can also create topics using the Confluent Kafka console, as shown in [Figure 7-6](#).

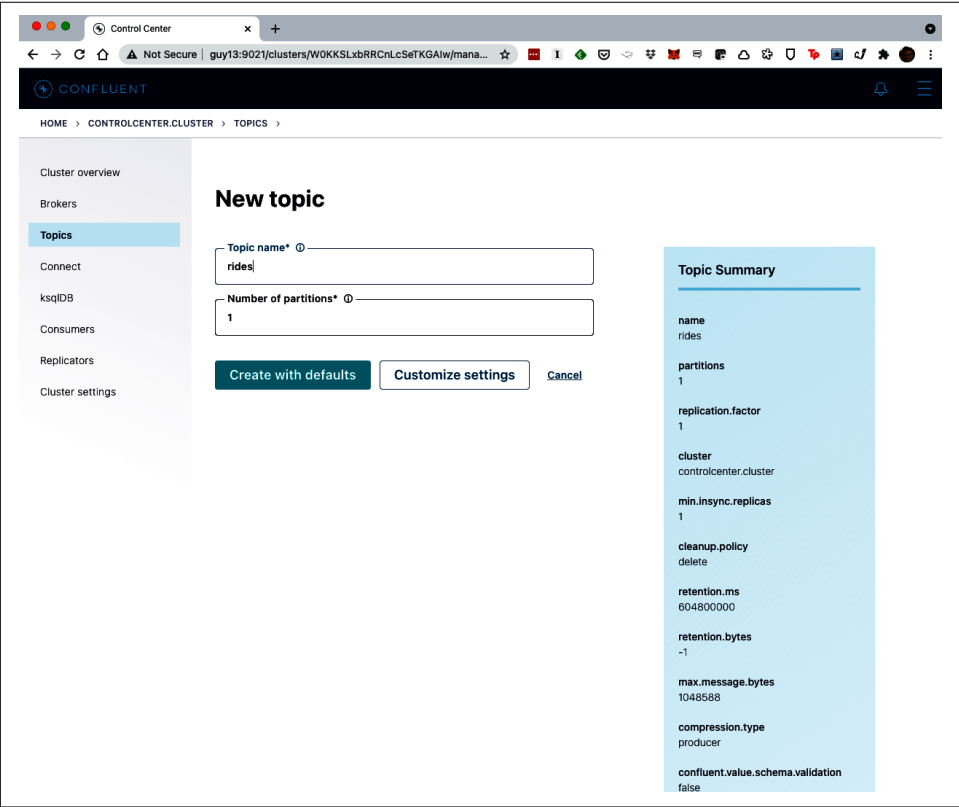


Figure 7-6. Creating a topic in the Confluent Kafka console

⁴ See the [CockroachDB documentation](#) for guidance on how to fine-tune topic names.

Once the Kafka topics are created, you can create a changefeed with a Kafka URL as the destination:

```
defaultdb> CREATE CHANGEFEED FOR TABLE movr.rides,movr.users INTO
'kafka://localhost:9092'
WITH updated, resolved='120s';
```

In this example, we have an unsecured Kafka server running on localhost. For enterprise deployments of Kafka, you would normally configure a connection to a broker and supply authentication credentials, as in this example:

```
'kafka://broker.address.com:9092?topic_prefix=bar&tls_enabled=true&
ca_cert=LS0tLS1CRUdJTiBDRVJUSUZ&sasl_enabled=true&sasl_user=petee&
sasl_password=bones&sasl_mechanism=SASL-SCRAM-SHA-256'
```

Avro is an alternative data format for changefeeds with Kafka. Like JSON, Avro documents are self-describing, but Avro data is more efficient and strongly typed. When using an Avro feed, specify the avro format and the location of the Confluent Kafka registry:

```
CREATE CHANGEFEED FOR TABLE movr.rides,movr.users
INTO 'kafka://localhost:9092'
WITH format = experimental_avro,
confluent_schema_registry = 'http://localhost:8081';

      job_id
-----
679859316344848385
(1 row)
```

Duplicate Change Stream Messages

Changefeeds can feed duplicate messages in some circumstances—the communication with the sink is not transactional, so in the event of a network error, a message may be retransmitted. Furthermore, unless you have an append-only workload, there could be many modification messages for a single row. You may have to implement some programmatic logic to ensure that the changefeed data is accurately represented in your target.

Change Data Capture to Pulsar

Changefeeds can also be sent to Apache Pulsar, a popular messaging/streaming platform. In this section, we'll configure a changefeed to send messages to Pulsar.

First, we'll need to create an instance of Pulsar. For this we'll use Docker, and for brevity we'll run Pulsar in standalone mode:

```
docker run \
-d \
```

```
-p 6650:6650 \  
--name pulsar \  
apachepulsar/pulsar:latest \  
bin/pulsar standalone
```

Next, we'll spin up CockroachDB:

```
cockroach demo --insecure --no-example-database
```

Then create a table and configure the changefeed (note that CDC queries can also be used, and—if you'd prefer—the `kafka://` scheme can be used instead of the `pulsar://` scheme):

```
CREATE TABLE orders (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  customer_id UUID NOT NULL,  
  total DECIMAL NOT NULL,  
  ts TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE CHANGEFEEED FOR TABLE orders INTO 'pulsar://localhost:6650';
```

Next, we'll create a Pulsar consumer. This will allow us to see the messages that CockroachDB's CDC produces.

```
docker run -it --rm \  
  apachepulsar/pulsar:latest \  
  bin/pulsar-client \  
    --url "pulsar://host.docker.internal:6650" consume orders -s test -n 0
```

With everything up and running, we're ready to start producing changefeeds. Let's create a variety of messages by way of INSERT, UPDATE, and DELETE:

```
INSERT INTO orders (id, customer_id, total) VALUES  
(  
  '2b74a3ad-864f-431d-8e3d-b8a20c6b4528',  
  '28eb37d5-fd06-4283-bac6-a6d690c5da0b',  
  10.99  
) ,  
(  
  '705bb573-cea9-4ffd-83e5-c2ab17121c28',  
  'e3ac589c-685d-4ead-a382-3fb1915ed166',  
  10.99  
) ,  
(  
  'b85f4c33-1685-4b74-9fb0-b62d1256ebf6',  
  '501fdb41-b0d1-431c-a474-e9f6848a409e',  
  10.99  
) ;  
  
UPDATE orders  
SET total = 20.99  
WHERE id = '2b74a3ad-864f-431d-8e3d-b8a20c6b4528';
```

```
DELETE FROM orders
WHERE id = '705bb573-cea9-4ffd-83e5-c2ab17121c28';
```

Once executed, our Pulsar consumer will receive changefeeds from CockroachDB and print them to the console (note that we've truncated UUIDs and removed empty fields for readability):

```
----- got message -----
content:{"Key":["b85f4c33..."],"Value":{"after":{"customer_id": "501fdb41...",
"id": "b85f4c33...", "total": 10.99, "ts": "2024-09-05T06:58:42.925856Z"}},
"Topic":"orders"}

----- got message -----
content:{"Key":["2b74a3ad..."],"Value":{"after":{"customer_id": "28eb37d5...",
"id": "2b74a3ad...", "total": 10.99, "ts": "2024-09-05T06:58:42.925856Z"}},
"Topic":"orders"}

----- got message -----
content:{"Key":["705bb573..."],"Value":{"after":{"customer_id": "e3ac589c...",
"id": "705bb573...", "total": 10.99, "ts": "2024-09-05T06:58:42.925856Z"}},
"Topic":"orders"}

----- got message -----
content:{"Key":["2b74a3ad..."],"Value":{"after":{"customer_id": "28eb37d5...",
"id": "2b74a3ad...", "total": 20.99, "ts": "2024-09-05T06:58:42.925856Z"}},
"Topic":"orders"}

----- got message -----
content:{"Key":["705bb573..."],"Value":{"after": null},
"Topic":"orders"}
```

As expected, we've consumed five messages, three for the INSERTs, one for the UPDATE, and another for the DELETE.

Change Data Capture to Snowflake

Once data from a changefeed is sent to Kafka or cloud storage, it can be consumed by a huge variety of downstream systems. Let's consider one of those possible integrations.

Like CockroachDB, Snowflake is a modern, cloud native SQL database. But where CockroachDB is optimized for transactional workloads, Snowflake is optimized for analytics. The two platforms are, therefore, very complementary.

We can use Enterprise CDC to automate the feeding of CockroachDB data changes into a Snowflake data warehouse. To start, we create a changefeed into an Amazon S3 bucket:

```
defaultdb>
CREATE CHANGEFEED FOR TABLE movr.rides,movr.users,movr.vehicles INTO
's3://cockroachdefinitiveguide/movrFeed?'
```

```

AWS_ACCESS_KEY_ID=my_access_key&
AWS_SECRET_ACCESS_KEY=my_secret_access_key'
  WITH updated, resolved='10s';
      job_id
-----
680913149274030081
(1 row)

```

Time: 373ms total (execution 373ms / network 0ms)

On the Snowflake side, we create a table to receive the changes. We use the VARIANT data type, which can store the JSON-structured documents from the changefeed:

```

GHARRISO#COMPUTE_WH@(no database).(no schema)>
CREATE DATABASE movrfeed;

```

```

+-----+
| status |
+-----+
| Database MOVRFEEED successfully created. |
+-----+
1 Row(s) produced. Time Elapsed: 0.309s

```

```

GHARRISO#COMPUTE_WH@MOVRFEEED.PUBLIC>
CREATE TABLE movr (changeFeed_record VARIANT);

```

```

+-----+
| status |
+-----+
| Table MOVRFEEED successfully created. |
+-----+
1 Row(s) produced. Time Elapsed: 0.327s

```

We associate our Amazon S3 bucket with a Snowflake *external stage*. A stage is an Amazon S3 bucket, Google Cloud Storage, or Azure container that contains data that will be loaded into Snowflake:

```

GHARRISO#COMPUTE_WH@MOVRFEEED.PUBLIC>
CREATE STAGE cdc_stage url='s3://cockroachdefinitiveguide/movrFeed/'
  credentials=(aws_key_id='my_access_key'
               aws_secret_key='kLkKzJS/my_secret_access_key')
  file_format = (type = json);

```

```

+-----+
| status |
+-----+
| Stage area CDC_STAGE successfully created. |
+-----+
1 Row(s) produced. Time Elapsed: 1.020s

```


The next step is to connect the stage with the table that will receive the changefeed. This is done by creating a Snowflake *pipe*. The pipe links the external stage with a Snowflake table:

```
GHARRISO#COMPUTE_WH@MOVRFEED.PUBLIC>
CREATE PIPE cdc_pipe auto_ingest = true
as
COPY INTO movr FROM @cdc_stage;
```

```
+-----+
| status |
+-----+
| Pipe CDC_PIPE successfully created. |
+-----+
1 Row(s) produced. Time Elapsed: 1.204s
```

We need to obtain the `notification_channel` associated with the pipe for the next section. Use the `SHOW PIPES` command to extract pipe information:

```
GHARRISO#COMPUTE_WH@(no database).(no schema)>
!set output_format=expanded;
GHARRISO#COMPUTE_WH@(no database).(no schema)>
show pipes;
*****[ 1 ]*****
created_on      | 2021-08-01 18:42:05.898 -0700
name            | CDC_PIPE
database_name   | MOVRFEED
schema_name     | PUBLIC
definition      | COPY INTO movr FROM @cdc_stage
owner           | SYSADMIN
notification_channel | arn:aws:sqs:ap-southeast-2:757948394836:
sf-snowpipe-AIDA3A6J6VVKNSPOHQ7NX-EswnTwnSGDYt8ve10d5hnA
comment         |
integration     | NULL
pattern         | NULL

1 Row(s) produced. Time Elapsed: 0.276s
```

The notification channel is used to trigger data loads from the Amazon S3 bucket into Snowflake. We want to configure the Amazon S3 bucket so that every time a change is made to the bucket, it notifies the pipe.

From the Amazon S3 bucket properties, find the “Event notifications” section and click “Create event notification” (Figure 7-7).

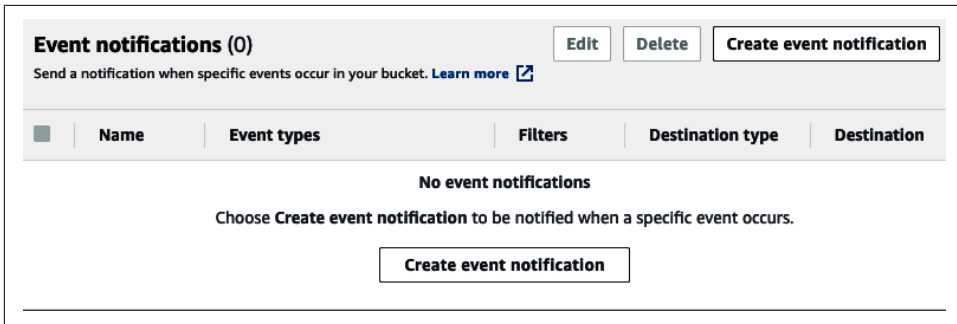


Figure 7-7. Creating a topic in the Amazon S3 console

Under “Events types,” specify all the object create events (Figure 7-8).

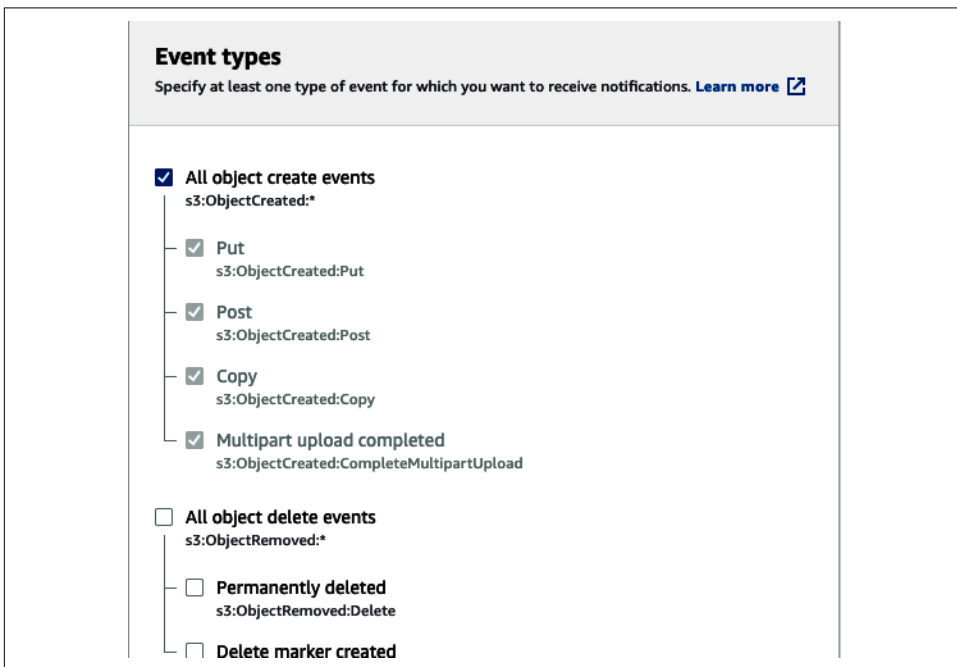


Figure 7-8. Amazon S3 event types for event notification

Finally, in the Destination section, specify “SQS queue” as the destination and paste in the `notification_channel` value from the Snowflake `SHOW PIPES` command (Figure 7-9).

Destination

ⓘ Before Amazon S3 can publish messages to a destination, you must grant the Amazon S3 principal the necessary permissions to call the relevant API to publish messages to an SNS topic, an SQS queue, or a Lambda function. [Learn more](#)

Destination

Choose a destination to publish the event. [Learn more](#)

☐ Lambda function

Run a Lambda function script based on S3 events.

☐ SNS topic

Send notifications to email, SMS, or an HTTP endpoint.

☒ SQS queue

Send notifications to an SQS queue to be read by a server.

Specify SQS queue

☐ Choose from your SQS queues

☒ Enter SQS queue ARN

SQS queue

arn:aws:sqs:ap-southeast-2:757948394836:sf-snowpipe-AIDA3A6J6VVKNSPOHQ7NX

Figure 7-9. Amazon S3 event destination

We’ve now hooked up a changefeed to an Amazon S3 bucket and the Amazon S3 bucket to a Snowflake pipe. If we query the Snowflake `movr` table, we can see it now contains the changefeed records that have been created as changes to the `movr` tables are processed:

```
GHARRISO#COMPUTE_WH@MOVRFEEED.PUBLIC>
select * from movr limit 10;
+-----+
| CHANGEFEED_RECORD |
+-----+
| {                  |
|   "after": {       |
|     "city": "rome", |
|     "end_address": null, |
|     "end_time": null, |
|     "id": "000288ee-56e9-42f3-9991-9831a668c76a", |
|     "revenue": null, |
|     "rider_id": "cfeb35ff-a826-439e-83da-c633a158315d", |
|     "start_address": "54948 Pool St Via Apt. 28", |
|     "start_time": "2021-07-02T07:47:58.526734", |
|     "vehicle_city": null, |
|     "vehicle_id": "e9f53976-9fc0-4a7a-b049-34b6ee286be7" |
|   },              |
+-----+
```

```
|   "key": [                               |
|       "rome",                             |
|       "000288ee-56e9-42f3-9991-9831a668c76a" |
|   ],                                       |
|   "updated": "1627871552959954000.000000000000" |
| }                                         |
```

Summary

In this chapter, we looked at how to migrate to CockroachDB and how to integrate CockroachDB with other systems.

CockroachDB's MOLT tools simplify the process of migrating to CockroachDB. They allow you to migrate schemas and data, verify the integrity of migrated data, and perform cutovers from your existing database to CockroachDB with minimal disruption.

CockroachDB provides import facilities that allow data to be loaded from flat files. These files can be loaded to storage locations within the cluster—userfile and node-local storage—or from cloud storage locations such as S3, Google Cloud Storage, or Azure containers.

Migrating from another SQL database to CockroachDB is generally a three-stage project. First, we extract DDL from the source systems and modify it to support CockroachDB syntax and best practices. Second, we dump data from the source system into a flat-file format. Finally, we import the data into a CockroachDB database. For a production system, there are usually additional configurations around synchronization and switchover.

We can export data from CockroachDB to flat files using the `EXPORT` command. Alternatively, we can capture changes to nominated tables with CDC. Core CDC allows these changes to be managed programmatically, while Enterprise CDC allows changefeeds to be automatically forwarded to cloud storage or to Kafka.

In the next chapter, we'll learn how to find and fix slow queries, harnessing constructs like indexes and computed columns to maximize query performance.